

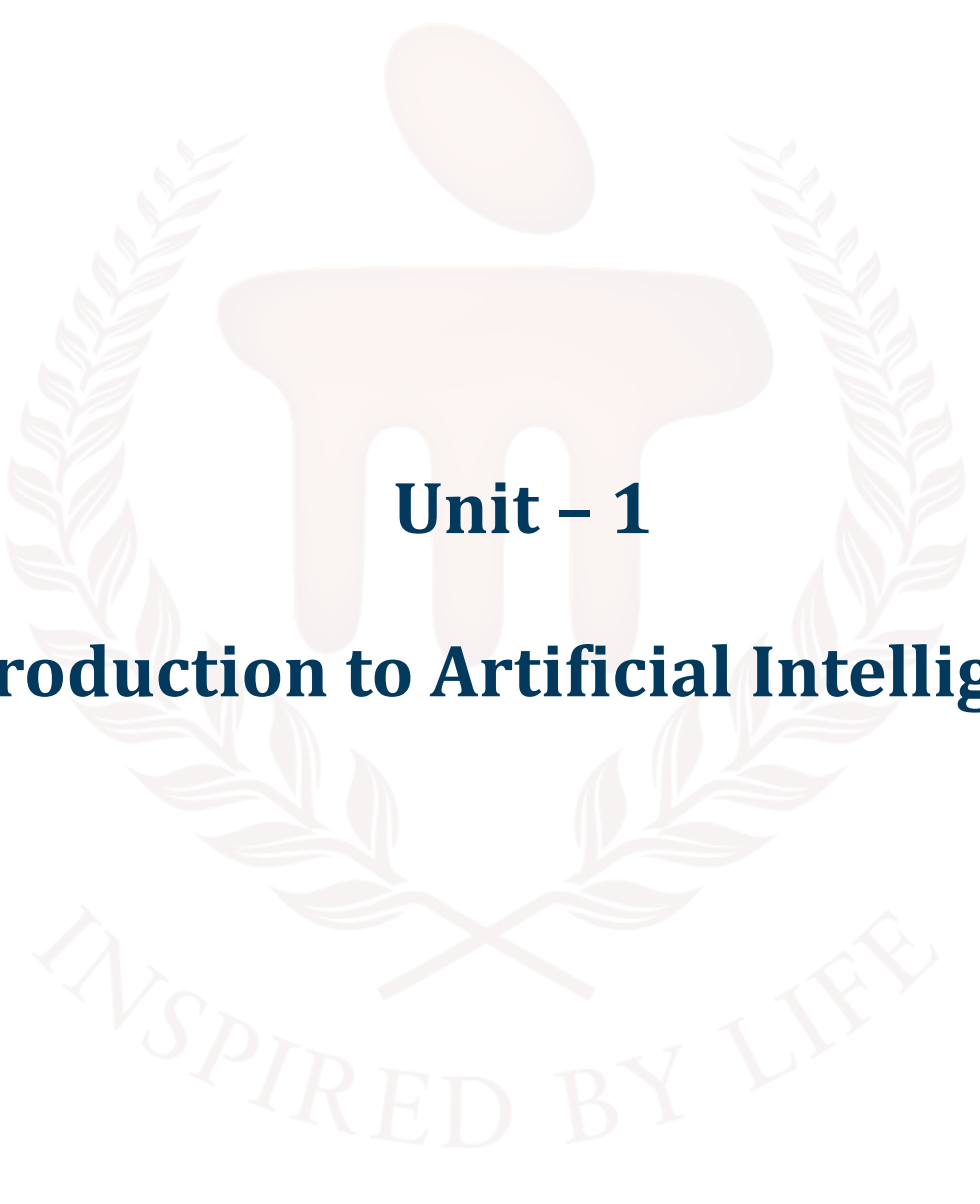
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 1

Introduction to Artificial Intelligence

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Importance of Artificial Intelligence	-	1	5 - 7
3	AI Approaches	1	2	8 - 11
4	Applications of AI	-	3	12 -14
5	Time- History of AI	-	-	15 -16
6	Intelligent System	-	-	17
7	Intelligence Types	-	-	18
8	The Building Blocks of Intelligence	2	-	19 -21
9	Agent and Environment	3, 4, 5, 6	4	22 - 27
10	The Nature of Environments	-	-	28
11	Summary	-	-	29
12	Glossary	-	-	30 -31
13	Terminal Questions	-	-	32
14	Answers	-	-	33 -35
15	References	-	-	36

1. INTRODUCTION

In this unit, learners will gain a foundational understanding of Artificial Intelligence (AI) by exploring its definition, significance, and wide-ranging applications by defining AI and explaining its role in modern technology, highlighting why AI is considered a revolutionary force in various industries. Learners will acquire knowledge regarding the importance of AI, particularly in automation, decision-making, and problem-solving, which have transformed fields such as healthcare, finance, and robotics. Learners will also explore applications of AI, showcasing real-world use cases in areas such as self-driving cars, virtual assistants, recommendation systems, and fraud detection. Through these examples, we will understand how AI is integrated into daily life and business processes to improve efficiency and accuracy.

Lastly, the notion of intelligent agents is introduced, explaining how AI-driven systems perceive their environment, make decisions, and take actions to achieve specific goals. Learners will explore different types of intelligent agents, including simple reflex agents, goal-based agents, and learning agents, gaining insight into how these agents function in AI-based systems.

1.1. Learning Objectives

After studying this unit, you should be able to:

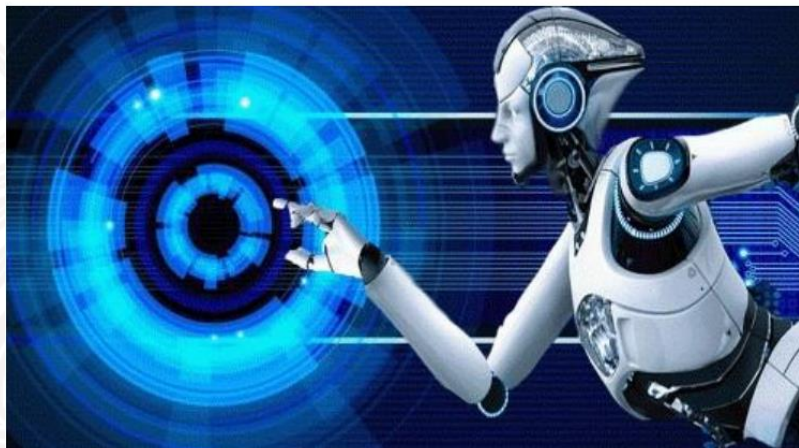
- Define Artificial Intelligence (AI) and its significance
- Describe the key components of AI
- Explain the goals of AI
- Identify the advantages and disadvantages of AI
- Outline the history of AI.



2. IMPORTANCE OF ARTIFICIAL INTELLIGENCE

Definition of Artificial Intelligence

A branch of computer science known as Artificial Intelligence (AI) aims to program machines to mimic human intelligence. When people talk about "the science and engineering of making intelligent machines, especially intelligent," they usually mean John McCarthy was the first to define AI. A subfield of computer science, artificial intelligence (AI) aims to give computers the ability to reason like humans when faced with difficult issues.



The term AI implies "a man-made thinking power" since it is a combination of the words "artificial" meaning "man-made" and "intelligence" meaning "thinking power."

As a result, we may say: "It is a branch of computer science by which we can create intelligent machines which can behave like humans, think like humans, and make decisions."

When a computer is able to learn, reason, and solve problems in the same way that a human would, we say the machine has artificial intelligence. The beauty of AI is that it eliminates the need to pre-program machines to accomplish certain tasks; yet, it is possible to program algorithms into machines that can perform tasks that require human intellect.

Some argue that AI has been around for a while, citing Greek mythology as evidence that mechanical men and their ability to mimic human behaviour existed in the distant past.

Importance of Artificial Intelligence

We need to know why we should learn about AI and its significance before we can study it. To understand AI for the following reasons:

- You may build apps or gadgets that address real-world problems (such as traffic, health, marketing, etc.) with ease and precision with the help of AI.
- You may create your very own virtual assistant with the use of artificial intelligence (AI), just like Siri, Google Assistant, Cortana, etc.
- You can create robots with artificial intelligence that can operate in hazardous environments where people might not be able to make it.
- Artificial intelligence paves the way for additional innovative products, technologies, and possibilities.
- From simplifying complicated decision-making to automating mundane chores, artificial intelligence (AI) is revolutionising many parts of contemporary life.

Artificial intelligence (AI) has several benefits, one of which is increased productivity and efficiency. AI-powered systems can handle massive volumes of data and execute repetitive activities at a far faster rate than people, which in turn reduces operational expenses and errors.

Enhancing decision-making in many businesses is another crucial use of AI. A number of medical tasks, including disease diagnosis, outcome prediction, and treatment plan customisation, can benefit from artificial intelligence (AI). Financial institutions may automate trade, manage risks, and detect fraud with the help of AI-driven algorithms. Autonomous vehicles and intelligent traffic control systems are two more ways that AI is improving transportation safety.

The part that AI plays in innovation and solving problems is another major influence. Intelligent virtual assistants, automated customer service, and improved security systems are all products of AI-enabled developments in domains like computer vision, robotics, and natural language processing. Online retailers, media streaming services, and marketers all benefit from AI-driven recommendation systems that tailor their services to each individual customer.

Climate modelling, energy efficiency, and conservation initiatives are just a few areas where AI has aided scientific inquiry and environmental sustainability. Industries can optimise resources, decrease waste, and develop sustainable solutions with the help of data powered by AI. The growing influence of AI on the future of many sectors and our daily lives is becoming more apparent as the technology develops further. Responsible research and regulation can ensure that the benefits of AI outweigh the

hazards, making it a transformative force for growth and innovation, despite the ethical and security issues it poses.

SELF-ASSESSMENT QUESTIONS - 1

Fill in the Blanks:	
1	Artificial Intelligence (AI) is the branch of computer science that focuses on creating machines capable of performing tasks that typically require _____.
Multiple Choice Questions	
2	Which of the following is a major benefit of AI in industries? a) Increased efficiency and automation b) Reduced accuracy in tasks c) Increased manual labour d) Slower decision-making
3	How does AI contribute to healthcare advancements? a) By replacing doctors entirely b) Through accurate diagnosis and personalised treatments c) By making medical decisions randomly d) By slowing down patient care services
4	Which of the following is an example of AI improving customer experience? a) Automated chatbots providing 24/7 support b) Delayed responses in customer service c) Manual data entry for queries d) Increasing the waiting time for customers

3. AI APPROACHES

One key distinction between artificial intelligence and human intelligence is the rationality with which humans think and act. Various individuals and groups have historically pursued each of the four aforementioned AI methodologies.

	Like humans	well
Think	GPS	Rational Agents
Act	Eliza	Heuristic System

Figure 1: AI approaches

Develop algorithmic representations of formal models of knowledge representation, reasoning, learning, memory, and problem solving.

Provably correct and guaranteeing systems are frequently emphasised, arriving at the best possible outcome

Act well

- Create a workable result, which may not be absolutely correct, given a set of inputs.
- Any approach, trick, simplification, rule of thumb, or other tool that significantly reduces the search for solutions in big problem spaces is called a heuristic.

In truth, heuristics don't promise any answer, much less an optimal one: A useful heuristic can only be described as providing solutions that are adequate the majority of the time.

Think like humans

Method based on cognitive science. Pay attention to the thinking process in addition to behaviour and I/O. "How" outcomes were obtained should be reflected in the computational model. Develop fresh ways of assessing cognitive theories and a new vocabulary to describe them. The Global Positioning System (GPS): The goal wasn't merely to have it act human (like ELIZA), but also to make it reason in a way that would be familiar to a human facing the same problem.

Act like humans

Take a behaviourist tack; they care only that your outcomes are comparable to human outcomes, regardless of the means by which you achieve them. One such example is the Turing Test, developed by Alan Turing in 1950.

Example:

ELIZA: a program that passed the Turing Test by simulating a psychotherapist's interaction with a patient. Joel Weizenbaum coded it at MIT from 1964 to 1966. Doctor was the first script. The script resembled regular expressions in that it was a straightforward set of syntactic patterns. The input bits could be included in the corresponding reply for each pattern following simple modifications (my → your). Reactions surprised Weizenbaum; psychiatrists saw promise. Humans clearly took on a more humanlike form. A lot of people thought it fixed the NL issue.

Goals of AI

Creating intelligent systems that can mimic human-like reasoning, learning, thinking, and decision-making is the main objective of artificial intelligence.

- Robots in the cloud hope to maximise productivity by decreasing manual labour by automating routine operations.
- Improve decision-making through data analysis of massive proportions.
- Find ways to solve problems and adapt that are similar to human intellect.
- Build systems that can learn from their own mistakes and get better with time.
- Facilitate human-machine contact through the application of robotics, computer vision, and natural language processing.

Advantages of AI

- Improved Efficiency & Automation: By eliminating human mistake and speeding up formerly laborious processes, AI is revolutionising the way we work.
- Better Decision-Making: Data-driven insights in healthcare, finance, and business strategy are provided by AI's rapid analysis of big datasets.
- 24/7 Availability: Artificial intelligence systems are perfect for monitoring and customer assistance because, unlike humans, they can work nonstop.
- Reduction in Human Error: Mistakes in areas like medical diagnosis and financial predictions can be drastically reduced when AI-powered systems are properly implemented.
- Enhancing Creativity & Innovation: AI is a game-changer in several fields, including healthcare, robotics, space travel, and personalised technologies.

- **Risk Reduction in Hazardous Jobs:** Artificial intelligence has the potential to greatly improve safety in high-risk fields, including space exploration, disaster relief, and the handling of hazardous materials.

Disadvantages of AI

- **High Implementation Costs:** AI development and upkeep necessitate substantial financial outlays for hardware, software, and human knowledge.
- **Job Displacement:** Some industries may see a rise in unemployment as a result of automation and AI-driven systems displacing human workers.
- **Lack of Creativity & Human Judgment:** The human abilities of intuition, creativity, and emotional intelligence are absent from AI, which is dependent on data and algorithms.
- **Ethical & Privacy Concerns:** There are ethical concerns with the usage of AI due to the potential for its misuse in areas such as biased decision-making, deepfakes, and spying.
- **Dependence on Data & Training:** Large volumes of accurate data are essential for AI systems, yet inaccurate or unjust results might result from biased or missing data.
- **Security Risks:** Security risks and data breaches could occur as a result of cyberattacks, hacking, or misuse of systems led by artificial intelligence

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
5	Which of the following is NOT a commonly used approach in AI? a) Logical Reasoning Approach b) Machine Learning Approach c) Quantum Computing Approach d) Expert System Approach
6	What is one of the major advantages of AI? a) Reduces human errors and increases accuracy b) Requires constant human intervention c) Slows down decision-making processes d) Makes jobs more difficult and inefficient
7	Which of the following is a disadvantage of AI?

	a) AI can lead to job automation, reducing employment opportunities b) AI makes human efforts more tedious c) AI reduces productivity in industries d) AI does not require data to function effectively
8	Which AI approach focuses on using past experiences to improve performance? a) Symbolic AI b) Machine Learning c) Rule-Based Systems d) Search-Based AI



4. APPLICATIONS OF AI

The theories and methods presented in AI literatures were finally embraced by industries in 1981, marking the beginning of AI's meteoric rise to the status of an industry. A wide range of industries make use of AI principles. Here are only a handful of them.

- **Game Playing:** Strategy games like chess, poker, tic-tac-toe, etc., rely heavily on AI since machines can generate a vast array of alternative moves using heuristic knowledge.
- **Speech Recognition:** When a person speaks to an AI, it can understand what they're saying in terms of complete phrases and their meanings. Various accents, slang terms, background noise, changes in human sounds caused by cold, and so on are all within its capabilities.
- **Understanding Natural Language:** In a world where computers can comprehend natural languages, people can finally communicate with one another in their own tongues. Currently, only a small subset of domains can have their corresponding knowledge imparted to computers in order for them to comprehend the text.
- **Computer Vision:** With the use of computer vision, the computer is able to interpret the signals and respond appropriately. One example of a system is the one used at airports, which uses facial recognition software. - Health evaluation
- **Expert Systems:** In order to embody domain experts' knowledge in a computer program for task execution, a "knowledge engineer" conducts interviews with them. Whether or if the necessary cognitive processes are within the capabilities of current AI determines how effectively this works. There were a lot of letdowns when this wasn't the case. In 1974, MYCIN was one of the earliest expert systems that could detect and recommend therapies for blood-borne bacterial illnesses. Assuming its limitations were acknowledged, it outperformed both medical students and actual doctors. To be more specific, its ontology encompassed germs, symptoms, and therapies but omitted patients, physicians, healthcare facilities, mortality, recuperation, and temporal occurrences. The consideration of just one patient dictated its interactions. It is evident that the knowledge engineers imposed a preconceived framework on the advice of the experts they interviewed, as the experts in question were knowledgeable about patients, physicians, mortality, rehabilitation, etc. Users' common sense determines the present expert systems' usefulness.

- **Heuristic Classification:** Using several sources of data to classify data into predetermined categories is one of the most practical types of expert systems available today in the field of artificial intelligence. Giving advice on whether to approve a planned credit card purchase is one example. There is data accessible on the credit card holder, their payment history, the product they are purchasing, and the store they are purchasing it from.
- **Intelligent Robots:** Light, heat, temperature, motion, sound, bumps, and pressure are just some of the physical data that robots may pick up with their sensors. They display intelligence through their efficient computers, many sensors, and vast memory.

Apart from regular application, there are some domain tasks covered by Artificial Intelligence which are listed as follows:

- Perception - Vision - Speech
- Natural Language - Understanding - Generation - Translation
- Common Sense Reasoning
- Robot Control
- Games - Chess - Backgammon - Checkers - Go
- Mathematics - Geometry - Logic - Integral Calculus
- Expert Tasks - Design - Fault finding - Manufacturing Planning
- Scientific Analysis
- Medical Diagnosis
- Financial analysis

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
9	Which of the following is a major application of AI in the healthcare industry? a) Virtual reality gaming b) AI-powered diagnosis and robotic surgery c) Traditional paper-based medical records d) Manual prescription writing
10	How is AI used in the finance sector? a) By automating fraud detection and risk assessment b) By manually processing all transactions c) By reducing automation and increasing paperwork d) By eliminating the need for online banking
11	Which of the following is an AI application in transportation? a) Self-driving cars and traffic management systems b) Manual ticket booking at railway stations c) Traditional roadmaps instead of GPS d) Human-driven taxis without AI assistance



5. THE TIME- HISTORY OF AI

Intelligence is the bedrock upon which every human technology and civilisation rests. The pursuit of knowledge, however, was thought to be an affront to divine or natural principles. Think about Prometheus from Greek mythology or Eve from the Bible. The reasonable place to begin, AI has its roots in the works of Aristotle. Careful investigation allowed him to formalise the discoveries, wonders, and worries about nature into a disciplined concept. He believed that all knowledge could be derived by studying thought itself. In his 'Logic', he sought to determine whether certain statements can be deemed "true" due to their relationships with other known truths. Notable figures from this school of thought include Gottlob Frege, Kurt Godel, Alan Turing, Bertrand Russell, and others. The significant finding that radically altered worldviews was the discovery of Copernicus, which proved that Earth is not the centre of the cosmos but rather one of its components. Even though it went against long-held religious beliefs and dogmas, the new insight that

- Our perceptions of the universe might not match up with reality was revolutionary.
- The human mind does not always correspond with the actual world.
- There is a difference between one's own ideas and those regarding the world around them.

Although distinct, the conventional wisdom holds that they are not inherently distinct from one another. Using formal mathematics or logic, mental processes can be described in the same way as physical processes.

The below given table provides a summary of the historical events:

Year	Event
1923	The word "robot" is first used in English in Karel Čapek's play "Rossum's Universal Robots" (RUR), which premieres in London.
1943	3 Foundations for neural networks laid.
1945	The term "robotics" was first used by someone who attended Columbia University, Isaac Asimov.
1950	In Computing Machinery and Intelligence, Alan Turing presented the Turing Test, a method for assessing cognitive abilities. Detailed Analysis of Chess Playing was produced by Claude Shannon in an effort to
1956	The term "artificial intelligence" was first used by John McCarthy. Launch of Carnegie Mellon University's inaugural artificial intelligence program.

1958	Programming language LISP for artificial intelligence is invented by John McCarthy.
1964	Computers can comprehend plain language to an adequate degree to accurately solve algebra word problems, according to Danny Bobrow's dissertation at MIT.
1965	An interactive puzzle called ELIZA was created by Joseph Weizenbaum at MIT. It can carry on a conversation in English.
1969	Researches at Stanford University Conceived of and built Shakey, a robot with mobility, vision, and intelligence
1973	Edinburgh University's Assembly Robotics group created Freddy, the Famous Scottish Robot, which can use vision to find and build models.
1979	Stanford Cart was constructed as the initial computer-controlled autonomous vehicle.
1985	The drawing program Aaron was developed and showcased by Harold Cohen.
1990	Impressive progress in every domain of artificial intelligence: Machine learning's major breakthroughs • Case-based reasoning • Game development • Vision and VR • Data mining and web crawling • Natural language processing and translation
1997	Former world chess champion Garry Kasparov is defeated by the Deep Blue Chess Program.
2000	Consumers can now purchase interactive robot pets. Kismet, an emoticon-displaying robot, is on display at MIT. Nomad, an autonomous robot, searches for meteorites in Antarctica's most inaccessible areas.

6. INTELLIGENT SYSTEM

The computational, cognitive, perceptual, memory-storing, problem-solving, and adaptability abilities that make up an intelligent system are the building blocks of artificial intelligence. AI-powered software systems, including chatbots, expert systems, and others, can also be considered intelligent systems.

To put it simply, anything that comes equipped with a useful computer that is not typically general-purpose and has access to the Internet. Even while embedded systems may handle and analyse data at a high level, they are often only used for tasks that are directly related to the host machine.

Intelligent systems can be found in a wide variety of everyday objects, including, but not limited to, point-of-sale (POS) terminals, digital televisions, traffic lights, smart meters, cars, digital signage, and aeroplane controls. The emerging internet of things (IoT) relies heavily on built-in intelligence, allowing for the tagging of virtually any object and its subsequent automated data transfer across networks, all without the need for any kind of human or computer interaction.

7. INTELLIGENCE TYPES

Intelligence	Detailed Description	Example
Linguistic intelligence	Fluency in spoken language, including phonology (the study of speech sounds), syntax, as well as semantics, which deals with meaning.	Narrators, Orators
Musical intelligence	The capacity to comprehend, produce, and convey auditory meanings; familiarity with rhythm, pitch.	Musicians, Singers, Composers
Logical-mathematical intelligence	The skill of making use of and comprehending connections even when no external object or action is present. Gaining a difficult concepts.	Mathematicians, Scientists
Spatial intelligence	The capacity to take in spatial or visual data, manipulate it, and then recreate images visually without locate the items, build three-dimensional pictures of them, and manipulate their position and orientation.	Map readers, Astronauts, Physicists
Bodily-Kinesthetic intelligence	Having command of fine motor skills, the capacity to move one's whole body or a limb to accomplish tasks or access fashionable items	Players, Dancers
Intra-personal intelligence	Being able to differentiate between one's own emotions, goals, and drives.	Gautam Buddha
Interpersonal intelligence	The skill of understanding and differentiating between other people's thoughts, feelings, and intentions.	Mass Communicators, Interviewers.

8. THE BUILDING BLOCKS OF INTELLIGENCE

The building blocks of Intelligence are:

- Reasoning
- Learning
- Problem Solving
- Perception
- Linguistic Intelligence

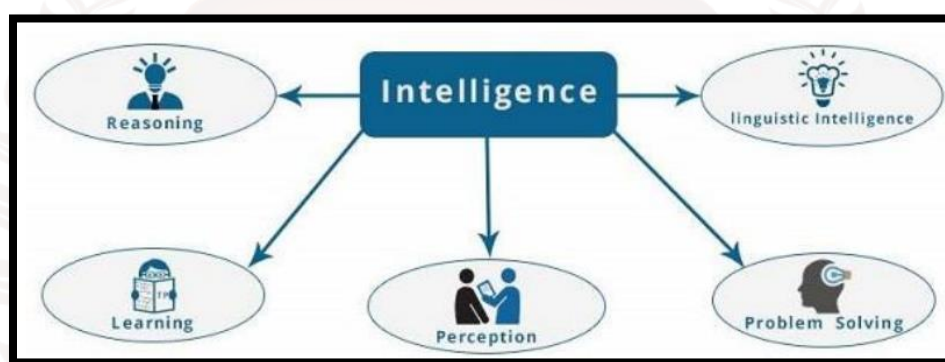


Figure 2: The Building Blocks of AI

a. Reasoning: We can give the groundwork for judgement, decision-making, and prediction through this set of activities. In general, two kinds –

- Inductive Reasoning
- Deductive Reasoning

Inductive Reasoning:

- To make broad generalisations, it performs particular observations.
- In inductive reasoning, the conclusion might be incorrect even if the premises are valid.
- Tajush is an educator, for instance. In his spare time, Tarush studies. That is why every educator is a student at heart.

Deductive Reasoning:

- It takes a broad claim as its starting point before narrowing in on a precise, logical conclusion by considering all of the possible outcomes.
- If something is true about a class generally, then it is true for all members of that class.

- For instance, all women who are 60 years old or older are grandparents. The age of Vijaya is 65. Thus, Vijaya is a grandparent.

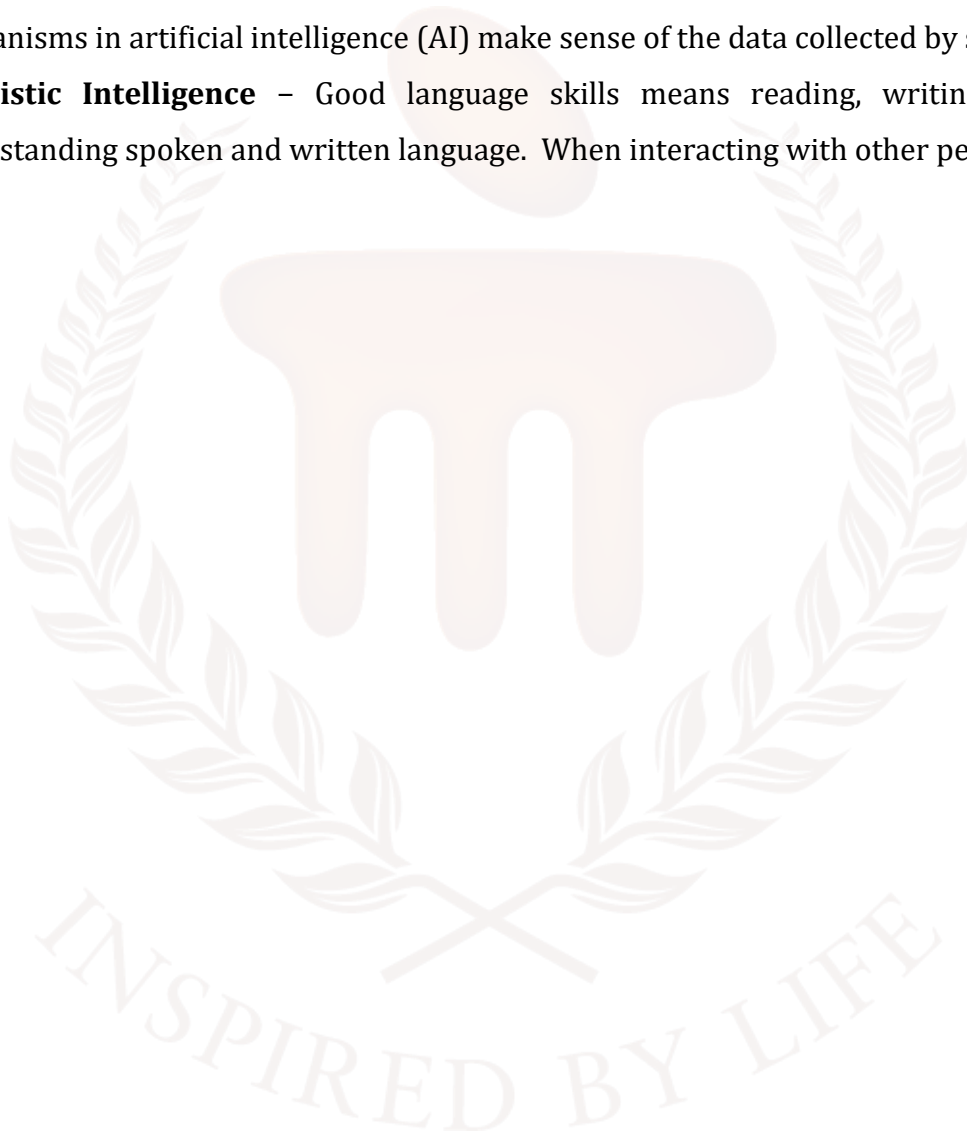
b. Learning – Literally, it's the process of learning something new, whether that's through books, videos, lectures, or hands-on experience. Subjects in the study gain a heightened sense of awareness through learning.

- Only humans, certain animals, and systems powered by artificial intelligence (AI) have the capacity to learn.
- One way that people learn is by auditory learning, which involves absorbing information through the sense of hearing. Listening to audio lectures that have already been recorded is one example.
- To gain knowledge through recalling sets of experiences that have been observed or encountered is known as episodic learning. Everything is neat and in its place.
- Learning through precise muscular action is known as motor learning. Examples include writing, selecting objects etc.
- Learning through observing and mimicking others is said to be as observational learning. A youngster may attempt to observe and mimic her parents in order to learn.
- Learning to identify previously viewed stimuli is said to be as perceptual learning. For instance, circumstances and several possibilities identified and categorised.
- Being able to differentiate between different stimuli based on relational features instead of absolute properties is what Relational Learning is all about. If your potatoes were overly salty the last time you cooked them, try cooking them with "little less" salt, such as half a tablespoon.
- Visual stimuli including images, colours, maps, etc. are used in spatial learning. As an example, before setting off on a journey, one can mentally map out the route.
- "Stimulus-Response Learning" refers to the process of learning to carry out a certain action in response to the presence of a specific stimulus. When a dog hears the doorbell, it will likely lift an ear to listen.

c. Problem-Solving – The process involves identifying a desired outcome from the current situation and then trying to achieve it through the pursuit of a path that is hindered by obstacles, some of which are known and others of which are unknown. An essential aspect of problem-

solving is determining, from a collection of alternatives, which one is best suited to accomplish a goal.

- d. Perception** It's the act of gathering, deciphering, choosing, and arranging data from the senses. To perceive, one must first sense. The sensory organs in humans help in perception. Perception mechanisms in artificial intelligence (AI) make sense of the data collected by sensors.
- e. Linguistic Intelligence** – Good language skills means reading, writing, speaking, and understanding spoken and written language. When interacting with other people, it's crucial.



9. AGENT AND ENVIRONMENT

Everything that can take in data from its surroundings using sensors and then manipulate that data using effectors is considered an agent.

- **Human Agent:** The skin, nose, ears, and eyes of a human being run perpendicular to the detectors, as well as other organs like the limbs, mouth, and hands that function as effectors.
- **Robotic Agent:** Rather than using traditional sensors like cameras and infrared range finders, a robotic agent can do all of these tasks using a variety of motors and actuators.
- **Software Agent:** Bit strings are the code for a software agent's programs and operations.

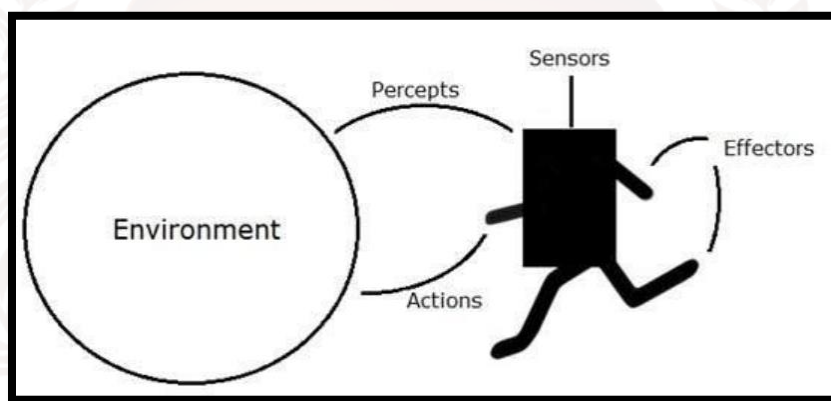


Figure 3: Agent and Environment

Agent Terminology

- **Performance Measure of Agent** – How effective an agent is is defined by these standards.
- **Behaviour of Agent** – It is the next step an agent takes in response to a specific set of perceptions.
- **Percept** – these are the visual cues that an agent receives at any particular time.
- **Percept Sequence** – All that an agent has perceived up to this point is documented in it.
- **Agent Function** – the map connects the precept sequence to an action.

Rationality

Being rational is nothing more than being sensible, having a strong sense of judgement, and being reasonable. Expected behaviours and results based on the agent's perceptions are fundamental to rationality. An integral aspect of reason is acting in a way that will yield meaningful information.

Ideal Rational Agent: To achieve optimal performance, a rational agent should operate in accordance with its internal database and perceptual sequence.

An agent's rationality is contingent upon -

- The success level is decided by looking at the performance indicators.
- Up until this point, the agent's perception sequence.
- The information the agent already has about its surroundings.
- The potential activities of the agent.

Any rational agent will always perform what's right; the right action is the one that maximises the agent's success in the current percept sequence. In order to identify the issue that the agent aims to fix, the PEAS framework is used.

Intelligent Agent Structure

The structure of the agent can be seen as -

- Architecture + Agent Program = Agent
- Agent programs are implementations of agent functions, while architecture refers to the machinery on which agents perform.

a. Simple Reflex Agents

- Their surroundings is entirely visible, and they are rational beings if and only if they make good decisions based on what they see right now.
- An action can be mapped to a state using the Condition-Action Rule.

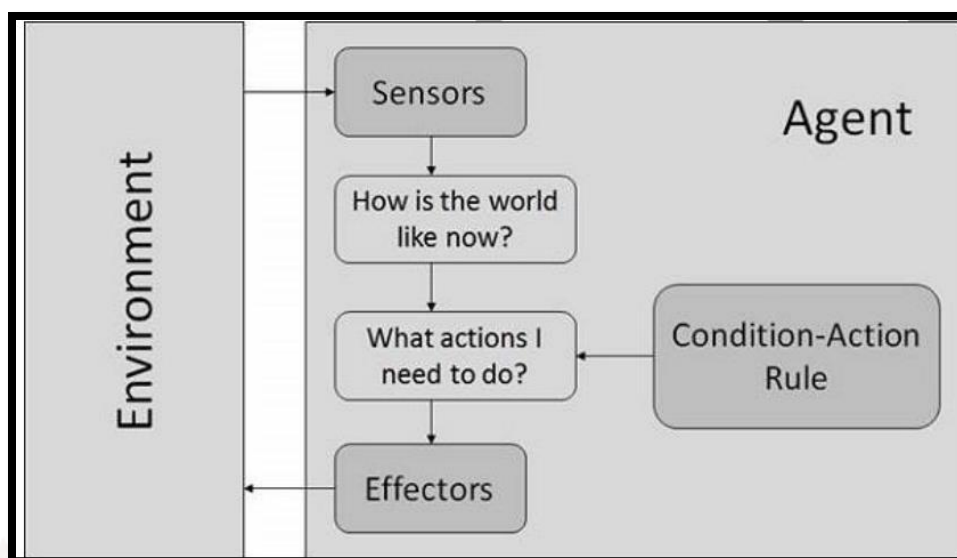


Figure 3a: Agents and Environments

Agents and Environments in Figure 3a illustrate how an intelligent agent operates within an environment by continuously sensing, processing, and acting.

On the left side of the diagram is the Environment, which represents the external world where the agent exists and interacts. The Agent receives input from the environment through Sensors. These sensors collect data and help the agent answer the question, "How is the world like now?" This step is crucial because the agent's understanding of its surroundings is based on real-time input.

Once the agent has this perception, it evaluates the situation and asks, "What actions do I need to do?" This decision-making process is influenced by a predefined set of Condition-Action Rules. These rules act like a guide, telling the agent what action to perform based on specific conditions detected in the environment.

After deciding the appropriate action, the agent uses its Effectors to carry out the chosen task. These effectors are responsible for interacting back with the environment, resulting in a response or change. The flow is continuous: the agent senses changes, processes the information, decides on an action using condition-action rules, and performs that action. This closed loop allows the agent to function effectively and adapt to different situations in its environment.

b. Model Based Reflex Agents

- To make decisions, they consult a world model. They have a condition inside themselves.
- Understanding "how the things happen in the world" is embodied by the model.
- Perceptual history determines the internal state, It stands for the attributes of the existing state that have not yet been observed.

- Data included regarding the following is necessary for updating the state:
- The rate of world evolution. The impact of the agent's actions on the world.

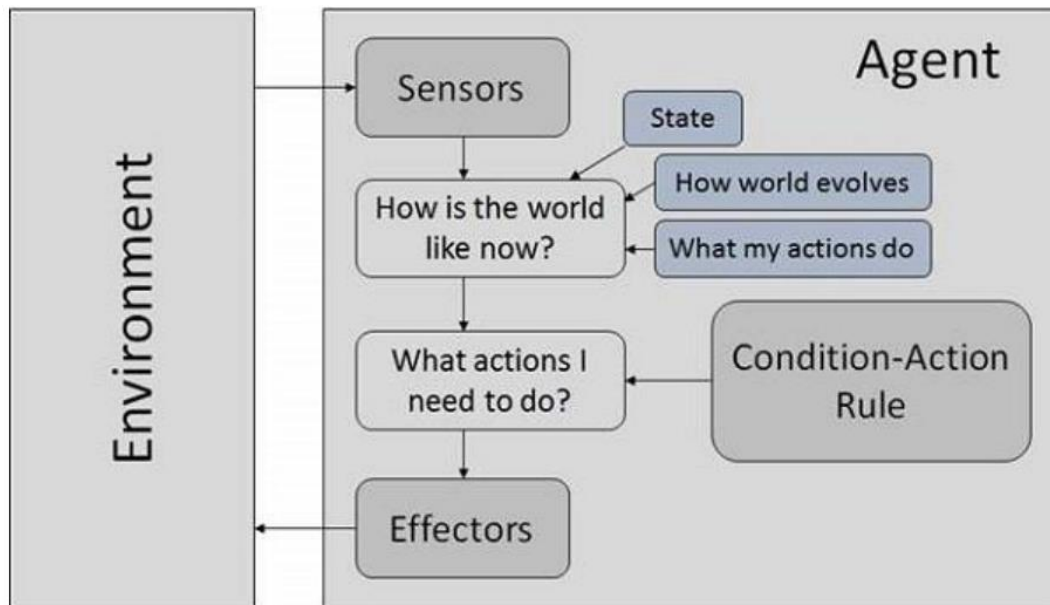


Figure 3b: Agents and Environments with updated Components

The three new components highlighted in Figure 3B has:

1. **State** – This represents the agent's internal memory or current understanding of the environment. It helps the agent keep track of relevant information gathered from the sensors over time.
2. **How world evolves** – This component allows the agent to predict or understand how the environment may change based on current conditions. It improves the agent's ability to plan actions in dynamic surroundings.
3. **What my actions do** – This block helps the agent estimate the outcome of its own actions. By knowing the effect of its behavior, the agent can make smarter decisions.

These added elements make the agent more intelligent and responsive. Instead of reacting only to the current situation, the agent now considers the state of the environment, anticipates future changes, and understands the impact of its actions. This leads to more informed and context-aware decision-making.

(c) Goal Based Agents

Their decision-making is driven by their desired outcomes. Compared to reflex-agent, goal-based methods are more adaptable since they clearly model the knowledge underlying a choice, which enables for modifications. Objective - It's the account of ideal circumstances.

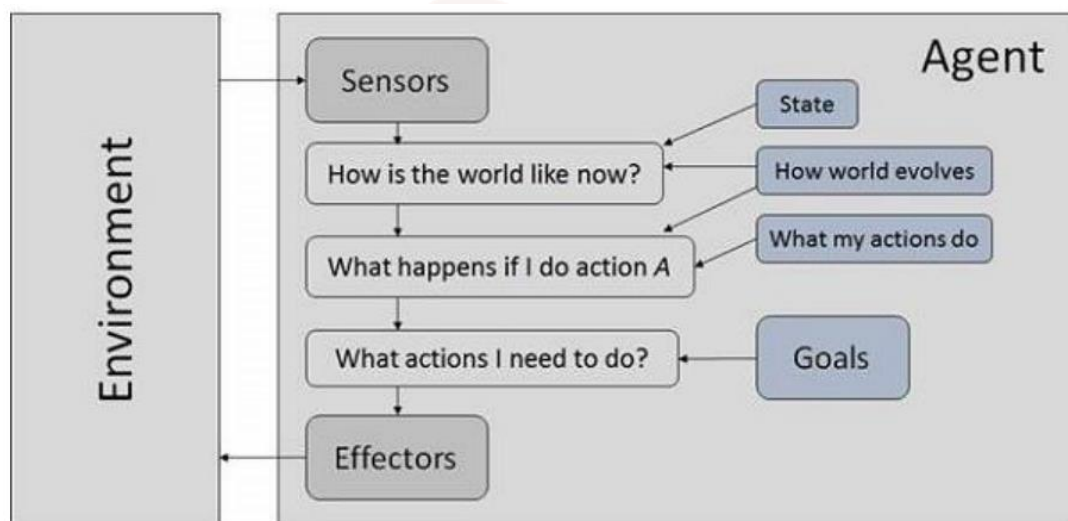


Figure 3c. Agents and Environments with Goals.

(d) Utility-Based Agents

For each state, they have a preference (utility) that guides their behaviour choices. Attempts fall short when –

- Only a handful of the competing objectives will ever be satisfied.
- The relevance of a goal should be considered in relation to the probability of its successful completion, since goals inherently contain an element of uncertainty.

(e) Turing Test

To determine whether a system can demonstrate intelligent activity, the Turing Test is an effective tool. Two humans and the machine under evaluation make up the exam. On one of the two people, you'll find the tester at work. Each of them is occupying a different room. It is unclear to the tester who is human and who is mechanical. He can acquire typed responses by sending his questions to the two intelligences via text.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
12	What is intelligence in the context of Artificial Intelligence? a) The ability to process and store large amounts of data b) The capability to learn, reason, and make decisions c) The ability to physically interact with the environment d) The execution of repetitive tasks without logic
13	Which of the following is NOT a type of intelligence? a) Logical Intelligence b) Linguistic Intelligence c) Artistic Intelligence d) Artificial Intelligence
14	Which of the following is a key component of intelligence? a) Memory and reasoning b) Random guessing c) Imitation without understanding d) Lack of problem-solving skills
15	What is an intelligent agent in AI? a) A software program that only stores data b) A system that perceives its environment and takes actions to achieve a goal c) A machine that operates without external inputs d) A program that performs tasks randomly without logic
16	Which of the following is a characteristic of the environment in which an AI agent operates? a) It is always fully observable b) It can be dynamic or static c) It does not provide feedback to the agent d) It always follows predefined rules

10. THE NATURE OF ENVIRONMENTS

Certain applications function solely within the constraints of a computer's database, file systems, and the text displayed on a screen, which constitute a wholly artificial world. On the other hand, there are software agents that exist in rich, limitless realms called softbots. The simulator's setting is intricate and detailed. In order to respond in real time, the software agent must select one action from a lengthy list. In both the real and virtual worlds, a softbot can analyse a customer's online preferences and recommend relevant products. The Turing Test is the most well-known artificial environment; it puts both real and artificial beings through their paces. It is quite unlikely that a software agent could achieve human-level performance in such a demanding setting.

Environmental Factors

The environment possesses multiple attributes.

- A discrete environment, as in chess, consists of a small number of well-defined states; a continuous environment, as in driving, lacks such divisions.
- An environment is considered fully observable if and only if its state can be determined at any particular moment by means of the senses; otherwise, it is considered partially observable.
- An agent's operational environment is said to be static if it does not change; alternatively, it is said to be dynamic.
- The agent could be the only agent in the environment, or there could be multiple agents present, either of the same or different type.
- To be considered accessible, an environment must be able to be perceived by the agent's sensory equipment as a whole. If they don't have it, they can't access it.
- The environment is considered deterministic if its future state can be entirely predicted from its present state and the activities of the agent, and non-deterministic otherwise.
- Episodic vs. non-episodic — In the former, the agent perceives and acts in each episode. The individual episodes determine the overall quality of their actions.
- What happens in one episode does not dictate what happens in the next. Since the agent is not required to plan ahead in episodic environments, they are far easier to navigate.

11. SUMMARY

- This chapter comprehensively explains Artificial Intelligence (AI), covering its definition, components, goals, advantages, disadvantages, history, and real-world applications.
- Machines that can mimic human intellect in terms of learning, reasoning, and decision-making are called artificial intelligence (AI). The healthcare, financial, automation, and entertainment industries are just a few that it impacts, all of which are crucial to modern technology.
- The components of AI include machine learning, deep learning, natural language processing, expert systems, and robotics, all of which contribute to creating intelligent machines capable of decision-making and automation. Creators of AI hope to create machines with brainpower comparable to that of humans, automate repetitive tasks, improve efficiency, and solve complex problems with minimal human intervention.
- While AI offers several advantages, such as enhanced productivity, reduced human errors, and risk mitigation in dangerous environments, it also comes with disadvantages, including job displacement, high costs, ethical concerns, and security risks.
- The origins of artificial intelligence can be found in the 1950s, when trailblazers such as Alan Turing and John McCarthy presented the core ideas of AI. Over time, AI has evolved from simple rule-based systems to sophisticated machine learning models, making significant progress in fields like robotics, natural language processing, and neural networks.
- AI is widely used in various applications, such as virtual assistants, self-driving cars, medical diagnosis, fraud detection, and recommendation systems, making it an integral part of everyday life.
- An intelligent system is a machine or software capable of learning from data, adapting to new situations, and making decisions autonomously. The types of intelligent systems include reactive machines, limited memory systems, theory of mind AI, and self-aware AI, each with different levels of intelligence and adaptability.
- AI operates within an agent-environment framework, where an agent (an AI system) perceives the environment through sensors and takes actions based on predefined rules or learning mechanisms. Agents can be simple reflex agents, goal-based agents, utility-based agents, or learning agents, depending on their complexity and decision-making capabilities.

12. GLOSSARY

Artificial Intelligence (AI)	- The simulation of human intelligence in machines that can learn, reason, and make decisions.
Machine Learning (ML)	- A subset of AI that enables computers to learn from data and improve performance without explicit programming.
Deep Learning	- A more advanced form of machine learning that uses neural networks to analyse complex patterns in large datasets.
Natural Language Processing (NLP)	- A branch of AI that enables machines to understand, interpret, and generate human language
Expert System	- A computer-based system that mimics human decision-making in specific fields using knowledge-based rules.
Robotics	- A field in AI that involves designing and programming robots to perform tasks autonomously.
Agent	- An AI system that perceives its environment and takes actions to achieve specific goals.
Environment	- The surrounding conditions in which an AI agent operates and interacts..
Intelligent System	- A machine or software that can adapt, learn, and make decisions based on its environment
Types of Intelligent Systems	- Includes reactive machines, limited memory systems, theory of mind AI, and self-aware AI, each with different levels of intelligence.
Reactive Agents	- AI systems that respond to current environmental conditions but do not store past experiences.
Goal-Based Agents	- AI systems that make decisions based on predefined goals and objectives.
Utility-Based Agents	- Intelligent agents that choose actions based on maximizing a defined utility function.

History of AI	- The evolution of AI from early symbolic reasoning in the 1950s to modern machine learning and neural networks.
AI Applications	- Various real-world implementations of AI, such as virtual assistants, self-driving cars, medical diagnosis, and fraud detection.
AI Goals	- Objectives of AI, including automating tasks, enhancing decision-making, and mimicking human intelligence
Advantages of AI	- Benefits such as increased efficiency, accuracy, risk reduction, and 24/7 operation.
Disadvantages of AI	- Challenges such as high costs, job displacement, ethical concerns, and security risks.
Turing Test	- A test proposed by Alan Turing to determine if a machine can exhibit human-like intelligence.

13. TERMINAL QUESTIONS

1. What is Artificial Intelligence? State its applications.
2. Discuss the historical evolution of Artificial Intelligence.
3. State the relationship between agent and environment.
4. What is an agent? State its types.
5. Explain the properties of the environment.
6. Explain different types of Intelligent Systems.
7. Write a note on the Turing Test.
8. Elucidate the building blocks of Intelligence.
9. Explicate the applications of AI.
10. Discuss the advantages and disadvantages of AI.

14. ANSWERS

Self-Assessment Questions

1. human intelligence
2. a) Increased efficiency and automation
3. b) Through accurate diagnosis and personalised treatments
4. a) Automated chatbots providing 24/7 support
5. c) Quantum Computing Approach
6. a) Reduces human errors and increases accuracy
7. a) AI can lead to job automation, reducing employment opportunities
8. b) Machine Learning
9. b) AI-powered diagnosis and robotic surgery
10. a) By automating fraud detection and risk assessment
11. a) Self-driving cars and traffic management systems
12. b) The capability to learn, reason, and make decisions
13. c) Artistic Intelligence
14. a) Memory and reasoning
15. b) A system that perceives its environment and takes actions to Achieve a goal
16. b) It can be dynamic or static

Terminal Questions Answers

Answer 1: Artificial Intelligence (AI) is the branch of computer science that focuses on creating machines capable of performing tasks that typically require human intelligence, such as learning, reasoning, and problem-solving. AI applications include virtual assistants, self-driving cars, medical diagnosis, robotics, fraud detection, and recommendation systems. (Refer to Sections 2 and 4)

Answer 2: AI has evolved through several stages, beginning in the 1950s with early symbolic reasoning and the Turing Test. In the 1980s, expert systems became popular, followed by the rise of machine learning and neural networks in the 1990s and 2000s. (Refer to Section 5)

Answer 3: An agent is an AI system that perceives its surroundings (environment) through sensors and takes actions using actuators to achieve a goal. The environment provides feedback based on the agent's actions, influencing its future decisions. (Refer to Section 10)

Answer 4: An agent is an entity that perceives and acts in an environment to achieve specific objectives. (Refer to Section 9)

Answer 5: The environment in AI can have various properties:

- If the environment can be divided into a small number of clearly defined states, like in chess, then it is discrete; otherwise, it is continuous, like in driving.
- If the entire condition of the environment can be ascertained at each given time point using the senses, then it can be said to be observable; otherwise, it can be said to be partially observable.

(Refer to Section 10)

Answer 6: Types of Intelligent System:

- a) Linguistic intelligence
- b) Musical intelligence
- c) Logical-mathematical intelligence
- d) Spatial intelligence
- e) Bodily-Kinesthetic intelligence
- f) Intra-personal intelligence
- g) Interpersonal intelligence (Refer to Section 7)

Answer 7: The Turing Test, proposed by Alan Turing in 1950, is a method to evaluate a machine's ability to exhibit human-like intelligence. If an AI can communicate in a way that a human cannot distinguish it from another human, it is considered intelligent. (Refer to Section 9)

Answer 8: The Building Blocks of Intelligence are: Reasoning • Learning • Problem Solving • Perception • Linguistic Intelligence (Refer to Section 8)

Answer 9: AI was adopted by industries in 1981, revolutionising fields like game playing, speech recognition, natural language processing, and computer vision.

It enabled systems like expert systems (e.g., MYCIN), heuristic classification (e.g., credit approvals), and intelligent robots that sense and react smartly.

These applications use AI to process complex data, simulate reasoning, and support human decision-making across various sectors. (Refer to Section 4)

Answer 10: Advantages of AI

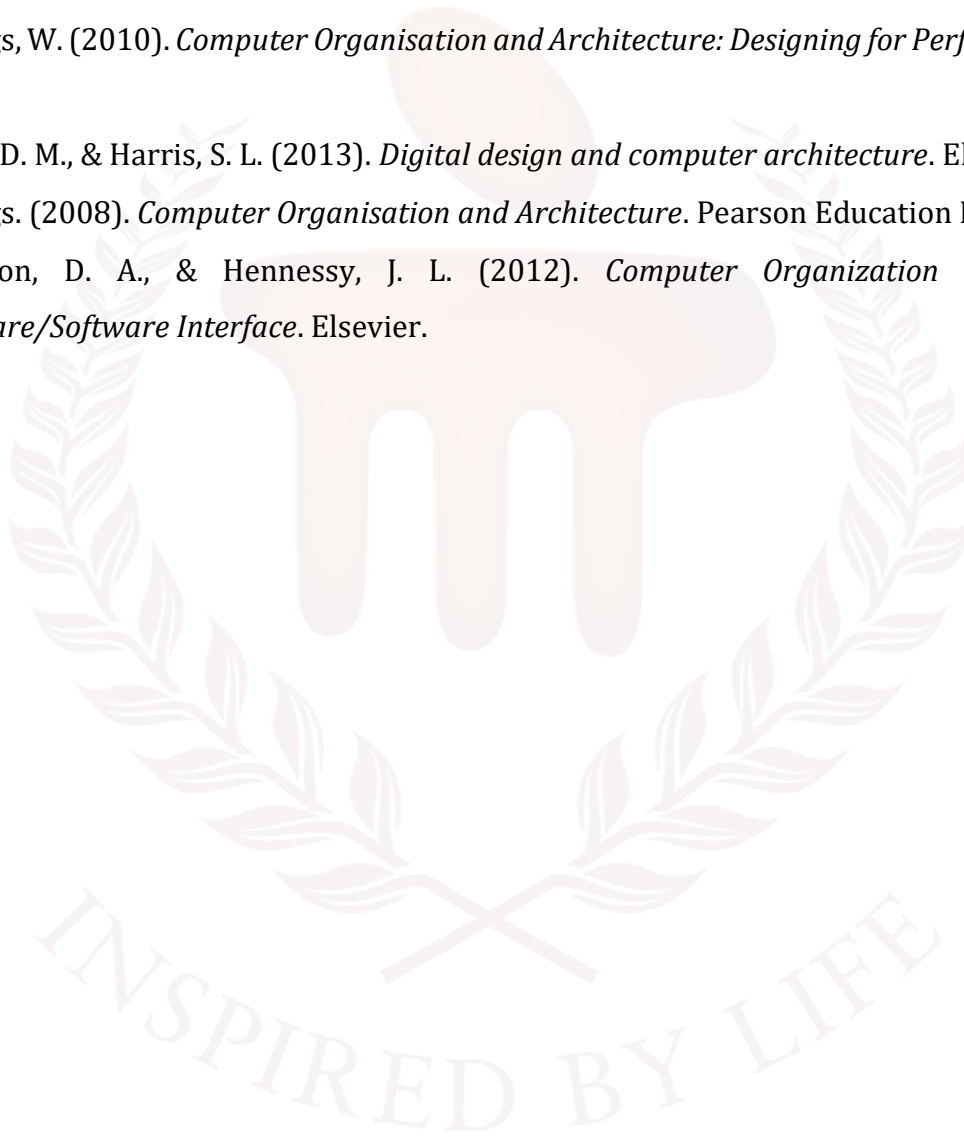
- **Improved Efficiency & Automation:** AI is revolutionising the way we work by eliminating human mistakes and speeding up formerly laborious processes.
- **Better Decision-Making:** AI's rapid analysis of big datasets provides data-driven insights in healthcare, finance, and business strategy.

Disadvantages of AI

- **High Implementation Costs:** AI development and upkeep necessitate substantial financial outlays for hardware, software, and human knowledge.
- **Job Displacement:** Some industries may see a rise in unemployment due to automation and AI-driven systems displacing human workers.

15. REFERENCES

- Mano, M. M. (1982). *Computer System Architecture*. Prentice Hall.
- Stallings, W. (2010). *Computer Organisation and Architecture: Designing for Performance*. Prentice Hall.
- Stallings, W. (2010). *Computer Organisation and Architecture: Designing for Performance*. Prentice Hall.
- Harris, D. M., & Harris, S. L. (2013). *Digital design and computer architecture*. Elsevier.
- Stallings. (2008). *Computer Organisation and Architecture*. Pearson Education India.
- Patterson, D. A., & Hennessy, J. L. (2012). *Computer Organization and design: The Hardware/Software Interface*. Elsevier.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3

DCA2109

ARTIFICIAL INTELLIGENCE FOR PROBLEM SOLVING

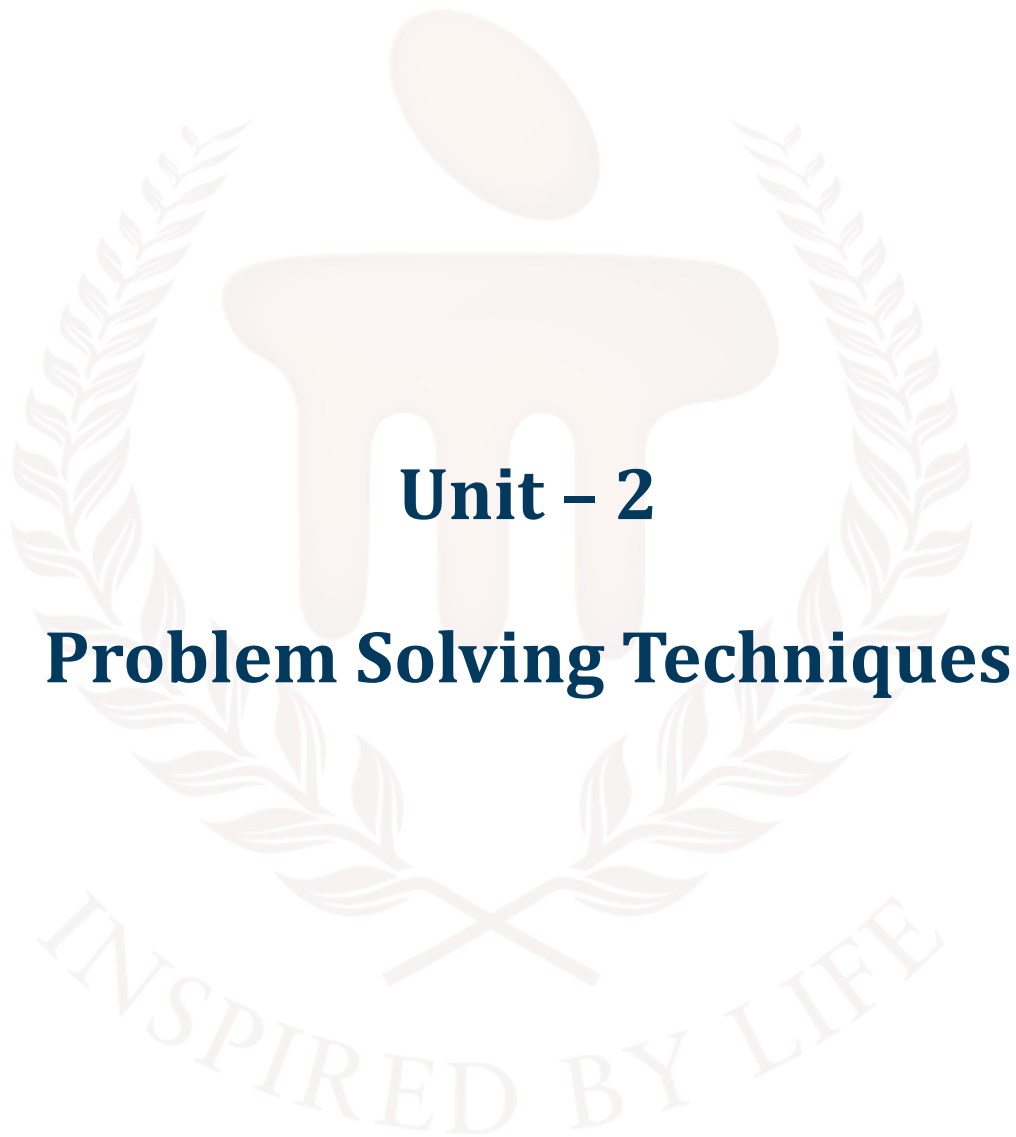


TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Overview of Problem solving Agents	-	-	6 - 9
3	Artificial Intelligence Problem Formulation Elements	1	1	10 -12
4	Different AI Issues in problem solving	-	2	13 -14
5	Goal formulation	-	3	15 - 17
6	Types of AI Agents for Solving Problems	-	4	18 - 21
7	Problem Solving Agents in AI	-	5	22 -25
8	Real-world Applications of Problem Solving Agents in AI	-	-	26 -27
9	Architecture of Agent	2, 3	-	28 -30
10	Role of Agent Program	-	6	31 -32
11	Summary	-	-	33
12	Glossary	-	-	34 -35
13	Terminal Questions	-	-	36
14	Answers	-	-	37 -39
15	References	-	-	40

1. INTRODUCTION

In previous unit, we had gain a foundational understanding of Artificial Intelligence (AI) by exploring its definition, significance, and wide-ranging applications. The unit began by defining AI and explaining its role in modern technology, highlighting why AI is considered a revolutionary force in various industries. We had acquired knowledge regarding the importance of AI, particularly in automation, decision-making, and problem-solving, which have transformed fields such as healthcare, finance, and robotics. Additionally, we delves into applications of AI, showcasing real-world use cases in areas such as self-driving cars, virtual assistants, recommendation systems, and fraud detection. Through these examples, we have understood how AI is integrated into daily life and business processes to improve efficiency and accuracy.

In this unit, we will study essential problem-solving techniques used in artificial intelligence, focusing on how intelligent agents identify and solve problems efficiently. The first topic, goal formulation, involves understanding how an agent defines its objectives based on the task at hand. This means transforming vague or high-level tasks into clear, actionable goals that can guide the agent's behaviour. Next, you will explore different types of problems that agents might face, such as single-state problems in fully observable environments, and multiple-state problems in partially observable or dynamic environments. We will also distinguish between deterministic and stochastic problems, and understand how the nature of a problem influences the method used to solve it. Finally, the unit covers problem-solving agents, which are designed to reason through their actions by representing the current state, considering possible actions, and planning a sequence of steps to reach a goal. These agents use search-based methods to make decisions, especially in complex environments like game playing or robotic navigation. Overall, the unit lays a foundation for understanding how AI systems approach tasks logically and systematically.

1.1. Objectives

By the end of this unit, you will be able to:

- Interpret Artificial Intelligence (AI) Problem-solving Agents
- Examine the key components of Problem Formulation Elements
- Recall the Characteristics of a well-formulated Goal
- Identify the role of the agent's program



2. OVERVIEW OF PROBLEM SOLVING AGENTS

To enable machines to make intelligent judgements and complete tasks autonomously, problem-solving agents are vital in Artificial Intelligence (AI). These agents are programmed to deliberate before acting, accomplishing objectives through the application of logic and strategy. Instead than reacting on the fly to stimuli, problem-solving agents take stock of their surroundings, identify the issue at hand, weigh their options, and ultimately settle on the best course of action. In order for them to function, they adhere to a systematic method that involves describing the present condition, deciding on the target state, and then looking for a set of steps that will bring about the desired change. Robotics, automated planning systems, and game playing are just a few examples of the many places this idea has found widespread use in artificial intelligence. We can learn a lot about how to model and implement intelligent behaviour in machines by studying how problem-solving agents function.

It may be required to employ many data structures and methodologies in artificial intelligence to address problems, which can be rather complex. One of the most important parts of computers that help solve problems is the Problem-Solving Agent. This post will go over what a problem-solving agent is, how it functions in AI systems, and a few interesting use examples that show what this technology can do.

What is problem solving agent?

Finding a solution to a problem is the first step in artificial intelligence problem solving. Every problem has its own unique characteristics, and there are likely to be many distinct types of difficulties, each of which will require a unique approach to resolution. The winner of most games is presented with a navigation puzzle or maze as their task.

Additionally, there are obstacles in the areas of finding solutions to systems of equations, predicting outcomes, and identifying patterns. Based on the nature of the problem at hand, a wide variety of approaches and tools are at one's disposal for finding a solution.

Primary Process for problem solving:

1. Grasping the problem: In order to solve an issue, one must first understand what the problem is.
To do this in artificial intelligence, one must first examine the surrounding environment,

determine the system's present state, and then provide the desired outcome (the goal state) for the agent to attain. Acquiring all pertinent data, comprehending limitations, and establishing success criteria are all part of this phase. The more an agent learns about the issue, the better it will be able to solve it. The agent needs to be aware of its origin, destination, potential routes, and environmental impediments in order to complete a navigation assignment, for instance. If the problem is not understood at this level, it can lead to wasteful solutions or routes. Problem formulation, which includes stating the starting point, the set of potential actions, the end state, and the path cost, must be precise and precise if the problem is to be solved.

2. Coming up with potential solutions:

The following stage, after identifying the issue, is to investigate all of the possible solutions. Here, the AI system comes up with a number of potential solutions by using search algorithms or logical reasoning. Some examples of these include A* and other heuristic-based approaches, as well as more complex methods like depth-first and breadth-first searches. During this phase, you should try to think about as many different possibilities as you can without passing judgement on any of them. This is known as divergent thinking. For every given problem, there is more than one answer that might take the agent from where it is now to where it needs to go. Finding an ideal or almost ideal answer is more likely with a longer list of possibilities. For the purpose of visualising and comparing options, AI systems frequently model or simulate these paths using data structures such as trees or graphs.

3. Deciding on the solution:

When fixing a problem, one of the most important steps is deciding on the best solution. Once the AI agent has developed multiple solutions, it must assess each one and choose the one that meets the goal the best within the limits and resources that are available. A number of criteria, including path cost (efficiency), execution speed, resource utilisation, and success likelihood, constitute this evaluation. The solution needs to be practical in terms of both time and effort, but it also needs to satisfy the criteria established when the problem was first stated. To do this in artificial intelligence, one may use heuristics or do simulations to determine which alternative is more cost-effective. When an optimal solution is unavailable, the agent may also take into account trade-offs between other options. The goal of choosing this approach is to improve the agent's performance in the given environment to its fullest potential.

A logical and organised framework for AI problem solving is formed by these three stages: comprehending the problem, creating potential solutions, and selecting a solution. Particularly in complicated or unpredictable settings, they aid intelligent agents in making systematic and successful decisions. By perfecting these steps, we may create AI systems that are smarter and more adaptive, able to solve a broad range of practical issues.

4. Execution and Monitoring:

The term "execution and monitoring" describes the process of putting the selected solution or strategy into action while keeping a close eye on how it does in the actual world. Accurate execution and response to unforeseen circumstances or plan deviations are essential components of artificial intelligence systems.

The Significance:

Any well-laid strategies can backfire if the agent isn't vigilant about monitoring their real-world surroundings, which are frequently unpredictable and dynamic. The agent's ability to recognise mistakes, respond to changes, and make required adjustments is ensured by monitoring. The maintenance of effective and intelligent behaviour relies on this feedback loop between action and observation. Take a robot Hoover as an example. They often have a set route they follow when cleaning a room. Surprisingly, it may run into a pet or a piece of furniture that wasn't originally mapped out as it starts to execute. It detects the obstruction, changes its trajectory, and continues cleaning autonomously thanks to its sensors and monitoring.

Important Parts:

- Reception and sensing for environmental feedback
- Mechanisms for detecting and handling errors
- Changing course or adjusting plans on the fly when the present strategy isn't working

5. Learning and Adaptation

A key component of learning and adaptation is the agent's capacity to continuously enhance its performance through the incorporation of lessons learnt from previous encounters, interactions, or data. Machine learning and reinforcement learning allow intelligent agents to improve their decision-making, model updating, and strategy discovery rather than just repeating the same old things.

Importance: This phase enables AI systems to adapt and grow wiser as time goes on, which is crucial for handling unexpected problems. Learning agents aren't content to merely respond; they use their past observations to optimise their future actions.

To avoid crowded places, a self-driving car may first take a longer route. It figures out which roads are usually faster at certain times of day after a few travels and traffic inspections. The system then adjusts its route planning to maximise travel time, regardless of external factors.

Customary Methods:

- Using incentives or punishments to guide trial and error is known as reinforcement learning.
- The process of improving prediction accuracy through learning from labelled datasets is known as supervised learning.
- Finding groups or patterns in data without labels is known as unsupervised learning.

3. ARTIFICIAL INTELLIGENCE PROBLEM FORMULATION ELEMENTS

1. Initial State

In order to solve a problem, one must first go to the initial state. It specifies the state or setup that the AI agent uses to start solving the problem.

The Significance:

Every step of solving a problem hinges on precisely identifying the beginning state. The outcome could be faulty or inaccessible if the starting point is wrong or missing information.

Take a chess game as an example. The first state is the usual board configuration, with each piece in its proper place. It may represent the agent's present position on a map in a navigation system.

2. Actions

A collection of operations or moves that an AI agent can do to go from one state to another is represented by actions. Operators or successors are other names for them.

The Significance:

For the agent to be able to investigate potential routes to the objective, it is crucial to define the correct set of activities. It is imperative that all actions be lawful, significant, and result in a legitimate new state.

For instance, "move north," "move south," "turn left," and "move forward" are all possible commands in a navigation system. Sliding tiles up, down, left, or right are some of the actions in puzzle games like the 8-puzzle.

The following information is based on the given text:

3. Transition Model

The transition model, which is also called the state transition function, explains how the agent's activities cause changes to the environment. The regulations for changing states are laid forth in it.

The Significance:

Planning and reasoning rely on the agent's ability to anticipate the outcomes of its actions, which is made possible by an accurate transition model.

A robotic arm's position is defined by the transition model for each action (such as rotating or extending). The placement of a knight on the board alters the chessboard in a predetermined manner.

4. Goal Test

As a function, the goal test checks if a state satisfies the goal conditions. It verifies that the issue has been adequately addressed.

The Significance:

In order to determine when to end the search and announce success, the agent employs the goal test. Finding the solution quickly and accurately is the purpose of a well-defined goal test.

As an illustration, the objective test of a puzzle game verifies the correct sequence of all tiles. It checks if the destination has been reached in a route-planning system.

5. Path Cost

The path cost, also known as the cost function, is a measure of the overall effort required to travel from a starting state to a goal using a predetermined set of actions.

The Significance:

There are often more than one answer to an AI problem. Using the path cost, one can choose the best solution—the one that gets them where they need to go as cheaply as possible. Distance, time, and fuel consumption are all components of the path cost of a delivery system. In a game, it could show the amount of movements or points gained/lost.

Component	Purpose	Example
Initial State	Starting point of the problem	Chess board setup
Actions	Possible moves/operations	Move north, slide tile
Transition Model	Result of an action	Moving a pawn changes the board
Goal Test	Checks if goal is achieved	Puzzle solved correctly
Path Cost	Total cost of a solution path	Distance in delivery system

Figure 1: Elements of AI problem formulation

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following is the correct order of steps for a problem-solving agent? A. Goal formulation → Perceive environment → Problem formulation → Search → Execute action B. Perceive environment → Goal formulation → Problem formulation → Search → Execute action C. Search → Goal formulation → Problem formulation → Execute action D. Goal formulation → Problem formulation → Execute action → Search
2	In AI, what is the purpose of problem formulation? A. To execute an action in the environment B. To define how the agent interacts with humans C. To structure the problem in a way that can be solved by the agent D. To identify the type of agent being used
3	Which of the following is NOT an element of problem formulation in AI? A. Initial state B. Goal test C. Internet connectivity D. Actions (successor function)
4	What is a goal test used for in a problem-solving agent? A. To find all possible solutions to a problem B. To determine whether a given state is a goal state C. To test the performance of an agent D. To evaluate the efficiency of the search algorithm
5	Which element of problem formulation defines how one state leads to another? A. Initial state B. Transition model C. Goal state D. Path cost

4. DIFFERENT AI ISSUES IN PROBLEM SOLVING

In artificial intelligence, issues are categorised according to their traits and the impact they have on the process of finding a solution. Having a grasp of these kinds aids in the development of efficient agents for solving problems.

Criteria for Classification

There are primarily three categories of AI problems:

1. Ignorable Problems: These are problems where skipping some steps won't change the result.
2. Problems that can be recovered are those in which errors can be fixed or reversed.
3. Problems when acts are permanent and require careful forethought are known as irrecoverable problems.

The design and strategy of AI agents are affected differently by each category.

1. Ignorable Problems

Problems that allow for the omission or skipping of specific solution steps without compromising the final result are defined as ignorable problems

Features:

- Less complex issue structure.
- Requiring fewer processing resources.

Things like fine-tuning a machine learning model's hyper parameters are examples of optimisation jobs where some variables have little to no effect.

2. Recoverable Problems

Agents have the ability to reverse or amend their actions in these types of problems, which allows for flexibility in solving them.

These activities can be undone and it is less danger than permanent issues.

One example is decision-making in AI games, where the AI can adapt its moves in response to the actions of its opponents.

3. Irrecoverable Problems

The problem at hand necessitates meticulous preparation because the acts taken cannot be undone.

Features:

- Solving problems with a high degree of uncertainty.
- Demands exactitude in carrying out.

Some examples includes like Navigation systems for autonomous vehicles, where a single mistake (such as running a red light) can have long-lasting effects

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which of the following is a major issue in AI problem solving related to handling incomplete or uncertain information? A. Modularity B. Knowledge representation C. Reasoning under uncertainty D. Natural language processing
7	Why is knowledge representation a critical issue in AI problem solving? A. It defines how to program loops in AI algorithms B. It helps in visualizing neural networks C. It determines how information about the world is stored and accessed by the agent D. It controls the physical movements of robots
8	Which of the following AI issues involves selecting the most efficient path to reach a goal state? A. Machine learning B. Search strategies C. Pattern recognition D. Robotics

5. GOAL FORMULATION

In order for an intelligent agent to accomplish its objectives, it must first formulate those objectives. The goal or state that the agent should strive for while solving the challenge is defined. An essential first step in artificial intelligence, this is because an agent can't make good decisions or do useful things unless it has a target it's trying to reach. The agent can learn what "success" means in its current setting by formulating goals, and then it can use that knowledge to direct its actions towards achieving those goals.

Why is it important to formulate goals?

- It enables progress evaluation as the agent interacts with its environment.
- It helps concentrate the agent's behaviour towards meaningful results.
- It turns broad, unattainable goals into more manageable, attainable targets.
- It serves as a benchmark for planning, learning, and decision-making.

The Process of goal formulation:

- One common step in goal formulation is the agent's perception of their environment, which entails gathering data about their surroundings.
- Knowing where the agent is at the moment is the second step.
- Defining the target state for following the completion of the work, how should the world appear?
- Fourth, formulate an objective by making a clear and quantifiable statement outlining the criteria for achievement.

Dynamic Nature of Goals

Objectives change throughout time in the actual world. When circumstances change, AI agents may have to:

- Rethink their objectives.
- Make a ranked list of all your objectives.

- Deal with goals that are at odds with one another.

Take a delivery robot as an example. Its original purpose is to deliver a product. However, when faced with unforeseen impediments or delays, it may need to reassess its priorities, pick a new route, or even postpone the delivery.

Examples of Goal Formulation

- The objective of an artificial intelligence chess program could be to "checkmate the opponent's king."
- "Safely reach the destination in minimum time" might be the objective of a self-driving vehicle.
- One possible objective for a robotic Hoover cleaner is to "clean all reachable floor areas while avoiding obstacles."

Characteristics of a Well-Formulated Goal

Good goals are:

- Unambiguous and easy to understand
- Realistic within the agent's power to accomplish
- For the agent to know when it's finished, it must be measurable.
- Appropriately connected to the agent's overarching goal

Formulating goals is similar to directing a GPS to a specific location. Without it, the agent has no idea what to accomplish or how to evaluate its progress. It's an essential component of smart behaviour that lets AI systems work efficiently, strategically, and with purpose in both basic and complex settings.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
9	What is the primary purpose of goal formulation in a problem-solving agent? A. To train the agent using historical data B. To define what the agent is trying to achieve C. To analyse all possible states in the environment D. To determine the shortest path between two nodes
10	Which of the following best describes a well-formulated goal in AI? A. A goal that changes dynamically over time B. A goal that has clear, achievable, and testable conditions C. A goal that is based on random user input D. A goal that focuses on improving hardware performance

6. TYPES OF AI AGENTS FOR SOLVING PROBLEMS

There are multiple approaches of classifying AI agents that solve problems. Types vary in how they do their tasks. The following are examples of artificial intelligence problem-solving agents:

i. Simple Reflex Agents

One of the simplest types of reflex agents is the simple reflex agent. They do nothing more than respond to their perceptions of the world around them. They have no concept of the passage of time, or more accurately, they disregard both the past and the future.

As an illustration of a straightforward reflex agent, consider a room thermostat. The only thing it considers when deciding whether to turn on the heat is the current room temperature.

How are they operated?

Automatons' reflex agents are their most fundamental and original form. They don't care about the past or the future; they only look at the data that is available to them at any given time. The correct course of action in each given situation is defined by condition-action protocols, which they faithfully follow.

Possible applications:

Not requiring much thought and requiring quick action in an easily predictable environment are good fits for these agents. They aren't good at learning logic or considering the consequences of their actions, so they should be reserved for highly specialised jobs. A thermostat, which activates the heating system in response to a drop in outside temperature, is a basic example of a reflex agent.

ii. Model based Agents:

Agents built from models perform far better. They construct functional representations of their surroundings. They may use this model to watch the world go by. They can use this model to brace themselves for other events that may take place in the future. Autonomous vehicles' ability to anticipate traffic patterns is one use case of model-based agents.

How are they operated?

For the purpose of representing the external environment, model-based reflex agents store a model of the world inside. This internal model allows them to monitor the planet's status and its history. They use condition-action principles to make judgements, but they can also take past data into account.

Possible applications:

Agents built on models are better equipped to adapt to changing conditions and do more complex tasks than their reflex-based predecessors. A model-based reflex agent in action would be a robot Hoover cleaner, which would use the room's sensors to navigate to previously cleaned spots.

iii. Goal-Based Agents

Agents that are goal-based are the most complicated. It empowers kids to decide what they want out of life and how they want to get it. While they accomplish their objectives, these agents are constantly learning new things. Digital assistants like Siri and Alexa are examples of responsive goal-based agents that help us with a variety of tasks. When it comes to the nuances of fixing a specific issue, any kind is better than the other. In some situations, reflex agents may be sufficient; but, when faced with complex problems, agents based on models or goals are necessary.

How are they operated?

Agents that are goal orientated work towards a certain objective or objectives. They are committed to their goal and have a strategy to achieve it. Taking stock of where they are in relation to their objectives, they pick the ones that will help them reach their targets. In order to accomplish their goals, they can simply "restructure" themselves, and they are also quite directive.

Possible applications:

In order to accomplish long-term goals, goal-based agents perform best in complex and dynamic environments where adaptation is essential. The unmanned delivery drones use route planning to avoid obstacles like traffic and poor weather on their way to deliver packages.

iv. Utility-Based Agents:

Intelligent agents that take into account both preferences and objectives are known as utility-based agents. In contrast to basic goal-based agents, utility-based agents use a utility function to quantify the desirability of a state, going beyond simply checking if it satisfies a goal.

An agent's degree of happiness or liking for a certain condition or result can be expressed numerically using a utility function. The agent then finds the optimal course of action by calculating the predicted value of each potential action and picking the one with the highest total utility. When faced with trade-

offs, uncertainty, or conflicting goals, this method enables the agent to make more complex judgements.

There may be two possible outcomes for an agent to consider: one that is moderately risky but has a moderate possibility of reward, and another that is safer but produces middling results. Taking these costs and benefits into account, a utility-based agent will choose the course of action with the highest expected utility, even if it implies forgoing some of its original objectives.

This line of thinking allows utility-based agents to adjust to dynamic and complicated contexts where it's not enough to just achieve a goal; the process and cost of doing so are of equal importance.

Cases in Which Utility-Based Agents Are Useful:

- In domains where balancing many objectives is essential and judgements cannot be made based on a single element alone, utility-based agents shine. Here are a few examples of notable use cases:
- **Financial Decision-Making:** When suggesting a portfolio, an AI financial advisor, for instance, needs to think about return, liquidity, risk, and investment horizon. Based on the investor's preferences, a utility-based agent can assess these competing considerations and select the most balanced investment strategy.
- **Decisions in autonomous vehicles** take things like speed, fuel economy, passenger comfort, and safety into account. Instead of focussing solely on getting to its goal, a utility-based agent assists the vehicle in navigating in a manner that takes all of these factors into account.
- **Healthcare Systems:** Utility-based agents can assess various treatment alternatives in medical diagnosis or treatment planning by taking into account cost, side effects, patient preferences, and effectiveness. They can then recommend the most appropriate course of action based on this evaluation.
- **Systems for the Smart Home:** Depending on the user's needs, a smart thermostat or home assistant can find the sweet spot between cost, energy efficiency, and comfort

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
11	Which type of AI agent selects actions based on current percepts without considering the future consequences? A. Model-based reflex agent B. Simple reflex agent C. Goal-based agent D. Utility-based agent
12	What type of problem is characterised by a clearly defined initial state, goal state, and set of actions? A. Real-world problem B. Ill-defined problem C. Well-defined problem D. Randomised problem
13	Which AI agent type uses internal models to track the world and plan future actions? A. Simple reflex agent B. Goal-based agent C. Utility-based agent D. Model-based reflex agent



7. PROBLEM SOLVING AGENTS IN AI

Entities built with artificial intelligence problem-solving capabilities can analyse data from a learning environment, draw conclusions, and implement solutions to complicated problems. These agents operate inside a specific environment and are capable of accomplishing specific goals by employing specific strategies for problem-solving. Essential features of problem-solving agents include the following:

➤ **Goal-Oriented:**

It was believed that problem-solving agents had a singular objective. There is a goal or aim that someone or some group is trying to accomplish in this particular context. The agents' primary concerns are defined by these objectives.

➤ **Perception:**

The agents' means of perceiving their surroundings are data sources or sensors. The conditions pertinent to the situation at hand, the resources accessible, and the current environment are all factors that they compile data on.

➤ **Representation:**

The subject-matter domain and the challenges that people are attempting to address are represented by people. Facts, rules, objectives, and constraints may be included in this representation.

➤ **Search and Planning:**

It is common practice for problem-solving agents to use planning strategies and search algorithms to consider and rank alternative possible actions that could lead to a desired outcome. They experiment with various scenarios and actions in an effort to find a solution.

➤ **Reasoning and Decision-Making:**

The agents use logic to choose the best plan to follow before deciding what to do next. The agent makes these decisions based on its expertise, the information at its disposal, and its objectives.

➤ **Action:**

By taking action within it, agents change the state of the environment. In order to solve the problem or reach the goal, these are the decisions that are intentionally taken.

➤ **Learning:**

Certain agents with problem-solving capabilities have the ability to learn, which allows them to gradually improve their methods. One way to accomplish this is by absorbing information from your surroundings or from experiences in the past.

➤ **Feedback and Evaluation:**

In order for agents to gauge the efficacy of their problem-solving endeavours, the environment furnishes them with feedback. Using this information, they can refine their strategies or respond to changing conditions.

➤ **Optimisation:**

The ultimate goal, which problem-solving entities may strive to attain, might be described as the most suitable answer, the most effective course of action, or the correct choice. In order to find these solutions, optimisation technologies are utilised. This approach makes use of optimisation techniques.

➤ **Problem Domains:**

Agents that solve problems are useful in many areas, including artificial intelligence, robotics, autonomous vehicles, recommendation systems, NLP, and neural networks. The agent must overcome the practical challenges and peculiarities of the problem domain.

The following are a few instances of problem-solvers:

- **Chabot's:** These chess-playing bots employ heuristics and search algorithms to figure out every possible move.
- **Find your way:** the most efficient way for a character to navigate the game's landscapes, logistics, and transportation is with the help of agents.

- Agents that prepare plans or schedules to carry out challenging tasks in fields such as manufacturing or project management are known as planning agents.
- Decisions and answers are provided by experts in the field using their subject-matter knowledge by expert systems. These systems are used in occupations such as technical support or disease detection.
- The term "reinforcement learning agent" refers to a person or thing that is always making decisions in an effort to maximise points and enhance the sequence of interactions with its environment. These find widespread application in autonomous robots and video games.

Components of a Problem-Solving Agent in AI

There are numerous interdependent components that make up a problem-solving agent. Here we will have a look:

➤ **Sensors**

Sensors function similarly to the eyes and ears of an agent. They collect information about the agent's environment. For example, a robot's camera and motion detectors serve as sensors.

➤ **Knowledge Base**

The Knowledge Base stores all of the agent's familiarity with rules, regulations, and facts. It appears as though the agent possesses an immense store of information within its brain. With this data, the agent may have a greater understanding of its environment and be able to make more informed decisions.

➤ **Reasoning Engine**

We refer to the agent's reasoning part as the Reasoning Engine. The data collected from the sensors is processed using the knowledge base. The next step is for the reasoning engine to assess the situation and decide on an appropriate course of action.

➤ **Actuators**

Last but not least, actuators are like the agent's limbs and hands. They put into action the judgements reached by the reasoning engine. A robot's wheels and robotic arms are its actuators.

Each of these parts is essential to the whole. Data is collected by sensors, plans are made by the reasoning engine, context is provided by the knowledge base, and the actual execution of the plans is carried out by the actuators

SELF-ASSESSMENT QUESTIONS - 5

Multiple Choice Questions	
14	What is the main function of a problem-solving agent in AI? A. To learn from experience and adapt B. To manipulate natural language C. To decide what actions to take in order to reach a specific goal D. To perform basic arithmetic calculations
15	Which of the following is NOT typically part of the problem-solving process in a problem-solving agent? A. Goal formulation B. Problem formulation C. Search for a solution D. Sensor calibration
16	In AI, a problem-solving agent searches through which of the following to find a solution? A. Neural networks B. State space C. Memory hierarchy D. Knowledge base

8. REAL-WORLD APPLICATIONS OF PROBLEM-SOLVING AGENTS IN AI

There is more to problem-solving agents than meets the eye. You can find them used in a variety of real-world contexts these days. First, we'll have a look at several examples:

➤ **Gaming Agents**

Video game applications often make use of Problem-Solving Agents. At any moment, they can evaluate the state of the game, formulate possible strategies, and put those plans into action. This gives them a chance to beat human players at complex games like chess and Go.

➤ **Robotics**

Warehouse and factory robots depend heavily on problem-solving agents. These agents perceive their environment around the robot through sensors. After that, the robot is instructed to take the most efficient paths, and it adjusts its movements and actions appropriately.

➤ **Virtual Assistants**

Google Home and Amazon Echo, among others, use goal-based agents to resolve issues in the smart home. When you reach out to them for assistance, they are able to understand your needs, find relevant information in their database, and provide you with useful responses.

➤ **Recommendation Systems**

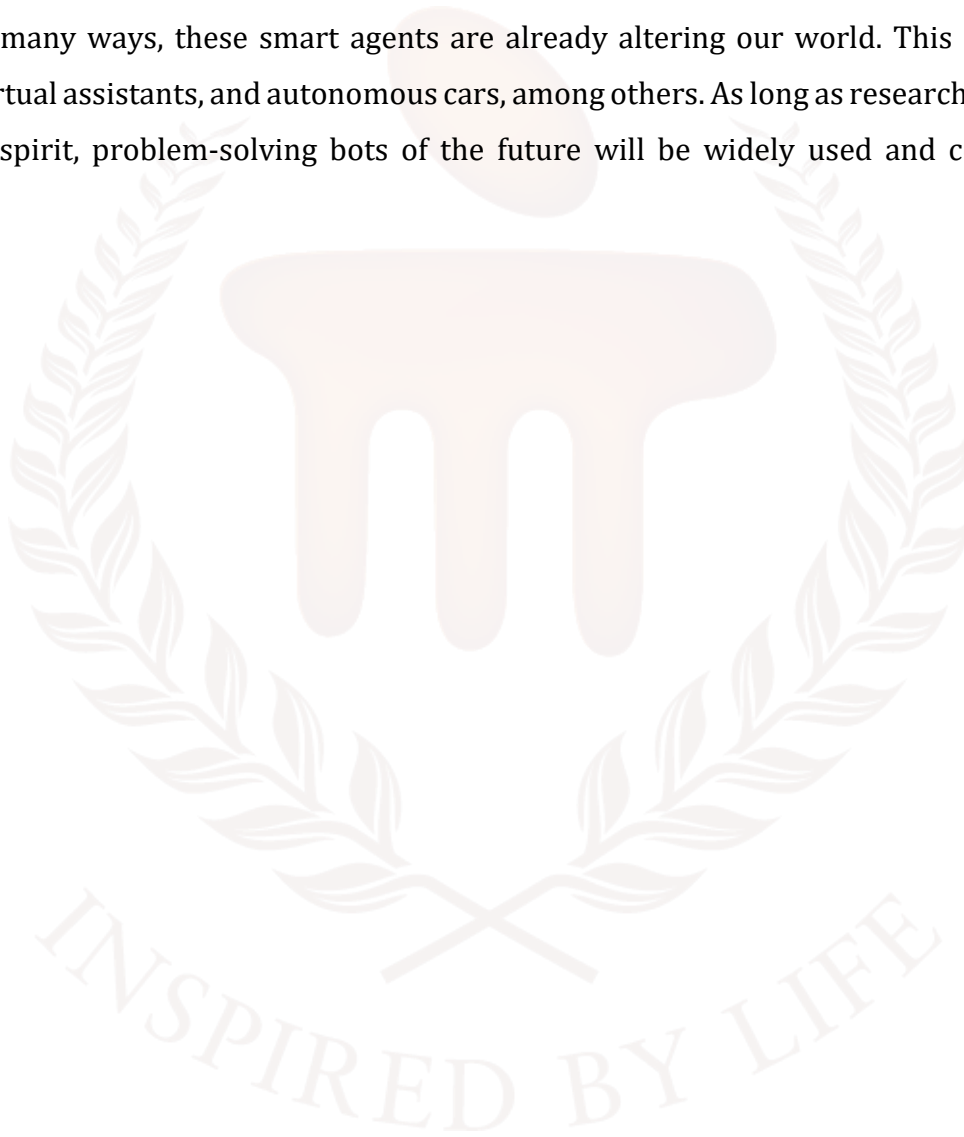
Businesses on the web employ product recommendations from problem-solving agents to provide you with options for products you might like. These agents consider your preferences and past purchases while making product recommendations.

➤ **Scheduling and Planning**

Make the most of your time management software by applying problem-solving techniques to your daily schedule. Your priorities, appointments, and travel time will be taken into consideration by the agents in order to optimise your daily schedule.

A few examples of the most advanced applications are self-driving cars. They have built-in problem-solving agents that monitor their surroundings, detect when other vehicles and objects are about to move, and drive themselves safely down the road.

AI revolves around computers that can solve problems by mimicking human reasoning and decision-making. In many ways, these smart agents are already altering our world. This is true in gaming, robotics, virtual assistants, and autonomous cars, among others. As long as researchers maintain their innovative spirit, problem-solving bots of the future will be widely used and considerably more advanced.



9. ARCHITECTURE OF AGENT

An architecture is a type of computing equipment that the agent application runs on. The architecture must be able to run the software we select. The architecture not only runs the program but also makes the sensor percepts available to it. It then feeds the instructions to the effectors in the program during their generation. It is possible to summarise the connection between agents, architectures, and programs in the following way:

Agent = Architecture + Program

The following figure shows the agent action process, as specified by architecture, which can also be termed as agent's structure



Figure 2: Agents Structure

1. **Environment:** Everything that the agent comes into contact with, whether it be external factors, things, or conditions, is part of the environment. In order to accomplish its objectives, the agent works within the environment.
2. **Sensors (Perception/Input)**
Agents can take in information about their surroundings through sensors, which can be either physical (like a robot's camera or microphone) or software-based (like a database or web page). What the agent perceives—its knowledge of the world as it is right now—is included in the data collected by its sensors.
3. **Percept (Current State Info)**
What the agent currently knows or believes about what's happening is represented by the percept, which is the processed information received from the environment. The agent's decision-making system receives this perception.

4. Agent Function (Decision-Making/Reasoning Unit)

This is the central component of the agent's design, this component evaluates the perception, considers objectives and past information, and decides on the optimal action to do. For decision-making, it might employ rules, ML, planning algorithms, or utility calculations.

5. Actuators (Output/Action Execution)

The agent then takes action in its environment by means of its actuators after making a decision. Physical agents, such as robots, may use wheels, arms, or lights as actuators. These could take the form of network communications, mouse clicks, or file writes in software agents.

6. Actions

Each action has the potential to alter the environment, which in turn can cause a change in perception in the subsequent cycle. The operations performed by the agent have an impact on the environment.

Cycle of Interaction

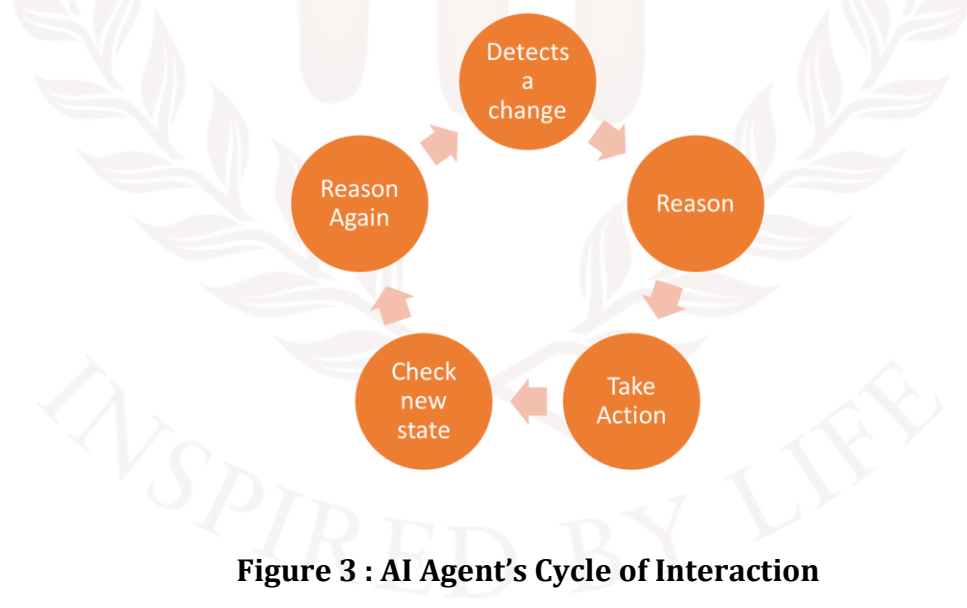


Figure 3 : AI Agent's Cycle of Interaction

- The process is iterative: the agent detects a change → reasons → takes action → checks the new state → reasons again, and the process continues in this loop.
 - This loop enables the agent to adapt, learn, and achieve its objectives.
-
- In intelligent agent architecture, this figure summarises the Sense-Think-Act cycle
 - Perceive their surroundings (via sensors), be able to Reason or make a decision about what to do?

- Use actuators to make changes to the environment.
- Do it all over again.

Every kind of agent relies on this structure, from basic reactive agents to advanced AI systems capable of learning and planning.



10. ROLE OF AGENT PROGRAM

A Program's Agent role is the internal implementation of an agent program is as an agent function. An agent program receives input from sensors in the form of the current percept and returns an action to the effectors, which are also known as actuators. Mild and Powerful Agents

Everything that can be seen as having sensors to take in its surroundings and effectors or actuators to change those surroundings is considered an agent.

a. Mild agent / Weak agent: A flimsy definition of an agent would be a computer system with the following characteristics, whether it be software or hardware:

1. **Autonomy:** Agents are able to manage their own behaviour and internal states and can function independently of humans.
2. **Social ability:** Agents are able to communicate with one another and, perhaps, with humans using an agent-specific language.
3. **Reactivity:** When agents notice changes in their environment, they react rationally and quickly.
4. **Being proactive:** Agents can do more than just react to their surroundings; they can also think of their own objectives and take steps to reach them.

b. Powerful Agent / Strong Agent: A more powerful country asserts that a person possesses mental qualities, including knowing, believing, intending, and having an obligation. Furthermore, an agent possesses additional qualities like: -

1. **Portability:** Agents are able to freely navigate across various machines and systems.
2. **Veracity:** Agents do not intentionally provide misleading information.
3. **Rationality:** Agents will behave rationally in an effort to accomplish their goals and will avoid doing anything that could hinder their progress.

SELF-ASSESSMENT QUESTIONS - 6

Multiple Choice Questions	
17	What is the core responsibility of the agent program in an AI agent? A. To visualise the data B. To design the environment C. To implement the agent function that maps percepts to actions D. To train the agent using reinforcement learning
18	In a typical agent architecture, what receives input directly from the environment? A. Actuators B. Percepts C. Sensors D. Agent program
19	Which of the following best describes the architecture of an intelligent agent? A. A single algorithm for all problem types B. A combination of sensors, actuators, and a control system C. A neural network only D. A passive structure with no interaction
20	What is the ultimate goal of an agent in an AI system? A. To process big data B. To maximise a performance measure by interacting with the environment C. To analyse trends in machine learning D. To operate independently of user input

11. SUMMARY

- AI revolves around computers that can solve problems by mimicking human reasoning and decision-making. In many ways, these smart agents are already altering our world.
- This is true in gaming, robotics, virtual assistants, and autonomous cars, among others. As long as researchers maintain their innovative spirit, problem-solving bots of the future will be widely used and considerably more advanced.
- The foundation of AI is problem-solving agents, which allow computers to tackle problems in various domains.
- Artificial intelligence agents can accomplish extraordinary flexibility and efficiency by learning the parts of issue formulation, classifying problems, and using the right techniques including optimisation, search algorithms, constraint satisfaction, and machine learning.
- Learning and Adaptation enable the agent to progress, leading to more effective judgements in the future, while Execution and monitoring make sure that planned actions are carried out appropriately and altered as needed. These parts allow AI systems to adapt to complicated real-world settings with ease by adding intelligence, growth potential, and robustness.
- A logical and organised framework for AI problem solving is formed by these three stages: comprehending the problem, creating potential solutions, and selecting a solution.
- Particularly in complicated or unpredictable settings, they aid intelligent agents in making systematic and successful decisions. By perfecting these steps, we may create AI systems that are smarter and more adaptive, able to solve a broad range of practical issues.
- Formulating goals is similar to directing a GPS to a specific location. Without it, the agent has no idea what to accomplish or how to evaluate its progress. It's an essential component of smart behaviour that lets AI systems work efficiently, strategically, and with purpose in both basic and complicated settings.
- In intelligent agent architecture, the Sense-Think-Act cycle-Perceive the surroundings (via sensors), make a choice or use reasoning to arrive at an action plan, take action to alter one's surroundings (by means of actuators). Every kind of agent relies on this structure, from basic reactive agents to advanced AI systems capable of learning and planning.

12. GLOSSARY

Goal Formulation	-	The process of defining a specific objective or outcome that an agent aims to achieve. It guides the agent's decision-making and planning.
Problem solving	-	A methodical approach used by an agent to find a sequence of actions that lead from an initial state to a goal state.
Initial State	-	The starting condition or configuration from which the agent begins the problem-solving process
Actions	-	The set of possible operations an agent can take to move from one state to another.
Transition Model	-	A function that describes how an action affects the current state and leads to a new state.
Goal Test	-	A procedure used to determine if the agent has reached the desired goal or final state.
Path Cost	-	A numeric value that represents the total cost incurred by the agent to reach the goal, used to evaluate and compare different solutions.
Problem Types	-	Categories of problems based on their complexity and structure, such as single-state problems, multiple-state problems, deterministic vs. stochastic, observable vs. partially observable.
Well-Defined Problem	-	A problem with clearly specified components: initial state, actions, transition model, goal test, and path cost.
Search Space	-	The set of all possible states and paths that can be explored by the agent to reach the goal.
Search Strategy	-	The method or algorithm used by the agent to explore the search space and find a solution (e.g., BFS, DFS, A*).
Problem-Solving Agent	-	An intelligent system that uses a structured approach (including goal formulation and search) to solve problems autonomously.

Rational Agent	- An agent that always selects actions expected to maximise its performance measure, given its knowledge and goals.
Percept	- The agent's observation or sensory input from the environment at a given moment.
Agent Function	- A function that maps a percept sequence to an action. It defines how an agent reacts based on what it perceives.
Performance Measure	- A criterion used to evaluate how well the agent achieves its goal, based on the outcomes of its actions.

13. TERMINAL QUESTIONS

1. Define a problem-solving agent in AI.
2. Explain the essential elements involved in AI problem formulation.
3. Identify two major issues faced by AI in problem-solving.
4. Describe the process of goal formulation in artificial intelligence.
5. Classify the different types of AI agents used for solving problems.
6. Summarise how a problem-solving agent functions within an AI system.
7. Illustrate a real-world scenario where a problem-solving agent is applied.
8. Interpret the meaning of agent architecture in the context of AI.
9. Explain the role played by the agent program in an intelligent agent's operation.
10. Distinguish between goal-based agents and utility-based agents with an example.
11. Analyse the importance of search techniques in problem-solving agents.
12. Evaluate the use of the 8-puzzle problem in demonstrating AI problem-solving strategies.

14. ANSWERS

Self-Assessment Questions

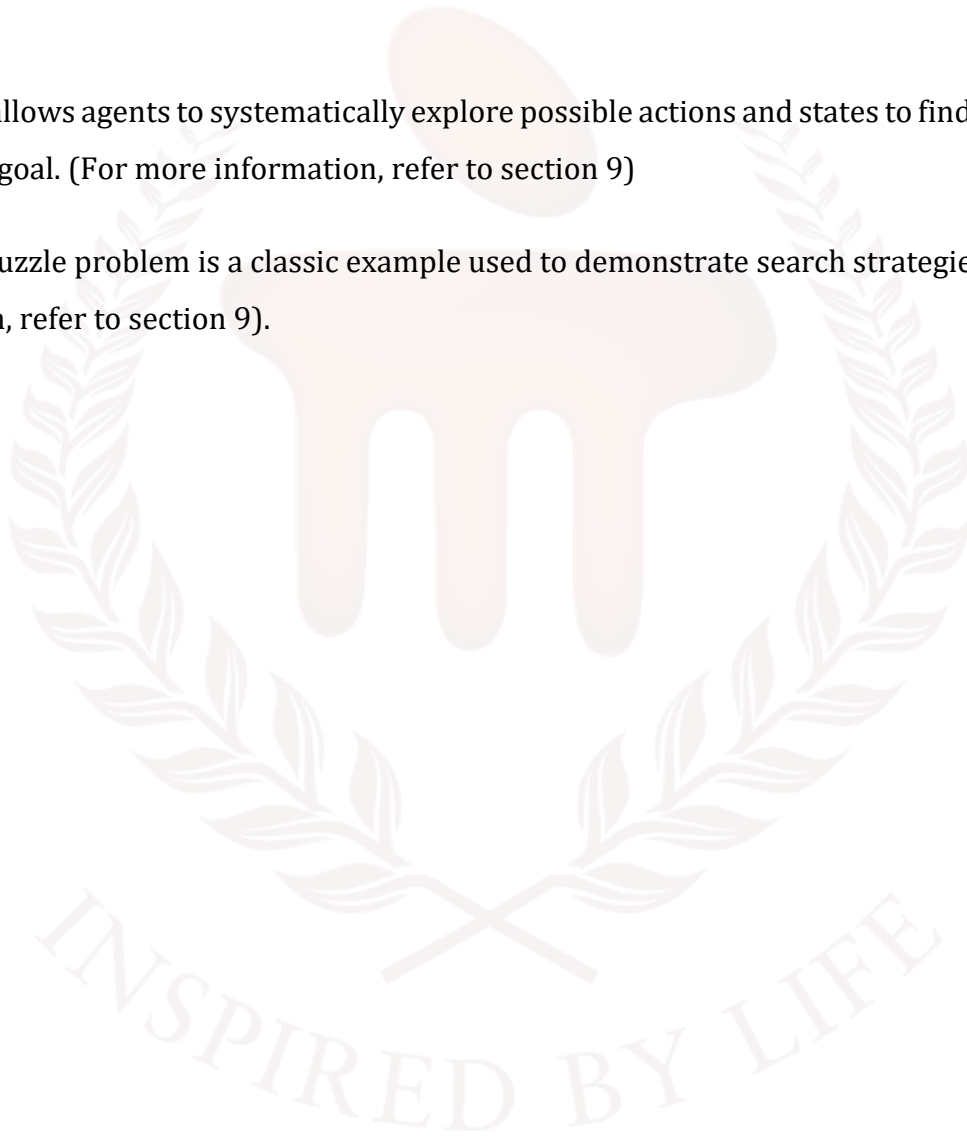
1. Answer: B. Perceive environment → Goal formulation → Problem formulation → Search → Execute action
2. Answer: C. To structure the problem in a way that can be solved by the agent
3. Answer: C. Internet connectivity
4. Answer: B. To determine whether a given state is a goal state
5. Answer: B. Transition model
6. Answer: C. Reasoning under uncertainty
7. Answer: C. It determines how information about the world is stored and accessed by the agent
8. Answer: C. It determines how information about the world is stored and accessed by the agent
9. Answer: B. To define what the agent is trying to achieve
10. Answer: B. A goal that has clear, achievable, and testable conditions
11. Answer: B. Simple reflex agent
12. Answer: C. Well-defined problem
13. Answer: D. Model-based reflex agent
14. Answer: C. To decide what actions to take in order to reach a specific goal
15. Answer: D. Sensor calibration
16. Answer: B. State space
17. Answer: C. To implement the agent function that maps percepts to actions
18. Answer: C. Sensors
19. Answer: B. A combination of sensors, actuators, and a control system
20. Answer: B. To maximise a performance measure by interacting with the environment

Terminal Questions Answers

1. A problem-solving agent is an AI agent that takes a goal and finds a sequence of actions to reach that goal from a given initial state. It uses search and planning techniques. (For more information, refer to section 2)
2. The essential elements involved in AI problem formulation are:

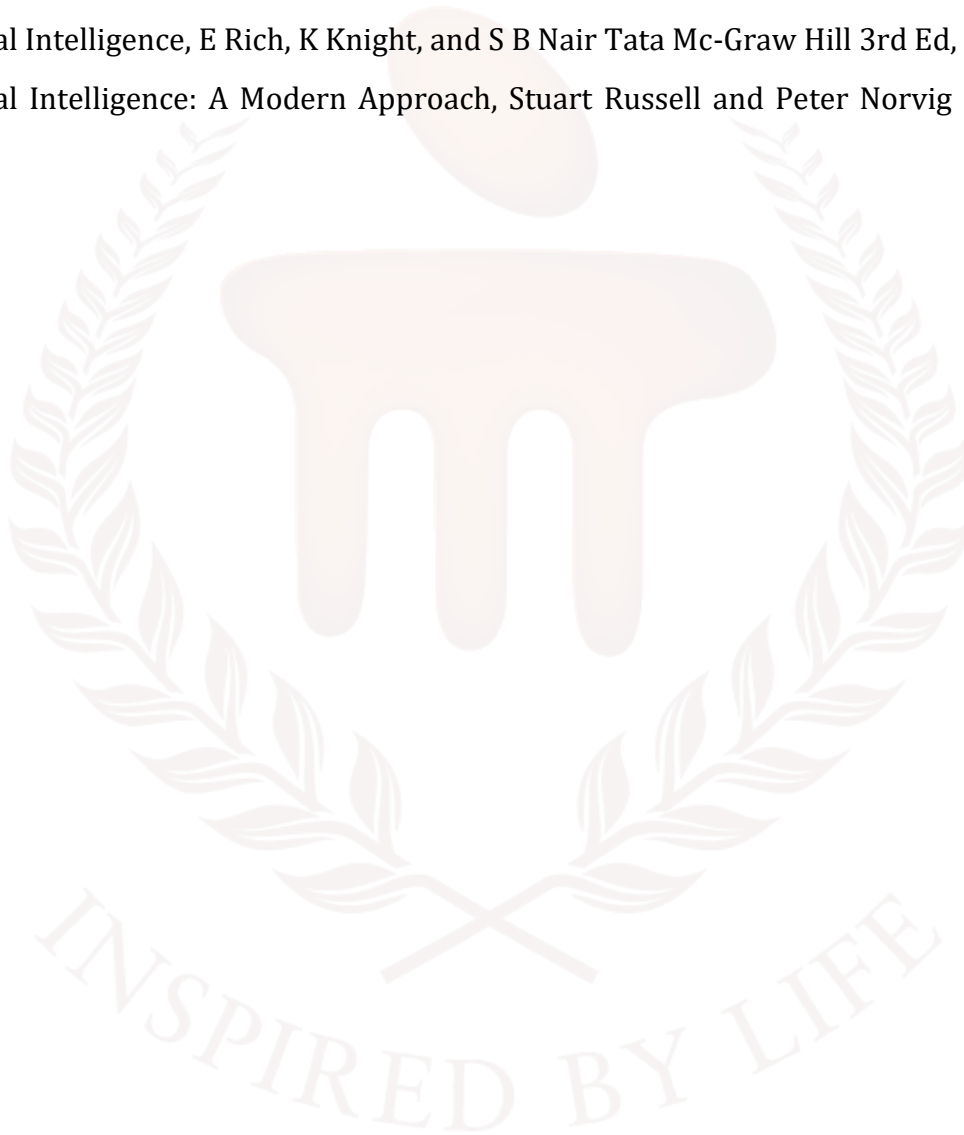
1. Initial State
 2. Actions (Operators)
 3. Transition Model
 4. Goal Test
 5. Path Cost (For more information, refer to section 3)
3. Combinatorial explosion, Incomplete or uncertain knowledge (For more information, refer to section 4)
4. Goal formulation is the process by which an agent decides what to achieve based on its current knowledge, desires, and environment. (For more information, refer to section 5)
5. The different types of AI agents used for solving problems
1. Simple Reflex Agents
 2. Model-Based Reflex Agents
 3. Goal-Based Agents
 4. Utility-Based Agents
 5. Learning Agents (For more information, refer to section 7)
6. They:
1. Perceive the environment
 2. Formulate a goal
 3. Formulate a problem
 4. Search for a solution
 5. Execute the plan (For more information, refer to section 7)
7. Autonomous navigation systems, such as self-driving cars, use problem-solving agents to plan optimal routes and avoid obstacles. (For more information, refer to section 7)
8. Agent architecture refers to the underlying structure of an agent, which defines how the agent perceives, decides, and acts (e.g., reactive, deliberative, hybrid). (For more information, refer to section 8)

9. The agent program maps percept sequences to actions. It embodies the logic that guides the agent's behaviour. (For more information, refer to section 8)
10. Goal-based agents choose actions that achieve a given goal. Utility-based agents consider how good an outcome is, offering flexibility and handling trade-offs. (For more information, refer to section 9)
11. Search allows agents to systematically explore possible actions and states to find the most suitable path to the goal. (For more information, refer to section 9)
12. The 8-puzzle problem is a classic example used to demonstrate search strategies in AI. (For more information, refer to section 9).



15. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.



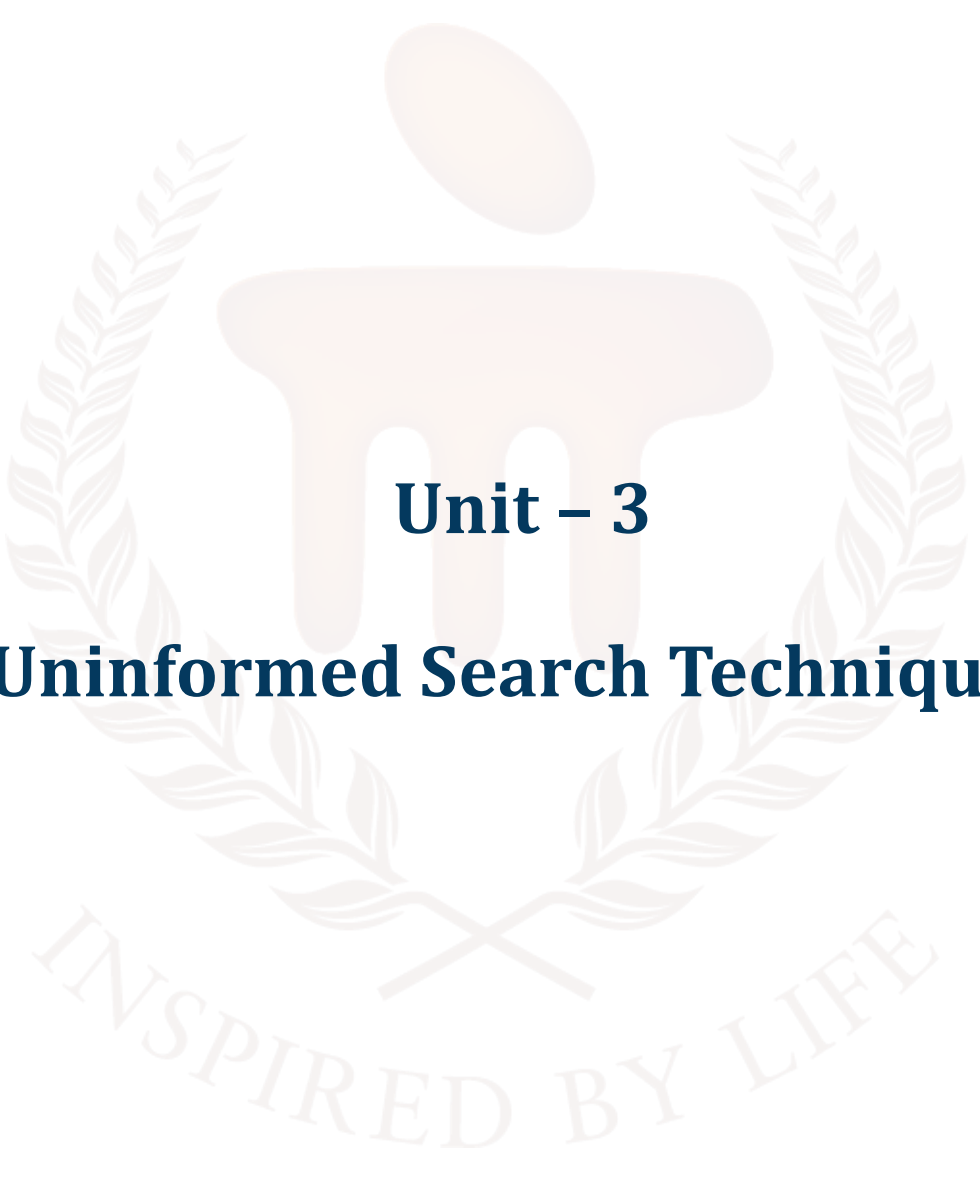
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 3

Uninformed Search Techniques

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 -5
1.1	Learning Objectives	-	-	
2	Introduction to Search Algorithms	-	-	6
3	Types of search algorithms	1	1	7 -10
4	Uninformed Search Algorithm	-	-	11
5	Breadth-First Search (BFS)	2, 3, 4	2	12 -21
6	Depth-First Search (DFS)	5, 6, 7, 8,	3	22 - 32
7	Iterative Deepening Depth-First Search	-	4	33 -38
8	Summary	-	-	39 -40
9	Glossary	-	-	41- 42
10	Terminal Questions	-	-	43
11	Answers	-	-	44 -45
12	References	-	-	46

1. INTRODUCTION

In previous unit, we had studied essential problem-solving techniques used in artificial intelligence, focusing on how intelligent agents identify and solve problems efficiently. The first topic, goal formulation, involves understanding how an agent defines its objectives based on the task at hand. This means transforming vague or high-level tasks into clear, actionable goals that can guide the agent's behaviour. Next, we had explored different types of problems that agents might face, such as single-state problems in fully observable environments, and multiple-state problems in partially observable or dynamic environments. We had also distinguished between deterministic and stochastic problems, and understand how the nature of a problem influences the method used to solve it. Finally, the unit covers problem-solving agents, which were designed to reason through their actions by representing the current state, considering possible actions, and planning a sequence of steps to reach a goal. These agents use search-based methods to make decisions, especially in complex environments like game playing or robotic navigation. Overall, the previous unit lays a foundation for understanding how AI systems approach tasks logically and systematically.

In this unit, you will master the art of solving problems systematically without any hints or special knowledge about the solution. You'll explore three powerful techniques that help an AI agent (or even a human) search through possibilities, even in the dark! Specifically, you will learn: Breadth-First Search (BFS): How to explore all possibilities level-by-level, ensuring you find the shortest path to the goal Depth-First Search (DFS): How to dive deep into the possibilities, following a path as far as possible before backtracking — a memory-efficient strategy. Iterative Deepening Search (IDS): How to cleverly combine the best of BFS and DFS — diving deep but still making sure you find the shortest solution with limited memory. We will also understand how AI agents search without prior knowledge of where the goal might be. Analyze trade-offs between memory usage, speed, and optimality. Visualize and design search trees for real-world problem scenarios. Choose the right strategy depending on the situation's demands (speed vs. accuracy). Apply these search techniques to basic AI problem-solving like pathfinding, puzzles, and robot navigation.

1.1. Objectives

By the end of this unit, you will be able to:

- Interpret the fundamental concepts of the uninformed search technique
- Compare the operational mechanisms of Breadth-First Search, Depth-First Search, and Iterative Deepening Search.
- Analyse the advantages and limitations
- Apply appropriate uninformed search strategies to solve simple AI search problems effectively



2. INTRODUCTION TO SEARCH ALGORITHMS

Research into creating intelligent machines that can reason is known as artificial intelligence. For the most part, these agents accomplish their goals by quietly running some sort of search algorithm.

The building blocks of every search problem are:

- A State Space: The set of all the potential states in which you could be.
- Getting Started: Originating state of the search.
- One Endpoint: You can check if the current state is the goal state by using a function that looks at it.

An approach that takes a starting state and ends it in a desired state is known as a plan, and it is the solution to a search issue. The strategy is carried out by the use of search algorithms.

A subset of search algorithms that don't make use of domain-specific information about the task at hand are called uninformed search algorithms or blind search algorithms.

- A problem statement should include the starting state, the actions possible in each state, and the desired end state for an uninformed search algorithm to work with.
- The lack of a heuristic function to systematically explore the search space towards the objective is what gives rise to the term "blind" when describing these.
- In situations when no information is provided other than the problem description, uninformed search algorithms offer fundamental search strategies for investigating problem areas.
- A variety of artificial intelligence issues, including pathfinding, puzzle solving, and state-space search, rely on these techniques.

Even if these algorithms aren't perfect, they help us comprehend and solve more complicated AI challenges.

3. TYPES OF SEARCH ALGORITHMS

To cover all of the sophisticated search algorithms in one essay would be an enormous undertaking. Here instead, we'll take a look at six of the most basic search algorithms, split down into two groups as illustrated below.

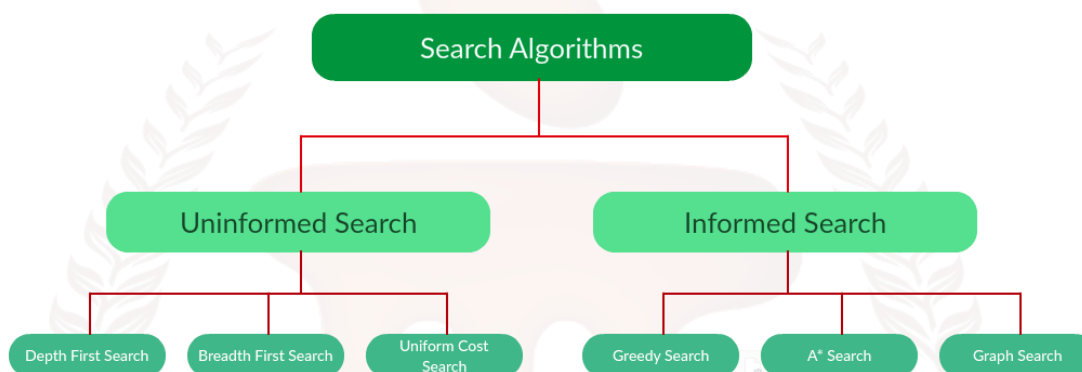


Figure 1: Types of search algorithms

The figure represents the different types of search algorithms, where it is broadly divided into two categories like uninformed search and informed search, again further division of these two categories has been represented

I. Uninformed Search Algorithms:

Only the information given in the problem description is used by the search algorithms in this section on the objective node. The sole difference between the various strategies to get from the starting state to the objective state is the sequence and duration of the actions. Blind search is another name for uninformed search. All these algorithms can do is distinguish between the goal state and non-goal states and create successors.

The following uninformed search algorithms are discussed:

1. Depth First Search
2. Breadth-First Search
3. Uniform Cost Search

For every algorithm, there will be:

- An issue graph with path S leading to destination G.
- A plan outlining the steps to take to reach G on the graph.
- An edge is a data structure that can hold all the alternative states (nodes) that can be reached from the present state.
- A tree that forms as a result of reaching the destination node.
- An approach to the problem, with a network topology that goes from S to G.

a. Depth First Search

One method for finding data structures like trees or graphs is depth-first search (DFS). In a graph, the algorithm begins at the root node (or any other node it chooses as the root node) and goes as far as it can along each branch before turning around. Because it employs the last-in, first-out technique, a stack is used to implement it.

b. Breadth First Search

When looking across graph or tree data structures, breadth-first search (BFS) is an algorithm to use. The process begins at the root of the tree (or any other node in the graph, commonly called a "search key") and proceeds to investigate all of the nodes at the current depth before going on to the nodes at the next depth level. A queue is used to implement it.

c. Uniform Cost Search:

Here, expenses play a role, which distinguishes UCS from BFS and DFS. To rephrase, it's possible that the cost of travelling via various edges is different. The objective is to identify a route with the lowest total cost.

II. Informed Search Algorithms:

In this case, the algorithms know the desired state, which allows for better search efficiency. Something called a heuristic is used to get this information.

Here, we'll go over some of the most common search algorithms.

1. Greedy Search

2. A* Tree Search
3. A*Graph Search

A search heuristic is a function that calculates the proximity of a state to the goal state in an informed search. The distance from Manhattan to the other end of the Earth, as measured in terms of the Euclidean distance, etc. (A closer look brings the target closer.) The following sections cover various informed algorithms and the heuristics used by them.

- a. Greedy Search:** The greedy search method involves expanding the node that is geographically nearest to the target node. The "closeness" is approximated using the heuristic $h(x)$.

Heuristic: The definition of a heuristic h is an estimate of the distance from one node to another, with x being any given node.

A lower $h(x)$ value indicates that the node is closer to the target.

- b. A*Tree Search:** Combining the best features of greedy search and uniform-cost search, A* Tree Search (or A* Search for short) is a powerful algorithm. Heuristic for this search is the product of two costs: $g(x)$ for UCS and $h(x)$ for greedy search. The formula $f(x)$ represents the total cost.

- c. A* Graph Search:**

By implementing the rule "do not expand the same node more than once," A* Graph Search (or just Graph Search) gets rid of this restriction.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following best describes a search algorithm in AI? A) A method to create random paths in a problem space B) A technique to systematically explore possible solutions to reach a goal C) A function that generates errors in decision-making D) A program that only classifies data
2	Which of the following is a characteristic of uninformed search algorithms? A) They use domain-specific knowledge B) They require no information other than the problem definition C) They guarantee the fastest solution always

	D) They use heuristics extensively
3	Which search strategy guarantees finding the shallowest solution if one exists? A) Depth-First Search (DFS) B) Greedy Best-First Search C) Breadth-First Search (BFS) D) Hill Climbing
4	What is the main difference between informed and uninformed search algorithms? A) Informed searches require less memory B) Uninformed searches always find optimal solutions C) Informed searches use additional knowledge (heuristics) about the goal D) Uninformed searches use more computational power
5	Which of the following is NOT a type of uninformed search? A) Depth-First Search (DFS) B) Breadth-First Search (BFS) C) A* Search D) Iterative Deepening Search (IDS)
6	In which situation would an informed search algorithm be preferred? A) When the search space is very small B) When we have no idea about the goal location C) When heuristic information about the goal is available D) When memory usage must be minimised

4. UNINFORMED SEARCH ALGORITHM

In an uninformed search, the search algorithms rely solely on chance rather than any information regarding the optimal search region. Regardless, they start simultaneously exploring the search space containing all possible solutions. Starting from the starting point, the search operation provides every feasible arrangement of next steps until the goal is accomplished. Although these search algorithms are generally the most basic, they might not work for complicated paths containing irrelevant or completely unrelated components. Basic processing algorithms are essential for accomplishing simple tasks or sending data to more powerful search algorithms that use prioritised information.

Key Characteristics of Uninformed Search

- No prior knowledge about the goal beyond the definition of the problem.
- Relies only on the information given in the problem (initial state, goal test, possible actions).
- Common metrics to evaluate performance:
 - Completeness (will it always find a solution if one exists?)
 - Time complexity (how long does it take?)
 - Space complexity (how much memory is used?)
 - Optimality (is the solution the best or shortest one?)

Applications of Uninformed Search Algorithm

- Robots and navigation systems use pathfinding to determine the quickest routes between two points. BFS and DFS look at the map to choose the best way.
- Solving puzzles: used to find a solution to challenges such as the Eight Puzzle or the Fifteen Puzzle.
- Game AI: Assists in finding winning strategies by exploring game trees. BFS and DFS are frequently used in conjunction with minimax or alpha-beta pruning for optimal performance.
- Robot Navigation: This feature helps robots navigate their environments and find the most efficient routes to avoid obstacles.

In web crawling, BFS is commonly utilised to index web pages in a methodical manner by following links.

5. BREADTH-FIRST SEARCH (BFS)

The Breadth-First Search (BFS) algorithm is a vital tool in artificial intelligence for exploring and navigating different problem domains. Graph or tree structures are the building blocks of BFS, which allows it to handle problems like pathfinding, network routing, and puzzles. Learn about BFS's foundational ideas, algorithms, and real-world AI uses in this in-depth essay.

When searching over a graph or tree, the breadth-first approach is usually used. This search method is known as breadth-first search because it traverses a graph or tree in a broad fashion. Before going on to the next level, investigates all nodes at the present depth. It finds the shortest path in an unweighted network by using a queue (FIFO) to keep track of nodes.

Before going on to the nodes of the next level, the BFS algorithm extends all successor nodes at the current level, starting its search at the root node of the tree. Among the many general-graph search algorithms is the breadth-first one. Using a FIFO queue data structure, a breadth-first search is executed.

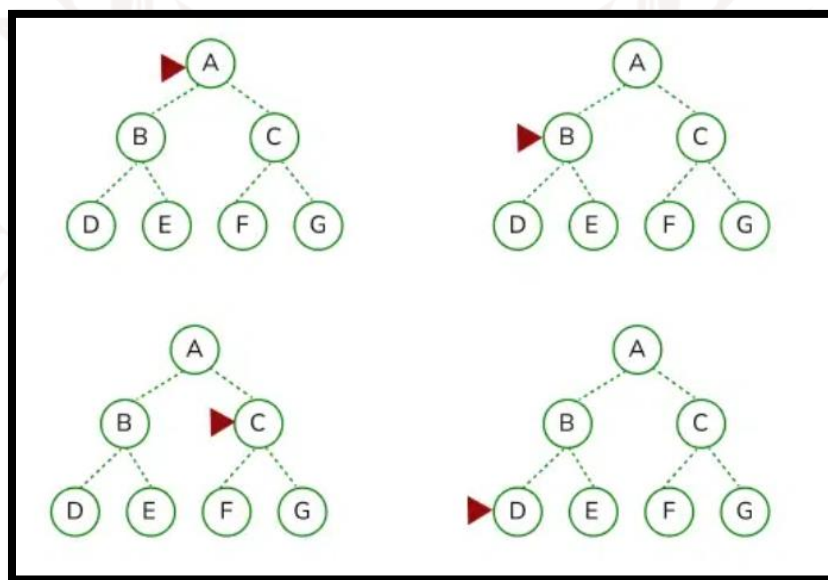


Figure 2: Representation of nodes in BFS

- Beginning at the root node, it meticulously investigates all of its neighbours' nodes before going on to the next level, expanding all of its successors along the way. (It grows its offspring B from the root node A, as seen in the preceding picture.)
- This iterative process of visiting every node in the graph—starting with the root node and working its way outwards—continues until either all of the nodes have been visited or a certain condition is satisfied. The preceding picture shows that it travels to node C after stopping at node B.
- After level 1 is finished, it goes on to level 2 and investigates node D. It will methodically make its way to nodes E, F, and G. It will end its journey at node G.

Key Features:

- It uses more memory because it keeps all the child nodes. It guarantees the shortest path if the cost is uniform.
- First-in, first-out (FIFO): In BFS, the FIFO queue is usually the way to go because it's faster than a priority queue and it usually ends up with the right order of nodes. Adding nodes further down the search tree to the rear of the queue is what happens when FIFO is applied to BFS. Prior to expanding new nodes, the older ones that can swallow more food are expanded.
- Early objective testing: Conventional BFS implementations keep track of the states the algorithm has reached while searching. Nevertheless, it selectively stores the set of obtained states that enable an early goal test, rather than saving all reached states. The purpose of this test is to verify, upon generation, whether the newly created node satisfies the objective criteria.
- Cost-optimal: Before generating nodes at a certain depth d , BFS explores and creates all the nodes at the depth $d-1$ below it, with the goal of finding a solution with the smallest cost by prioritising the shortest path. If a solution is present in a search area, BFS can find it after it reaches a certain depth level. This leads many to believe that BFS is the most economical option.

How does it Work:

- Start from the initial state.
- Expand the shallowest (earliest) nodes first.
- Use a queue (FIFO - First In, First Out) to store the nodes to be expanded.

Algorithm:

```
BREATH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node

  frontier ← a FIFO queue, initialized with node
  reached ← {problem.INITIAL}

  while not IS-EMPTY(frontier) do
    node ← POP(frontier)

    for each child in EXPAND(problem, node) do
      s ← child.STATE

      if problem.IS-GOAL(s) then return child

      if s is not in reached then
        add s to reached
        add child to frontier

  return failure
```

Python for Implementing BFS Algorithms

The BFS algorithm for pathfinding is implemented via the `breadth_first_search` function. Beginning with the beginning state, the BFS algorithm systematically explores neighbouring states in a breadth-first manner until either the goal state is identified or all reachable states have been investigated. It keeps track of the visited nodes in a FIFO queue named `frontier`. Additionally, it remembers the visited states and doesn't go back to them by using the set called `reached`.

```
def breadth_first_search(problem):
    node = Node(problem.initial)
    if problem.is_goal(node.state):
        return node

    frontier = deque([node])
    reached = {problem.initial}

    while frontier:
        node = frontier.popleft()

        for child in problem.expand(node):
            s = child.state

            if problem.is_goal(s):
                return child
            if s not in reached:
                reached.add(s)
                frontier.append(child)
    return None
```

Real-World Applications of BFS for Robot Pathfinding

Take a look at the example below, which shows a robot trying to find its way to a destination. At first, the robot stretches out its descendants, which assess potential routes to its objective. As the robot moves through the nodes, it must be careful not to run into anything that could block its way. Let's take it one step at a time and learn about pathfinding.

Step 2: Create a class for Grid Problem and Node

The **GridProblem** class represents the problem of pathfinding on a grid. It contains attributes for the initial state, goal state, and obstacles on the grid. The three typical methods are fitted in the code such as

1. **is_goal** - This method checks if the given state is the goal state.
2. **is_valid_cell** - This functions checks that a cell is valid to move or obstacles
3. **expand** - This method is in charge of generating child nodes (states) from the given parent node by considering obstacles and grid boundaries.

```
class Node:
    def __init__(self, state, parent=None,
action=None):
        self.state = state
        self.parent = parent
        self.action = action
```

The grid pathfinding problem is modelled by the GridProblem class. Along with the starting point, endpoint, and grid barriers, it also includes properties for the current state. The code incorporates the three common approaches, like as

- determines whether the current state is the target state using the `is_goal` method.
- The `is_valid_cell` function verifies whether a cell can actually move or encounter obstacles.
- From the provided parent node, this method is responsible for creating child nodes (states) taking grid borders and obstacles into account.

```
from collections import deque

class GridProblem:
    def __init__(self, initial_state, goal_state, grid):
        self.initial_state = initial_state
        self.goal_state = goal_state
        self.grid = grid

    def is_goal(self, state):
        return state == self.goal_state

    def is_valid_cell(self, row, col):
        return 0 <= row < len(self.grid) and 0 <= col < len(self.grid[0]) and
self.grid[col][row] == 0

    def expand(self, node):
        row, col = node.state
        children = []
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            new_row, new_col = row + dr, col + dc
            if self.is_valid_cell(new_row, new_col):
                child_state = (new_row, new_col)
                child_node = Node(child_state, parent=node)
                children.append(child_node)
        return children
```

Example:

We have showed the BFS algorithm's traversal of the tree from root node S to target node K in the following tree form. Since the BFS search algorithm works in layers, it will follow the dotted arrow's course, which will be:

Step 3: Steps to restore the Solutions path

Define the function to reconstruct the solution path and print the path

```
def reconstruct_path(node):  
    path = []  
    while node:  
        path.append(node.state)  
        node = node.parent  
    return list(reversed(path))  
  
# Function to print the complete path  
def print_complete_path(path):  
    for step, point in enumerate(path):  
        print("Step {}: {}".format(step, point))
```

Step 4: Define the Traversal Algorithm

These are the key components of the answers. Here, too, we make use of the aforementioned breadth_first_search algorithm function.

Fully Automated BFS Pathfinding for Robots

Here, we set the starting point and endpoint states to 0 and 6, respectively, and we place grid-based obstacles. By passing these parameters into the GridProblem constructor, it is able to locate a solution for the issue instance through the BREATH_FIRST_SEARCH method. No solution discovered is the result returned if it is unable to locate a solution for the given problem instance.

Grid with Obstacles						
Robot (0,0)	Block	(2,0)	(3,0)	Block	(5,0)	Goal
(0,1)	Block	(2,1)	(3,1)	Block	(5,1)	(6,1)
(0,2)	(1,2)	(2,2)	(3,2)	Block	(5,2)	(6,2)
(0,3)	(1,3)	Block	(3,4)	Block	(5,3)	(6,3)
(0,4)	(1,4)	Block	(3,5)	(4,4)	(5,4)	(6,4)
(0,5)	(1,5)	Block	(3,6)	(4,5)	(5,5)	(6,5)
(0,6)	(1,6)	Block	(3,6)	(4,6)	(5,6)	(6,6)

Figure 4: output representation with grid obstacle

Output explantion:

1. We first start at the initial position (0,0), it is where the robot begins its exploration.
2. The robot moves south along below row such as (0,1), and (0,2).
3. Now, the robot moves east along the column to the position of (1,2).
4. The robot follows a path to the east (2,2), and (3,2), then south (3,3), and again (3,4).
5. Again, the robot moves east to towards (4,4), (5,4) and (6,4).
6. Now the robot moves towards the North along row from (6,4) to (6,3),(6,2),(6,1) and (6,0).
7. Finally, the robot reaches the goal node i.e (6,0).



Figure 6: Output representation BFS Pathfinding for Robots

The Breadth_First_Search algorithm solved the pathfinding problem, as seen in the output. Use cases in artificial intelligence where the robot must reach a specific location are given by the goal state, which is (6, 0).

To sum up, the breadth-first-search algorithm is an essential AI tool for systematising the exploration and navigation of various problem domains. Among the many AI applications, BFS is particularly useful for pathfinding, network routing, and puzzle solving thanks to its systematic exploration of tree and graph structures.

Advantages:

- BFS will offer a resolution if one is available.
- If multiple solutions exist for a certain problem, BFS will offer the simplest one that involves the fewest steps.
- Additionally, it aids in determining the quickest route to the destination state, as it requires all nodes to be at the same hierarchical level prior to accessing nodes at lower levels.
- We can use this to determine which path types are more important, and it's also quite straightforward to understand.

Disadvantages:

- A lot of RAM is needed since expanding one level of the tree necessitates saving the previous level to RAM.
- If the answer is distant from the root node, BFS will take a long time to find it.
- Searching over densely layered environments can be a real pain with this method because it has to go over every single node at every level before going on to the next.

Applications of BFS in Graphs

Among the many uses of BFS in the fields of computer science and graph theory are:

- Finding the shortest path between any two nodes in an unweighted graph is one application of BFS. To find the shortest path, it is necessary to remember which node is which along the traverse.
- One use of BFS is the detection of cycles in graphs. A cycle is present if the traversal visits the same node repeatedly.
- Connected Components: BFS can be utilised to detect graph components that are connected. A group of interconnected nodes is each component's building block.
- Directed acyclic graphs (DAGs) can undergo topological sorting with the help of BFS. By applying a linear sorting to the nodes, topological sorting ensures that, for every edge (u, v) , u will always come before v .
- You can use BFS to do a level-order traverse of a binary tree, which is useful for many reasons. Prior to going to the next level, this traversal visits all nodes at the current level.
- Network Routing: BFS is helpful for routing data packets in network protocols because it can determine the shortest path between two network nodes.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
7	What is the time complexity of the Breadth-First Search algorithm? a) $O(n)$ b) $O(n^2)$ c) $O(V + E)$ d) $O(V * E)$
8	In BFS, which data structure is used to explore nodes level by level? a) Stack b) Queue c) Array d) Linked List
9	What is the primary goal of the Breadth-First Search algorithm? a) To find the shortest path from the source node to all other nodes b) To find the longest path in a graph c) To detect cycles in a graph d) To sort the vertices of the graph
10	In BFS, what happens when a node is visited for the first time? a) It is added to the stack b) It is added to the queue and its neighbors are explored c) It is marked as visited but not processed d) It is removed from the graph
11	Which of the following is true about BFS in an unweighted graph? a) BFS will always give the longest path b) BFS will always give the shortest path c) BFS is not suitable for finding the shortest path d) BFS is not capable of traversing all the nodes

6. DEPTH-FIRST SEARCH (DFS)

One factor that enhances its efficacy and optimisation in AI is depth-first search. When it comes to improving AI systems, DFS is crucial, both in terms of algorithmic insights and practical applications. Now, we will explore DFS's foundations, its importance in AI, and its practical applications.

Data structures like trees and graphs often employ the traversal algorithm known as depth-first search. The process often begins with a search of the frontier's deepest node. The algorithm searches all the way down the search tree, beginning at the root node, until it reaches a node without any successors. If the search reaches a node without expanding successors, it will go back to the deepest node in the tree to find other possible paths.

The depth-first search recursive method is one way to explore trees and graphs. Starting with the root node and going through each path until it reaches the node with the highest depth is how the depth-first search gets its name.

A stack data structure is utilised in the construction of DFS.

A process similar to that of the BFS method is followed by the DFS algorithm. Before turning around, it finds everything along each route. It uses a stack (LIFO) and is more memory efficient than Breadth-First Search (BFS), but it doesn't guarantee the shortest path.

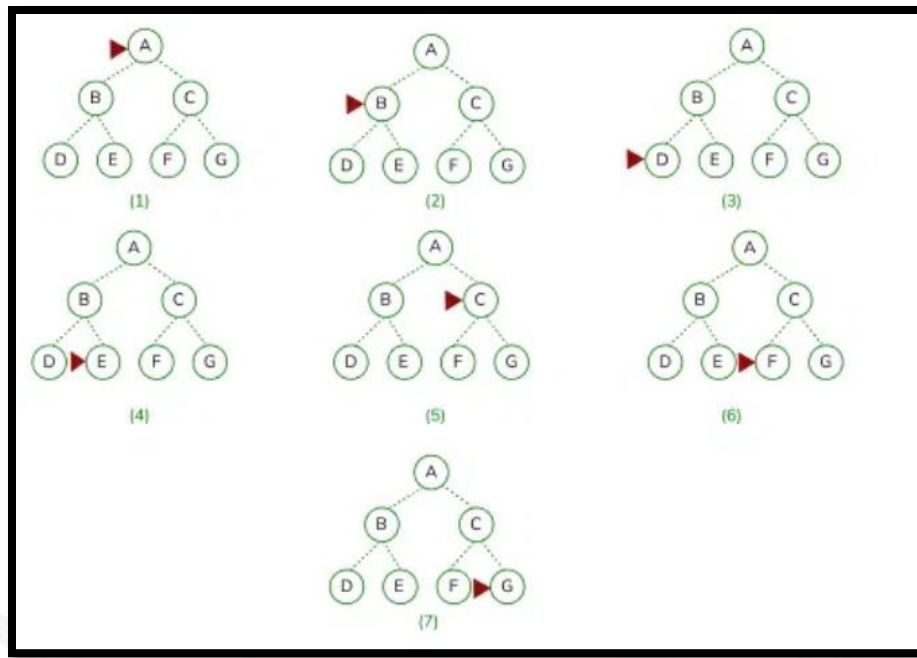


Figure 7: Node Traversal representation for DFS Algorithm

The depth-first search (DFS) algorithm selects a path across a graph, travels along it to its maximum length, and then turns around.

- The process starts at the root node and continues until all nodes have been visited or a condition is met. It does this by first traversing the single branch to its end, and then returning to the most recent undiscovered node. Beginning at node A, DFS examines its successor B, continues on to its offspring, and eventually encounters a wall at node D, as seen in the previous image. The process then loops back to node B to examine the nodes that followed it (E).
- Until the search is ended or all nodes have been searched, this systematic exploration will proceed. (Having investigated every node in B, we conclude in our example that... The DFS process begins at node C and continues to the right-hand nodes F and G. Now let's have a look at node G. Every node has been visited. We will conclude the process.

Key Features:

To put it simply, DFS algorithms in AI have the ability to explore all conceivable alternatives by first exploring the existing path to its deepest extent.

- Due to its inability to ensure the discovery of the shortest paths, DFS is inefficient in terms of cost.
- To maintain track of which nodes have been visited, DFS employs a basic principle: To facilitate the graph's backtracking, it employs a stack to record the visited nodes thus far. The DFS inserts a new node into the stack whenever it finds one, allowing it to probe its immediate vicinity. When it reaches a node that does not have any successors, also called a leaf node, it will go backwards by removing nodes from the stack in order to find other paths.
- Search that goes backwards: Lower memory consumption compared to conventional depth-first search is achieved by a version of DFS known as backtracking search. The DFS is able to generate a single successor at a time instead of all of them thanks to the backtracking search. Instead of assigning memory to a new state, this method enables dynamic state modification, which is useful for tasks like creating successors by directly editing the existing state description. Consequently, storing a single state description and set of instructions requires less memory.

Edge Classes in a depth-first search tree based on a spanning tree

Based on the spanning tree, the depth-first search tree's edges can be categorised into four groups:

- A forward edge is a connection between a node in a tree and one of its descendants.
- Back edges: The ability to trace a branch's lineage back to an ancestor is possessed by the back edge.
- An edge is called a tree edge when DFS explores a new vertex from the current vertex. The pathways to be taken throughout the traversal are critical while building a DFS spanning tree.
- Edges that join two nodes in the DFS tree but are not related to each other in any way are called cross edges.

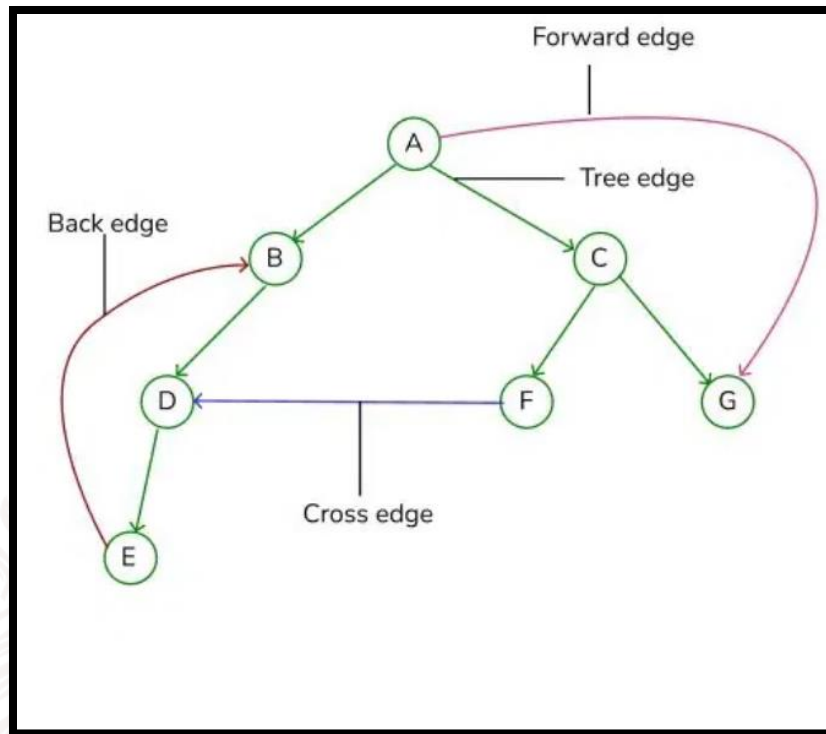


Figure 8: Representation of edge class

Algorithm:

Take a look at the below Pseudocode which explains the working of DFS

```
function findPath(robot, start, goal):
    stack ← empty stack
    visited ← empty set
    stack.push(start)

    while not stack.isEmpty():
        current ← stack.pop()
        if current == goal:
            return "Path found"

        visited.add(current)
        neighbors ← robot.getNeighbors(current)

        for neighbor in neighbors:
            if neighbor not in visited:
                stack.push(neighbor)
    return "Path not found"
```


A Step-by-Step Guide to DFS Pseudocode

- Get the stack and visited vertices sets started with nothing.
- You should add the beginning vertex to the stack.
- While there is at least one element on the stack, pop the current vertex.
 - Make sure to return "Path found" if that vertex is the target.
 - Join the visited set with the current vertex.
 - Find out who the current vertex's neighbours are.
 - Place an item onto the stack for every neighbour that has not been visited.
- "Path not found" should be returned if the loop finishes without locating the target.

How DFS Works:

- Start from the initial state.
- Expand the deepest unexpanded node first.
- Use a **stack (LIFO - Last In, First Out)** or recursion.

Time complexity of DFS:

Time complexity is an explicit consequence of DFS's one-time traversal of every vertex and edge in the worst-case situation. This level of complexity is reserved for once-and-done explicit DFS traversal. A typical way to express the time complexity of DFS is as

$$O(|V| + |E|)$$

Where,

- V-represents the number of vertices,
- E-represents the number of edges.

Implicit Time Complexity:

If we change our focus from the number of edges and vertices in a graph to the number of nodes visited in a tree, we can more accurately calculate the implicit time complexity. As a simple representation of implicit traversal of DFS,

$$O(b^d)$$

Where,

- **b-represents the branching factor,**
- **d-represents the depth of the search.**

Space complexity of DFS:

$$O(|V|)$$

Where,

- **V-represents the number of vertices.**

This is because DFS typically uses additional data structures like stack to keep track of visited vertices

Implicit Space complexity: For an implicit search in a graph, DFS's space complexity can be represented as follows

$$O(bd)$$

Where,

- **b-represents the branching factor,**
- **d-represents the depth of the search.**

The Action of DFS in Various State Space Models

- Sets of finite states represented by trees: In this case, DFS works well and finishes the job. To make sure it traverses the full state space methodically, it explores all conceivable states without revisiting any.
- Even in the absence of cycles, DFS may become trapped on an infinite path due to the fact that it is not a systematic one in the infinite state space. Because it is not possible to systematically explore all possible infinite state spaces, DFS cannot be considered complete for such spaces; it may never terminate or it may not cover the full space. The depth-first search becomes insufficient as a result.
- Problems with cyclic state spaces might cause DFS to become stuck indefinitely. To solve this problem and avoid becoming stuck in an endless loop, some DFS implementations include a cycle-checking method.

Although it can traverse all possible state spaces, acyclic state spaces might cause the same state to be expanded several times via different paths.

DFS Implementation in Robotics Pathfinding

Using DFS, one can navigate a grid or maze to reach a destination node. Starting at the starting node, the DFS algorithm methodically investigates each branch in search of a way to the goal. The maze depicted in the figures below includes the robot's starting point, obstacles, and final destination. To reach the objective node, which is located at (0,5) in this case, the robot must navigate the maze.

The maze's dimensions are defined by a `maze_size`; for this example, it's a 6x6 grid. It also specifies a set of coordinates that stand for the locations of obstacles in a labyrinth. From its starting point at (0,0), the robot is attempting to go to its target state at (0,5).

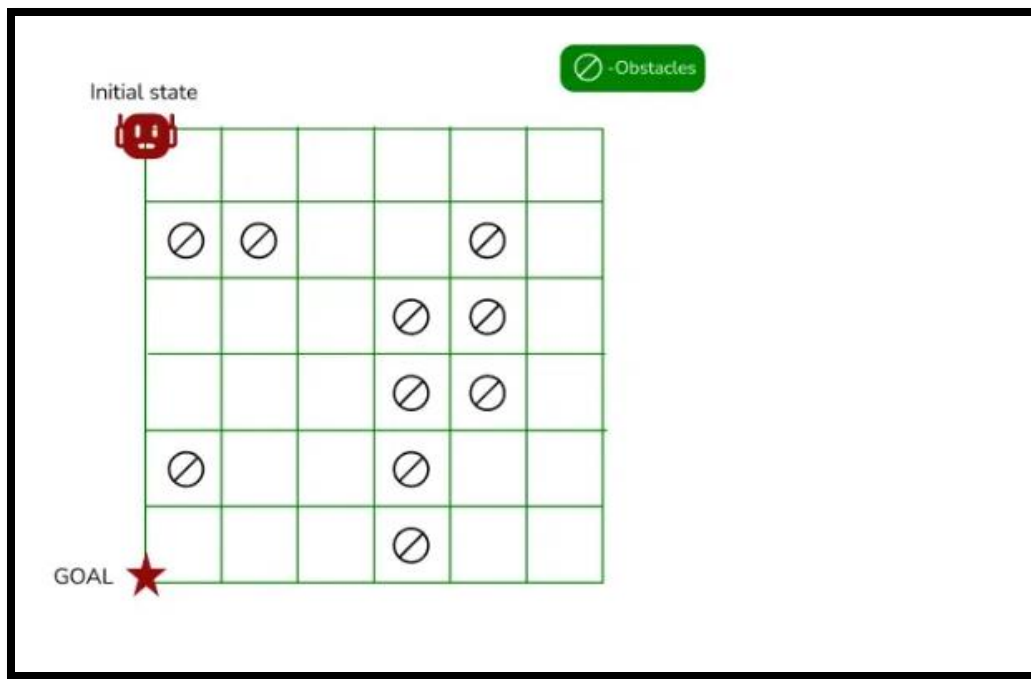


Figure 9: DFS Implementation in Robotics Pathfinding

Output explanation:

Step 1: We first start at the initial position (0,0), it is where the robot begins its exploration.

Step 2: The robot moves right along the top row of the maze such as (1,0), (2,0), (3,0).

Step 3: Now, the robot moves down to the second row to the position of (3,1).

Step 4: The robot follows a path to the left (2,1), then down (2,2), then right (1,2), and then down again (0,2).

Step 5: Now, the robot moves right to reach the third row to the position (0,3).

Step 6: The robot moves right along the third row such as passing the positions of (1,3) and (2,3).

Step 7: The robot moves down to the fourth row (2,4) and then it moves left (1,4).

Step 8: The robot moves right to the last column (1,5).

Step 9: Finally, the robot reaches the goal node at the bottom-left corner (0,5) of the maze

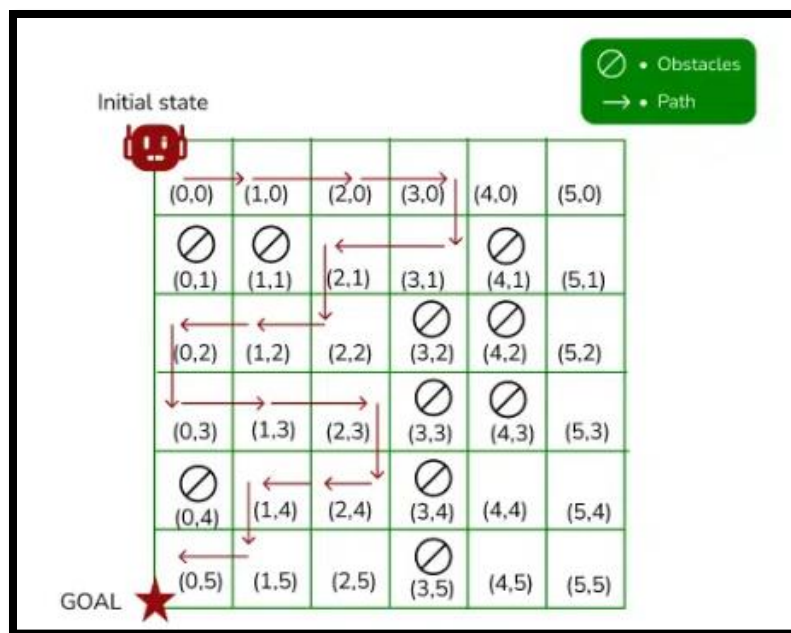


Figure 10 :Output of DFS Implementation in Robotics Pathfinding

Advantage:

- DFS uses a negligible amount of RAM since it merely keeps a stack of all the nodes that have been traversed from the root node to the present node.
- Assuming it follows the correct route, it is faster than the BFS algorithm in getting to the destination node.
- We can save time by storing the tracked route in memory with this, as it only needs to keep one at a time.

Disadvantage:

- There's no assurance of discovering a solution, and it's possible that numerous states keep happening.
- The DFS algorithm conducts in-depth searches, which can lead to an endless loop at times.
- Finding the shortest route to a solution is not guaranteed by the depth-first search (DFS) technique.

Applications of DFS in AI

- Designing the layout of corridors and walls within a maze is what the Maze generation is all about. Because it uses the recursive method and stack, this maze generation employs a randomised approach of the Depth-first search algorithm in its work. For illustration's sake, let's pretend the room is really a huge grid of cells, with the four walls constituting each cell. At the outset, the DFS chooses a neighbour at random that hasn't been visited yet. After connecting the two unconnected cells, it takes down the wall between them and adds the third cell to the stack. When no further solutions can be found, the process terminates, leaving behind a fully-formed labyrinth.
- Puzzle-solving: solving puzzles, which may involve, to methodically explore probable answers, Japanese nonograms can use depth-first search. Using DFS, Japanese nonograms can experiment with alternative grid configurations of filled and empty cells.
- Robotic pathfinding: DFS can be used for robotic pathfinding, particularly when adaptability, memory efficiency, and simplicity are key factors.

With its theoretical underpinnings and practical uses, depth-first search is quickly becoming an important AI algorithm. DFS servers are essential for exploring intricate state spaces. Even as AI research and applications continue to push the envelope, DFS will continue to be an essential algorithm for driving improvements in the field.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
12	What is the main data structure used in Depth-First Search (DFS)? a) Queue b) Stack c) Array d) Linked List
13	What is the time complexity of the Depth-First Search algorithm for a graph with V vertices and E edges? a) $O(V)$ b) $O(E)$

	c) $O(V + E)$ d) $O(V * E)$
14	In which order does Depth-First Search explore the nodes of a graph? a) Level by level b) From the root node to the farthest leaf node c) From the starting node, exploring as deep as possible along each branch before backtracking d) From the starting node, exploring each node at the same level before moving to the next level
15	Which of the following is a characteristic of Depth-First Search in a directed graph? a) It can detect cycles in the graph b) It always finds the shortest path from the source node c) It processes nodes level by level d) It guarantees to visit all nodes in the graph
16	What happens in DFS when a node is visited for the first time? a) It is added to the queue and processed b) It is added to the stack and all its neighbors are recursively visited c) It is immediately removed from the graph d) It is marked as visited but not explored

7. ITERATIVE DEEPENING DEPTH-FIRST SEARCH

In order to efficiently solve problems, Artificial Intelligence (AI) incorporates a variety of search techniques. Iterative Deepening Search (IDS), which is also called Iterative Deepening Depth-First Search (IDDFS), is an algorithm that takes the best features of two different types of search algorithms and applies them together. When the solution's depth is unknown, it becomes especially beneficial.

Artificial intelligence (AI) search algorithms like Iterative Deepening Search (IDS) combine the depth-first search (DFS) and breadth-first search (BFS) to find the best results. In order to find the target node in a graph or tree, IDS iteratively increases the depth limit, effectively carrying out a succession of DFS procedures.

When we want to get optimality and completeness without using too much memory and the solution depth is unknown, this method really shines.

Iterative deepening uses a hybrid of deep learning and binary filtering techniques. By progressively increasing the limit until a goal is found, this search method identifies the optimal depth limit. An initial "depth limit" is set by this technique, which then uses depth-first search iterations to find the target node.

The speed of breadth-first search and the memory efficiency of depth-first search are combined in this search algorithm.

When the search space is huge and the depth of the goal node is unknown, the iterative search technique is useful for uninformed search. A combination of BFS and DFS, DS is a powerful tool. Up to a specific depth limit, it executes DFS. In the event that no resolution is discovered, the depth limit is raised and DFS is reset.

Imagine doing this several times, each time delving slightly deeper into the ocean.

How IDS Works:

- Set depth = 0.

- Perform DFS up to this depth.
- Increase depth and repeat until the solution is found.

Repetition of a depth-limited DFS up to progressively higher depth levels is key to IDS. Iteratively increasing the depth limit from zero is the starting point. A DFS search is carried out by each iteration till the present depth limit is reached.

The algorithm is described in detail here:

1. Start at the Root Node: Begin the search from the root node (or starting point).
2. Perform DFS with Depth Limit (L): In each iteration, perform a DFS with a depth limit L.
3. Increment Depth: After each iteration, increment L by 1.
4. Repeat: Continue this process until the goal node is found or the search space is exhausted.

```
IterativeDeepeningSearch(root, goal)
  for depth = 0 to  $\infty$  do
    result = DepthLimitedSearch(root, goal,
                                depth)
    if result == found then
      return result
```

Robot Navigation using Iterative Deepening Search

Step 1: First, we import the Python libraries that are needed. We make use of matplotlib for path and grid visualisation, and numpy for numerical array operations on the grid.

Step 2: Establish Paths of Motion We lay out all the potential paths the robot can take when moving. On top of that, the robot has full control over its movement in all four directions. The changes in the (x, y) coordinates are shown as tuples.

- Number one: tell the robot which way to go (up, down, left, right).
- These are the directions: [(0, 1), (0, -1), (1, 0), (-1, 0)]

Step 3: Determine Movement Routes We map out every possible route the robot can take while it moves. Plus, it can move freely in any of the four directions that the robot can sense. In this case, tuples represent the changes in the (x, y) coordinates.

Step 4: Fourth, put the Iterative Deepening Depth-First Search (IDDFS) algorithm into action.

The search is carried out by the iddfs function. It searches the grid to a depth you specify using the helper function dls (Depth-Limited Search), and then it returns the path if it finds the target.

Step 5: Add Obstacles to the Grid

On a grid, we put 0 for a clear path and 1 for any impediments. It is a difficult navigation task for the robot to solve on the more complicated grid.

Step 6: The sixth step is to search for the route.

The next step is to execute the search from the beginning to the endpoint and print out the results by calling the iddfs function.

Step 7: Picture the Route

Our code concludes with a method called visualize_path that draws the grid and the robot's path using matplotlib. The red dots represent the journey, the green dots represent the starting point, and the blue dots represent the endpoint.

Iterative deepening search: why use it?

The drawbacks of both BFS and DFS are what motivated the development of IDS. I'll explain why it's unique:

- Because it uses just $O(bd)$ space, where b is the branching factor and d is the depth of the goal node, IDS allows for efficient use of space. This is because the method it employs, DFS, restricts itself to storing the current path.
- **Optimality:** Just like BFS, IDS ensures that the shortest path (least depth solution) is found in uniform-cost or unweighted graphs.
- In the case of a finite search tree, IDS ensures completeness by promising to locate the solution, should one exist.

Advantages:

- When it comes to memory efficiency and quick search, it combines the best features of the BFS and DFS search algorithms.

- It is an easy method that is commonly utilised because it expands upon the traditional depth-first search technique.
- Assuming that the costs of all edges in the search space are equal, this search technique ensures that the optimal solution will be found.
- Because it is a full algorithm, it will never fail to find a solution—provided that one exists.
- Because it just keeps the current path in memory instead of the full search tree, the Iterative Deepening Depth-First Search (IDDFS) algorithm requires less memory than Breadth-First Search (BFS).

Disadvantages:

- A major issue with IDS is that it re-explores the same nodes multiple times in order to increase the depth limitations, which requires computing effort. As the depth limit rises, the root node, for instance, is probed more and more. Duplicate calculations result from this.
- The space complexity is minimal, but the temporal complexity is greater than that of DFS and BFS. With b being the branching factor and d being the solution depth, the time complexity of IDS is $O(b^d)$. Overhead time is added to the total computing time by the repetitive exploration.
- Since IDS prioritises uniform depth over path cost, it underperforms on weighted graphs, where the cost of traversing edges varies.

Examples of AI Applications of Iterative Deepening Search

When memory efficiency and determining the quickest solution path are of the utmost importance in artificial intelligence applications, IDS becomes invaluable. As an example, consider the following:

- Game AI (Chess, Go): IDS is commonly utilised in game AI due to the large search space and uncertain solution depth. The use of IDS allows for the more efficient exploration of game trees with millions of different states.
- Algorithms for Pathfinding: In real-time pathfinding situations, such as robot navigation, IDS can be used, particularly when the depth of the destination in the search space is unknown.
- Crawling techniques for search engines can benefit from IDS since they must strike a balance between memory consumption and the discovery of all relevant results, which is crucial.

- Because it effectively manages the exponential rise of potential states, IDS is useful for solving puzzles in artificial intelligence (AI) where the ideal solution involves traversing numerous states, such as 8-puzzle.

An effective method that combines the best features of both Depth-First and Breadth-First Search is Iterative Deepening Search. An crucial component of AI search algorithms, it ensures memory efficiency without sacrificing completeness or optimisation. Nevertheless, there are costs and benefits to consider, especially when it comes to computing time. Wherever optimal solutions are needed yet depth is unknown, IDS displays its worth in areas like game AI and pathfinding.

Algorithm:

```
// Returns true if target is reachable from
// src within max_depth
bool IDDFS(src, target, max_depth)
    for limit from 0 to max_depth
        if DLS(src, target, limit) == true
            return true
    return false
bool DLS(src, target, limit)
    if (src == target)
        return true;
    // If reached the maximum depth,
    // stop recursing.
    if (limit <= 0)
        return false;
    foreach adjacent i of src
        if DLS(i, target, limit-1)
            return true
    return false
```

Keep in mind that we repeatedly visit the top-level nodes. Each level is visited once, starting with the last one (or max depth level). Since the majority of a tree's nodes are located in the base, what appears to be an expensive endeavour turns out to be really inexpensive. As a result, making repeated trips to the top levels is largely irrelevant. The algorithm's implementation is shown below.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
17	What is the primary advantage of Iterative Deepening over Depth-First Search (DFS)? a) It uses less memory b) It always finds the shortest path in an unweighted graph c) It explores all nodes before backtracking d) It avoids the need for a stack
18	In Iterative Deepening Depth-First Search (IDDFS), what happens during each iteration? a) The search depth is increased incrementally, and DFS is run for each depth limit b) The depth is fixed, and BFS is run at each level c) The entire graph is explored, and the depth is not restricted d) The search is conducted in random order
19	Which of the following is true about Iterative Deepening? a) It combines the advantages of both BFS and DFS b) It always uses a queue for storing nodes c) It is slower than BFS and DFS for all graphs d) It cannot handle infinite depth graphs
20	What is the time complexity of Iterative Deepening Depth-First Search (IDDFS) in a graph with V vertices and E edges? a) $O(V)$ b) $O(E)$ c) $O(V + E)$ d) $O(V * E)$

8. SUMMARY

- Uninformed search techniques are fundamental strategies in Artificial Intelligence where the agent searches the problem space without using any domain-specific knowledge about the location of the goal. These methods are also referred to as blind search techniques. The three primary uninformed search strategies are Breadth-First Search (BFS), Depth-First Search (DFS), and Iterative Deepening Search (IDS). In Breadth-First Search, the agent explores the search space level by level, expanding all nodes at a given depth before moving deeper. BFS guarantees the shortest path to the goal if all actions have equal cost but requires significant memory, especially in wide or deep search trees.
- In contrast, Depth-First Search dives deep into one branch of the tree before backtracking, using much less memory. Although DFS is memory-efficient, it can get stuck exploring infinite paths and does not guarantee the shortest solution. Iterative Deepening Search combines the advantages of both BFS and DFS.
- It conducts repeated depth-limited searches, gradually increasing the depth limit until the goal is found. IDS is both memory-efficient and finds the optimal solution, making it highly effective when the solution depth is unknown. Mastering these three techniques is essential because they lay the groundwork for understanding more advanced AI search algorithms and solving real-world problems effectively. Choosing the right search strategy often depends on factors like available memory, the importance of finding the shortest path, and whether the depth of the solution is known.
- To sum up, the breadth-first-search algorithm is an essential AI tool for systematising the exploration and navigation of various problem domains. Among the many AI applications, BFS is particularly useful for pathfinding, network routing, and puzzle solving thanks to its systematic exploration of tree and graph structures.
- With its theoretical underpinnings and practical uses, depth-first search is quickly becoming an important AI algorithm. DFS servers are essential for exploring intricate state spaces. Even as AI research and applications continue to push the envelope, DFS will continue to be an essential algorithm for driving improvements in the field.
- An effective method that combines the best features of both Depth-First and Breadth-First Search is Iterative Deepening Search. An crucial component of AI search algorithms, it ensures memory efficiency without sacrificing completeness or optimisation. Nevertheless, there are costs and

benefits to consider, especially when it comes to computing time. Wherever optimal solutions are needed yet depth is unknown, IDS displays its worth in areas like game AI and pathfinding.



9. GLOSSARY

Breadth-First Search (BFS)	- An algorithm that explores all nodes at the present depth before moving on to nodes at the next level. It is widely used for pathfinding, network routing, and solving puzzles, ensuring the shortest path is found when step costs are uniform.
Depth-First Search (DFS)	- An algorithm that explores as far as possible along each branch before backtracking. It is crucial for navigating large, intricate state spaces and remains fundamental in advancing AI research and applications.
Iterative Deepening Search (IDS)	- A hybrid search strategy that combines the memory efficiency of DFS with the completeness of BFS. IDS incrementally increases depth limits and is highly useful when the solution depth is unknown, particularly in game AI and pathfinding.
Pathfinding	- The process of finding the shortest or most efficient path from a start point to a goal, often used in robotics, video games, and network navigation.
Network Routing	- The method of selecting paths in a network along which to send data packets. BFS is particularly suited for routing protocols in communication networks.
Puzzle Solving	- Applying search algorithms to solve structured problems like mazes, Sudoku, or the 8-puzzle, where a clear start and goal state are defined.
State Space	- The set of all possible configurations or states that can be reached from the initial state by any sequence of actions.
Completeness (in Search Algorithms)	- A property of an algorithm that guarantees it will find a solution if one exists.
Memory Efficiency	- An important characteristic where an algorithm uses minimal memory resources, which is critical for large and complex problems.
Optimality	- Refers to an algorithm's ability to find the best or least-cost solution among all possible solutions.

Systematic Exploration	- An organized method of exploring all possible options or paths in a structured manner without missing any potential solutions.
-------------------------------	--

Computational Cost	- The method or algorithm used by the agent to explore the search space and find a solution (e.g., BFS, DFS, A*).
---------------------------	---



10. TERMINAL QUESTIONS

1. Identify what is meant by an uninformed search technique in AI.
2. Specify the data structure typically associated with Breadth-First Search (BFS).
3. Explain under what conditions BFS guarantees the shortest path to the goal.
4. Analyse the main limitation of Depth-First Search (DFS).
5. Identify the data structure commonly used in Depth-First Search (DFS).
6. Justify why Iterative Deepening Search (IDS) is considered memory efficient.
7. Compare the working strategies of Breadth-First Search (BFS) and Depth-First Search (DFS).
8. Evaluate the major drawback of Iterative Deepening Search (IDS).
9. Assess why IDS is preferred over BFS and DFS when solution depth is unknown.
10. Classify a real-world application where Breadth-First Search is commonly used.

11. ANSWERS

Self-Assessment Questions

1. Answer: B) A technique to systematically explore possible solutions to reach a goal
2. Answer: B) They require no information other than the problem definition
3. Answer: C) Breadth-First Search (BFS)
4. Answer: C) Informed searches use additional knowledge (heuristics) about the goal
5. Answer: C) A* Search
6. Answer: C) When heuristic information about the goal is available
7. Answer: c) $O(V + E)$
8. Answer: b) Queue
9. Answer: a) To find the shortest path from the source node to all other nodes
10. Answer: b) It is added to the queue and its neighbors are explored
11. Answer: b) BFS will always give the shortest path
12. Answer: b) Stack
13. Answer: c) $O(V + E)$
14. Answer: c) From the starting node, exploring as deep as possible along each branch before backtracking
15. Answer: a) It can detect cycles in the graph
16. Answer: b) It is added to the stack and all its neighbors are recursively visited
17. Answer: b) It always finds the shortest path in an unweighted graph
18. Answer: a) The search depth is increased incrementally, and DFS is run for each depth limit
19. Answer: a) It combines the advantages of both BFS and DFS
20. Answer: c) $O(V + E)$

Terminal Questions Answers

Answer 1: Uninformed search techniques explore the problem space without any domain-specific knowledge, relying solely on the structure of the search space.(Refer to Section 2 for detailed information)

Answer 2: BFS utilizes a queue (First-In-First-Out) to manage the nodes to be explored.(Refer to section 3 for more details)

Answer 3: BFS guarantees the shortest path when all step costs are equal, as it explores nodes level by level. (Refer to section 3 for more details)

Answer 4: DFS can get trapped in infinite paths in cyclic graphs and doesn't guarantee the shortest route. (Refer to section 4 for more details)

Answer 5: DFS typically uses a stack (Last-In-First-Out) or recursion to manage node exploration. (Refer to section 4 for more details)

Answer 6: IDS combines the space efficiency of DFS with the completeness of BFS, using linear memory relative to the depth of the solution. (Refer to section 4 for more details)

Answer 7: Breadth-First Search (BFS) explores all nodes at the current depth level before moving on to the next level. It systematically checks neighbours first and guarantees the shortest path if all steps have equal cost. BFS uses a queue data structure (First-In-First-Out).

Depth-First Search (DFS), on the other hand, explores as far as possible along one branch before backtracking. It dives deep into one path without checking all the neighbours first. DFS uses a stack data structure (Last-In-First-Out) or recursion.

- BFS is wide and level-wise, memory-intensive, but finds the shortest path.
- DFS is deep and branch-focused, memory-efficient, but may miss the shortest solution. (Refer to Sections 3 & 4)

Answer 8: IDS may incur additional computation due to repeated explorations at each depth level. (Refer to section 5 for more details)

Answer 9: IDS is preferred as it ensures completeness and optimality with lower memory usage, especially when the depth of the solution is unknown. (Refer to section 5 for more details)

Answer 10: BFS is commonly applied in network routing to find the shortest path between nodes. (Refer to section 3 for more details)

12. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.



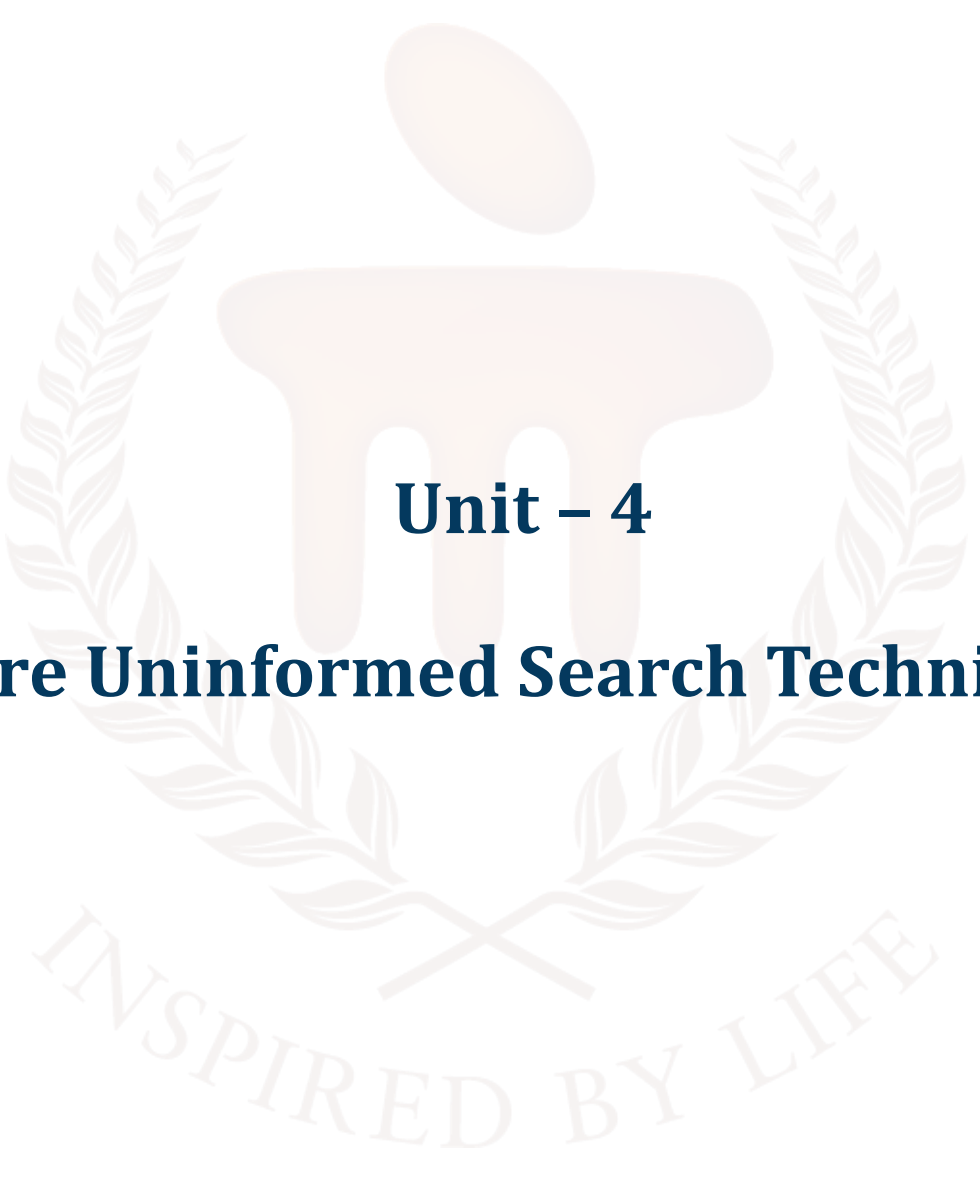
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 4

More Uninformed Search Techniques

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Introduction to Bidirectional Search	-	1	6
3	Importance of the bidirectional approach	-	-	7
4	Working of Bidirectional Search	-	-	8 - 14
5	Hill Climbing in Artificial Intelligence	-	-	15
6	Working of the Hill Climbing Algorithm	-	-	16 - 17
7	Types of Hill Climbing in Artificial Intelligence	1	2	18 - 22
8	Uniform Cost Search (UCS) in AI	-	-	23
9	Working of Uniform Cost Search Work	-	3	24 - 29
10	Summary	-	-	30
11	Glossary	-	-	31 - 32
12	Terminal Questions	-	-	33
13	Answers	-	-	34 - 36
14	References	-	-	37

1. INTRODUCTION

In previous unit, you have mastered the art of solving problems systematically without any hints or special knowledge about the solution. You had explored three powerful techniques that help an AI agent (or even a human) search through possibilities, even in the dark! Specifically, you have learnt: Breadth-First Search (BFS): How to explore all possibilities level-by-level, ensuring you will find the shortest path to the goal Depth-First Search (DFS): How to dive deep into the possibilities, following a path as far as possible before backtracking — a memory-efficient strategy. Iterative Deepening Search (IDS): How to cleverly combine the best of BFS and DFS — diving deep but still making sure that you have found the shortest solution with limited memory. We also understood how AI agents search without prior knowledge of where the goal might be. Analyse trade-offs between memory usage, speed, and optimality. Visualise and design search trees for real-world problem scenarios.

In this unit, we will explore three important search strategies that are fundamental to AI problem solving: Bidirectional Search, Hill Climbing, and Uniform Cost Search. Bidirectional Search introduces an efficient approach where the search begins simultaneously from the start and goal states, meeting somewhere in between to drastically reduce the search time compared to traditional methods. Hill Climbing, a heuristic-based search technique, teaches students how to make decisions based on moving toward neighbouring states that appear more promising, while also highlighting challenges like local maxima and plateaus. Finally, Uniform Cost Search emphasises cost optimisation by always expanding the path with the lowest cumulative cost, ensuring the discovery of the least-cost solution. By studying these techniques, students will gain a deeper understanding of how different search algorithms handle complexity, optimise resources, and find solutions effectively in real-world AI applications.

1.1. Objectives

By the end of this unit, you will be able to:

- Interpret the fundamental concepts of uninformed search techniques
- Compare the operational mechanisms of various advanced search techniques
- Analyse the advantages and limitations
- Apply appropriate uninformed search strategies to solve simple AI search problems effectively



2. INTRODUCTION TO BIDIRECTIONAL SEARCH

In artificial intelligence, a bidirectional search is an algorithm that meets halfway between the starting and ending states to search for a solution, thus reducing the search time.

This unit goal is to teach learners all about bidirectional search algorithms, including how they function, the advantages they offer, and the real-world uses they have in AI.

A search method that works in both directions is called a bidirectional search. In this setup, two searches are executed in parallel: one goes from the source to the goal, while the other goes in the opposite direction, from the goal back to the source. When everything is running well, the two searches will intersect at the centre of the data structure.

Finding the shortest path from the source (beginning node) to the goal (end node) is the job of the bidirectional search algorithm, which operates on a directed graph. When both searches reach a node, the algorithm will terminate. Each search will begin at a different location. When it comes to artificial intelligence (AI), bidirectional search is a powerful search technique for discovering the shortest path between two states—the beginning and the objective. It works by doing two searches at once, one starting from the starting point and the other working backwards from the desired endpoint. At the point where the two processes intersect, the search is terminated. In big search areas, where Depth-First Search (DFS) and Breadth-First Search (BFS) might not be the best options, this approach comes in handy.

3. IMPORTANCE OF THE BIDIRECTIONAL APPROACH

It's a more efficient method that shortens the time needed to traverse the graph. This method works well when both the beginning and the end nodes are distinct and well-defined. In both directions, the branching factor is the same.

- Efficiency in Reducing Search Time
 - Starts searching from both the initial state and the goal state simultaneously.
 - **Cuts the problem in half**, meeting in the middle, which significantly reduces search time.
- Reduces Computational Complexity
 - Compared to unidirectional searches like BFS, it explores fewer nodes by working from both ends of the search space.
 - This makes it much faster and more efficient, especially in large search spaces.
- Faster Pathfinding
 - Particularly effective for problems like GPS navigation, where finding the shortest route between two points is critical.
 - Improves speed in real-world applications like route planning and navigation.
- Effective for Large, Finite Search Spaces
 - Works best when the goal state is known in advance and the search space is large but finite.
 - Speeds up problem-solving in areas like robotics, puzzle solving, and game AI.
- Optimal for Speed and Resource Efficiency

By halving the search space, it ensures solutions are found quickly without using excessive computational resources.

4. HOW BIDIRECTIONAL SEARCH WORKS

An advanced search algorithm, Bidirectional Search works by investigating both the starting and ending points at the same time. The goal is to find a compromise that will cut down on search time and space. I'll walk you through each step by step.

1. Two Searches Simultaneously:

- Bidirectional Search begins by initiating **two separate searches**:
 - One search starts from the **initial state** and explores all possible paths.
 - The other starts from the **goal state** and explores all possible paths in reverse.

2. Meeting in the Middle:

- The key concept of bidirectional search is that these two searches work **towards each other** from opposite ends.
- They explore the state space, with the goal of **meeting in the middle**.

3. Stopping Criteria:

- Once the two searches meet, a **path between the start and goal state** is found.
- The solution is formed by combining the paths from the initial state to the meeting point and from the goal state to the meeting point.

4. Efficiency:

- Since the algorithm searches from both directions, it reduces the overall number of states that need to be explored.
- If the search depth is **d** and the branching factor is **b**, then a unidirectional search would explore **$O(b^d)$** nodes. However, bidirectional search only explores **$O(b^{(d/2)})$** nodes from each direction, which is **much faster**.

5. Requirements:

- Bidirectional search works best when the goal state is **known** in advance.
- It assumes that there is a **well-defined path** from the start state to the goal state.
- The graph must be **undirected** or allow traversing in both directions (i.e., the reverse search must be possible).

Visualising the Process:

- Imagine you're trying to find the shortest path between two points on a map. Instead of starting at point A and searching all the way to point B, you start at both A and B. Each search finds nodes closer to the middle, and when they meet, you have the shortest route.

Key Points:

- **Two simultaneous searches:** One from the start, one from the goal.
- **Meets in the middle:** Reduces the search space by half.
- **Efficiency:** Reduces time complexity from $O(b^d)$ to $O(b^{(d/2)})$.
- **Works best with known goal states:** Assumes the solution is reachable and the graph is undirected.

Bidirectional Search for Maze Navigation

Step1: Bring in Required Library Files

At this point, the necessary libraries are imported into the code, including collections, numpy, and matplotlib, which are used for visualisation and array manipulation, respectively. deque for creating a parallel queue

```
import matplotlib.pyplot as plt
import numpy as np
from collections import deque
```

Step2: Make a Function to Verify Legal Motions

This function verifies the validity of a specified maze move. For a move to be legitimate, it must stay inside the maze's boundaries and not touch any walls (1).

```
def is_valid_move(x, y, maze):
    return 0 <= x < len(maze) and 0 <= y < len(maze[0]) and maze[x][y] == 0
```

Step 3: Establish a Breadth-First Search (BFS) Method

Beginning at its current location, the BFS function investigates the labyrinth in four directions: up, down, left, and right. To facilitate subsequent path reconstruction, it modifies the parent mapping, visited nodes, and queue.

```
def bfs(queue, visited, parent):
    (x, y) = queue.popleft()
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right moves
    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if is_valid_move(nx, ny, maze) and (nx, ny) not in visited:
            queue.append((nx, ny))
            visited.add((nx, ny))
            parent[(nx, ny)] = (x, y)
```

Step 4: Execute the Bidirectional Search Algorithm

Here, we begin two BFS searches, one from the beginning and one from the end. Both searches will keep going until they reach an intersection. It returns the junction node and the pathways in both directions if an intersection is found.

```
def bidirectional_search(maze, start, goal):
    if maze[start[0]][start[1]] == 1 or maze[goal[0]][goal[1]] == 1:
        return None, None, None

    queue_start = deque([start])
    queue_goal = deque([goal])
    visited_start = set([start])
    visited_goal = set([goal])
    parent_start = {start: None}
    parent_goal = {goal: None}
```

```
while queue_start and queue_goal:
    bfs(queue_start, visited_start, parent_start)
    bfs(queue_goal, visited_goal, parent_goal)

    # Check for intersection
    intersect_node = None
    for node in visited_start:
        if node in visited_goal:
            intersect_node = node
            break

    if intersect_node is not None:
        return (intersect_node, parent_start, parent_goal)

return (None, None, None)
```

Step 5: Figure Out How to Get There

Starting at the junction node, this method retraces the path to its endpoint. The two paths, one leading to the beginning and one leading to the intersection, are joined together.

```
def reconstruct_path(intersect_node, parent_start, parent_goal):
    if intersect_node is None:
        return []

    path = []
    # from start to intersection
    step = intersect_node
    while step is not None:
        path.append(step)
        step = parent_start[step]
    path.reverse()

    # from intersection to goal
```

```
step = parent_goal[intersect_node]
while step is not None:
    path.append(step)
    step = parent_goal[step]

return path
```

Step 6: See the Labyrinth and the Way Through It

With matplotlib, this function draws a picture of the labyrinth and the way out. It shows the beginning point (green), the end point (red), the solution path (gold), and the cells of the maze.

Step 7: Assign Beginning and Ending Locations to the Maze

The starting and ending locations of the maze are defined at this stage.

```
# Define the maze
```

```
maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 1, 0]
]
```

```
start = (0, 0)
```

```
goal = (4, 4)
```

Step 8: Check Both Ways, Make a New Route, and See the Results

The code then uses the previously described methods to visualise the results of the bidirectional search and recreate the path if one is identified.

Two-Way Search's Advantages

- **Efficient:** Its speed is the most striking benefit. It narrows the search space considerably, which usually results in faster results, by searching from both ends towards the centre at the same time.
- Finding an optimal answer is assured when using bidirectional search with uniform-cost search techniques.
- **Less Memory Needed:** Particularly in highly linked graphs or expansive search areas, it frequently uses less memory than more conventional algorithms such as BFS.

Obstacles and Things to Think About

- When compared to unidirectional techniques, managing two simultaneous searches and making sure they meet optimally might be a complex implementation.
- **Memory Management:** Although bidirectional search typically uses less memory than other options, it is still important to carefully control both search frontiers.
- **Usefulness:** It may not work for every issue. In searchable spaces, for example, when forward and backward growth are relevant and practicable, it works best.

The Real-World Use of Bidirectional Search for Artificial Intelligence

There are several real-world uses for bidirectional search in artificial intelligence, thus it's not only a theoretical construct:

- Useful in many contexts, including GPS navigation and robotic movement planning, pathfinding is a component of artificial intelligence navigation systems.
- When solving a puzzle or game with a clearly defined end state, like a Rubik's Cube, this strategy works well.

Social network analysis makes good use of network analysis since it allows for the rapid establishment of linkages between elements (such as the degree of separation between two people).

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the primary goal of Bidirectional Search? A. To use less memory than Depth-First Search B. To increase the depth of the search tree C. To reduce the time complexity by searching from both directions D. To avoid backtracking
2	In Bidirectional Search, where does the search begin? A. Only from the initial node B. Only from the goal node C. From the middle of the search space D. From both the initial and goal nodes simultaneously
3	What is the time complexity of Bidirectional Search in terms of branching factor b and depth d? A. $O(bd)O(b^d)O(bd)$ B. $O(bd/2)O(b^{\{d/2\}})O(bd/2)$ C. $O(db)O(d^b)O(db)$ D. $O(\log_{fo} bd)O(\log b^d)O(\log bd)$
4	Which of the following is a key requirement for Bidirectional Search to work effectively? A. The branching factor must be constant B. The goal state must be unknown C. The graph must allow traversal in both directions D. It should avoid using a heuristic function
5	What is the main drawback of Bidirectional Search? A. It cannot find the optimal path B. It does not terminate C. It requires both directions of search to be feasible and well-defined D. It uses excessive backtracking

5. HILL CLIMBING IN ARTIFICIAL INTELLIGENCE

In the field of artificial intelligence (AI), hill climbing is a popular optimisation strategy for discovering optimal solutions to problems. It is a member of the family of local search algorithms and is commonly used to find the best answer out of several options while solving optimisation problems.

Mathematical optimisation problems in AI often use the heuristic search process known as hill climbing. Because it is a local search method, its primary goal is to discover the best solution by iteratively improving upon an existing one and seeing if the results improve it. The procedure is like climbing a hill: you keep trying to get better and better until you reach the peak, or local maximum, and then there's no going back.

When it comes to optimisation problems or finding answers in enormous search spaces, hill climbing is a basic idea in AI due to its simplicity, efficiency, and efficacy.

6. WORKING OF THE HILL CLIMBING ALGORITHM

The Hill Climbing method starts with a basic solution and refines it through iteratively adding and removing minor pieces. To find out how good the answer is, a heuristic function looks at these modifications. At some point, the algorithm will have reached a local maximum, at which point it will no longer be able to improve using the present set of movements.

Basic Concepts of Hill Climbing Algorithms

This is the procedure for ascending a hill:

- The initial state is to begin with a solution that is completely at random.
- Using minor changes (mutations or tweaks), locate nearby states of the present solution.
- Transfer to Neighbouring State: Transfer to a neighbouring state if, according to an evaluation function, it provides a better solution.
- Stop: Keep going till there isn't a neighbouring state that's better than this one. Whether you're trying to maximise or minimise, you've hit a local maximum or minimum now.

Hill Climbing as a Heuristic Search in Mathematical Optimisation

Algorithms like hill climbing are commonly employed in artificial intelligence to resolve mathematical optimisation issues. Hill Climbing can quickly discover a decent solution with a big set of inputs and a solid heuristic function, but it might not always find the global optimal maximum.

Hill Climbing is a popular tool in mathematical optimisation for solving issues where maximising or minimising a real function is required. For instance, one such problem is the Travelling Salesman Problem, where the goal is to reduce the salesman's journey distance between many places.

What is a Heuristic Function?

At each branching stage of a search algorithm, heuristic functions rank the options according to the information that is available. This aids the algorithm in efficiently steering the search towards a good result by selecting the best route among several feasible paths.

Features of the Hill Climbing Algorithm

1. Hill climbing is an example of a variant of algorithms used for both generation and testing. Here are the steps:
 - Thinking of potential answers: Possible answers in the search space are generated by the algorithm.
 - We test each created solution to see if it satisfies our requirements.
 - Iteration: The algorithm continues until it finds a good solution; if not, it starts over from the beginning with generation.

Using data from past evaluations to guide future moves in the search space, Hill Climbing is able to refine its search through this iterative feedback process.

2. The Hill Climbing algorithm takes what is known as a greedy approach, which means that it optimises the objective function at each step. This approach seeks to efficiently discover the optimum answer by picking the greatest option right away, without thinking about the bigger picture of the problem.

7. TYPES OF HILL CLIMBING IN ARTIFICIAL INTELLIGENCE

1. Simple Hill Climbing Algorithm

The simplest kind of hill climbing, known as simple hill climbing, involves the algorithm iteratively evaluating each neighbouring node and picking the first one that provides an improvement over the current one.

Algorithm for Simple Hill Climbing

- Take a look at the starting point. Make sure to return success if it's a goal state.
- Put the starting point back into play.
- Repeat until either an answer is obtained or no operators are available:
 - Choose a different state that hasn't been applied to the present one yet.
 - Assess the altered condition.
 - "Return success" if getting to the new state is your aim.
 - Just make the new state the current state and keep going if it's better.
 - Keep looking in nearby states if things don't get better.
- If a better state cannot be found, exit the function.

2. Steepest-Ascent Hill Climbing

Longest Gradient An improved variant of the original hill climbing game is Hill Climbing. By evaluating all neighbours and going to the one with the biggest improvement, it avoids going to the first neighbouring node that improves the state. This approach is called steepest climb.

Algorithm for Steepest-Ascent Hill Climbing

1. Take a look at the starting point. Make sure to return success if it's a goal state.
2. Put the starting point back into play.
3. Continue until either a solution is discovered or the existing situation is maintained:
 - Pick a different state that hasn't been used for this one yet.
 - Make sure a "best state" variable is initialised and then assess all nearby states.
 - Make sure to update the best state if a better one is found.
 - Make it a success if getting to the best state is your aim.
 - Replace the present state with the greatest possible one if it improves upon it.
4. If a better state cannot be found, exit the function.

3. Stochastic Hill Climbing

Stochastic Hill Climbing makes searching more unpredictable. As an alternative to considering all neighbours or picking the first improvement, it chooses a neighbouring node at random and bases its decision to move on how much better it is than the present state.

Algorithm for Stochastic Hill Climbing:

- Take a look at the starting point. Make sure to return success if it's a goal state.
- Put the starting point back into play.
- Continue until either a solution is discovered or the existing condition remains unchanged:
 - Take the current state and run it through the successor function to get all the states that are nearby.
 - Use a probability function to pick a neighbouring state at random.
 - Put the selected state into effect as the current state if it is superior.
 - Return success if the chosen neighbour is the target state.
- If a better state cannot be found, exit the function.

State-Space Diagram in Hill Climbing: Key Concepts and Regions

The state-space diagram plots the values of the objective function (the function we intend to maximise) against all potential states that the Hill Climbing algorithm can reach.

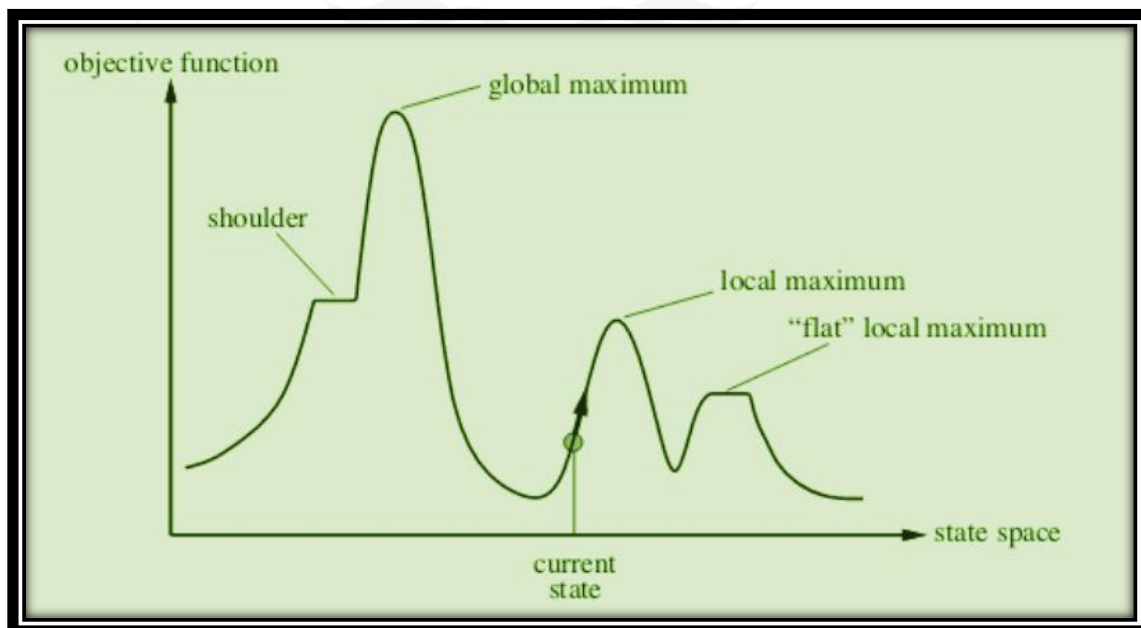


Figure 1: State space diagram

In the above given figure1, which represent the state space diagram :

- On the X-axis we see the state space, which contains every conceivable configuration or state that the algorithm can achieve.
- The values of the objective function that correspond to each state are shown on the y-axis.

At the point where the objective function hits its highest value—also called the global maximum—in a state-space diagram, we have the best possible solution.

Key Regions in the State-Space Diagram

1. When a state is better than its neighbours but isn't the best overall, it is called a local maximum.
2. Even though it has a greater goal function value than neighbouring states, there might be a global maximum somewhere.
3. The optimal state in a state-space diagram is called the global maximum since it is at this point that the objective function is maximised. The algorithm is trying to find the best possible answer here. When the objective function values of nearby states are equal, creating a plateau or flat local maximum, the algorithm has a hard time determining which way to go.
4. A ridge is a gently sloping, elevated area that may resemble a peak in shape. This could lead the algorithm to terminate too soon, preventing it from finding neighbouring solutions that are superior.
5. Where the algorithm is in its quest to find the best solution is represented by its current state in the state-space diagram.
6. Shoulder: If the algorithm keeps searching beyond the plateau, it will eventually find better solutions; a shoulder is a plateau with an uphill edge.

Advantages of Hill Climbing Algorithm

1. **Simplicity and Ease of Implementation:** Developers and researchers alike will find Hill Climbing to be an approachable algorithm due to its simplicity and intuitiveness.
2. **Versatility:** Optimisation issues with complex constraints and huge search areas are just two examples of the many types of optimisation problems that the algorithm can handle. Particularly helpful in scheduling, route planning, and resource allocation.

3. **Efficiency in Finding Local Optima:** If you need a decent answer fast, Hill Climbing is a fantastic option because it finds local optima very efficiently.
4. **Customisability:** More specific optimisation methods are possible with the algorithm's ease of modification and extension to include new heuristics or limitations.

Difficulties with Hill Climbing Algorithm • Regional Peak, Flat Area, and Hill

- **Local Maximum Problem:** When every neighbouring state has worse values than the current state, we say that there is a local maximum. Hill Climbing will not proceed to a worse state since it is greedy; thus, the algorithm will end even if a better solution is further down the line.

How to Get Beyond the Local Maximum?

The employment of backtracking techniques is one viable option for resolving the local maximum issue. If the algorithm becomes stuck in an unpleasant state, it may easily return to a previous configuration and try different paths because it keeps track of all the states it has visited.

- **Plateau Problem:** When all of the states adjacent to a plateau have the same value, we say that this area of the search space is flat. The algorithm finds it challenging to determine the optimal course of action due to this.

What to Do When You Hit a Wall?

Random Jumps: If the algorithm reaches a plateau, it can jump significantly to a random state that is far away from where it is now. The chances of touching down in an area where development can be achieved, rather than a plateau, are enhanced in this way.

- **Ridge Problem:** A ridge is a peak-like area where it appears that every potential direction of movement leads downward. There may be better solutions out there, but the Hill Climbing algorithm may stop too soon thinking it has found the best one.

How to Pass Ridge?

Using multi-directional search, the algorithm can apply many rules before testing a solution, allowing it to overcome ridges more effectively. This method increases the algorithm's chances of discovering a better path by letting it go in numerous directions at once.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	What is the main strategy of the Hill Climbing algorithm? A. Explore all nodes before selecting the best B. Move in the direction that improves the current state C. Randomly jump between states D. Always move toward the farthest node
7	Which of the following problems is Hill Climbing most likely to encounter? A. Infinite loops B. Out of memory errors C. Getting stuck in a local maximum D. Finding multiple solutions simultaneously
8	In Hill Climbing, which of the following best describes a plateau? A. A point with better neighbors B. A flat area where all neighbors have the same value C. The highest peak in the search space D. The lowest point in the search space
9	Which of the following is NOT a variant of Hill Climbing? A. Simple Hill Climbing B. Steepest Ascent Hill Climbing C. Stochastic Hill Climbing D. Breadth-First Hill Climbing
10	What is a common technique to avoid local maxima in Hill Climbing? A. Use BFS instead B. Restart from a random state C. Avoid checking neighbor nodes D. Limit the search depth
11	Which of the following statements is TRUE about Hill Climbing? A. It guarantees the optimal solution B. It uses a heuristic to guide the search C. It explores all paths equally D. It stores all visited states in memory

8. UNIFORM COST SEARCH (UCS) IN AI

Among the many search algorithms employed by AI, Uniform Cost Search (UCS) stands out for its ability to locate the graph's least cost path. This version of Dijkstra's algorithm finds the shortest total cost path from a starting node to a destination node when every edge in the graph has a different weight.

To determine the shortest route to a given destination, Uniform Cost Search iteratively grows the node with the lowest cost. For weighted graphs, where the cost of each edge is different, UCS is an appropriate search method since it considers the cost of each path, unlike Breadth-First Search (BFS).

Key Concepts of Uniform Cost Search

- 1. Priority Queue:** For node storage, UCS employs a priority queue. Priority is given to expanding the node that has the lowest cumulative cost. By doing so, we can be sure that the search will prioritise investigating the best options.
- 2. Path Cost:** Amount needed to get from the starting node to a specific node. To determine which nodes to prioritise, UCS adds up all the costs from the beginning to the present node.
- 3. Exploration:** When exploring nodes, UCS starts by expanding the node with the lowest cost and keeps going until it reaches the node with the highest cost. You can be certain that the route to your destination will be the most economical one.
- 4. Termination:** As soon as the objective node is expanded, the algorithm stops, guaranteeing that the best path is reached the first time.

9. WORKING OF UNIFORM COST SEARCH WORK

UCS is based on a straightforward principle: from the starting node, choose the extension that has the minimum overall cost among all potential paths. Based on the overall cost from the root node, a priority queue is used to implement this and keep the partial pathways in order.

The procedure for using UCS is as follows:

1. **Initialisation:** Every UCS system begins with a root node. Due to the lack of action, its cumulative cost is zero as it gets added to the priority queue.
2. **Node Expansion:** To make room in the priority queue for other nodes, we eliminate the one with the shortest path. The next step is to enlarge this node and investigate its neighbours.
3. **Exploring Neighbours:** The algorithm determines the total cost from the start node to each neighbour through the current node for each neighbour of the enlarged node. A neighbouring node is added to the priority queue along with its computed cost if it is not already there. If a neighbour is already in the queue but a cheaper way to reach them is discovered, the cost in the queue is changed accordingly.
4. **Goal Check:** Once the algorithm has expanded a node, it verifies if it has arrived at the target node. Upon successful completion, the algorithm will provide the total cost and path used to reach the specified node.
5. **Repetition:** This procedure is repeated until either the priority queue is cleared or the target is accomplished.

Fixing the Pathfinding Issue with Uniform Cost Search

Step 1: First, import the libraries you'll need.

To implement Uniform Cost Search (UCS) and display the graph, this step imports the required libraries.

```
import heapq
import networkx as nx
import matplotlib.pyplot as plt
```

Step 2: Explain the Uniform Cost Search Algorithm

In a weighted network, this function uses the UCS algorithm to determine the shortest path between two nodes.

```
def uniform_cost_search(graph, start, goal):
    # Priority queue to store the frontier nodes, initialized with the start node
    priority_queue = [(0, start)]
    # Dictionary to store the cost of the shortest path to each node
    visited = {start: (0, None)}

    while priority_queue:
        # Pop the node with the lowest cost from the priority queue
        current_cost, current_node = heapq.heappop(priority_queue)

        # If we reached the goal, return the total cost and the path
        if current_node == goal:
            return current_cost, reconstruct_path(visited, start, goal)

        # Explore the neighbors
        for neighbor, cost in graph[current_node]:
            total_cost = current_cost + cost
            # Check if this path to the neighbor is better than any previously found
            if neighbor not in visited or total_cost < visited[neighbor][0]:
                visited[neighbor] = (total_cost, current_node)
                heapq.heappush(priority_queue, (total_cost, neighbor))

    # If the goal is not reachable, return None
    return None
```

Step 3: Give an Explanation of the Path Reconstruction Method

After visiting each node along the way, this function can piece together the route that led from the beginning to the end.

```
def reconstruct_path(visited, start, goal):  
    # Reconstruct the path from start to goal by following the visited nodes  
    path = []  
    current = goal  
    while current is not None:  
        path.append(current)  
        current = visited[current][1] # Get the parent node  
    path.reverse()  
    return path
```

Step 4: Explain the Visualisation Process

Using networkx to generate graphs and matplotlib to display them, this tool makes the graph and UCS-found path visually appealing.

```
def visualize_graph(graph, path=None):  
    G = nx.DiGraph()  
  
    # Adding nodes and edges to the graph  
    for node, edges in graph.items():  
        for neighbor, cost in edges:  
            G.add_edge(node, neighbor, weight=cost)  
  
    pos = nx.spring_layout(G) # Positioning the nodes  
  
    # Drawing the graph  
    plt.figure(figsize=(8, 6))  
    nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=2000, font_size=15,  
font_weight='bold', edge_color='gray')  
    labels = nx.get_edge_attributes(G, 'weight')  
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels, font_size=12)  
  
    if path:
```

```
# Highlight the path in red
path_edges = list(zip(path, path[1:]))
nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='red', width=2.5)

plt.title("Uniform Cost Search Path Visualization")
plt.show()
```

Step 5: Launch UCS After Graph Definition

In this phase, the UCS algorithm is executed, the start and end nodes are defined, and a sample graph is defined as an adjacency list. Next, the discovered path and graph are displayed.

```
# Example graph represented as an adjacency list
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 1), ('E', 3)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

# Example usage of the UCS function
start_node = 'A'
goal_node = 'G'
result = uniform_cost_search(graph, start_node, goal_node)

if result:
    total_cost, path = result
    print(f"Least cost path from {start_node} to {goal_node}: {' -> '.join(path)} with total cost {total_cost}")
    visualize_graph(graph, path)
```

else:

```
print(f"No path found from {start_node} to {goal_node}")
```

Applications of UCS in AI

There are many areas of artificial intelligence that can benefit from Uniform Cost Search:

1. **Pathfinding in Maps:** Finding the least expensive path between two points on a map takes many factors into account.
2. **Network Routing** Locating the data or communication network's least expensive path
3. **Puzzle Solving:** Making progress in puzzles where there is a cost to each action, like the sliding tiles problem.
4. **Resource Allocation** Activities requiring the correct distribution of resources, with varying costs linked to various allocation procedures.

Advantages of Uniform Cost Search

- **Optimality:** If the cost of each step is more than zero, UCS will identify the path to the goal state that is least expensive.
- This algorithm is comprehensive in the sense that it will discover a solution if one exists.

Challenges with UCS

- To maximise efficiency, UCS will find the cheapest way to the target state if the cost of each step is non-zero.
- Because it will find a solution if one exists, this method is thorough.

When the objective is to minimise the total cost and pathways have different costs, a sophisticated AI approach called Uniform Cost Search can be used. Its efficacy and adaptability are demonstrated by its use in different fields. To put it into practice, though, one must be aware of the computing resources it will need, particularly in situations involving big data sets or scarce computing power.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
12	What is the main criterion used by Uniform Cost Search to choose the next node? A. Depth of the node B. Alphabetical order C. Total path cost from the start node

	D. Number of successors
13	Uniform Cost Search is guaranteed to find the optimal solution if: A. The graph is acyclic B. The path cost increases with depth C. All edge weights are equal D. All step costs are non-negative
14	Which data structure is typically used to implement Uniform Cost Search? A. Stack B. FIFO Queue C. Priority Queue D. Set
15	What happens if there are multiple paths to the same node in UCS? A. It randomly picks one B. It always picks the last one added C. It picks the one with the lowest cumulative cost D. It ignores repeated paths
16	Uniform Cost Search is a type of: A. Depth-limited search B. Informed search C. Uninformed search D. Randomized search
17	The time complexity of Uniform Cost Search depends on: A. Only the depth of the shallowest goal B. The number of leaf nodes C. The branching factor and the cost of the optimal solution D. The number of goals in the graph

10. SUMMARY

- The three essential uninformed search strategies—**Bidirectional Search**, **Hill Climbing**, and **Uniform Cost Search**—each offering unique strengths for solving AI problems.
- **Bidirectional Search** is designed to drastically cut down search time by running two simultaneous searches: one forward from the start state and the other backward from the goal state.
- The major advantage here is the exponential reduction in the number of nodes explored, which is especially crucial for very large or deep problem spaces. It relies on the goal being well-defined and accessible, making it a preferred method in applications like pathfinding and navigation systems. Moving to
- **Hill Climbing**, although technically considered a heuristic search method, it offers an intuitive strategy where decisions are made step-by-step by choosing the neighbor with the best immediate improvement towards the goal. Hill Climbing is fast and memory-efficient, but it faces limitations like getting stuck in local optima, encountering plateaus where no progress is evident, or ridges that require complex moves.
- Despite these challenges, it is frequently used in optimisation problems and robotics where quick, reasonable decisions are necessary. Finally, **Uniform Cost Search** focuses on expanding the node with the lowest cumulative cost rather than the shallowest depth, ensuring the discovery of the least expensive path.
- Unlike depth-first or breadth-first approaches, UCS adapts to problems where different actions have different costs, guaranteeing both completeness and optimality if the cost of all actions is non-negative. It is particularly effective for problems like route planning with varied travel costs or resource management tasks.
- Together, these techniques deepen the understanding of AI search mechanisms, offering practical methods to tackle real-world challenges that require balancing between time, space, and cost efficiency.

11. GLOSSARY

Uninformed Search	- A type of search strategy that has no additional information about the goal's location beyond the problem definition. It relies purely on the structure of the problem space
Search Space	- The set of all possible states or configurations that can be reached in the process of solving a problem
Bidirectional Search	- A search strategy that runs two simultaneous searches—one from the initial state and one from the goal state—and stops when the two searches meet, significantly reducing time complexity.
Frontier	- A data structure (like a queue or priority queue) that holds all the states that have been discovered but not yet expanded.
Hill Climbing	- A heuristic search algorithm that continuously moves in the direction of increasing value (or decreasing cost), hoping to reach the global maximum (or minimum)..
Local Maximum	- A point in the search space where the algorithm gets stuck because all neighboring states appear worse, even though it is not the best possible solution.
Plateau	- A flat area of the search space where neighboring states have the same evaluation, causing difficulty in choosing a direction.
Ridge	- A challenging terrain in the search space where moving toward the goal requires a sequence of sideways moves, often causing simple hill-climbing algorithms to fail.
Uniform Cost Search (UCS)	- A search algorithm that always expands the node with the lowest total path cost from the start state. It is optimal and complete when costs are non-negative.
Cumulative Path Cost	- The total cost from the initial state to a particular node, considering all the actions taken to reach that node.
Branching Factor (b)	- The average number of child nodes for each node in the search tree.

Search Depth (d)	- The number of steps from the root node to the goal node.
Completeness	- A property of a search algorithm that guarantees finding a solution if one exists.
Optimality	- A property that ensures the algorithm finds the best possible solution, usually the one with the lowest cost.
Time Complexity	- A measure of the time taken by an algorithm in terms of the number of nodes it needs to explore.
Space Complexity	- A measure of how much memory an algorithm uses during execution.

12. TERMINAL QUESTIONS

1. Q: What is Bidirectional Search in AI?
2. Q: Why is Bidirectional Search considered efficient and important in AI
3. Q: How does the Bidirectional Search algorithm actually function?
4. Q: What is Hill Climbing in AI and when is it used?
5. Q: What are the steps involved in the Hill Climbing algorithm?
6. Q: What are the different types of Hill Climbing techniques?
7. Q: What is Uniform Cost Search, and how is it different from BFS or DFS?
8. Q: How does the UCS algorithm function step-by-step?
9. Q: What is a plateau in Hill Climbing?
10. Q: What is Steepest Ascent Hill Climbing?

13. ANSWERS

Self-Assessment Questions

1. Answer: C. To reduce the time complexity by searching from both directions
2. Answer: D. From both the initial and goal nodes simultaneously
3. Answer: B. $O(b^{d/2})O(b^{d/2})O(b^{d/2})$
4. Answer: C. The graph must allow traversal in both directions
5. Answer: C. It requires both directions of search to be feasible and well-defined
6. Answer: B. Move in the direction that improves the current state
7. Answer: C. Getting stuck in a local maximum
8. Answer: B. A flat area where all neighbours have the same value
9. Answer: D. Breadth-First Hill Climbing
10. Answer: B. Restart from a random state
11. Answer: B. It uses a heuristic to guide the search
12. Answer: C. Total path cost from the start node
13. Answer: D. All step costs are non-negative
14. Answer: C. Priority Queue
15. Answer: C. It picks the one with the lowest cumulative cost
16. Answer: C. Uninformed search
17. Answer: C. The branching factor and the cost of the optimal solution

Terminal Questions Answers

Answer 1: Bidirectional Search is a graph search algorithm that runs **two simultaneous searches** — one **forward from the start node** and one **backward from the goal node** — and stops when the two searches meet. This approach can significantly reduce the search time by focusing efforts from both directions. (Refer to Section 2 for information)

Answer 2: In large search spaces, traditional search algorithms like BFS or DFS may take a long time. Bidirectional Search reduces the effective branching factor because instead of exploring all paths from start to goal, it only needs to explore paths up to halfway. This means that for a problem with solution depth d and branching factor b , Bidirectional Search explores **$O(b^{d/2})$** nodes instead of **$O(b^d)$** — an exponential time saving. (Refer to Section 3 for information)

Answer 3: The algorithm starts two simultaneous searches:

- A **forward search** from the initial state.
- A **backward search** from the goal state.

Both searches expand nodes level-by-level (like BFS), and they keep track of visited nodes. The algorithm terminates when the two searches meet — i.e., when a node is found that is common to both search frontiers. The final path is then reconstructed by connecting the paths from the start and goal through this intersection point. (Refer to Section 4 for information)

Answer 4: Hill Climbing is a **local search algorithm** used for optimisation problems. It starts from an arbitrary solution and **iteratively makes small changes** to the solution, **choosing the change that improves the objective function the most**. It is used when you want to find a good (not necessarily perfect) solution quickly — like in pathfinding, scheduling, or feature selection. (Refer Section 5 for information)

Answer 5 : The basic steps are:

1. Start with a **random initial state**.
2. Evaluate the **neighbouring states**.
3. Move to the neighbour with the **highest improvement** (or "height").
4. Repeat steps 2–3 until **no better neighbours** exist — a **local maximum**.

The algorithm uses a heuristic function to evaluate how good a state is, aiming to maximise (or minimise) this value. (Refer to Section 6 for information)

Answer 6: There are three main types:

1. **Simple Hill Climbing** – Only evaluates the current neighbour and moves if it is better.
2. **Steepest-Ascent Hill Climbing** – Looks at **all neighbours** and chooses the one with the greatest improvement.
3. **Stochastic Hill Climbing** – Selects a **random better neighbour**, which introduces randomness and helps escape local optima. (Refer to Section 7 for information)

Answer 7: Uniform Cost Search is a search algorithm that **expands the node with the lowest total cost from the start node**, not just the shallowest (as in BFS). It uses a **priority queue** (often a min-

heap) to always explore the most cost-effective path first. It is useful when path costs vary and you want the **optimal path**. (Refer to Section 8 for information)

Answer 8: Start by adding the **start node** to the priority queue with cost 0.

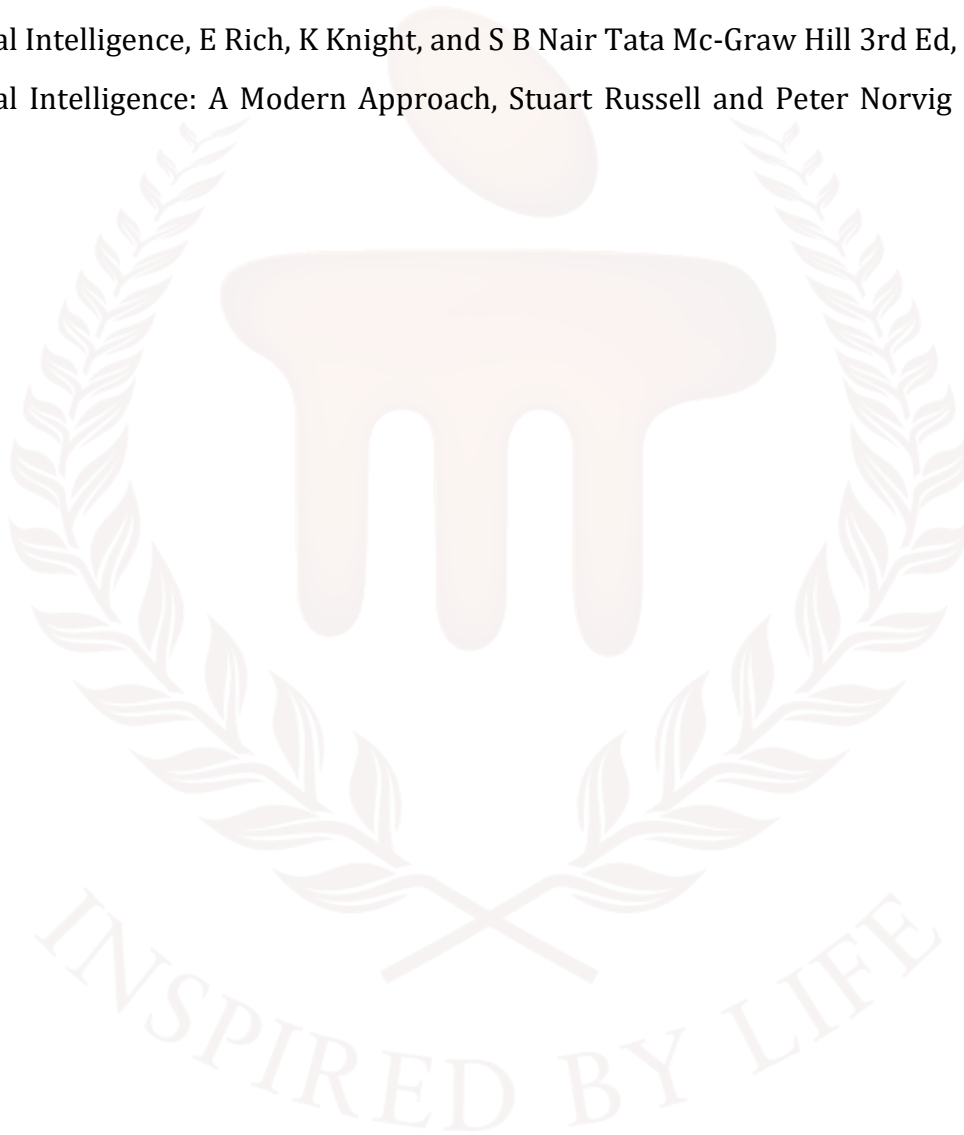
1. While the queue is not empty:
 - Remove the node with the **lowest cost**.
 - If it is the goal node, return the path.
 - Otherwise, expand it by adding all unvisited neighbours with **updated path costs** to the queue.
2. The algorithm guarantees to find the **least-cost path** to the goal, assuming costs are non-negative. (Refer to section 9 for more information)

Answer 9: A flat region with no gradient, making the direction of improvement unclear. (Refer to Section 7)

Answer 10: A variant that evaluates all neighbours and chooses the one with the best improvement. Refer to section 5)

14. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.



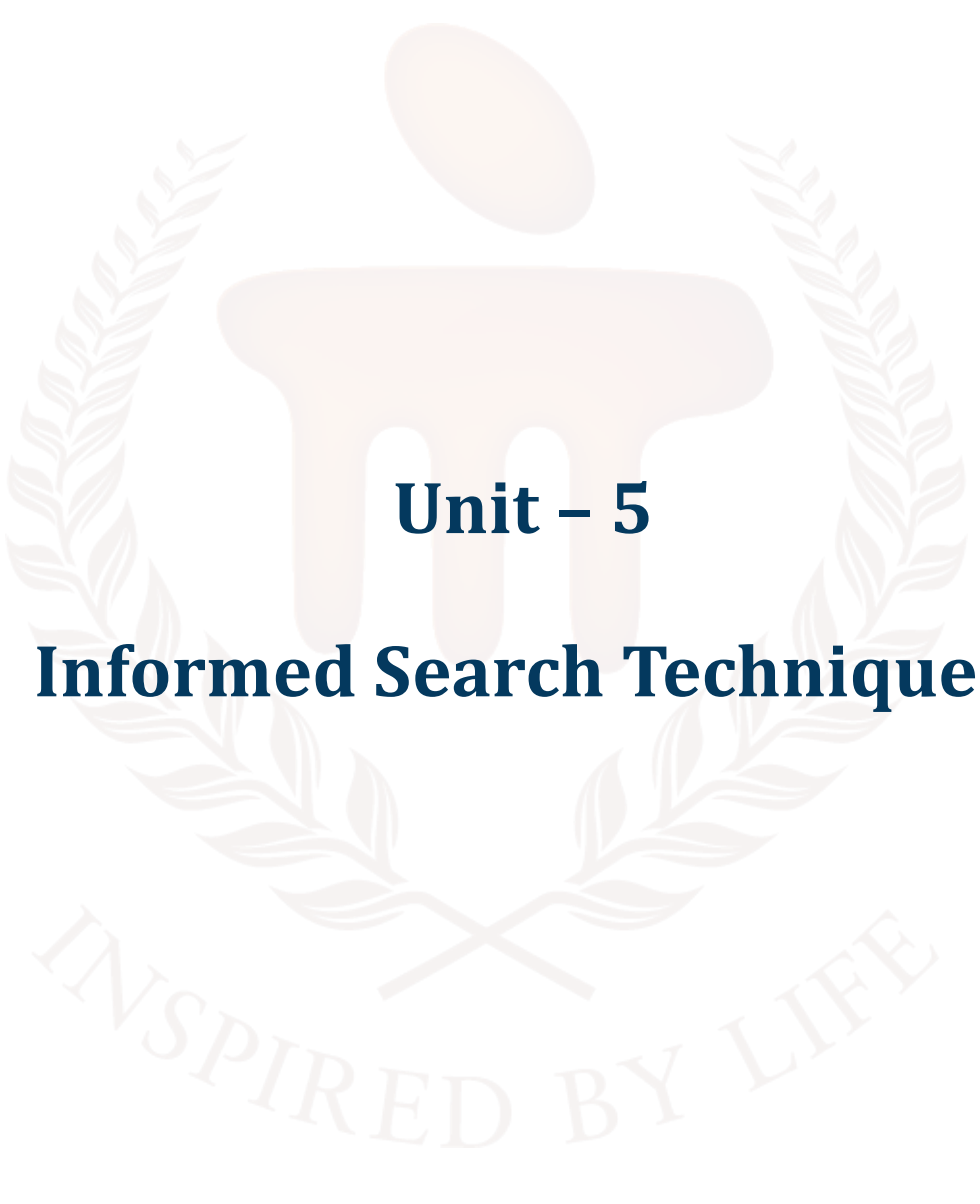
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 5

Informed Search Techniques

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Introduction to Informed Search Techniques	-	1	6
3	Commonly Used Intelligent Search Algorithms in AI	-	-	7 - 11
4	Heuristics Search Technique	-	-	12 - 13
5	Types of Heuristic Search Techniques	-	2	14 - 17
6	Greedy Best-First Search in AI	-	-	18
7	Heuristics in Greedy Best-First Search	1	3	19 - 26
8	A* Search	-	-	27 - 29
9	Pathfinding using A* Algorithm	-	-	30 - 36
10	Summary	-	-	37
11	Glossary	-	-	38 - 39
12	Terminal Questions	-	-	40
13	Answers	-	-	41 - 43
14	References	-	-	44

1. INTRODUCTION

In the previous unit, we explored three important search strategies that were fundamental to AI problem solving: bidirectional search, hill climbing, and uniform cost search. Bidirectional Search introduced an efficient approach where the search begins simultaneously from the start and goal states, meeting somewhere in between to drastically reduce the search time compared to traditional methods. Hill Climbing, a heuristic-based search technique, taught us how to make decisions based on moving toward neighbouring states that appear more promising, while also highlighting challenges like local maxima and plateaus. Finally, Uniform Cost Search emphasises cost optimisation by always expanding the path with the lowest cumulative cost, ensuring the discovery of the least-cost solution. By studying these techniques, we gained a deeper understanding of how different search algorithms handle complexity, optimise resources, and find solutions effectively in real-world AI applications.

In this unit, we will explore informed search techniques, which improve the efficiency of searching in complex problem spaces by using additional knowledge known as heuristics. Unlike uninformed (blind) search methods that explore the search space without any domain-specific guidance, informed search techniques leverage heuristic information to make smarter decisions about which paths to follow. We will begin by understanding what heuristics are—rules or strategies based on experience or domain knowledge that help estimate the cost to reach the goal from any given state. Then, we will study Greedy Search, a method that uses heuristics to always pick the next node that appears to be closest to the goal, although it does not guarantee the optimal path. Finally, we will dive into A* Search, one of the most powerful and widely used search algorithms in AI. A* combines the strengths of Uniform Cost Search and Greedy Search by considering both the cost to reach a node and the estimated cost to reach the goal. It guarantees optimality if the heuristic used is admissible (never overestimates the actual cost). Through this unit, we will learn how to design heuristics, implement and evaluate these algorithms, and understand their advantages, limitations, and real-world applications in problem-solving, such as in navigation systems, game AI, and robotics. This knowledge forms a critical foundation for building intelligent systems capable of efficient decision-making.

1.1. Objectives

By the end of this unit, you will be able to:

- Interpret the fundamental concepts of informed search techniques
- Compare the operational mechanisms of various informed search techniques
- Analyse the advantages and limitations
- Apply appropriate informed search strategies to solve simple AI search problems effectively.



2. INTRODUCTION TO INFORMED SEARCH TECHNIQUES

Heuristic search algorithms, sometimes referred to as informed search algorithms, are a crucial part of artificial intelligence (AI). Compared to ignorant search techniques, these algorithms employ domain-specific information to increase search efficiency, producing quicker and better results. The time and processing resources needed to locate a solution can be decreased by using heuristics to help informed search algorithms make educated assumptions about which paths to investigate in a search area.

In artificial intelligence, an informed search algorithm—also referred to as a heuristic search—is a kind of search algorithm that effectively finds a solution by utilising extra knowledge about the problem's state space. This extra data, usually in the form of a heuristic function, aids in estimating the distance or cost between the destination node and a specific node in the search space. The application of heuristics sets informed search algorithms apart from uninformed ones, which don't make use of domain-specific information.

Key Characteristics of Informed Search Algorithms

- i. **Heuristic Function:** To estimate the least cost from node n to the goal, informed search algorithms employ a heuristic function $h(n)$. This feature aids the algorithm in determining which nodes, depending on their likelihood of producing the best answer, should be investigated first.
- ii. **Efficiency:** Especially in large or complex search spaces, educated search algorithms tend to locate solutions faster than ignorant ones by concentrating on more promising paths.
- iii. **Optimality and Completeness:** Informed search algorithms can be both optimal and complete, depending on the heuristic employed. An algorithm is considered optimal if it consistently finds the best solution, and complete if it is assured to discover a solution if one exists. For example, when the heuristic function is acceptable (i.e., it never overestimates the true cost), the A^* search method is both complete and optimal.

3. COMMONLY USED INTELLIGENT SEARCH ALGORITHMS IN AI

The commonly used intelligent Search Algorithms are:

- a. Greedy Best First Search
- b. A Search*
- c. IDA (Iterative Deepening A)**
- d. Beam Search

a. Greedy Best First Search:

A heuristic-driven method called Greedy Best-First Search ranks nodes according to how much they are expected to cost the objective. Using a heuristic function $h(n)$, the algorithm chooses the node that seems the most promising. $h(n)$, which calculates the approximate cost between node n and the objective.

Consider a robot moving from a starting point to a destination over a grid. To determine the most effective route, A* would take into account the distance previously covered and an estimate of the amount that remains (such as the straight-line distance).

How Does It Operate?

Beginning at the first node, the algorithm assesses its neighbours.

After selecting the neighbour with the lowest heuristic value, $h(n)$, it proceeds with the next step. Since Greedy Best-First Search always selects the path that appears to be the best right now, it is susceptible to becoming stuck in local optima and cannot ensure that the best path will be found.

b. A Search*

One of the most popular informed search algorithms is A* Search. By employing a composite cost function $f(n)$, it combines the advantages of Greedy Best-First Search with uniform-cost search.

$$f(n)=g(n)+h(n)$$

where:

The precise cost from the start node to node n is $g(n)$.

The heuristic estimate of the cost from n to the objective is $h(n)$.

Without taking the total distance into account, a greedy best-first search may select the road that appears to go the closest to the destination when a GPS is attempting to determine the shortest route.

How Does It Operate?

A^* assesses nodes using the heuristic estimate plus the path cost added together. As long as the heuristic function is acceptable (it never overestimates the true cost), the algorithm guarantees that the path discovered is the shortest and most economical.

c. IDA (Iterative Deepening A) **

IDA* (Iterative Deepening A*) combines the optimality and completeness of A^* with the memory efficiency of depth-first search. It is especially helpful in situations where it is impractical to store every node in memory (like in A^*) due to enormous state spaces. In situations like solving puzzles (like the sliding tile puzzle), where memory efficiency is important but an ideal solution is still sought, IDA* can be helpful.

How Does It Operate?

With progressively higher cost thresholds, IDA* does a sequence of depth-first searches using the $f(n)=g(n)+h(n)$ function.

- To make sure the algorithm discovers the shortest path without using too much memory, the search iterates, each time deepening the threshold, until the target is discovered within the threshold.

d. Beam Search

By only keeping a certain number of the best nodes at each search level, Beam Search, a variation of Best-First Search, restricts the amount of nodes stored in memory. This method compromises efficiency and optimality.

In applications involving natural language processing, like machine translation, beam search is frequently employed because it strikes a balance between search depth and computing constraints.

How Does It Operate?

- The algorithm keeps searching the search space but prunes the nodes that don't seem promising according to the heuristic; only the best kk nodes, as defined by the heuristic function, are kept at each level of the search, where kk is the beam width.

Applications of Informed Search Algorithms

Numerous applications make considerable use of informed search algorithms, including:

1. Pathfinding in navigation systems: This technique determines the quickest path between two points on a map.
2. Playing games: By analysing the most promising movements first, it assists in figuring out the best moves in games like chess or go.
3. Robotics: Used in autonomous navigation, in which a robot must determine the most efficient route through a given area.
4. AI Problem Solving: Used to solve a variety of issues, including logistics, resource allocation, and scheduling and planning.

Benefits of Knowledgeable Search Algorithms

- Enhanced Efficiency: Knowledgeable search algorithms can significantly narrow the search space, producing quicker answers.
- Domain-Specific Optimisation: These algorithms can be customised to address particular problem areas by utilising heuristics.
- Optimality and Efficiency in Balance: Algorithms such as A^* provide optimal answers while using an acceptable amount of resources.

Obstacles and Things to Think About

- Heuristic Design: The calibre of the heuristic function has a significant impact on how well-informed search algorithms perform. Suboptimal performance may result from a heuristic that is poorly constructed.
- Memory Restrictions: Although powerful, intelligent search algorithms such as A^* can use a lot of memory, particularly for large search spaces.

- Trade-offs: Memory consumption and solution optimality are traded off by algorithms such as beam search.

A key component of AI is informed search algorithms, which allow for effective and frequently ideal answers to challenging issues. These algorithms outperform their ignorant counterparts in navigating vast search spaces by utilising heuristic knowledge. Informed search algorithms are essential components of the AI toolbox, whether they are utilised for pathfinding, gaming, or natural language processing.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the main advantage of informed search techniques over uninformed search techniques? A. They guarantee faster hardware performance B. They use no additional information C. They use heuristics to improve search efficiency D. They always find all possible solutions
2	Which of the following is an example of an informed search algorithm? A. Breadth-First Search B. Depth-First Search C. Greedy Best-First Search D. Depth-Limited Search
3	What does a heuristic function estimate in an informed search? A. The exact cost of reaching the goal B. The memory usage of the algorithm C. The minimum cost from the current node to the goal D. The number of nodes expanded
4	Which function is used in A* Search to evaluate nodes? A. $f(n) = h(n)$ B. $f(n) = g(n)$ C. $f(n) = g(n) + h(n)$ D. $f(n) = g(n) - h(n)$
5	In which of the following applications is informed search commonly used?

	A. Word processing B. Email communication C. Route/pathfinding in maps and games D. Database indexing
6	What is an admissible heuristic? A. One that overestimates the cost to the goal B. One that randomly selects a cost C. One that never overestimates the actual cost D. One that ignores the goal
7	Which of the following is true about Greedy Best-First Search? A. It guarantees optimal solutions B. It considers only $g(n)$ C. It considers only $h(n)$ D. It uses both $g(n)$ and $h(n)$
8	Why is A* Search considered better than Greedy Search? A. It uses more memory B. It only explores depth-first paths C. It balances actual cost and heuristic cost D. It ignores path costs completely

4. HEURISTICS SEARCH TECHNIQUE

When it comes to solving problems, AI systems rely on heuristic search approaches. In areas like navigation systems, game playing, and optimisation difficulties, these methods are crucial for determining the best way to move from one place to another.

- In a problem's search space, heuristic search uses systematic methods to discover the best or near-optimal solution.
- Heuristic search uses heuristic information to direct the search towards more plausible paths, as opposed to brute-force methods that assess all possible answers exhaustively.

A heuristic is a collection of criteria or guidelines that give an approximation of the best possible answer in this situation. Complex problems would be computationally expensive to solve without heuristic algorithms, which efficiently balance exploration and exploitation to find novel answers.

Why Heuristic Search Is Crucial for AI

Heuristic search algorithms have a leg up in artificial intelligence because of how well they handle big search spaces. Through the use of heuristics, the number of potential paths can be drastically reduced by prioritising the most promising ones. Not only does this make search times much shorter, but it also allows AI systems to tackle difficult issues that precise algorithms just can't handle.

Heuristic Search Elements

There are usually a few key parts to heuristic search algorithms:

1. The first is the "state space," which is the sum of all conceivable configurations or states that are thought to constitute the answer to the problem at hand.
2. Starting Point: The starting point of the current problem is the highest level instance in the search tree that does not have any null values.
3. Thirdly, check if the current condition is a terminal or consenting state where the problem is addressed. This is done during the exploration phase.
4. Successor Function: This sets up a scenario where different states, representing potential movements or solutions in the issue space, replace the current state.

5. A heuristic's job is to provide an approximation of the value or distance between a current state and the desired target state. This aids in narrowing the procedure to states or regions with a better chance of success.



5. TYPES OF HEURISTIC SEARCH TECHNIQUES

Heuristic search algorithms have seen numerous improvements and application to new problem fields throughout their history. A few well-known methods encompass:

1. A* Search Algorithm

Among heuristic search algorithms, A* Search Algorithm may have the greatest name recognition. It takes an input of a starting node and utilises a best-first search to determine the shortest route to the destination node. The function $f(n)=g(n)+h(n)$ is a heuristic that estimates the cost of the cheapest path from n to the target. Here, $g(n)$ is the cost from the start node to n , and $h(n)$ is a heuristic. Pathfinding and graph traversal make extensive use of A*.

2. Greedy Best-First Search

A greedy best-first search will use a heuristic function to determine which node is closest to the objective, and then extend that node. Greedy best-first search ignores path cost in favour of anticipated remaining cost, in contrast to A*, which takes both into account. This makes it go faster, but it isn't always the best choice and usually ends up with less-than-ideal results.

3. Hill Climbing

In mathematical optimisation problems, hill climbing is employed as a heuristic search. This technique is an adaptation of the gradient ascending approach. By selecting the optimal neighbouring state, it iteratively progresses from a randomly generated starting point towards higher values (local maxima). Nevertheless, it is not always successful in finding the global optimum and can become stuck in local maxima.

4. Simulated Annealing

Using a probabilistic approach that draws inspiration from metallurgical annealing, simulated annealing can discover the global optimum. To temporarily avoid local optima, it permits the search to accept poorer options, unlike hill climbing. Because its probability of acceptance declines with time, it is able to converge on the optimal answer.

5. Beam Search

One graph-based search method, known as beam search, restricts its exploration to a small subset of potentially relevant nodes. The search's efficiency and precision are greatly affected by the beam width, which controls the amount of nodes that may be held in memory.

Heuristic Search and Its Uses

Heuristic search strategies have extensive application in a multitude of practical contexts, such as:

- Using heuristic search, one may determine the quickest or most efficient way to go from one location to another, which is useful for pathfinding in many contexts, such as city navigation and game route planning.
- When it comes to scheduling and resource allocation, heuristic methods help optimise efficiency and make the most of what is available.
 - Gaming: Heuristic search is employed by AI in strategic games such as Go and chess to assess potential moves and foresee future outcomes.
 - Robotics: Heuristic search helps autonomous robots navigate, avoid obstacles, and get work done faster.
 - NLP, or natural language processing: Search algorithms are fundamental to NLP because they allow AI to comprehend and mimic human language in contexts such as parsing, semantic analysis, and text generation.

Advantages of Heuristic Search Techniques

The benefits of heuristic search strategies are numerous:

- Efficient use of time and computing resources: heuristic search narrows the search to the most likely solutions, saving a lot of time and effort.
- Optimality: Algorithms like A* can ensure the best possible answer when utilising admissible heuristics.
- Adaptability: Heuristic approaches can be used for a variety of tasks, including pathfinding, optimisation, gaming AI, and robotics, thanks to their versatility.

Limitations of Heuristic Search Techniques

- There are a few drawbacks to heuristic search algorithms, notwithstanding their benefits:

- **The Quality of the Heuristic Function:** A high-quality heuristic function is crucial to the success of heuristic search. Suboptimal or inefficient solutions might result from ill-designed heuristics.
- **Complexity of Space:** Heuristic algorithms that deal with huge search spaces, in particular, can be memory intensive, rendering them impractical in settings with limited resources.
- **Domain-Specificity:** A lack of generalisable heuristic approaches can be a result of the domain-specific information that is typically necessary for heuristic design.

The importance of heuristic search approaches in AI-driven problem-solving was covered in this unit. We spoke about how these techniques prioritise the most promising paths, allowing us to quickly browse enormous search spaces. Heuristic methods, which include optimisation techniques like Beam Search and Simulated Annealing as well as algorithms like A* Search and Greedy Best-First Search, strike a balance between the two extremes. Heuristic quality, spatial complexity, and domain specialisation are some of the obstacles that come with these techniques, despite their efficiency and versatility.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
9	What is the main purpose of using a heuristic in search algorithms? A. To store data in memory B. To randomly select the next node C. To estimate the cost or distance to the goal D. To increase the number of nodes expanded
10	Which of the following is a type of heuristic search technique? A. Linear Search B. Binary Search C. A* Search D. Hashing
11	A heuristic function is considered admissible if: A. It always overestimates the cost to the goal B. It uses random values C. It never underestimates the cost to the goal D. It never overestimates the cost to the goal

12	Which heuristic search algorithm uses only the heuristic value $h(n)$ to decide the next node? A. Depth-First Search B. Greedy Best-First Search C. Breadth-First Search D. A* Search
13	Which of the following is a key component in both A* Search and Greedy Best-First Search? A. Path cost $g(n)$ B. Stack-based memory C. Heuristic function $h(n)$ D. Random node selection
14	Which heuristic search technique guarantees optimal solutions if the heuristic is admissible? A. Greedy Best-First Search B. Depth-First Search C. A* Search D. Hill Climbing



6. GREEDY BEST-FIRST SEARCH IN AI

With the rapid advancement of AI, search algorithms have become indispensable tools for tackling difficult problems. One of the most well-known approaches to pathfinding and problem-solving is Greedy Best-First Search, which is known for its simplicity and effectiveness in specific contexts. This unit explores the Greedy Best-First Search (GBFS), including how it works, its benefits, drawbacks, and real-world AI uses.

Introduction to Greedy Best-First Search

In artificial intelligence, the greedy best-first search (GBFS) graph traversal technique is employed to determine the shortest path between two points or to resolve issues with numerous solutions. Because it uses an evaluation function to decide what to do next, with an emphasis on getting near the goal as fast as feasible, it is called a heuristic search algorithm. When generating decisions, GBFS uses what it knows about the problem to direct its search, making it an educated search algorithm.

Greedy Best-First Search: How Does It Operate?

As it traverses a graph, the greedy best-first search method gives preference to nodes that are relatively near to the target state. The "greedy" part stems from its strategy of constantly taking the heuristic function-guided path that appears to go to the goal the quickest. While the algorithm is quick, it may not be able to find a solution if the path it chooses does not lead to one. This is because it does not revisit nodes that are less ideal or go backwards.

Algorithmic Steps of Greedy Best-First Search Algorithm

The Greedy Best-First Search method is described here in a simplified, step-by-step format:

1. **Begin at the beginning:** This usually means beginning at the root or an initial node. Put this node in a queue with higher priority.
2. **Extend Nodes:** Assess every node that is next to the present node. Use a heuristic function to assign a value to each node; this value should reflect the expected distance to the target.
3. **Third, choose the best node** by looking at the priority queue and picking the node that seems to be closest to the objective (the one with the lowest heuristic value).
4. **Verify the Goal:** If the chosen node is the target, then stop the search.
5. **Keep going:** If it doesn't, continue to the next node and repeat steps 2–4 until you accomplish the objective or the queue is empty.

7. HEURISTICS IN GREEDY BEST-FIRST SEARCH

It is the heuristic function that determines GBFS's success or failure. From the current node, a heuristic can be defined as a function that estimates the cost to attain the goal. Here are some common heuristics:

- (Euclidean Distance): The distance in a straight line from the present node to the target, as employed in pathfinding tasks.
- The Manhattan Distance: Used in grid-based applications, it is the total of the absolute differences between two places' coordinates.

The algorithm's behaviour and its ability to efficiently identify an optimal solution are affected by the heuristic decision.

Unfair Prioritisation Search for Hierarchical Routing

Step 1: Define the Node Class:

For every node in the graph, we establish a Node class. The name and heuristic value of the node are stored in the class, and the `__lt__` function permits comparison of nodes according to their heuristic values. The priority queue cannot be maintained without this

```
# Node class to store information about each node
class Node:
    def __init__(self, name, heuristic):
        self.name = name
        self.heuristic = heuristic
    def __lt__(self, other):
        return self.heuristic < other.heuristic
```

Step 2: Implement Greedy Best-First Search Algorithm

The Greedy Best-First Search algorithm is executed by this function. Starting at the first node, it moves it up a priority queue and investigates nodes according to their heuristic values. Before moving on to nodes in other regions, it gives priority to those in the same region.

Greedy Best-First Search for Hierarchical Routing

```
def greedy_best_first_search_hierarchical(graph, start, goal, heuristic, region_map):
    priority_queue = [] # Priority queue to hold nodes to explore, sorted by heuristic value
    heapq.heappush(priority_queue, Node(start, heuristic[start])) # Add start node

    visited = set() # To keep track of visited nodes
    path = {start: None} # Path dictionary to track explored paths

    while priority_queue:
        current_node = heapq.heappop(priority_queue).name # Get node with lowest heuristic

        if current_node == goal: # Check if goal is reached
            return reconstruct_path(path, start, goal)

        visited.add(current_node) # Mark current node as visited

        # Explore neighbors in the same region first
        current_region = region_map[current_node]
        for neighbor in graph[current_node]:
            if neighbor not in visited and region_map[neighbor] == current_region:
                heapq.heappush(priority_queue, Node(neighbor, heuristic[neighbor]))
                if neighbor not in path:
                    path[neighbor] = current_node

        # Explore neighbors in other regions
        for neighbor in graph[current_node]:
            if neighbor not in visited and region_map[neighbor] != current_region:
                heapq.heappush(priority_queue, Node(neighbor, heuristic[neighbor]))
                if neighbor not in path:
                    path[neighbor] = current_node

    return None # If no path is found
```

Step 3: Reconstruct the Path

This helper function iteratively traverses the path dictionary to re-create the route from the beginning node to the destination after the objective has been reached.

```
# Helper function to reconstruct the path from start to goal
def reconstruct_path(path, start, goal):
    current = goal
    result_path = []
    while current is not None:
        result_path.append(current) # Add node to path
        current = path[current] # Move to the parent node
    result_path.reverse() # Reverse the path to get the correct order
    return result_path
```

Step 4: Visualise the Graph and Path

This function visualises the graph and highlights the path found by the search algorithm using networkx and matplotlib. Additionally, it annotates the nodes with labels that denote their region.

```
# Function to visualise the graph and the path
def visualize_graph(graph, path, pos, region_map):
    G = nx.Graph()

    # Add edges to the graph
    for node, neighbors in graph.items():
        for neighbor in neighbors:
            G.add_edge(node, neighbor)

    plt.figure(figsize=(10, 8)) # Set figure size

    # Draw the nodes and edges
    nx.draw(G, pos, with_labels=True, node_size=4000, node_color='skyblue',
            font_size=15, font_weight='bold', edge_color='gray')
```

```
# Highlight the path
if path:
    path_edges = list(zip(path, path[1:]))
    nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='green', width=3)
    nx.draw_networkx_nodes(G, pos, nodelist=path, node_color='lightgreen')

# Display region information on the graph
for node, region in region_map.items():
    plt.text(pos[node][0], pos[node][1] - 0.2, f"Region {region}", fontsize=12, color='black')

plt.title("Greedy Best-First Search for Hierarchical Routing", size=20)
plt.show()
```

Step 5: Define the Graph and Heuristic Values

The structure of the graph is defined, with nodes linked to their neighbours. Furthermore, the search is guided by heuristic values that are set for each node and the areas.

```
# Complex graph with hierarchical regions
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': ['I', 'J'],
    'F': ['K', 'M', 'E'],
    'G': ['L', 'M'],
    'H': [], 'I': [], 'J': [], 'K': [], 'L': [], 'M': []
}

# Heuristic values (assumed for this example)
heuristic = {
    'A': 8, 'B': 6, 'C': 7, 'D': 5, 'E': 4, 'F': 5, 'G': 4,
    'H': 3, 'I': 2, 'J': 1, 'K': 3, 'L': 2, 'M': 1
}
```

```
}

# Define regions for the hierarchical routing (nodes belonging to different regions)
region_map = {
    'A': 1, 'B': 1, 'C': 1, 'D': 2, 'E': 2, 'F': 3, 'G': 3,
    'H': 2, 'I': 2, 'J': 2, 'K': 3, 'L': 3, 'M': 3
}
```

Step 6: Execute the Search and Visualise the Result

Deciding on a starting and ending node, as well as visualising the resultant path using the given graph and heuristics, we launch the greedy best-first search.

```
# Define positions for better visualisation layout (can be modified)
pos = {
    'A': (0, 0), 'B': (-1, 1), 'C': (1, 1), 'D': (-1.5, 2),
    'E': (-0.5, 2), 'F': (0.5, 2), 'G': (1.5, 2), 'H': (-2, 3),
    'I': (-1, 3), 'J': (0, 3), 'K': (1, 3), 'L': (2, 3), 'M': (3, 3)
}

# Perform Greedy Best-First Search for hierarchical routing
start_node = 'A'
goal_node = 'M'
result_path = greedy_best_first_search_hierarchical(graph, start_node, goal_node, heuristic,
region_map)

print("Path from {} to {}: {}".format(start_node, goal_node, result_path))

# Visualise the graph and the found path
visualize_graph(graph, result_path, pos, region_map)
```

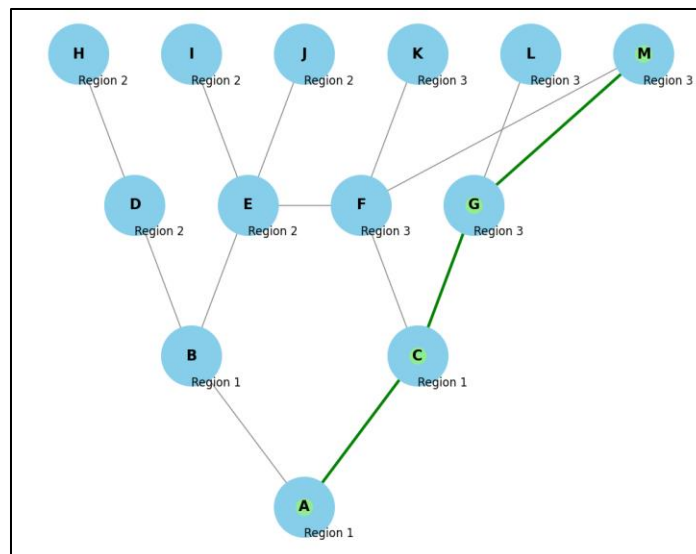


Figure 1: Greedy Best-First Search for Hierarchical Routing

Advantages of Greedy Best-First Search

1. Speed: By utilising heuristic knowledge, GBFS is typically quicker than uninformed search algorithms such as Breadth-First Search.
2. Ease of Use: When time is of the essence rather than perfect performance, this algorithm is a good pick because it is easy to deploy and comprehend.
3. Third, it's efficient with resources; unlike algorithms like A*, it just stores data for the most promising paths.

Limitations of Greedy Best-First Search:

1. First, it isn't perfect; GBFS doesn't take retracing into account, therefore it might not discover a solution if it hits a dead end or local optimum.
2. Second, it is not ideal; the algorithm might not always locate the quickest route. A less-than-ideal approach could result from its narrow focus on short-term benefits (i.e., picking the closest node greedily).
3. Thirdly, GBFS is very dependent on the heuristic's quality for its efficiency and performance. Ineffective search results could be the result of a bad heuristic.

Applications of Greedy Best-First Search in AI

A wide range of artificial intelligence and computer science domains can benefit from greedy best-first search:

Map navigation systems and artificial intelligence in video games rely on pathfinding to get players from one location to another.

- **Puzzle Solving:** Used frequently in games where the objective is to place tiles in a specified sequence, such as the 8-puzzle or 15-puzzle.
- **Robot navigation** aids autonomous robots in their exploration of environments by directing them to the most direct path to a predetermined objective.
- **AI Planning:** When faced with an issue with various solutions and the need for an efficient method to solve it, GBFS is employed.

In situations where time and resources are limited, the strong and efficient search algorithm known as Greedy Best-First Search can swiftly find solutions. Nevertheless, it occasionally veers off course due to its insatiable appetite, leading it to overlook ideal answers or even bomb in intricate situations. When coupled with a precise heuristic function, its utility is highly problem-specific.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
15	What does Greedy Best-First Search primarily rely on to make decisions? A. Total path cost $g(n)$ B. Heuristic estimate $h(n)$ C. Sum of $g(n)$ and $h(n)$ D. Random choice
16	Which of the following is a major drawback of Greedy Best-First Search? A. It uses too much memory B. It guarantees optimal solutions C. It may get stuck in local minima and miss the optimal path D. It considers both actual and estimated costs
17	In Greedy Best-First Search, how are nodes selected for expansion? A. Based on the smallest value of $g(n)$ B. Based on the highest priority value C. Based on the lowest heuristic value $h(n)$ D. In a first-come, first-served order
18	Which of the following statements is true about Greedy Best-First Search?

- | | |
|--|--|
| | <ul style="list-style-type: none">A. It always finds the shortest pathB. It uses a queue in FIFO orderC. It is complete and optimal in all casesD. It can be fast but is not guaranteed to be optimal |
|--|--|



8. A* ALGORITHM SEARCH ALGORITHM

When it comes to finding the best possible route between nodes in a network, computer scientists turn to the robust and flexible A* (A-star) algorithm. A* is still a fundamental method in AI and algorithmic thinking, and it has many uses outside of AI and video game pathfinding, such as network routing and AI.

Understanding the A* algorithm's inner workings, heuristic search method, and what makes it unique among pathfinding algorithms is the focus of this unit.

Origins and Fundamentals of A*

An improvement and expansion of Dijkstra's algorithm—also famous for determining the shortest path between nodes in a graph—the A* method was created in 1968 by Bertram Raphael, Nils Nilsson, and Peter Hart. A* employs heuristics to estimate the cost from a node to the objective, which optimises the search process and reduces computational burden, in contrast to Dijkstra's method, which uniformly investigates all directions surrounding the beginning node.

The A* Algorithm's Basic Workings

Heuristics and cost functions are the basis of the A* algorithm.

Two primary parameters are utilised:

1. The function $g(n)$: How much it really costs to go from the initial node to any other node n .
2. The heuristic cost from node n to the target is denoted by $h(n)$. To direct the search, A* now incorporates information outside the graph.

The sum, $f(n)=g(n)+h(n)$, represents the total estimated cost of the cheapest solution through n . One way the A* algorithm works is by keeping track of all the potential paths along the graph and sorting them by their f values. This is called a priority queue or open set. Here are the algorithmic steps:

1. The first step in initialisation is to add the initial node to the open set using its function $f(n)$.
2. The second loop removes the node from the queue with the lowest $f(n)$ value, even though the open set is not empty.
3. Checking for the Goal: If this node is indeed the goal, the algorithm will exit and provide the found path.

4. In every other case, locate all of the node's neighbours and extend it by determining the values of g, h, and f for each of them. If a neighbour is not already in the open set, or if a more direct route to this neighbour is discovered, then add it.
5. Reiterate: The loop iterates until either the goal is accomplished or the open set is exhausted, meaning no path is available.

Heuristic Function in A* Algorithm

Choosing the right heuristic is crucial to the success of the A* algorithm. How well and fast an algorithm works is heavily dependent on the heuristic used. For an algorithm to discover the shortest path, it needs a heuristic that allows it to explore the fewest possible nodes. Some characteristics of a heuristic are:

- **Admissibility:** If a heuristic consistently fails to overstate the effort required to do a task, it is not admissible. The straight-line distance in a spatial map is the basic example of an acceptable heuristic.
- **Monotonicity:** A heuristic is considered consistent if and only if the predicted cost from the present node to the objective is consistently less than or equal to the sum of the projected costs from all nearby nodes plus the step cost from the present node to each adjacent node.

For grid-based maps (seen in games and city plans), a common heuristic is the Manhattan distance; for direct point-to-point distance measurement, the Euclidean distance is the go-to.

Applications of A*

Because it can determine the optimal path using a given heuristic, the A* algorithm has many real-world uses:

- **A* finds widespread application in robotics** for determining the best way to get from one location to another, and in video games for controlling characters in procedurally generated landscapes.
- **Network Routing:** A* is useful in the field of telecommunications for finding the shortest route for data packets to travel in order to reach their destination.

- Planning and decision-making algorithms can benefit from A* in the realm of artificial intelligence and machine learning when evaluating various phases of decisions and motions becomes necessary.



9. PATHFINDING USING A* ALGORITHM

Step 1: Library Imports

Importing the appropriate Python libraries for working with priority queues, manipulating graphs, and visualising results is the first step.

```
import heapq
import networkx as nx
import matplotlib.pyplot as plt
```

Step 2: Heuristic Function Definition

Make an approximation of the distance from the present node to the destination using a heuristic function. Here, the heuristic in question is the Manhattan distance.

```
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

Step 3: Algorithm Implementation

To determine the shortest path in a network between two nodes, the A* algorithm uses structures for open sets, closed sets, and score tracking.

```
def a_star(graph, start, goal):
    open_set = []
    heapq.heappush(open_set, (0, start))
    came_from = {}
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0
    f_score = {node: float('inf') for node in graph}
    f_score[start] = heuristic(start, goal)

    while open_set:
        _, current = heapq.heappop(open_set)
```

```
if current == goal:
    return reconstruct_path(came_from, current)

for neighbor, cost in graph[current].items():
    tentative_g_score = g_score[current] + cost
    if tentative_g_score < g_score[neighbor]:
        came_from[neighbor] = current
        g_score[neighbor] = tentative_g_score
        f_score[neighbor] = g_score[neighbor] + heuristic(neighbor, goal)
        heapq.heappush(open_set, (f_score[neighbor], neighbor))

return None
```

Step 4: Establish the Function for Path Reconstruction

After the A* algorithm finishes, you need to make a function that can retrace the steps to get from the beginning node to the end node.

```
def reconstruct_path(came_from, current):
    total_path = [current]
    while current in came_from:
        current = came_from[current]
        total_path.append(current)
    total_path.reverse()
    return total_path
```

Step 5: Establish a Graph and Evaluate

Use `networkx` to define the graph, then run the A* algorithm to identify the path. Finally, use `matplotlib` to visualise both the graph and the path.

```
# Define the graph
graph = {
```

```

(0, 0): {(1, 0): 1, (0, 1): 1},
(1, 0): {(0, 0): 1, (1, 1): 1, (2, 0): 1},
(0, 1): {(0, 0): 1, (1, 1): 1},
(1, 1): {(1, 0): 1, (0, 1): 1, (2, 1): 1},
(2, 0): {(1, 0): 1, (2, 1): 1},
(2, 1): {(2, 0): 1, (1, 1): 1, (2, 2): 1},
(2, 2): {(2, 1): 1}
}
start = (0, 0)
goal = (2, 2)

# Use NetworkX to create the graph
G = nx.DiGraph()
for node, edges in graph.items():
    for dest, weight in edges.items():
        G.add_edge(node, dest, weight=weight)

# Get the path from A* algorithm
path = a_star(graph, start, goal)

# Plotting
pos = {node: (node[1], -node[0]) for node in graph} # position nodes based on grid coordinates
nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=2000, edge_color='gray',
width=2)
nx.draw_networkx_edges(G, pos, edgelist=path_to_edges(path), edge_color='red', width=2)
plt.title('Graph Visualization with A* Path Highlighted')
plt.show()

```

Complete for A* Pathfinding Problem:

```

import heapq
import networkx as nx

```

```
import matplotlib.pyplot as plt
```

```
def heuristic(a, b):
```

```
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
def a_star(graph, start, goal):
```

```
    open_set = []
```

```
    heapq.heappush(open_set, (0, start))
```

```
    came_from = {}
```

```
    g_score = {node: float('inf') for node in graph}
```

```
    g_score[start] = 0
```

```
    f_score = {node: float('inf') for node in graph}
```

```
    f_score[start] = heuristic(start, goal)
```

```
    while open_set:
```

```
        _, current = heapq.heappop(open_set)
```

```
        if current == goal:
```

```
            return reconstruct_path(came_from, current)
```

```
        for neighbor, cost in graph[current].items():
```

```
            tentative_g_score = g_score[current] + cost
```

```
            if tentative_g_score < g_score[neighbor]:
```

```
                came_from[neighbor] = current
```

```
                g_score[neighbor] = tentative_g_score
```

```
                f_score[neighbor] = g_score[neighbor] + heuristic(neighbor, goal)
```

```
                heapq.heappush(open_set, (f_score[neighbor], neighbor))
```

```
    return None
```

```
def reconstruct_path(came_from, current):
```

```
    total_path = [current]
```

```
    while current in came_from:
```

```
        current = came_from[current]
```



```
total_path.append(current)
total_path.reverse()
return total_path

def path_to_edges(path):
    return [(path[i], path[i + 1]) for i in range(len(path) - 1)]

# Define the graph
graph = {
    (0, 0): {(1, 0): 1, (0, 1): 1},
    (1, 0): {(0, 0): 1, (1, 1): 1, (2, 0): 1},
    (0, 1): {(0, 0): 1, (1, 1): 1},
    (1, 1): {(1, 0): 1, (0, 1): 1, (2, 1): 1},
    (2, 0): {(1, 0): 1, (2, 1): 1},
    (2, 1): {(2, 0): 1, (1, 1): 1, (2, 2): 1},
    (2, 2): {(2, 1): 1}
}

start = (0, 0)
goal = (2, 2)

# Use NetworkX to create the graph
G = nx.DiGraph()
for node, edges in graph.items():
    for dest, weight in edges.items():
        G.add_edge(node, dest, weight=weight)

# Get the path from A* algorithm
path = a_star(graph, start, goal)

# Plotting
pos = {node: (node[1], -node[0]) for node in graph} # position nodes based on grid coordinates
```

```
nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=2000, edge_color='gray',  
width=2)  
nx.draw_networkx_edges(G, pos, edgelist=path_to_edges(path), edge_color='red', width=2)  
plt.title('Graph Visualization with A* Path Highlighted')  
plt.show()
```

Output:

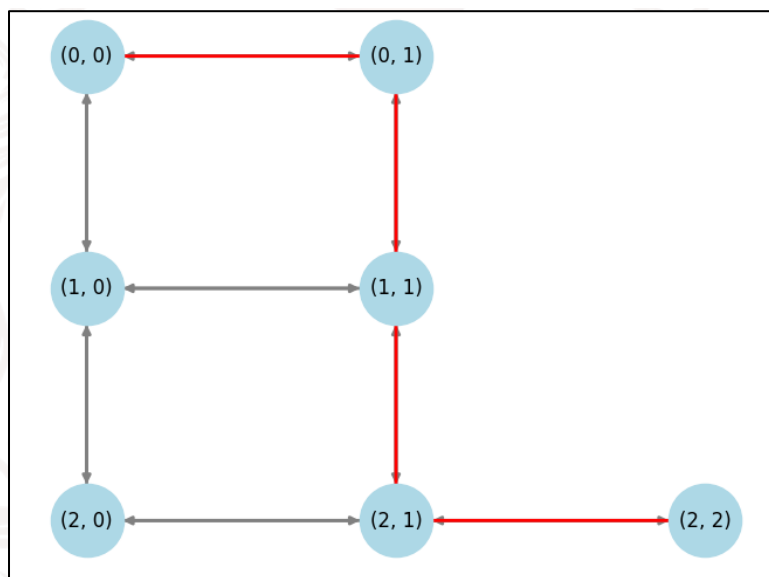


Figure: Graphical Visualisation with A* path Highlighted

The output represents:

- Nodes, denoted by circles with coordinate labels. The starting point and ending point of a network are (0, 0) and (2, 2), respectively.
- The edges are the lines that link the nodes. All potential transitions are shown by the grey lines, and the A* algorithm's path is highlighted by the red lines
- Red-Highlighted Path: This presents the A*-determined best path from the beginning node to the endpoint. Starting at (0, 0), the road descends to (0, 1), continues to (1, 1), (2, 1), and (2, 2), where it achieves the target.

You can easily follow the algorithm's decisions and see how A* uses the graph to discover the best path to the objective according to the heuristic and graph structure that have been established with this visualisation.

Benefits of A*

- **Optimality:** A* will always discover the shortest path to the goal when given an admissible heuristic.
- **Fullness:** If a solution exists, A* will always discover it.
- **Versatility:** A* can be fine-tuned to handle a broad variety of issue situations and limitations by modifying its heuristics.

Things to Keep in Mind and Restrictions

Although A* possesses great power, it does have its limitations. Because it must keep track of all nodes, investigated and unexplored, in memory, the memory consumption could be substantial. In addition, the algorithm's efficiency is greatly affected by the heuristic choice; using an ineffective heuristic might result in wasted computation and wasteful exploration.

10. SUMMARY

- Informed search techniques are advanced search strategies in artificial intelligence that use additional domain-specific knowledge—called heuristics—to make the search process more efficient.
- A heuristic is a function that estimates how close a given state is to the goal. It helps guide the search algorithm in selecting the most promising path at each step. Unlike uninformed (or blind) search methods, which explore blindly, informed techniques use these heuristics to make smarter decisions, reducing the number of nodes expanded and speeding up the search process.
- One of the most basic informed search algorithms is Greedy Best-First Search, which selects the node that appears to be closest to the goal based on the heuristic function $h(n)$.
- It always chooses the path with the lowest estimated cost to the goal, ignoring the cost already taken to reach the current node.
- While Greedy Search can be fast and memory-efficient, it is not guaranteed to find the shortest path and may even get stuck in loops if not implemented carefully.
- In contrast, the A* (A-Star) Search algorithm is a more robust and reliable informed search method. It combines the actual cost from the start node to the current node ($g(n)$) with the estimated cost to the goal ($h(n)$) to form an evaluation function $f(n) = g(n) + h(n)$. This makes A* more balanced, as it considers both the path already travelled and the remaining estimated distance.
- If the heuristic function used is admissible (never overestimates the true cost) and consistent, A* is guaranteed to find the optimal solution. Due to its effectiveness, A* is widely used in areas such as route planning, robotics, and game development.
- In summary, informed search techniques provide a significant improvement over uninformed methods by using heuristics to guide the search intelligently.
- Greedy Search is fast but may not be optimal, while A* offers a more reliable and often optimal solution, making it one of the most popular algorithms in problem-solving and artificial intelligence applications.

11. GLOSSARY

Informed Search	- A type of search algorithm that uses additional information (heuristics) to make decisions about which node to explore next.
Heuristic	- A problem-specific rule or function that estimates the cost or distance from a node to the goal. Helps in guiding the search efficiently.
Heuristic Function (h(n))	- A function that provides an estimate of the minimum cost from a node n to the goal.
Greedy Best-First Search	- An informed search strategy that selects the node with the lowest heuristic value h(n) at each step. It aims to reach the goal as quickly as possible, but may not find the shortest path.
A* Search	- A powerful informed search algorithm that uses both the cost to reach a node g(n) and the estimated cost to the goal h(n). Evaluates nodes using $f(n) = g(n) + h(n)$.
Evaluation Function (f(n))	- A function used in A* to rank nodes: $f(n) = g(n) + h(n)$. Combines actual and estimated cost.
g(n)	- The actual cost incurred to reach node n from the start node.
Admissible Heuristic	- A heuristic that never overestimates the true cost to reach the goal. Ensures optimality in A*..
Consistent Heuristic (Monotonic)	- A heuristic where the estimated cost is always less than or equal to the estimated cost from any neighboring node plus the cost to reach that neighbor. Guarantees optimality in A*.
Optimality	- The property of an algorithm to always find the best (least-cost) solution. A* with admissible heuristics is optimal.
Completeness	- The ability of a search algorithm to find a solution if one exists. A* is complete when using admissible heuristics.
Search Tree	- A representation of all possible paths or sequences of actions that can be taken from the initial state to the goal.

State Space - The entire set of possible states or configurations that a problem can have.



12. TERMINAL QUESTIONS

1. What is informed search in Artificial Intelligence?
2. List any two commonly used intelligent search algorithms in AI.
3. What is a heuristic function? Give an example.
4. What are the advantages of using heuristic search techniques?
5. Name and briefly describe any two types of heuristic search techniques.
6. What is the evaluation function used in A* Search?
7. Explain the term "admissible heuristic".
8. How does Greedy Best-First Search differ from A* Search?
9. What are the limitations of Greedy Best-First Search?
10. What properties make A* Search optimal and complete?
11. What is the role of heuristics in Greedy Best-First Search?
12. Describe the application of A* algorithm in pathfinding.

13. ANSWERS

Self-Assessment Questions

1. Answer: C. They use heuristics to improve search efficiency
2. Answer: C. Greedy Best-First Search
3. Answer: C. The minimum cost from the current node to the goal
4. Answer: C. $f(n) = g(n) + h(n)$
5. Answer: C. Route/pathfinding in maps and games
6. Answer: C. One that never overestimates the actual cost
7. Answer: C. It considers only $h(n)$
8. Answer: C. It balances actual cost and heuristic cost
9. Answer: C. To estimate the cost or distance to the goal
10. Answer: C. A* Search
11. Answer: D. It never overestimates the cost to the goal
12. Answer: B. Greedy Best-First Search
13. Answer: C. Heuristic function $h(n)$
14. Answer: C. A* Search
15. Answer: B. Heuristic estimate $h(n)$
16. Answer: C. It may get stuck in local minima and miss the optimal path
17. Answer: C. Based on the lowest heuristic value $h(n)$
18. Answer: D. It can be fast but is not guaranteed to be optimal

Terminal Questions Answers

Answer 1: Informed search refers to search strategies that use problem-specific knowledge, typically in the form of heuristics, to find solutions more efficiently than uninformed (blind) search methods. (Refer to section 2 for more details)

Answer 2 : Two commonly used intelligent search algorithms are:

1. Greedy Best-First Search
2. A* (A-Star) Search (Refer to Section 5.3 for more details)

Answer 3: A heuristic function is an estimate of the minimum cost from a given node to the goal.

Example: In a map-based problem, the straight-line distance from a location to the destination can be used as a heuristic. (Refer to Section 5.4 for more details.)

Answer 4: Heuristic search techniques reduce the number of nodes explored, improve search efficiency, and often lead to faster solutions by focusing on promising paths (Refer to Section 4 for more details)

Answer 5:

1. **Greedy Best-First Search:** Selects the path with the lowest estimated cost to the goal (based only on $h(n)$).
2. **A* Search:** Considers both the cost from the start node ($g(n)$) and the estimated cost to the goal ($h(n)$), using $f(n) = g(n) + h(n)$. (Refer to section 5 for more details)

Answer 6: The evaluation function is $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach node n and $h(n)$ is the estimated cost from n to the goal. (Refer to Section 8 for more details.)

Answer 7: An admissible heuristic is one that never overestimates the true cost to reach the goal. It ensures that algorithms like A* find an optimal solution (Refer to Section 9 for more details.)

Answer 8: Greedy Best-First Search uses only the heuristic function $h(n)$ to guide its search, while A* Search combines both $g(n)$ (path cost) and $h(n)$ (heuristic estimate), making A* more balanced and optimal. (Refer to Section 6 for more details.)

Answer 9: Greedy Best-First Search can get trapped in local minima, may not find the optimal path, and can be misled by inaccurate heuristics. (Refer to Section 6 for more details.)

Answer 10: A* Search is optimal and complete when the heuristic used is both **admissible** and **consistent**. It guarantees the best solution if one exists. (Refer to Section 8 for more details.)

Answer 11: In Greedy Best-First Search, heuristics guide the algorithm by selecting the node with the lowest estimated cost to the goal ($h(n)$), thus attempting to reach the goal faster. Refer to Section 5.6 for more information. (Refer to Section 7 for more details.)

Answer 12: In pathfinding, A^* is used to determine the shortest path from a starting point to a destination. It considers both the distance already travelled and the estimated remaining distance to the goal, making it suitable for navigation systems, games, and robotics. (Refer to Section 8 for more details.)



14. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.



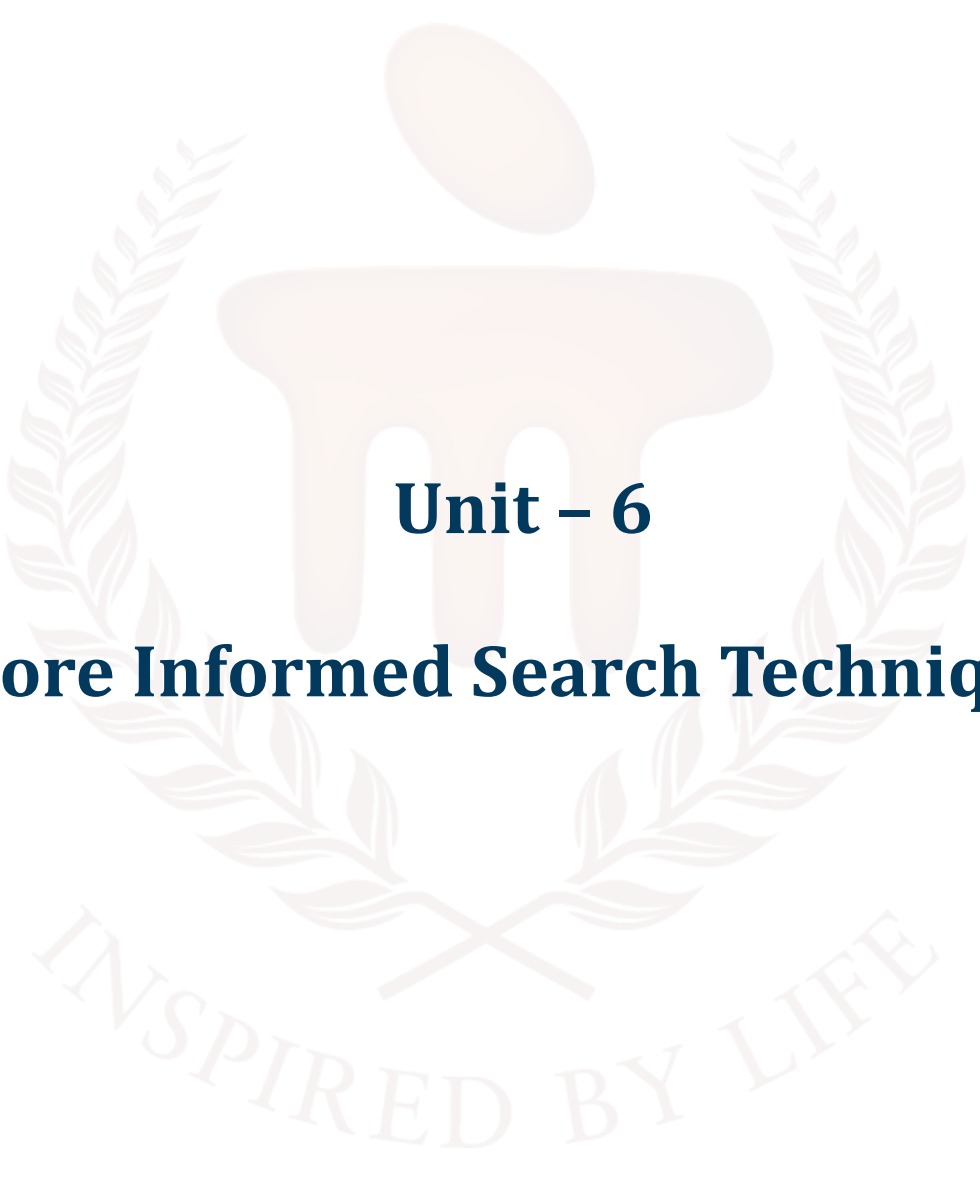
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 6

More Informed Search Techniques

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Introduction to AO* Search Techniques	1	-	6 - 8
3	Working Principles of AO* Algorithm	2	-	9 - 10
4	Illustration of AO* Algorithm with AND-OR Graph	3, 4, 5, 6	1	11 - 17
5	Constraint Satisfaction Problems in Artificial Intelligence	-	-	18 - 19
6	Efficiently Resolving Problems using Constraint Satisfaction	-	-	20 - 21
7	Solving Sudoku with CSP	-	2	22 - 29
8	Summary	-	-	31
9	Glossary	-	-	32 - 33
10	Terminal Questions	-	-	34
11	Answers	-	-	35 - 36
12	References	-	-	37

1. INTRODUCTION

In previous unit, we had explored informed search techniques, which improves the efficiency of searching in complex problem spaces by using additional knowledge known as heuristics. Unlike uninformed (blind) search methods that explore the search space without any domain-specific guidance, informed search techniques leverage heuristic information to make smarter decisions about which paths to follow. We were able to understand what heuristics are—rules or strategies based on experience or domain knowledge that help estimate the cost to reach the goal from any given state. Then, we had explored the study Greedy Search, a method that used heuristics to always pick the next node that appears to be closest to the goal, although it does not guarantee the optimal path. Finally, we divided into A* Search, one of the most powerful and widely used search algorithms in AI. A* combines the strengths of Uniform Cost Search and Greedy Search by considering both the cost to reach a node and the estimated cost to reach the goal. It guaranteed optimality if the heuristic used is admissible (never overestimates the actual cost). Throughout the unit, we had learnt how to design heuristics, implement and evaluate these algorithms, and understood their advantages, limitations, and real-world applications in problem-solving, such as in navigation systems, game AI, and robotics. This knowledge forms a critical foundation for building intelligent systems capable of efficient decision-making.

In this unit, we will explore advanced informed search strategies that go beyond basic algorithms like A* to handle more complex problem-solving scenarios. First, we will study the AO* (And-Or Star) search algorithm, which is designed for problems involving decision trees with both AND and OR nodes—commonly found in planning and reasoning tasks. AO* helps in finding optimal solutions in domains where achieving a goal may require satisfying multiple sub-goals simultaneously. Next, we will delve into Constraint Satisfaction Problems (CSPs), a powerful framework used for solving problems defined by a set of variables, each with possible values, and constraints that restrict the combinations of values. Applications of CSPs range from scheduling and assignment tasks to puzzles like Sudoku and map coloring. Together, these topics will deepen your understanding of how intelligent systems reason and plan under structured conditions.

1.1. Objectives

After studying this unit, you should be able to:

- Interpret the fundamental concepts of AO star algorithm
- Examine the operational mechanisms of Constraint Satisfaction Problems
- Analyse the advantages and limitations
- Apply appropriate informed search strategies to solve simple AI search problems effectively.



2. INTRODUCTION TO AO* SEARCH

Artificial intelligence applications of the AO* algorithm include enhanced search, with a focus on decision-making and problem-solving. To deal with more complicated problems involving handling numerous pathways and making judgements at each node, it is an expansion of the A* method. Learn the fundamentals of the AO* algorithm as well as its uses, benefits, and drawbacks in this introductory unit.

By integrating the best features of both Uniform Cost Search and Greedy Best-First Search, the A* search algorithm creates a robust and popular informed search method. Using prior knowledge of the problem and other relevant information, it seeks for the shortest, most efficient route from an initial state to a final destination in a state space.

A* uses a cost function:

$$f(n)=g(n)+h(n)$$

- $g(n)$ represents the actual cost from the start node to the current node.
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal.
- $f(n)$ is the total estimated cost of the cheapest path through node n .

A* achieves a healthy equilibrium between exploratory and goal-directed behaviour by integrating the actual cost up to this point ($g(n)$) with the predicted future cost ($h(n)$). For heuristics that never overstate the real cost, it becomes both optimum (it finds the best solution) and complete (it discovers a solution if one exists).

The accuracy and reliability of A* make it a popular choice for real-world applications like as robots, gaming pathfinding, navigation systems, and network routing.

The AO* Algorithm—A Brief Overview

Aptly named the "And-Or Star" algorithm, the "AO" is a robust best-first search technique for directed acyclic graph (DAG) representations of problems. The AO* algorithm assesses numerous paths concurrently, in contrast to more conventional algorithms such as A*, which only investigate one path. This improves its efficiency for AND-OR node problems. If you have a job that requires all of the child nodes to be satisfied, you may represent it as an AND node. On the other hand, if you have many paths that only require one child node to be satisfied, you can represent them as OR nodes. Artificial intelligence's AO* (And-Or) search is an educated search method for solving issues that can be divided

into smaller ones using AND and OR nodes. Like A* search, it estimates the cost of attaining a target from a particular node using a heuristic function. AO* outperforms blind search approaches in finding solutions by concentrating on the most promising paths, making it ideal for issues with several paths to a solution.

How does it work:

1. **Breaking the Problem Down:** We can see the problem laid out as an AND-OR graph, with AND nodes representing subproblems that must be solved at the same time and OR nodes representing subproblems that can be solved separately, resulting in several possible solutions.
2. **A Function for Heuristics:** A heuristic function, $h(n)$, is comparable to A* search in that it assesses the cost of attaining a target from a particular node.
3. when expanding nodes, AO* prioritises those that show the most promise, directing the search towards the target using the heuristic function.
4. By expanding an AND node requires solving all of its children. The expansion of an OR node requires the solution of just one of its children.
5. The search process is ongoing until an answer is discovered by AO* that meets all the requirements of the problem.

Key terminologies used in this search are as follows:

- **AND nodes:** The AND nodes stand for the subproblems that, when solved, will lead to the overall solution.
- **OR nodes:** Conversely, nodes that stand in for discrete problems that, when addressed separately, can be combined to form a legitimate whole.
- **Heuristic function $h(n)$:** An estimate of the cost to achieve the target from a given node is provided by the heuristic function ($h(n)$), which allows for more informed decision-making.

Example of AO* Algorithm

We take into account the procedure of purchasing a vehicle in the figure. With the use of an AND-OR graph, we may divide this process into smaller tasks. You can find information about getting a loan or securing finances in the AND area, and then you can find other ways to buy the car, like paying cash or getting a loan, in the OR section. Before the main issue can be resolved, all of the subproblems in the AND-OR graph must be resolved.

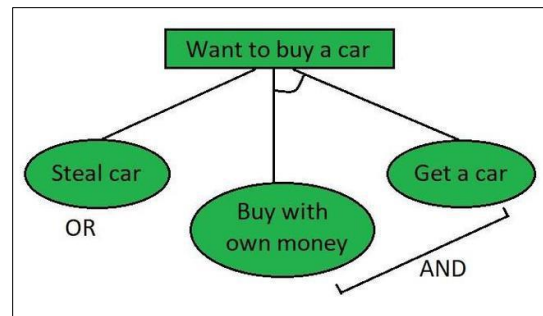


Figure 1: Example for AO star

AND-OR Graph

With preset beginning and ending states, the AO* algorithm employs knowledge-based search. Finding the optimal route while drastically cutting down on time complexity is achieved by utilising heuristics. When solving problems represented as AND-OR graphs, AO* outperforms the A algorithm* because it examines several paths simultaneously.

Complex problems that may be broken into interdependent tasks are perfect for the AO algorithm* since it combines best-first search, DAG representation, and heuristic optimisation. Problems are solved more quickly and effectively when these methods are used.

3. WORKING PRINCIPLES OF AO* ALGORITHM

A tree structure, with each node representing a state in the problem space, is utilised by the AO* algorithm to accomplish its tasks. The algorithm's main parts are:

Node Types

AND Nodes: In order to accomplish a goal, it is necessary to satisfy all child nodes in a given state. An AND node would be used to describe a task that requires multiple conditions to be met.

OR Nodes: Show the states where a goal can't be achieved without satisfying at least one child node. When there are several possible ways to solve a problem, this kind is helpful.

Heuristic Function

From any given node, the method estimates the cost to reach the target using a heuristic function that is comparable to A*. Finding the best avenues to pursue is much easier with this capability. One possible heuristic is $h(n)$. The cost to reach the target from node n is estimated by $h(n)$. $h(n)$ = the expected cost to reach the destination from the current node $h(n)$ = expected distance from node to destination

Search Process

Starting with the root node, the search iteratively investigates its offspring nodes according to their type (AND or OR). In order to determine the related costs of nodes, the following principles are applied:

For OR Nodes: The algorithm takes into account the child node with the lowest cost. One way to put the price of an OR node into words is:

The formula $C(n) = \min\{C(c_1), C(c_2), \dots, C(c_k)\}$ stands for the cost of node n , where $c_1, c_2, \dots, c_k$ are the child nodes of n and $C(n)$ represents the total cost.

For AND Nodes: Due to the necessity of meeting all conditions, the algorithm determines the maximum cost by computing the costs of all child nodes. One way to put the price of an AND node is as

$$C(n) = \max\{C(c_1), C(c_2), \dots, C(c_k)\}$$

where $C(n)$ is the cost of node n , and $c_1, c_2, \dots, c_k$ are the child nodes of n .

Total Estimated Cost

The total estimated cost $f(n)$ at any node n is given by:

$$f(n) = C(n) + h(n)$$

where:

- $C(n)$ is the real price tag for getting from the starting node to node n .
- $h(n)$ is the intended expense beginning at node n and ending at the target.
- $f(n) = \text{Actual cost} + \text{Estimated cost}$

The search iteratively keeps going until either the target is found or all possible outcomes are used up.

Table 1: Comparison between A* Algorithm and AO* algorithm

Aspect	<i>A Algorithm*</i>	<i>AO Algorithm*</i>
Search Type	Best-first search	Best-first search
Type of Search	Informed search using heuristics	Informed search using heuristics
Solution Optimality	Always gives the optimal solution	Does not guarantee an optimal solution
Path Exploration	Explores all possible paths	Stops exploring once a solution is found
Memory Usage	Uses more memory	Uses less memory
Endless Loop	May go into an endless loop without proper checks	Cannot go into an endless loop

4. ILLUSTRATION OF AO* ALGORITHM WITH AND-OR GRAPH

Here, we'll use an AND-OR graph to show how the AO algorithm* operates. Assuming an edge length of 1, a heuristic value $h(n)$ is assigned to each node in the network.

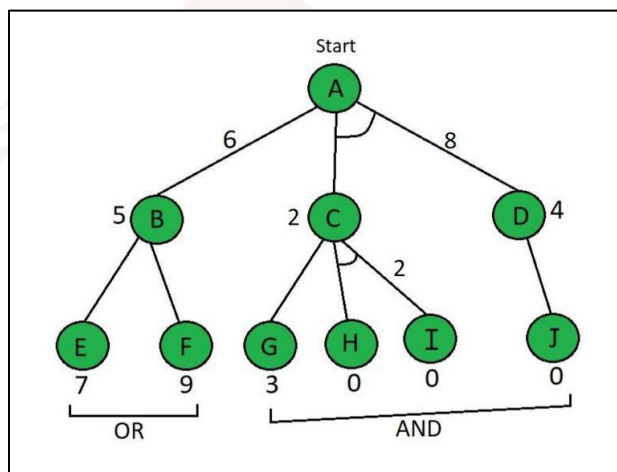


Figure 2: Demonstration of each node assigned by the heuristic

AO* Algorithm - Question tree

Step 1: Initial Evaluation Using $f(n) = g(n) + h(n)$

Starting from node A, we use the evaluation function:

$$\begin{aligned}
 f(A \rightarrow B) &= g(B) + h(B) \\
 &= 1 + 5 \quad (g(n) = 1 \text{ is the default path cost}) \\
 &= 6
 \end{aligned}$$

For the path involving AND nodes (C and D):

$$\begin{aligned}
 f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\
 &= 1 + 2 + 1 + 4 \quad (C \text{ \& } D \text{ are AND nodes}) \\
 &= 8
 \end{aligned}$$

We choose the A->B option because $f(A \rightarrow B) = 6$ is less than $f(A \rightarrow C + D) = 8$.

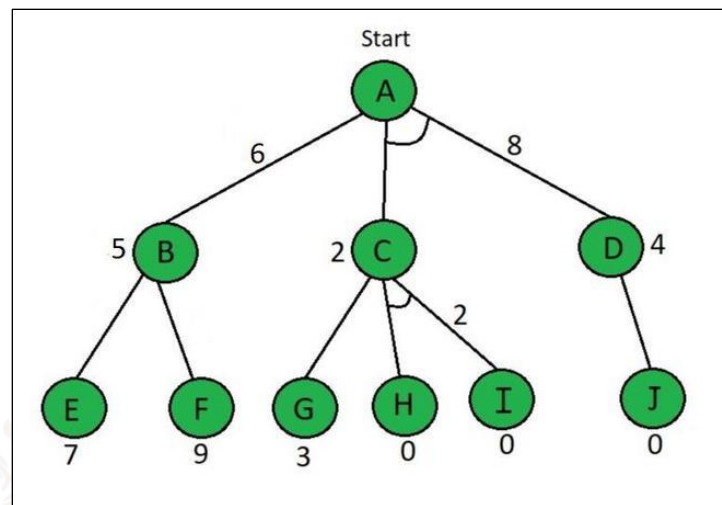


Figure 3: Representation of the Step1

Step 2: Investigate Node B

After that, we'll determine the values of nodes E and F after investigating node B:

$$f(B \rightarrow E) = g(E) + h(E)$$

$$= 1 + 7$$

$$= 8$$

$$f(B \rightarrow F) = g(F) + h(F)$$

$$= 1 + 9$$

$$= 10$$

Since the heuristic value of B differs from its actual value, the minimum path, denoted as $f(B \rightarrow E)$, is selected using the preceding calculation. We update the heuristic and choose the minimal cost path. For our case, the bare minimum is 8. Since B's heuristic has changed, A's heuristic also needs updating. Therefore, we must recalculate it.

The ideal route is thus $f(B \rightarrow E) = 8$. Because the estimated cost of B is different from its real cost, we revise A's heuristic.

$$f(A \rightarrow B) = g(B) + \text{updated } h(B)$$

$$= 1 + 8$$

$$= 9$$

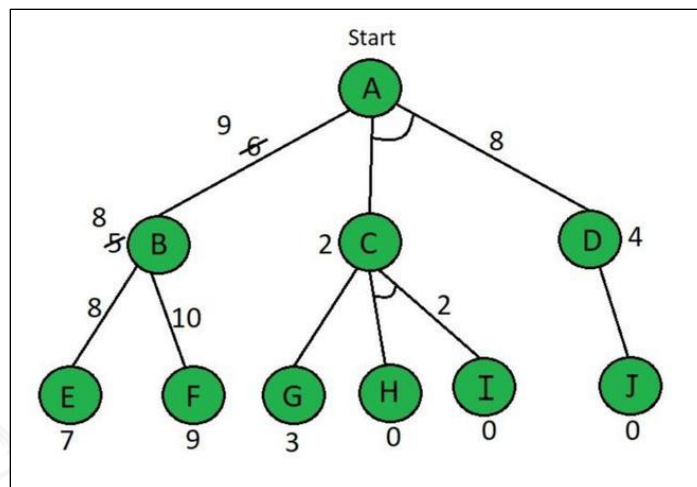


Figure 4: Representation of step-2 AO* Algorithm

Step 3: Check Out Different Routes

The function $f(A \rightarrow B) = 9$ is compared to $f(A \rightarrow C + D) = 8$ now. We probe this route and proceed to node C since $f(A \rightarrow C + D)$ is less.

With regard to node C:

$$f(C \rightarrow G) = g(G) + h(G)$$

$$= 1 + 3$$

$$= 4$$

$$f(C \rightarrow H + I) = g(H) + h(H) + g(I) + h(I)$$

$$= 1 + 0 + 1 + 0 \text{ (H \& I are AND nodes)}$$

$$= 2$$

The heuristic is also kept constant since it aligns with the actual cost, and $f(C \downarrow H + I)$ is chosen as the path with the lowest cost. The heuristic for pathways H and I is 0, therefore they are solved. However, Path $A \downarrow D$ needs to be calculated because it has an AND.

With a value of 2, we choose the path $f(C \rightarrow H + I)$. These paths are deemed solved since the heuristic for H and I is equal to the real cost, which is zero. The value for $A \rightarrow D$ is then determined because it, too, contains an AND node.

For node D:

$$f(D \rightarrow J) = g(J) + h(J)$$

$$= 1 + 0$$

$$= 1$$

We refine our calculations after revising the heuristic for D:

$$f(A \rightarrow C + D) = g(C) + h(C) + g(D) + h(D)$$

$$= 1 + 2 + 1 + 1$$

$$= 5$$

Since $f(A \rightarrow C + D)$ is the cheapest, it has solved the AND-OR graph and is now the only path that remains.

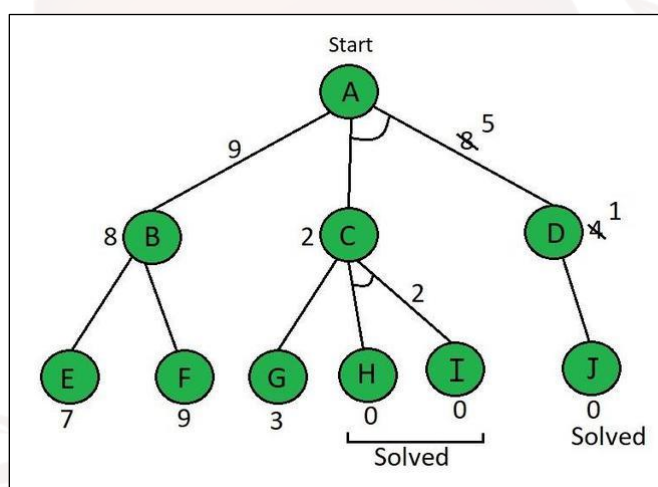


Figure 5: Representation of step3 in AO* Algorithm

First, the AO algorithm* computes heuristic values for each level; subsequently, from the root node upwards, these values are updated. In this example, we changed the values of every node in the tree in a methodical way.

A Cost-Effective Algorithm for Pathfinding in AND-OR Graphs

Optimising pathfinding in a problem with complicated dependencies is illustrated below using the AO* (AND-OR) technique. The method iteratively updates node costs to find the least-cost solution while it investigates paths specified by AND and OR conditions. The AND requirements necessitate taking into account all linked nodes, whereas the OR criteria permit choosing the most cost-effective choice.

Practical Uses of the AO* Algorithm in Artificial Intelligence

1. **Vehicle Routing Problem:** The shortest routes for a fleet of vehicles can be found using the AO* algorithm, which minimises the overall distance travelled and time taken to return to the depot after visiting a set of clients.
2. **Portfolio Optimization:** When it comes to optimising financial portfolios, the AO* algorithm can assist in selecting the optimal investments that maximise returns while minimising risks. It searches for the best possible investment portfolio that meets both goals by experimenting with various combinations of assets.

Whether it's maximising profits while minimising risks in portfolio management or minimising time and distance in vehicle routing, the AO* algorithm manages many conflicting goals. The algorithm iteratively improves an initial answer until it finds the optimal solution that satisfies the objectives.

Why the AO* Algorithm Is Beneficial for AI

1. **Efficient:** Applying AO* to complex decision trees allows for the simultaneous evaluation of several paths, drastically reducing the search area.
2. **Adaptability:** The algorithm's versatility is enhanced by its ability to handle AND and OR nodes, allowing it to be applied to a wide range of problems.
3. **AO* seeks optimal solutions,** like the A* algorithm, by making use of heuristic functions.
4. **Apt for Complex Problems:** AO* excels at tasks like planning, problem-solving, and game-playing that naturally break down into smaller components.
5. **Able to Discover Ideal Answers:** If the heuristic function is carefully selected, AO* will be able to discover the best solutions.

Restrictions on AI's AO* Algorithm

1. **Difficulty:** The algorithm has to consider a lot of options, which might make it computationally expensive, particularly in complicated graphs with many of nodes and links.
2. **AO*'s memory usage** can be an issue in contexts with limited memory because it stores the nodes and paths that are being analysed.
3. **Heuristic Dependency:** AO*'s performance is quite sensitive to the heuristic function's quality. Suboptimal answers and longer search times can result from a heuristic that is not well-designed.

When faced with complicated issues with several possible solutions, the AO* algorithm stands out as a potent AI tool. Natural language processing, robotics, and gaming are just a few of the many areas

that benefit from its versatility and efficiency when dealing with AND and OR nodes. Its computational complexity and memory utilisation are two areas where practitioners should be wary. All things considered, AO* is still a great tool for AI researchers to have at their disposal when solving problems.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What type of graph does the AO* algorithm work on? A. Linear Graph B. Tree C. AND-OR Graph D. Uninformed Graph
2	What is the primary difference between A* and AO* algorithms? A. AO* is faster than A* B. AO* handles AND-OR relationships; A* does not C. A* uses no heuristic D. AO* only uses breadth-first search
3	In an AO* graph, an OR node represents: A. All subgoals must be solved B. A choice between multiple paths C. A leaf node D. A failure node
4	What does an AND node in an AND-OR graph signify? A. One subgoal must be true B. Only one successor is required C. All child nodes must be solved D. It is not supported by AO*
5	What cost function is used by AO* search? A. $h(n)$ – heuristic only B. $f(n) = g(n)$ – cost so far C. $f(n) = g(n) + h(n)$ D. $f(n) = h(n) - g(n)$
6	The AO* algorithm selects the next node to expand based on:

	A. Random choice B. Lowest heuristic value only C. Maximum number of children D. Minimum cost path estimate ($f(n)$)
7	What happens if an AO* node has multiple children and is an AND node? A. Only the cheapest child is expanded B. Only one child is randomly expanded C. All children must be expanded D. The node is skipped
8	AO* can be considered a combination of: A. Depth-first and breadth-first search B. A* and BFS C. A* and dynamic programming D. Heuristic search and AND-OR reasoning
9	In AO*, how is the best partial solution updated? A. From goal nodes to root B. From root to leaves C. Randomly across all nodes D. Using a stack
10	Why is AO* considered an "informed" search algorithm? A. It uses random values B. It uses only path cost C. It uses heuristics to guide the search D. It avoids using any cost function

5. CONSTRAINT SATISFACTION PROBLEMS (CSP) IN ARTIFICIAL INTELLIGENCE

In mathematics, a problem whose solution satisfies all of the requirements is called a constraint satisfaction problem. Assigning values to variables in a way that satisfies all constraints is the goal of CSP. For decision-making tasks involving the management or arrangement of resources under stringent rules, CSPs are utilised by numerous AI applications. Some common uses for CSPs are:

- **Scheduling:** It involves allocating resources, such as personnel or machinery, taking into account limitations in terms of both time and availability.
- **Planning:** Arrange activities in a logical order with due dates.
- **Allocation of Resources:** Making the Most of Available Assets While Minimising Waste

Elements of Problems with Constraint Satisfaction The three main components of CSPs are:

1. **Variables:** These are the items for which we must establish monetary worth. Something needs to be determined by each variable. In a Sudoku problem, for instance, each blank cell represents a variable that requires a value. There are many different kinds of variables; some examples are boolean (yes/no) options, integers (complete numbers), and categories (names, colours, etc.).
2. **Domains:** A variable's range of possible values is this. The range of possible values for each variable is informed by the domain. The numbers 1 through 9 make up the domain of each cell in Sudoku, as every cell is required to contain at least one of these numbers. While some domains are little and finite, others can be enormous and endless.
3. **Constraints:** These are the regulations that limit the possible values for variables. The permitted value combinations are defined by constraints. Various kinds of limitations exist:
 - Examples of unary limitations include "this cell cannot be 5" which apply to a single variable.
 - Statements like "these two cells cannot have the same number" illustrate the two-variable nature of binary constraints.
 - Examples of higher-order restrictions include "each row in Sudoku must have all numbers from 1 to 9 without repetition" and other similar examples involving three or more variables.

Many Forms of Problems with Constraint Satisfaction

The limitations and issue features of CSPs allow for their classification into distinct types

1. Problems with binary CSPs: In these cases, there are only two variables involved in each

constraint. Just like with a scheduling issue, the restriction may state that job A has to be finished before job B

2. There are problems known as non-binary CSPs, and they require restrictions involving more than two variables. As an example, a restriction could specify that no three persons can be seated adjacent to one another in a seating arrangement problem.
3. Strictly Satisfying Hard Constraints and Soft requirements: Strictly satisfying hard constraints requires absolute adherence, while violated soft requirements incur costs. In the actual world, this is commonly employed when certain limitations hold more weight than others.

Methods for Representing CSPs

The interplay of domains, constraints, and variables is central to CSP. The formulation of CSP is illustrated below in a structured form:

1. Finite Set of Variables ($V_1, V_2, \dots, V_n$): There are a number of variables in the problem that must be assigned a value that meets the constraints that are provided.
2. Non-Empty Domain for Each Variable ($D_1, D_2, \dots, D_n$): The range of values that a variable can take is called its domain. In a Sudoku puzzle, for instance, the domain could consist of the numbers 1 through 9 for every cell
3. Finite Set of Constraints ($C_1, C_2, \dots, C_m$): Variables are limited in the range of values they can take by constraints. With each restriction, a rule or connection between variables is defined.
4. Constraint Representation: The collection of variables involved in a constraint is called the scope, and for each constraint C_i , there is a corresponding pair of variables called the relation.

A relation is a set of all possible permutations of variables that are both valid and able to meet the requirement.

Example: For example, suppose you have two variables V_1 and V_2 . This indicates that the values given to these variables cannot be equal, as one such constraint could be $V_1 \neq V_2$. In that place:

- Area of interest: V_1 and V_2 .
- If V_1 is not equal to V_2 , then there is a valid value combination that is part of the relation.
- Explicit combinations may be part of certain relations while abstract relations that are dynamically checked for validity may be the basis of others.

6. EFFICIENTLY RESOLVING PROBLEMS USING CONSTRAINT SATISFACTION

To find solutions that fit the given restrictions, CSP use a variety of methods to explore and optimise the search space. A rundown of the most popular CSP algorithms is as follows:

1. Backtracking Algorithm

To methodically investigate potential solutions in CSPs, the backtracking algorithm employs a depth-first search strategy. Assigning values to variables is how it works, and if an assignment violates a constraint, it will backtrack.

How it works:

- A variable is chosen and its value is set by the algorithm.
- Variables are assigned values in a recursive fashion.
- If an issue occurs, meaning a variable cannot be given a legitimate value, the algorithm will attempt a new value by retracing to the preceding variable.
- The procedure is repeated until either a workable answer is discovered or every possible option has been considered.

Although this approach is popular because of how easy it is to use, it may not be the best choice for complex issues with many variables.

2. Forward-Checking Algorithm

To further narrow the search space, the forward-checking technique applies local consistency tests; this is an improvement over the backtracking algorithm.

How it works:

An algorithm is implemented to keep track of the remaining valid values for each unassigned variable.

- Upon assigning a value to a variable, nearby variables are subject to local constraints, which remove any values that are inconsistent from their domains.
- The method will retrace its steps if a neighbor's forward-checked values are invalid.
- Since this approach avoids some conflicts in the first place, it reduces the need for needless computations, making it more efficient than pure backtracking.

3. Constraint Propagation Algorithms

By mandating that all variables be consistent on a local level, constraint propagation methods significantly narrow the search area.

How it works:

- Interrelated variables propagate constraints.
- Data from other variables is used to remove inconsistent values from domains of variables.
- These algorithms exclude values that could cause conflicts by drawing inferences and removing them from the search space.

A combination of constraint propagation and other CSP techniques, such as backtracking, is employed to expedite the search.

7. SOLVING SUDOKU WITH CONSTRAINT SATISFACTION PROBLEM (CSP) ALGORITHMS

Step 1: Define the Problem (Sudoku Puzzle Setup)

To begin, we must establish that a 9x9 grid with 0 for an empty cell is the standard for a Sudoku puzzle. To make the puzzle more readable for humans, we also define a `print_sudoku` method.

```
puzzle = [[5, 3, 0, 0, 7, 0, 0, 0, 0],  
          [6, 0, 0, 1, 9, 5, 0, 0, 0],  
          [0, 9, 8, 0, 0, 0, 0, 6, 0],  
          [8, 0, 0, 0, 6, 0, 0, 0, 3],  
          [4, 0, 0, 8, 0, 3, 0, 0, 1],  
          [7, 0, 0, 0, 2, 0, 0, 0, 6],  
          [0, 6, 0, 0, 0, 0, 2, 8, 0],  
          [0, 0, 0, 4, 1, 9, 0, 0, 5],  
          [0, 0, 0, 0, 8, 0, 0, 7, 9]]
```

```
def print_sudoku(puzzle):  
    for i in range(9):  
        if i % 3 == 0 and i != 0:  
            print("-----")  
        for j in range(9):  
            if j % 3 == 0 and j != 0:  
                print(" | ", end="")  
            print(puzzle[i][j], end=" ")  
        print()  
print("Initial Sudoku Puzzle:\n")  
print_sudoku(puzzle)
```

Output:

Initial Sudoku Puzzle:

5	3	0		0	7	0		0	0	0
6	0	0		1	9	5		0	0	0
0	9	8		0	0	0		0	6	0
- - - - -										
8	0	0		0	6	0		0	0	3
4	0	0		8	0	3		0	0	1
7	0	0		0	2	0		0	0	6
- - - - -										
0	6	0		0	0	0		2	8	0
0	0	0		4	1	9		0	0	5
0	0	0		0	8	0		0	7	9

Initial Sudoku Puzzle

Step 2: Create the CSP Solver Class

To manage the CSP algorithm's logic, we create a class called CSP. The functions for choosing variables, giving them values, and verifying that they are consistent with constraints are all part of this.

class CSP:

```

def __init__(self, variables, domains, constraints):
    self.variables = variables
    self.domains = domains
    self.constraints = constraints
    self.solution = None

def solve(self):
    assignment = {}
    self.solution = self.backtrack(assignment)
    return self.solution

def backtrack(self, assignment):
    if len(assignment) == len(self.variables):
        return assignment

    var = self.select_unassigned_variable(assignment)
    for value in self.order_domain_values(var, assignment):
        if self.is_consistent(var, value, assignment):

```

```

    assignment[var] = value
    result = self.backtrack(assignment)
    if result is not None:
        return result
    del assignment[var]
    return None

def select_unassigned_variable(self, assignment):
    unassigned_vars = [var for var in self.variables if var not in assignment]
    return min(unassigned_vars, key=lambda var: len(self.domains[var]))

def order_domain_values(self, var, assignment):
    return self.domains[var]

def is_consistent(self, var, value, assignment):
    for constraint_var in self.constraints[var]:
        if constraint_var in assignment and assignment[constraint_var] == value:
            return False
    return True

```

Step 3: Implement Helper Functions for Backtracking

As an added convenience, we've included methods to sort domain values, pick unassigned variables, and verify compatibility with constraints. These procedures guarantee that the algorithm for backtracking is effective.

```

def select_unassigned_variable(self, assignment):
    unassigned_vars = [var for var in self.variables if var not in assignment]
    return min(unassigned_vars, key=lambda var: len(self.domains[var]))

def order_domain_values(self, var, assignment):
    return self.domains[var]

def is_consistent(self, var, value, assignment):
    for constraint_var in self.constraints[var]:
        if constraint_var in assignment and assignment[constraint_var] == value:
            return False
    return True

```


Step 4: Define Variables, Domains and Constraints

We then provide the variables, their domains of potential values, and the constraints that the Sudoku problem must adhere to. Domains stand for potential values while variables stand for cells. No more than one instance of any number in every given row, column, or 3x3 subgrid should be allowed under the constraints.

```
variables = [(i, j) for i in range(9) for j in range(9)]

domains = {
    var: set(range(1, 10)) if puzzle[var[0]][var[1]] == 0 else {puzzle[var[0]][var[1]]}
    for var in variables
}

constraints = {}

def add_constraint(var):
    constraints[var] = []
    for i in range(9):
        if i != var[0]:
            constraints[var].append((i, var[1]))
        if i != var[1]:
            constraints[var].append((var[0], i))
    sub_i, sub_j = var[0] // 3, var[1] // 3
    for i in range(sub_i * 3, (sub_i + 1) * 3):
        for j in range(sub_j * 3, (sub_j + 1) * 3):
            if (i, j) != var:
                constraints[var].append((i, j))

for var in variables:
    add_constraint(var)
```

Step 5: Apply CSP to the Sudoku Problem

We find the answer to the Sudoku puzzle by creating an instance of the CSP class and calling the solve function. After then, the last puzzle with the answer is written out.

```
csp = CSP(variables, domains, constraints)
sol = csp.solve()
```

```

solution = [[0 for _ in range(9)] for _ in range(9)]
for (i, j), val in sol.items():
    solution[i][j] = val

print("\n***** Solution *****\n")
print_sudoku(solution)

```

Output:

```
***** Solution *****
```

```

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
- - - - -
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
- - - - -
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9

```

Artificial intelligence uses of CSPs

The versatility and efficiency with which CSPs may resolve real-world problems have led to their widespread deployment. Some typical uses are as follows:

1. **Scheduling:** They are useful for arranging things like university schedules, flight itineraries, and staff shifts. The objective is to delegate work in accordance with predetermined criteria, such as availability, deadlines, and priority.
2. **CSPs are useful for solving a wide variety of logic problems,** including the N-Queens problem, Sudoku, and crosswords. Constraints guarantee adherence to the rules of the puzzle.
3. **Issues with Configuration:** They facilitate the selection of appropriate product or system components. For instance, it checks that the computer components you're buying are compatible with one another.
4. **Planning and Robots:** CSPs help robots plan their moves, avoid impediments, and finish tasks effectively. To avoid collisions and conserve energy, a robot navigating a warehouse, for instance, must pay close attention.

5. NLP (Natural Language Processing): They're useful in NLP for things like using language rules to deconstruct sentences into their component parts.

Advantages of CSPs in AI

1. Standardised Representation: CSPs offer an organised and comprehensible method for defining issues using variables, their potential values, and rules.
2. Efficiency: By utilising smart search techniques like as backtracking and forward-checking, the time required to identify solutions can be significantly reduced.
3. Adaptability: By utilising CSP approaches in many domains, specialised expertise is not required for each application.

Problems with CSP Solutions

1. One issue with scalability is that it can become exceedingly complicated and time-consuming to discover a solution when there are an excessive number of variables and rules.
2. Adapting to New Information (Dynamic CSPs): In practice, parameters and limitations can evolve over time, necessitating revisions to the CSP solution.
3. Difficult or Extremely Strict Issues: On occasion, a CSP could not be able to find a solution due to the excessively stringent regulations. Sometimes, in order to discover a solution that works, you have to be flexible or make some sacrifices.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
11	What is the goal in solving a CSP? A. Maximising constraints B. Assigning values to variables without violating constraints C. Minimising variables D. Selecting only one constraint
12	What is the goal in solving a CSP? A. Maximising constraints B. Assigning values to variables without violating constraints C. Minimising variables D. Selecting only one constraint

13	In CSPs, what does the term "domain" refer to? A. Set of possible values for each variable B. Set of constraints C. Type of search algorithm D. Memory location
14	Which of the following is a real-world example of a CSP? A. Image classification B. Sudoku puzzle C. Web browsing D. File compression
15	What is the purpose of forward checking in CSP? A. To assign values to all variables immediately B. To prevent backtracking C. To eliminate inconsistent values from domains of unassigned variables D. To guess values randomly
16	What heuristic is used to choose the variable with the fewest legal values left? A. Degree heuristic B. Most Constrained Value C. Least Constraining Value D. Minimum Remaining Values (MRV)
17	In Sudoku as a CSP, what are the constraints? A. All cells must be filled B. No digit may repeat in any row, column, or 3x3 block C. All numbers must be even D. Each cell must be a different color
18	Which technique helps reduce the CSP search space by enforcing local consistency? A. Heuristic pruning B. Random sampling C. Constraint propagation D. Constraint expansion
19	What is the Degree heuristic used for in CSPs? A. Selects variable with the smallest domain

	B. Selects variable with the fewest constraints C. Selects variable with most constraints on other unassigned variables D. Selects variable with highest domain value
20	What technique is commonly combined with backtracking to improve CSP solving? A. Clustering B. Forward checking C. Random sampling D. Greedy selection



8. SUMMARY

- Informed search techniques are a crucial area of artificial intelligence that leverage problem-specific knowledge, often in the form of heuristics, to guide the search process more intelligently and efficiently than uninformed (or blind) methods.
- Two significant approaches in this category are the AO* (And-Or Star) search algorithm and Constraint Satisfaction Problems (CSPs). The AO* search algorithm is specifically designed for problems represented as AND-OR graphs, which are common in scenarios that involve both sequential decisions and parallel subgoals.
- Unlike traditional A* which handles linear paths, AO* must evaluate multiple branches where some nodes (AND) require all child nodes to be satisfied, and others (OR) require only one to be successful.
- This makes AO* particularly suitable for solving complex decision-making problems such as automated planning, diagnosis systems, and hierarchical task management in robotics and expert systems.
- AO* works by recursively evaluating nodes using a cost function that combines the actual cost of reaching a node with a heuristic estimate of the cost to reach the goal, enabling it to find the most cost-effective solution path.
- On the other hand, Constraint Satisfaction Problems are a powerful framework used for solving problems that involve assigning values to a set of variables under specific constraints.
- Each CSP consists of variables, domains (possible values), and constraints that define acceptable combinations of values. Examples include timetabling, scheduling tasks or exams, resource allocation, map coloring, and puzzles like Sudoku.
- CSPs are typically solved using backtracking search along with optimizations like forward checking, constraint propagation, and heuristic variable selection techniques such as Minimum Remaining Value (MRV) or Degree Heuristic. These enhancements help reduce the search space and improve efficiency.
- While CSPs are highly versatile and declarative, they can become computationally expensive as the number of variables and constraints increases. Similarly, AO* can be resource-intensive due to its recursive structure and the need to manage complex AND-OR relationships.
- Despite these limitations, both techniques offer robust and effective methods for problem-solving in AI.

- AO* is preferred where goal states depend on combinations of actions and subgoals, while CSPs excel in structured decision environments involving variable constraints.
- Together, they form the foundation for intelligent systems capable of planning, reasoning, and decision-making in real-world domains.



9. GLOSSARY

Informed Search	- A type of search that uses problem-specific knowledge (heuristics) to make better decisions during the search process.
Heuristic	- An estimate or rule of thumb that guides the search algorithm toward the goal more efficiently.
AO* Search	- An informed search algorithm used to solve AND-OR graphs, where nodes may require multiple or alternative subgoals to be solved.
AND-OR Graph	- A graph structure that includes both AND nodes (requiring all child nodes to be solved) and OR nodes (requiring only one).
$g(n)$	- The actual cost from the start node to the current node.
$h(n)$	- The heuristic or estimated cost from the current node to the goal.
$f(n)$	- The total cost function used in A* and AO* algorithms: $f(n) = g(n) + h(n)$.
Backtracking	- A search method that tries values one by one and undoes choices when a constraint is violated.
Constraint Satisfaction Problem (CSP)	- A problem defined by variables, domains, and constraints that must be satisfied simultaneously.
Variable	- An element of a CSP whose value needs to be determined.
Domain	- The set of possible values that a variable can take in a CSP.
Constraint	- A rule that restricts the values that variables can simultaneously take.

Forward Checking	- A technique in CSPs that eliminates inconsistent values from domains ahead of time to prevent future conflicts.
Constraint Propagation	- A method of deducing variable domains by enforcing constraints throughout the CSP as early as possible.
Arc Consistency	- A condition where for every value of a variable, there exists a consistent value in a connected variable's domain.
Minimum Remaining Values (MRV)	- A heuristic that selects the variable with the fewest legal values left.
Degree Heuristic	- A CSP strategy that chooses the variable involved in the largest number of constraints with other unassigned variables.
Optimality	- A property of a search algorithm where it guarantees finding the best (least-cost) solution.
Completeness	- A guarantee that a search algorithm will find a solution if one exists.
Search Space	- The set of all possible states or configurations that can be explored to find a solution.

10. TERMINAL QUESTIONS

1. What is the AO* search algorithm, and how is it different from A*?
2. In which types of problems is AO* search most useful?
3. What cost function does AO* use for node evaluation?
4. How does AO* handle AND and OR nodes differently during the search process?
5. What is an AND-OR graph?
6. How does AO* propagate costs in an AND-OR graph?
7. What are the main components of a Constraint Satisfaction Problem (CSP)?
8. Give two real-world examples of CSPs.
9. What is forward checking in CSPs?
10. Name two heuristics used to improve CSP solving efficiency.
11. How is Sudoku modeled as a CSP?
12. Which CSP techniques are typically used to solve Sudoku efficiently?

11. ANSWERS

Self-Assessment Questions

1. C. AND-OR Graph
2. B. AO* handles AND-OR relationships; A* does no
3. B. A choice between multiple path
4. C. All child nodes must be solve
5. C. $f(n) = g(n) + h(n)$
6. D. Minimum cost path estimate ($f(n)$)
7. C. All children must be expanded
8. D. Heuristic search and AND-OR reasoning
9. A. From goal nodes to root
10. C. It uses heuristics to guide the search
11. C. Actions
12. B. Assigning values to variables without violating constraint
13. A. Set of possible values for each variable
14. B. Sudoku puzzle
15. C. To eliminate inconsistent values from domains of unassigned variables
16. D. Minimum Remaining Values (MRV)
17. B. No digit may repeat in any row, column, or 3x3 block
18. C. Constraint propagation
19. C. Selects variable with most constraints on other unassigned variables
20. B. Forward checking

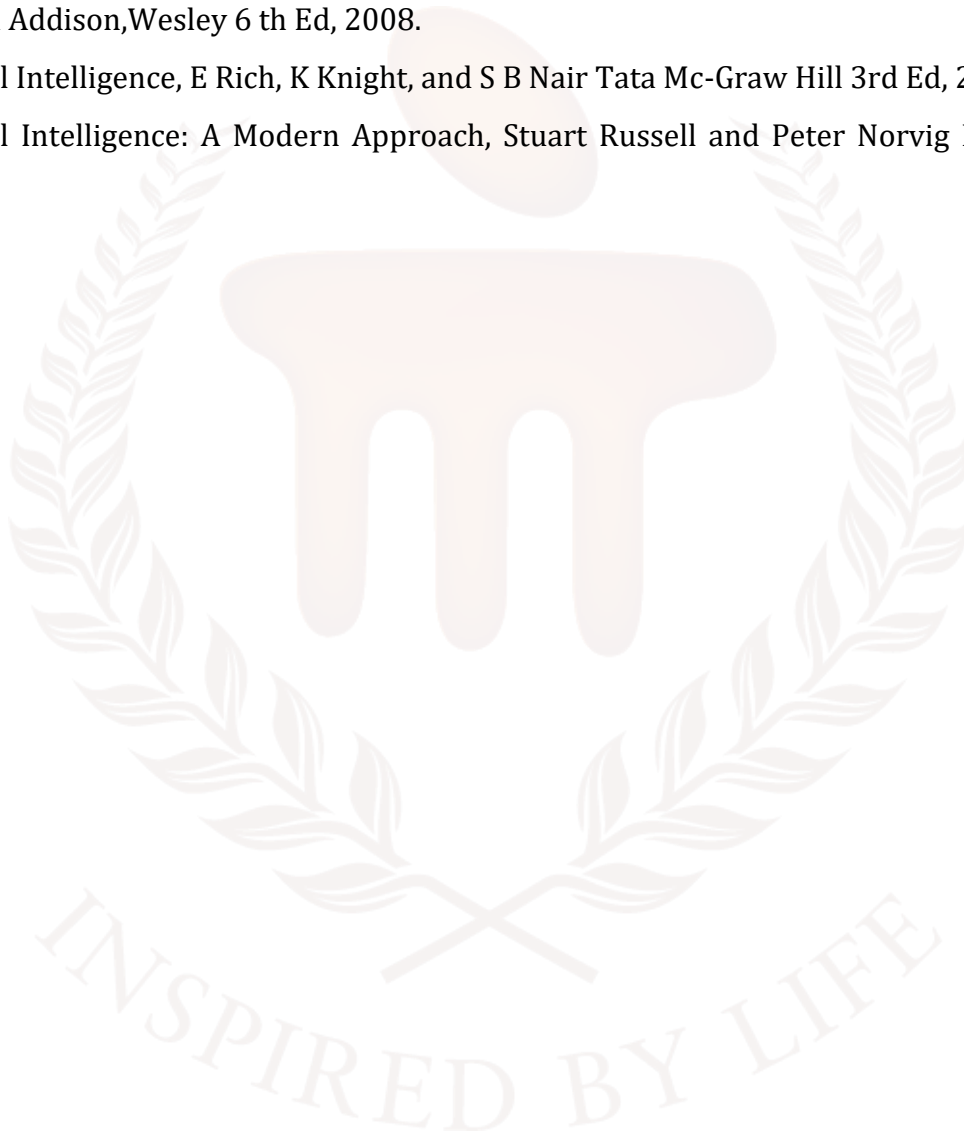
Terminal Questions Answers

1. Answer: AO* (And-Or Star) search is a best-first search algorithm designed for solving problems represented as AND-OR graphs. Unlike A*, which deals with linear paths, AO* handles hierarchical decision trees where achieving a goal may require satisfying multiple subgoals (AND nodes) or choosing among alternatives (OR nodes). (Refer to Section 2 for more information)
2. Answer: AO* is most useful in domains like expert systems, planning, robotics, and diagnostic systems—any scenario where solutions involve both alternative and combinational decisions. (Refer to Section 2 for more information)

3. Answer: AO* uses a cost function similar to A*: $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start to the current node, and $h(n)$ is the heuristic estimate of the cost from the current node to the goal. (Refer to Section 3 for more information)
4. Answer: For OR nodes, AO* selects the child with the minimum estimated cost. For AND nodes, it evaluates the combined cost of all children that must be solved together, considering the total estimated path cost. (Refer to Section 3 for more information)
5. Answer: An AND-OR graph is a tree or graph where nodes may be connected through either OR links (choose one of the branches) or AND links (all branches must be followed to reach a solution). (Refer to Section 4 for more information)
6. Answer: AO* propagates costs from leaf nodes to the root. In OR nodes, it selects the minimum-cost child, while in AND nodes, it sums the costs of all children paths to compute the parent node's value. (Refer to Section 4 for more information)
7. Answer: A CSP consists of three main components: a set of variables, the domain of possible values for each variable, and a set of constraints that specify allowable combinations of values. (Refer to Section 5 for more information)
8. Answer: Common CSP examples include scheduling tasks (e.g., exam timetabling) and solving puzzles like Sudoku, where constraints must be satisfied for a correct solution. (Refer to Section 5 for more information)
9. Answer: Forward checking is a technique where, after assigning a value to a variable, inconsistent values are removed from the domains of connected variables to avoid future conflicts early. (Refer to Section 6 for more information)
10. Answer: (i) Minimum Remaining Values (MRV): chooses the variable with the fewest legal values. (ii) Degree Heuristic: selects the variable involved in the most constraints on unassigned variables. (Refer to Section 6 for more information)
11. Answer: In Sudoku, each cell is a variable with a domain of 1–9. The constraints ensure no repetition of values in any row, column, or 3x3 grid. CSP algorithms enforce these constraints while assigning values. (Refer to Section 7 for more information)
12. Answer: Techniques such as backtracking, forward checking, and constraint propagation (like maintaining arc consistency) reduce the search space and solve Sudoku puzzles efficiently. (Refer to Section 7 for more information)

12. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 7

Game Playing

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Game Playing in AI	-	1	6 - 9
3	Core Functions of AI in Gaming	-	2	10 - 13
4	Mini-Max Algorithm in Artificial Intelligence	1, 2	-	14 - 15
5	Working of Min-Max Process in AI	-	3	16 - 22
6	Alpha-Beta pruning in AI	3, 4, 5, 6, 7, 8, 9	4	23 - 30
7	Summary	-	-	31 - 32
8	Glossary	-	-	33 - 34
9	Terminal Questions	-	-	35
10	Answers	-	-	36 - 37
11	References	-	-	38

1. INTRODUCTION

In previous unit, we had explored advanced informed search strategies that go beyond basic algorithms like A* to handle more complex problem-solving scenarios. Firstly, we had studied the AO* (And-Or Star) search algorithm, which is designed for problems involving decision trees with both AND and OR nodes—commonly found in planning and reasoning tasks. AO* helps in finding optimal solutions in domains where achieving a goal may require satisfying multiple sub-goals simultaneously. Next, we delved into covered Constraint Satisfaction Problems (CSPs), a powerful framework used for solving problems defined by a set of variables, each with possible values, and constraints that restrict the combinations of values. Applications of CSPs range from scheduling and assignment tasks to puzzles like Sudoku and map coloring. Together, these topics had deepened our understanding of how intelligent systems reason and plan under structured conditions.

In this chapter, learners will study how Artificial Intelligence (AI) is applied in gaming, particularly in two-player, turn-based games such as Tic-Tac-Toe, Chess, and Checkers. They will learn how AI models these games using game trees, which represent all possible moves and their outcomes. The chapter introduces the Minimax Algorithm, a foundational concept in game-playing AI, which allows the AI to make decisions by evaluating the best possible moves assuming the opponent also plays optimally. This helps the AI choose moves that maximise its chance of winning while minimizing the opponent's.

Learners will also explore Alpha-Beta Pruning, an advanced technique that improves the efficiency of the Minimax Algorithm by eliminating branches in the game tree that don't influence the final decision. This greatly reduces the computation time, making the AI faster and more practical for complex games. The chapter will include examples, diagrams, and code implementations to help learners understand how these algorithms work in real scenarios.

By the end of this chapter, learners will understand how AI makes strategic decisions in games, how the Minimax Algorithm works, and how Alpha-Beta Pruning makes it faster and smarter. They will also gain hands-on experience by applying these techniques to simple games.

1.1. Objectives

After studying this unit, you should be able to:

- Interpret the fundamental concepts of AO star algorithm
- Examine the operational mechanisms of Constraint Satisfaction Problems
- Analyse the advantages and limitations
- Apply appropriate informed search strategies to solve simple AI search problems effectively.



2. GAME PLAYING IN AI

Playing games has become an essential part of AI, demonstrating its capacity to handle intricate, strategic issues. Playing games is a good way to test AI models because it mimics decision-making processes, which allow the systems to assess several options. Demonstrating the enormous potential of artificial intelligence to outperform human reasoning in limited contexts, early accomplishments include AI systems beating human champions in games such as Go and Chess.

Dota 2 and StarCraft II are examples of modern video games that use AI. In these games, players compete against AI in real-time environments that are constantly changing, necessitating quick thinking and decision-making. All of these successes highlight the ways in which AI systems learn methods, evaluate massive decision trees, and foretell the best solutions.

What is Game Playing

Creating algorithms and models that allow computers to play and win games that demand strategy, decision-making, and problem-solving is known as "game playing" in the field of artificial intelligence. These games are great for gauging AI capabilities because they include clear rules, organised surroundings, and measurable consequences.

The use of artificial intelligence (AI) into gaming has several potential applications, such as:

- Single-player games: Robots play against themselves or predetermined obstacles in games like solitaire or riddles.
- Multi-player games: Artificial intelligence competing against humans or other AI in games like Go or Chess.
- Deterministic games: The results of games like chess are entirely dependent on the player's actions; such games do not include randomness.
- Stochastic games: Card games, dice games, and other games where the outcome is unknown or subject to random chance (e.g., Poker).

Decisions in the real world rely on weighing several options, studying opponents, and making the most of limited resources; this is where gaming comes in handy. Game playing is a crucial topic in AI research and development because by overcoming these difficulties, AI systems improve their information processing efficiency, strategy optimisation, and adaptability to changing contexts.

Types of Game Playing in Artificial Intelligence

The structure of the game and the amount of information available determine the type of AI game played. Here are the main categories:

1. Deterministic vs. Stochastic Games:

- Everything that happens in a deterministic game is directly related to the actions that the players take. Games like chess and tic-tac-toe are examples.
- Events involving chance or unpredictability, such rolling dice or shuffling cards, are at the heart of stochastic games. Games like Poker and Backgammon are examples.

2. Perfect vs. Imperfect Information Games:

- Game types like chess and go are examples of perfect information games where every player always has full knowledge of the current situation.
- In games with imperfect information, like poker, you can't see your opponent's hand.

3. Zero-Sum Games:

- One player's success is another's failure in these games because of the inherent competitive nature of them. Chess and other strategic games and competitive multiplayer games are good examples.

The Role of Search Algorithms in Game Playing

By allowing the methodical investigation of potential actions and outcomes, search algorithms are crucial in AI game playing. With the help of these algorithms, AI systems can assess current game conditions, foretell their opponents' actions, and devise winning strategies.

Minimax Algorithm:

Chess and other two-player zero-sum games make heavy use of minimax. It takes into account every potential move made by each player and assumes the other would play optimally, reducing their losses and increasing their wins.

Alpha-Beta Pruning:

By reducing the number of nodes assessed, Alpha-Beta Pruning improves efficiency and is an optimisation of Minimax. For games that use deep decision trees, it is perfect because it removes irrelevant search tree branches.

Monte Carlo Tree Search (MCTS):

When deciding what to do, MCTS uses a mix of statistical analysis and random simulations. It shines

in complicated video games and games like Go, where searching the state space exhaustively would be a huge pain.

Benefits of Using AI in Games

Among the several advantages of artificial intelligence in video games are the following:

1. **Less expensive labour:** Artificial intelligence (AI) functions similarly to cheaper labour in games; it speeds up common tasks, like correcting bugs, and is thus useful for improving the overall gaming experience. In the past, this kind of labour would have required a large team of game developers, but now, with AI, you can accomplish the same amount of work in minutes for a fraction of the cost.
2. **Creative Ideas:** Second, AI can be employed to generate ideas for the games. Games may grow old fast, so this is a good way to come up with fresh concepts. The games can't succeed until the companies come up with fresh concepts.
3. **Level design:** Artificial intelligence can also be employed to create better levels, adding to the game's playability and player enjoyment. As far as design tools go, it's top-notch.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the primary objective of game playing in AI? A. To improve game graphics B. To simulate intelligent decision-making C. To reduce memory usage D. To control hardware performance
2	Which of the following is a characteristic of a zero-sum game? A. Both players win B. One player's gain is another player's loss C. There is no winner or loser D. Players play independently
3	What type of game is Chess in AI terminology? A. Deterministic and perfect information B. Stochastic and imperfect information C. Real-time multiplayer D. Action-adventure

4	What is a Non-Player Character (NPC) in AI gaming? A. A game character controlled by AI B. A secondary player C. The main human player D. A network administrator
5	In AI game playing, the AI evaluates ____ to make decisions. A. graphics B. possible moves and outcomes C. network speed D. music volume



3. CORE FUNCTIONS OF AI IN GAMING

Non-Player Characters (NPCs) and Game World Behaviours are elements of online multiplayer games. A brief review of the main points you have brought up is as follows:

1. Characters Who Are Not Playable (NPCs)

- **AI for the enemy:** this includes programming opponents to use fighting-specific strategies and manoeuvres. To do this, can, for instance, adopt combat strategies that are well-suited to the current battle, take frequent cover, or slide for a sidetrack. An additional advantage of AI is that it allows difficulty levels to scale and adapt to the player's skill level. It takes the player's performance into account and makes it harder as they get better.
- **AI Ally:** AI can generate allies that devise plans of action and strategies to defeat the mission's objectives alongside you. AI Ally has the potential to be a fully realised character in the role of a playable partner, complete with a distinct personality and set of quirks that expand the player's capacity for empathy and immersion.

2. Game World Behavior

- **Procedural content generation:** AI may be taught to create dynamic game environments, paths to advancement, and scenarios. That manner, a fresh vision may be conveyed to the gamers in a way that keeps the gameplay exciting and consistent. Envision a cosmos with an infinite number of planets. They all have their own unique traits and are using AI to tackle distinct problems.
- **Adaptive level design:** Artificial intelligence components could be employed to construct levels that portray distinct changes based on player activities. Incorporating this element that brings everything to a more individual level has the potential to foster a sense of individual accountability for each player's choices.
- **Game optimisation and balancing:** With the help of AI, the game may rapidly adjust the difficulty or other aspects of the game based on user data, eliminating the need for human intervention. This ensures that every player has an equal chance to enjoy and challenge themselves. This ensures that both new and experienced players can enjoy themselves without ever feeling unfairly disadvantaged.

Therefore, artificial intelligence (AI) can only be useful in the gaming industry for one thing: making games better, more dynamic, and more immersive. Since AI is at the core of this dynamic gaming

industry, it is assuming the reins and dictating how non-player characters (NPCs) and the virtual world act.

Top Artificial Intelligence Video Games

A great deal of games incorporate AI to enhance the level of interaction and realism in their games. Here are a few examples:

1. **GTA Series:** Grand Theft Auto Every Grand Theft Auto game has been ground breaking in its own unique way. It was very much like an open-world game in that it let the player do whatever they wanted. Numerous NPCs in GTA enhance the game's realism by facilitating player-character interactions.
2. **Fortnite:** Second, there's Fortnite, a widely played game that incorporates AI to make battles more intense. It modifies the entire gaming element, including the environment, by using AI to introduce new occurrences and changes.
3. **Civilisation Series:** Thirdly, the Civilisation series allows you to create a wide range of prehistoric civilisations. The game's objectives are made more engaging and unique with the usage of AI.

Application of AI in the Game Industry

The gaming industry has numerous uses for AI.

- Integrating AI into games has been a trend since the dawn of the industry, all the way back to Pacman, and it's only going to keep growing.
- AI, short for artificial intelligence, has numerous uses. Levels can be made more challenging by incorporating new aspects.
- Visually, an older game can be remastered to look much better for today's standards.
- AI has the potential to personalise your gaming experience based on your preferences and needs.

Limitation of AI in the Game Industry

Just as there are numerous limits to AI in general, the gaming business is no exception. Whether AI can conquer them or not will be shown as its growth progresses.

- AI, despite its impressive capacity, does have its limitations. You cannot:
- Create an entire game by simply instructing the AI;
- Make drastic changes to the game's mechanics in the middle of the action. Additionally, AI is susceptible to bias.

- AI also raises ethical concerns. Without clear instructions, AI has the potential to introduce unwanted elements into the game.

Future of AI in gaming

Additionally, as technology progresses, artificial intelligence in games will likewise undergo a radical transformation. Games like Cyberpunk 2077 incorporate AI technology into different aspects of its gameplay. Artificial intelligence (AI) is rapidly improving to the point where it may soon be able to create complex levels for an entire game in just a few hours. Changing the gaming business as a whole will be more enjoyable with this. The material will give rise to newer developments like personalised gaming experiences, which will provide an experience unique to each player. A more lifelike gaming experience is on the horizon, thanks to improved natural language processing.

The ability to converse with in-game characters in the same way as one would with a real person is a direct result of this language processing. Thanks to AI-driven graphical rendering, the entire gaming experience will appear increasingly lifelike. When it comes to improving sound design for various levels, AI is also a fantastic alternative.

As time has progressed, AI has become increasingly sophisticated. The gaming industry has already begun to integrate AI in a number of contexts, and this trend will likely continue. Even small studios are utilising AI to make their games easier to build and better.

The game business is undergoing a sea change as a result of the widespread adoption of AI. Gamers can take advantage of AI to make games more immersive and exciting. The gaming business will find new opportunities as technology continues to progress. Once a technical improvement leaves the beta phase, the industry will quickly adopt it because of how well it adapts to new technologies.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which of the following is not a core function of AI in gaming? A. Pathfinding B. Decision-making C. Printing game manuals D. Environment adaptation
7	Procedural content generation allows AI to: A. Manually code levels

	B. Automatically generate new game levels C. Decrease game size D. Control multiplayer modes
8	AI helps improve gameplay by: A. Shortening the game B. Adapting difficulty based on player skill C. Increasing power usage D. Limiting content
9	AI-based ally characters in games are used to: A. Support the player with strategies B. Win the game alone C. Break the storyline D. Stop game progress
10	AI behavior trees are used for: A. Saving files B. Making decisions for NPCs C. Rendering shadows D. Playing music

4. MINI-MAX ALGORITHM IN ARTIFICIAL INTELLIGENCE

In two-player games, the minimax algorithm aids players in determining the optimal move. When trying to figure out what the best move is for the current player, it's important to pretend that the opposing player is doing the same. Everyone in the game is trying to limit how much they can lose. Everything you need to know about the minimax algorithm, including how it works and what its attributes are, will be covered in this article. The Mini-Max algorithm is an AI decision-making tool with applications in computer games and game theory. With its "min" design, it aims to minimise the worst-case loss and "max" design, respectively, in order to maximise the potential benefit.

For optimal play on the assumption that your opponent is also playing optimally, decision-makers and game-theorists employ minimax, a form of backtracking algorithm. Many two-player turn-based games rely on it, including chess, Tic-Tac-Toe, backgammon, mancala, and many more.

Maximiser and minimiser are the names of the two participants in Minimax. The goal of the maximiser is to achieve the maximum possible score, whereas the objective of the minimiser is to achieve the worst possible score. There is a monetary value linked to each board state. The board's score will typically be positive in a given stage if the maximiser has the upper hand. As a general rule, it will be negative if the minimiser is in a stronger position in that state of the board. Heuristics that are specific to each game genre determine the board values.

Example:

Take the following example of a game with four possible outcomes: the player can progress from the root state to each of the four leaves of a perfect binary tree. Pretend for a second that you and your opponent are both playing at the root level; you have the advantage of being the maximiser and having the first opportunity to move. While playing optimally, what would your next move be if your opponent did the same?

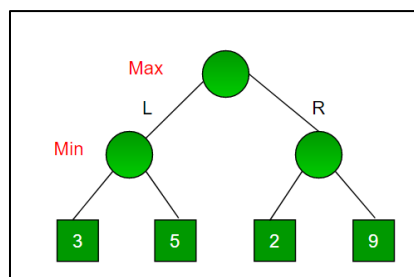


Figure 1: Demonstration of Maximiser for a game tree, initial step

Being a backtracking based algorithm, it explores every potential move before making a choice. Maximiser moves to the left, and the minimisers take over. Now the minimiser can choose between three and five options. As the minimiser, it will select the lower of the two options, which is 3, without a doubt.

The moment has come for the minimisers as the maximiser moves to the right. Two numbers, two and nine, are now available to the minimiser. Since 2 is the lesser of the two figures, he will select it.

You, as the maximiser, would pick the bigger number, three. Therefore, going to the left is the best move for the maximiser, and the best value is 3.

The game tree now appears as follows:

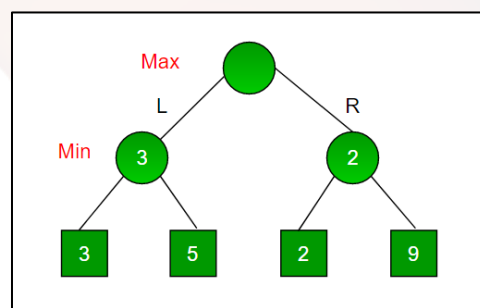


Figure 2: Demonstration of Maximiser for a game tree

5. WORKING OF MIN-MAX PROCESS IN AI

Both the maximiser and the minimiser are involved in the Min-Max algorithm, and their goal is to maximise their own outputs.

Players Involved

a. Optimal Player (Max):

- Strives to achieve the highest possible score or utility value for the player.
- Assuming the opponent will play optimally, choose the move that leads to the maximum potential utility value.

b. Minimising Player (Min)

- Assuming the opponent will play optimally, Minimising Player (Min):
- Chooses the move that minimises the maximiser's utility value.
- Aims to minimise the maximiser's score or utility value. The Min-Max algorithm revolves around the dynamic between these two actors, as they try to out-plan and counter-act one another.

Procedures Followed by the Mini-Max Method:

In order to find the best course of action, the Min-Max algorithm iteratively executes a series of critical steps. This is a detailed explanation:

Step 1: Generate the Game Tree

- The first step is to generate the game tree. The goal is to create a tree structure that represents all possible moves starting from the current state of the game.
- Information: A game state is represented by each node, and a possible move is represented by each edge.

Step 2: Evaluate Terminal States

- Goal: Give the game tree's end nodes monetary prices that reflect its utility.
- Specifics: These numbers indicate whether the game was won, lost, or tied.

Step 3: Propagate Utility Values Upwards

- The goal is to increase the utility values as you go up the tree, beginning at the terminal nodes.
- Each non-terminal node's details are:

- Choose the highest value from the child nodes if it is the player's turn to maximise.
- If the player whose turn it is to minimise, then choose the lowest value from the child nodes.

Step 4: Choose the Best Action

- At the very beginning of the game tree is the objective, which is to maximise utility by choosing the path that will lead to the most possible value.

Min-Max Formula

The following recursive formulas are used to get the Min-Max value of a node in the game tree:

1. Maximizing Player's Turn:

$$\text{Max}(s) = \max_{a \in A(s)} \text{Min}(\text{Result}(s, a))$$

Here:

- $\text{Max}(s)$ is the maximum value the maximising player can achieve from state s .
- $A(s)$ is the set of all possible actions from state s .
- $\text{Result}(s, a)$ is the resulting state from taking action a in state s .

Terminal States

- For terminal states, the utility value is directly assigned:

Example Calculation

Think of a child's game where the value of each terminal condition is provided in terms of utility. The Min-Max calculations can be seen in this example:

1. Determine the utility values beginning with the terminal states.
2. Apply the Min-Max formulae to propagate these values up the tree.

When the terminal states are associated with useful things, for instance

- For the maximizing player's node: $\text{Max}(s) = \max(U_1, U_2, \dots, U_n)$
- For the minimizing player's node: $\text{Min}(s) = \min(U_1, U_2, \dots, U_n)$

Pseudocode for Min-Max Algorithm

This pseudocode shows how the Min-Max algorithm is recursive; it evaluates utility values by

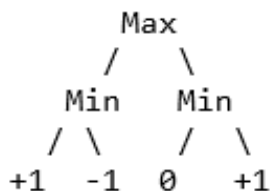
switching between the maximising and minimising players until the best move is found.

```
def minmax(state, depth, maximizing_player):
    if is_terminal(state) or depth == 0:
        return utility(state)

    if maximizing_player:
        max_eval = -infinity
        for action in actions(state):
            eval = minmax(result(state, action), depth - 1, False)
            max_eval = max(max_eval, eval)
        return max_eval
    else:
        min_eval = infinity
        for action in actions(state):
            eval = minmax(result(state, action), depth - 1, True)
            min_eval = min(min_eval, eval)
        return min_eval
```

Example of Min-Max in Action

Think of a streamlined game where every player gets two moves per turn. A simple game tree like this:



- Utilities are 1, -1, 0, and +1 at the nodes that serve as leaves.
- The smallest value from the left subtree's child nodes, -1, and the right subtree's minimum value, 0, will be chosen by the minimising player.
- The player aiming to maximise their score will select the highest possible value between -1 and 0, which happens to be 0.

With the Minimiser's ideal play in mind, the maximising player's best move results in zero utility.

Mini-Max Algorithm Optimisation for Alpha-Beta Pruning

The Min-Max algorithm is improved with alpha-beta pruning since it removes branches that do not contribute to the final conclusion. The alpha-beta pruning formulas that are most important are:

- Alpha(α): The best value that the player who is maximising can promise thus far is represented by alpha (α).
- Beta (β): The maximum value that the player who is minimising can promise up to this point.

During the search:

- If $\alpha \geq \beta$, prune the remaining branches.

Alpha-Beta Pseudocode

The mathematical foundation of the Min-Max algorithm and its optimisations are these formulae and notions, which offer a systematic way to make decisions in competitive setting

```
def alpha_beta_minmax(state, depth, alpha, beta, maximizing_player):
    if is_terminal(state) or depth == 0:
        return utility(state)
    if maximizing_player:
        max_eval = -infinity
        for action in actions(state):
            eval = alpha_beta_minmax(result(state, action), depth - 1, alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break # Beta cut-off
        return max_eval
    else:
        min_eval = infinity
        for action in actions(state):
            eval = alpha_beta_minmax(result(state, action), depth - 1, alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
```

```
if beta <= alpha:  
    break # Alpha cut-off  
return min_eval
```

Advantages of the Minimum-Maximum Technique

1. **Optimal Decision Making:** By taking into account all potential moves and their outcomes, the Min-Max algorithm guarantees optimal decision making. By foreseeing the opponent's optimal moves and selecting them, it gives the player a strategic edge.
2. The Min-Max method is straightforward and easy to grasp on a conceptual level. It is one of the most well-known and easily-taught AI algorithms due to its simple method of assessing and propagating utility values across a game tree.

Issues with the Min-Max Method

1. **The Difficulty of Computing:** The computational complexity is the main issue with the Min-Max algorithm. There are an exponentially increasing number of nodes to analyse as the game tree's depth and branching factor increase. Games with deep and complicated trees, such as Go, find it computationally expensive and impracticable because of this.
2. **Depth Limitations:** In order to control the amount of processing power required, the Min-Max algorithm frequently restricts the level of the game tree. suboptimal choices may result, nevertheless, if crucial actions are located outside the selected depth. It is quite difficult to strike a balance between computational practicality and depth.
3. **Dealing with Uncertain circumstances:** In circumstances where the outcomes of each move are not known, the Min-Max method may not work as well. The basic Min-Max approach typically has to be modified for real-world settings that entail ambiguity and insufficient knowledge.

Evaluation Against Competing Algorithms

- a. **Min-Max vs. Monte Carlo Tree Search (MCTS)**
 - **Exploration vs. Exhaustive Search:** Comparing Min-Max to Monte Carlo Tree Search (MCTS) and Exploration to Exhaustive Search, we find that Min-Max thoroughly investigates all potential moves up to a specific depth, guaranteeing that judgements made within that scope are optimal. Contrarily, MCTS strikes a balance between exploration and exploitation by using statistical analysis and random sampling to investigate the most promising movements.

- Scalability: because of its selective exploration, MCTS scales well to high-complexity games like Go, whereas Min-Max has problems with exponential increase in the size of the game tree.
- Use cases: MCTS shines in complicated, probabilistic settings, whereas Min-Max is better in chess and other games with obvious utility values and small tree sizes.

b. Min-Max vs. Reinforcement Learning

- Learning vs. Planning: The Min-Max algorithm is designed for planning purposes and requires a full game tree and utility values. Methods such as Q-learning and policy gradients are used in Reinforcement Learning (RL) to learn the best strategies by interacting with the environment.
- Adaptability: Min-Max uses pre-computed evaluations, RL learns from new experiences continuously, allowing it to adjust to unpredictable and dynamic contexts.
- Use Cases: RL is commonly employed in situations that demand adaptive, real-time decision-making, such as autonomous systems and robotics, while Min-Max is best suited for deterministic, adversarial games.

AI's Mini-Max Algorithm in History

- a) Deep Blue Chess: First, there's Deep Blue chess, the infamous computer from IBM that beat world champion Garry Kasparov in 1997 using the Min-Max algorithm with alpha-beta pruning. The effectiveness of Min-Max in strategic game playing was demonstrated by Deep Blue's capacity to analyse millions of positions per second.
- b) AlphaZero: Made by DeepMind, AlphaZero achieved superhuman performance in Go, chess, and shogi by combining deep learning and reinforcement learning with Min-Max search. The neural networks used by AlphaZero assess board positions and direct the Min-Max search, demonstrating the complementary nature of classical algorithms and contemporary AI methods.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	The Minimax algorithm is best used in: A. Two-player zero-sum games B. Single-player puzzle games C. First-person shooters D. Sandbox building games

12	What does the "min" player try to do in the Minimax algorithm? A. Maximise the score B. Minimise the opponent's score C. End the game D. Make random moves
13	Minimax assumes both players play: A. Carelessly B. Optimally C. Randomly D. Aggressively
14	What type of tree does Minimax use for decision-making? A. Game tree B. Binary search tree C. AVL tree D. Hash tree
15	AI What is the result of a perfect Minimax decision in a game? A. Optimal move selection B. Faster graphics rendering C. Random move selection D. Non-replayable game

6. ALPHA-BETA PRUNING IN AI

Pruning refers to the process of removing unwanted branches and leaves. A technique used in AI called "alpha-beta pruning" removes superfluous nodes from decision trees. Researchers in the 1900s separately discovered this alpha-beta pruning technique.

One way to optimise search results and make the minimax algorithm work better is via alpha beta pruning. One method for determining decisions in zero-sum games with two players, such as chess, is the minimax algorithm. Two players compete in these games, with the goal of maximising their score and the other minimising theirs. The minimax method finds the optimal solution by iteratively investigating all conceivable game states, which are visualised as a tree structure, and allocating values to the nodes at the top of the tree according to the possible outcomes.

After that, the algorithm uses these values to determine the best next step by propagating them up the tree. Nevertheless, the computing costs become prohibitive as the game's complexity rises since the number of alternative states grows exponentially. Alpha beta pruning is quite important in this situation. It "prunes" branches that won't have any impact on the minimax algorithm's final choice, reducing the number of nodes it needs to analyse. It streamlines the decision-making process by removing superfluous computations, allowing for more efficient and quicker evaluations. Consequently, Alpha Beta Pruning works well for time-sensitive applications like AI gaming, where efficiency and speed are paramount.

To represent and solve issues where two or more players compete against each other, adversarial search algorithms are utilised in artificial intelligence, especially in game playing and decision-making. The alpha-beta pruning method is among the best known in this field.

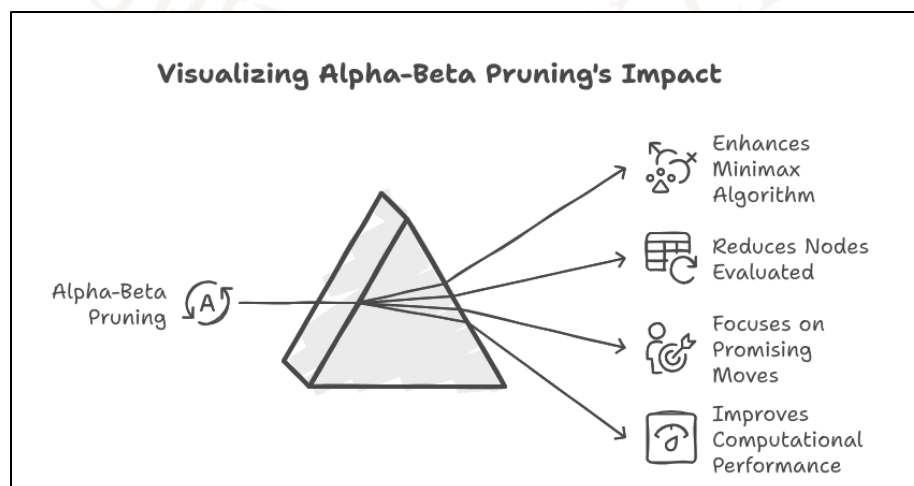


Figure 3: Alpha-Beta Pruning

The Process of Alpha Beta Pruning

One of the main ideas behind Alpha Beta Pruning is to not use the values found during the search to evaluate branches of the game tree that won't affect the final choice. Two numbers, Alpha and Beta, are used to accomplish this.

Beta and Alpha Values(s)

- Alpha: The optimal player (often an AI) can guarantee the best (highest-value) value up to this point, and that's what Alpha stands for. As so, it serves as a floor. Alpha starts with the value $-\infty$.
- Beta: Beta stands for the best (lowest) value that the opponent, who is minimising, can guarantee up to this point. As such, it serves as a ceiling. Beta starts with a value of $+\infty$.

The Pruning Process

- The AI monitors the tree's Alpha and Beta values as it explores it. It uses these values as a benchmark when examining a node.
- The current branch will have no impact on the final decision if Alpha ever becomes equal to or larger than Beta. This is because the opponent will shun this path in favour of a better one. Consequently, the algorithm prunes this branch and advances to the next one.
- The method can bypass substantial portions of the tree through this process, drastically decreasing the amount of nodes that need to be assessed.

Working of Alpha-beta Pruning

1. The first move will be handled by us first. At first, we will set the worst case $\alpha = -\infty$ and the best case $\beta = +\infty$. We shall prune the node if alpha is equal to or larger than beta.

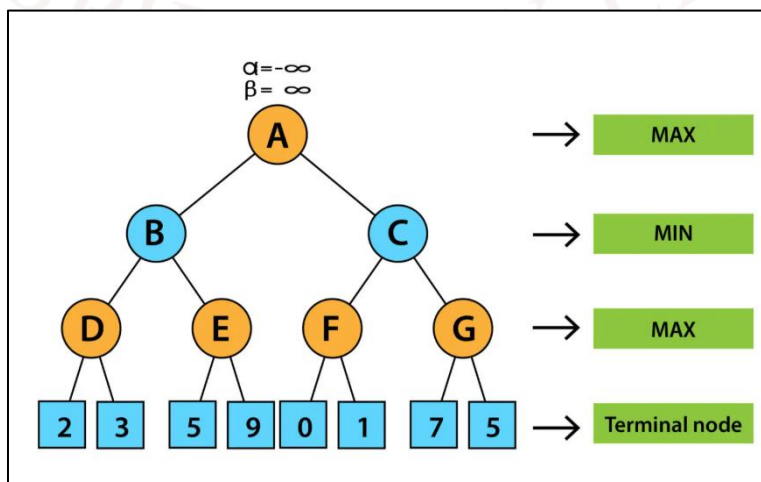


Figure 4: Step-1 Alpha-Beta Pruning

2. Since beta is larger than alpha at the outset, we refrained from pruning it. Let Max take the stage now. For that reason, we shall determine alpha at node D. The maximum value of alpha at node D will be 3 (maximum of 2, 3).
3. The next node to be MIN is B. This indicates that at node B, the value of alpha beta will be minimum (3, infinity). Hence, at node B, alpha will be $-\infty$ and beta will be 3.

The values of $\alpha = -\infty$ and $\beta = 3$ will then be passed on to node E, the next successor of node B, using the algorithms.

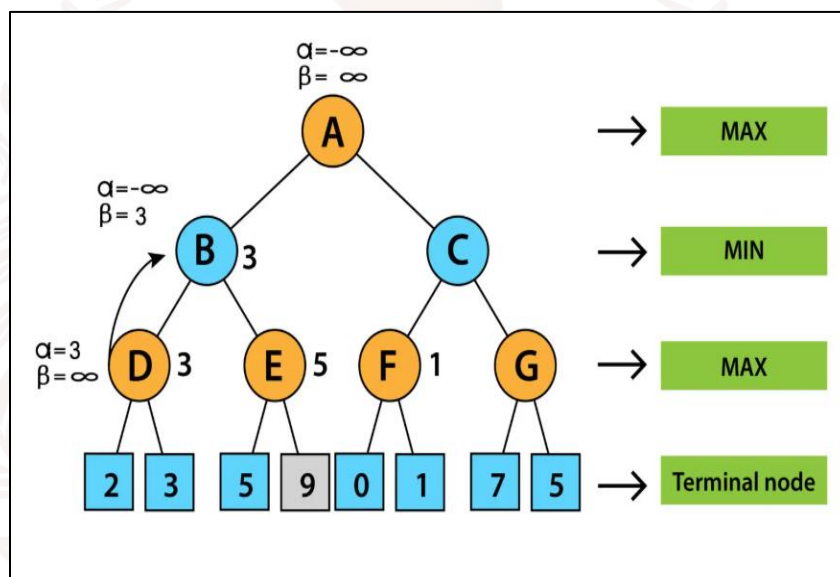


Figure 5: Step-3 Alpha-Beta Pruning

4. MAX is now in the spotlight. To find MAX, we shall therefore go to node E. When comparing with 5, the alpha value at E is $-\infty$. Thereby, the maximum value between $-\infty$ and 5 will be 5. Alpha = 5 and Beta = 5 at node E.

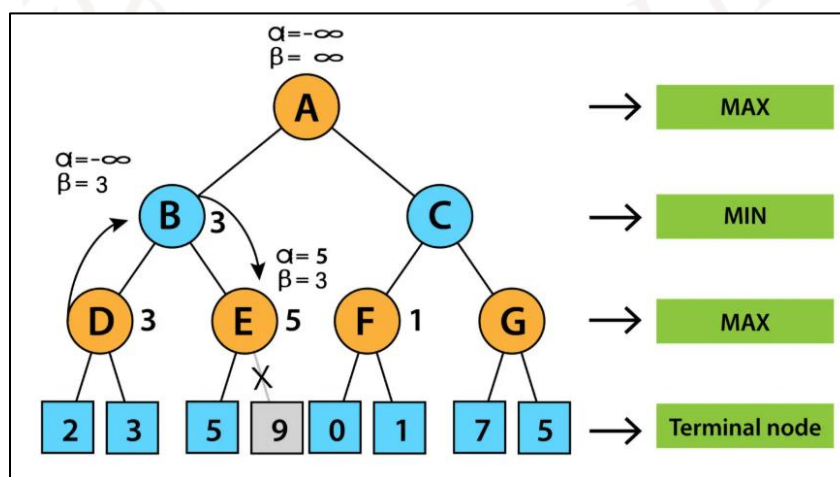


Figure 6: Step-4 Alpha-Beta Pruning

5. Node E's value will be 5, the algorithm will not be visited, and we can prune the right successor of node E because alpha is bigger than beta and is also a pruning condition. The method then loops back around to node A from node B. The maximum value of $\text{MAX}(-\infty, 3)$ is used to alter the alpha of node A. Node C will get the alpha and beta values from node A, which are $(3, +\infty)$. Node F will receive the identical values.
6. At node F, the left branch has a value of 0, and this value will be compared to alpha. Currently, when comparing the right child (1) with $\text{MAX}(3, 1) = 3$, α remains at 3, but F's node value will be 1.

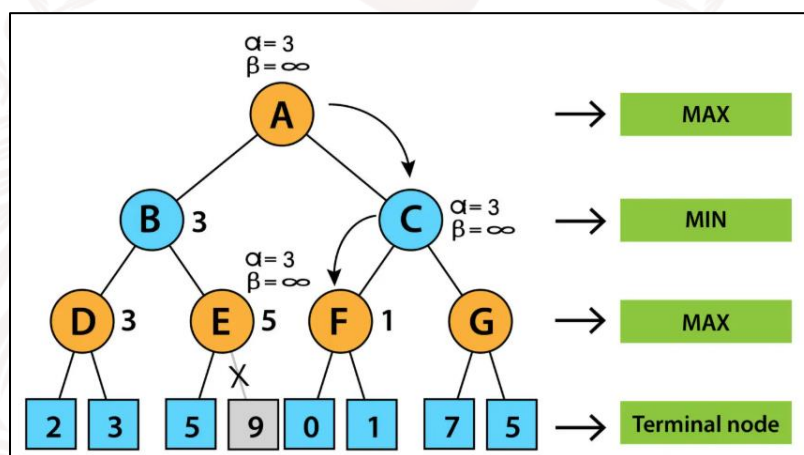


Figure 7: Step-6 Alpha-Beta Pruning

7. Then, node F will compare the node value 1 with the beta value at node C after passing it to node C. It is MIN's turn now. Hence, the minimum value between zero and infinity plus one will be one. Once again, the pruning criterion is satisfied at node C since $\alpha = 3$, $\beta = 1$, and alpha is bigger than beta. The method did not compute the entire subtree G, hence it will be trimmed after node C.

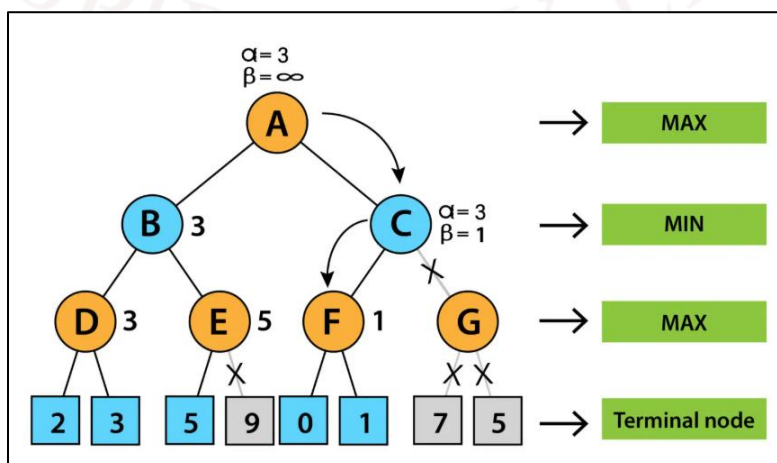


Figure 8: Step-7 Alpha-Beta Pruning

A will now compute $\text{MAX}(1,3)$, which yields 3, after receiving the node value from C.

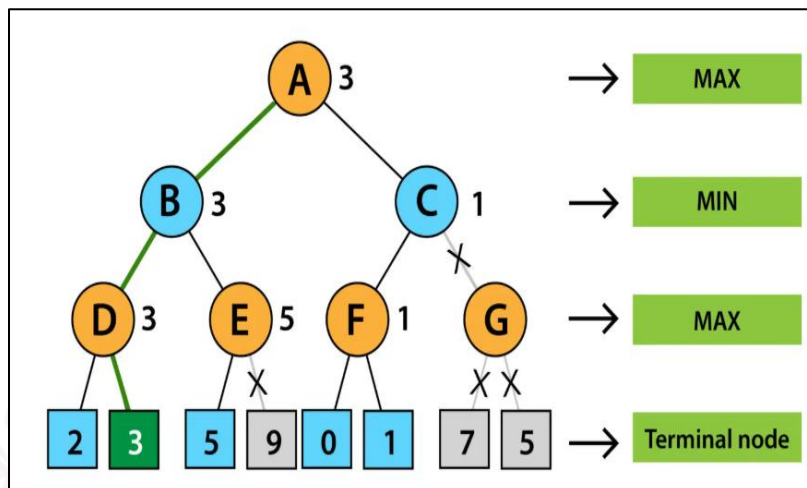


Figure 9: Alpha-Beta Pruning -Final output

You can see the last tree with computed and non-computed nodes in the image above. Thus, 3 would be the maximiser's ideal value in this case.

Theoretical approach for How Alpha-Beta Pruning

Like Minimax, the Alpha-Beta pruning algorithm iteratively explores the game tree, but it eliminates unnecessary branches. Below, we will go over the steps:

1. Initialisation: The default values for alpha and beta are negative infinity and positive infinity, respectively, to begin.
2. Max Node Evaluation:

For every child node of a Max node, do the following:

 - Apply the Minimax algorithm with Alpha-Beta pruning to evaluate the child node.
 - Revise the alpha by setting $\alpha = \max(\alpha, \text{child value})$.
 - Remove offspring (beta cutoff) if alpha is equal to or higher than beta.
3. Min Node Evaluation:
 - Apply the Minimax algorithm with Alpha-Beta pruning to each child node of a Min node.
 - The beta has been updated to $\beta = \min(\beta, \text{childvalue})$.
 - Cut off the offspring (the alpha cutoff) if beta is equal to or less than alpha.

Alpha-Beta pruning applied to adversarial search algorithms

In order to minimise the number of nodes examined in the game tree, the given code employs the alpha-beta pruning algorithm, a variation of the minimax algorithm that is used in decision-making

and game theory. Every section of the code is broken down into its component parts here.

- Node class: In the game tree, a node is defined by its class, which includes the name, children, and value.
- Functions that are useful:
 - evaluate(node): Gets the value of the node.
 - Is_terminal (node): this function determines whether a given node is a terminal node.
 - Get_children(node): This function returns the children of a node called "node".
 - Alpha-Beta Pruning Function: When the depth is zero or the node is terminal, the alpha_beta_pruning function returns the node's value. The gamer that aims to maximise their score constantly updates their alpha and checks for the beta cut-off. Minimising player requires checking for alpha cut-off, updating beta, and attempting to minimise score.
- Game Tree Creation:
 - Assigns values to the terminal nodes DE, FE, G, H, and I. generates offspring nodes (B, C) within the network.
 - Makes a root node (A) by joining progeny B and C.
- Run Algorithm:

Applying the alpha-beta pruning technique to root node A. Initial_beta is set to infinity and initial_alpha to negative infinity puts depth at 3, and outputs the best possible value.

```
class Node:
    def __init__(self, name, children=None, value=None):
        self.name = name
        self.children = children if children is not None else []
        self.value = value

    def evaluate(node):
        return node.value
```

Output: The optimal value is: 3

The Alpha-Beta Pruning Method in Action-Example

Think of a basic game tree with a depth of 3 and a branching factor of 2.

1. To begin, go to the very top, the root node. Start with alpha set to $-\infty$ and beta set to ∞ .

2. Return to the initial child node (Min node) and assess the subsequent children nodes (Max nodes).
3. Evaluate the children (leaf nodes) of each Max node as you traverse them. Make the necessary updates to alpha and beta.
4. For Max nodes, prune branches where $\alpha \geq \beta$. For Min nodes, prune branches where $\beta \leq \alpha$.

Advantages of Alpha-Beta Pruning

- Effectiveness: Alpha-beta pruning speeds up the search process by drastically reducing the number of nodes considered in comparison to the basic minimax algorithm.
- Optimality: Alpha-beta pruning ensures discovering the optimal step even when it doesn't impact the final decision.

Limitations of Alpha-Beta Pruning

The complexity may remain high for really large game trees, despite the fact that it decreases the number of evaluations.

- Alpha-beta pruning's efficacy may be quite sensitive to the evaluation sequence of nodes, a phenomenon known as heuristic dependency.
- If you have good heuristic functions or know how to order moves, it will perform much better.

Applications of Alpha-Beta Pruning

When it comes to artificial intelligence, two-player games like:

- Chess: In chess, it improves the performance of chess engines so that they can assess more complex moves in less time.
- Checkers: Makes AI opponents more difficult by optimising move evaluation in checkers.
- Othello: Strengthens AI techniques by enhancing decision-making in Othello.

In gaming and decision-making contexts, alpha-beta pruning is an effective optimisation method for adversarial search algorithms. It improves the minimax algorithm's efficiency without sacrificing the solution's optimality by removing branches that do not affect the ultimate outcome. But things like move ordering and game tree complexity can affect how well it works.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
16	Alpha-beta pruning is used to: A. Reduce the number of nodes evaluated B. Increase game speed C. Improve audio quality D. Design game graphics
17	Alpha refers to: A. Best value that the maximiser can guarantee B. Worst value for the Minimiser C. Maximum sound pitch D. Minimum CPU usage
18	When is a node pruned in Alpha-Beta pruning? A. When it cannot affect the final decision B. When it's too close to the root C. When it's too deep D. When it's already visited
19	Alpha-Beta pruning is a/an ____ of the Minimax algorithm. A. Replacement B. Optimisation C. Error D. Alternative game
20	What is the benefit of using Alpha-Beta pruning? A. Improved efficiency B. Better screen resolution C. Easier coding D. More multiplayer options

7. SUMMARY

- Artificial Intelligence (AI) plays a pivotal role in the modern gaming industry, enhancing both game design and player experience. Game playing in AI focuses on enabling machines to play games either against human players or other AI systems. It involves building systems that can make decisions, learn from experience, and adapt their strategies.
- This has led to the development of highly intelligent game opponents and dynamic game environments that respond to a player's actions, creating a more immersive and challenging experience.
- The core functions of AI in gaming include pathfinding, decision-making, pattern recognition, and natural language processing. These functions are used to control non-player characters (NPCs), predict player behavior, and even generate new content dynamically. For example, AI helps NPCs navigate through complex environments using algorithms like A* for pathfinding, and can simulate human-like behavior to improve realism.
- Additionally, AI systems are employed to adjust game difficulty in real time, create personalised gameplay, and automate quality testing during development
- One of the fundamental strategies in game AI is the Mini-Max algorithm, which is used in turn-based games like chess and tic-tac-toe.
- The Mini-Max algorithm is a recursive decision-making process where one player tries to maximise their score (MAX), while assuming the opponent will try to Minimise it (MIN). This helps simulate rational decision-making between two opposing players. The algorithm evaluates all possible moves and chooses the one that leads to the best possible outcome, considering the worst-case scenario.
- The working of the Mini-Max process involves constructing a game tree where each node represents a game state, and each level alternates between the MAX and MIN player.
- The algorithm begins at the terminal nodes (end-game states), assigns them values based on who is winning, and then backtracks through the tree, selecting the optimal move at each level. This process ensures that the player chooses a move that leads to the best guaranteed outcome, assuming the opponent plays optimally.
- To improve the efficiency of this process, Alpha-Beta pruning is used to eliminate branches in the game tree that do not affect the final decision. It keeps track of two values: alpha (the best already explored option along the path to the root for the maximiser) and beta (the best for the Minimiser).

- If the algorithm finds that a node cannot possibly influence the final decision, it prunes or cuts off that branch, reducing the number of nodes evaluated. This optimisation significantly reduces computation time, allowing deeper exploration in complex games.



8. GLOSSARY

Artificial Intelligence (AI)	-	The simulation of human intelligence in machines that are programmed to think, learn, and make decisions.
Game Playing in AI	-	A branch of AI that focuses on developing systems capable of playing games intelligently against human players or other AI agents.
Non-Player Character (NPC)	-	A character in a game that is not controlled by a player but by the game's AI to simulate human-like behavior.
Pathfinding	-	An AI function that determines the shortest or most efficient route for characters to move from one point to another in a game environment.
Decision Making	-	The process by which AI selects actions based on evaluating different outcomes to achieve specific goals.
Dynamic Difficulty Adjustment (DDA)	-	An AI technique that automatically changes the game's difficulty level in real-time based on the player's performance.
Mini-Max Algorithm	-	A decision-making algorithm used in two-player turn-based games where one player maximises the score while the other Minimises it, assuming both play optimally.
MAX Player	-	The player in a Mini-Max algorithm who aims to maximise the outcome or score.
MIN Player	-	The player in a Mini-Max algorithm who tries to Minimise the opponent's outcome.
Game Tree	-	A graphical representation of all possible moves in a game, starting from the current state and branching out into potential future states.
Terminal Node	-	A node in the game tree that represents a final game state (win, loss, or draw) where no further moves are possible.
Backtracking	-	The process of returning from the end nodes up the game tree to determine the best possible move by comparing outcomes.

Alpha-Beta Pruning	- An optimisation technique for the Mini-Max algorithm that eliminates branches of the game tree that won't influence the final decision, reducing computation time.
Alpha (α)	- The best value that the maximiser can guarantee at that level or above.
Beta (β)	- The best value that the Minimiser can guarantee at that level or above.
Pruning	- Cutting off branches in the game tree that do not need to be evaluated because they won't affect the final decision.
Heuristic Evaluation Function	- A function used in game AI to estimate the value of a game state when the terminal state is not reached, guiding the algorithm's choices.
Adversarial Search	- A search method used in competitive environments (games) where multiple agents compete and the goal is to defeat the opponent.
Turn-Based Games	- Games where players alternate taking turns (e.g., chess, tic-tac-toe), often modeled using Mini-Max and related AI strategies.

9. TERMINAL QUESTIONS

1. What is the role of AI in game playing?
2. Name any three core functions of AI in gaming.
3. What is the Mini-Max algorithm used for in AI?
4. How does the Mini-Max algorithm evaluate game states?
5. Define Alpha-Beta pruning.
6. What are the terms Alpha and Beta in Alpha-Beta pruning?
7. Explain one advantage of using Alpha-Beta pruning over Mini-Max.
8. What is a terminal node in a game tree?
9. Why is AI important for dynamic difficulty adjustment in games?
10. Give one real-world example where Mini-Max or Alpha-Beta pruning is applied.

10. ANSWERS

Self-Assessment Questions

1. B. To simulate intelligent decision-making
2. B. One player's gain is another player's loss
3. A. Deterministic and perfect information
4. A. A game character controlled by AI
5. B. possible moves and outcomes
6. C. Printing game manuals
7. B. Automatically generate new game levels
8. B. Adapting difficulty based on player skill
9. A. Support the player with strategies
10. B. Making decisions for NPCs
11. A. Two-player zero-sum games
12. B. Minimise the opponent's score
13. B. Optimally
14. A. Game tree
15. A. Optimal move selection
16. A. Reduce the number of nodes evaluated
17. A. Best value that the maximiser can guarantee
18. A. When it cannot affect the final decision
19. B. Optimisation
20. A. Improved efficiency

Terminal Questions Answers

1. Answer:

AI in game playing is used to create intelligent behaviour in non-player characters (NPCs), enable adaptive difficulty levels, and simulate strategic thinking to provide a more challenging and engaging experience for players. (Refer to section 2 for more information)

2. Answer:

- Pathfinding
- Decision Making
- Opponent Behaviour Simulation (Refer to section for more information)

3. Answer:

The Mini-Max algorithm is used in two-player turn-based games to determine the best possible move by assuming both players play optimally. One player aims to maximise the score while the other tries to minimise it. (Refer to section 5 for more information)

4. Answer:

It builds a game tree of possible moves and backtracks from terminal states, assigning values to each move. The maximising player selects the move with the highest value, while the minimising player selects the move with the lowest value. (Refer to section 5 for more information)

5. Answer:

Alpha-beta pruning is an optimisation technique for the Mini-Max algorithm. It prunes away branches in the game tree that cannot influence the final decision, thus improving efficiency by reducing the number of nodes evaluated. (Refer to section 6 for more information)

6. Answer:

- Alpha (α): The best score that the maximiser is currently guaranteed.
- Beta (β): The best score that the Minimiser is currently guaranteed. (Refer to section 6 for more information)

7. Answer:

Alpha-Beta pruning reduces the number of nodes that need to be evaluated in the game tree, making the search process faster without affecting the outcome. (Refer to section 6 for more information)

8. Answer:

A terminal node represents a final game state (such as win, lose, or draw) where no further moves are possible.

9. Answer:

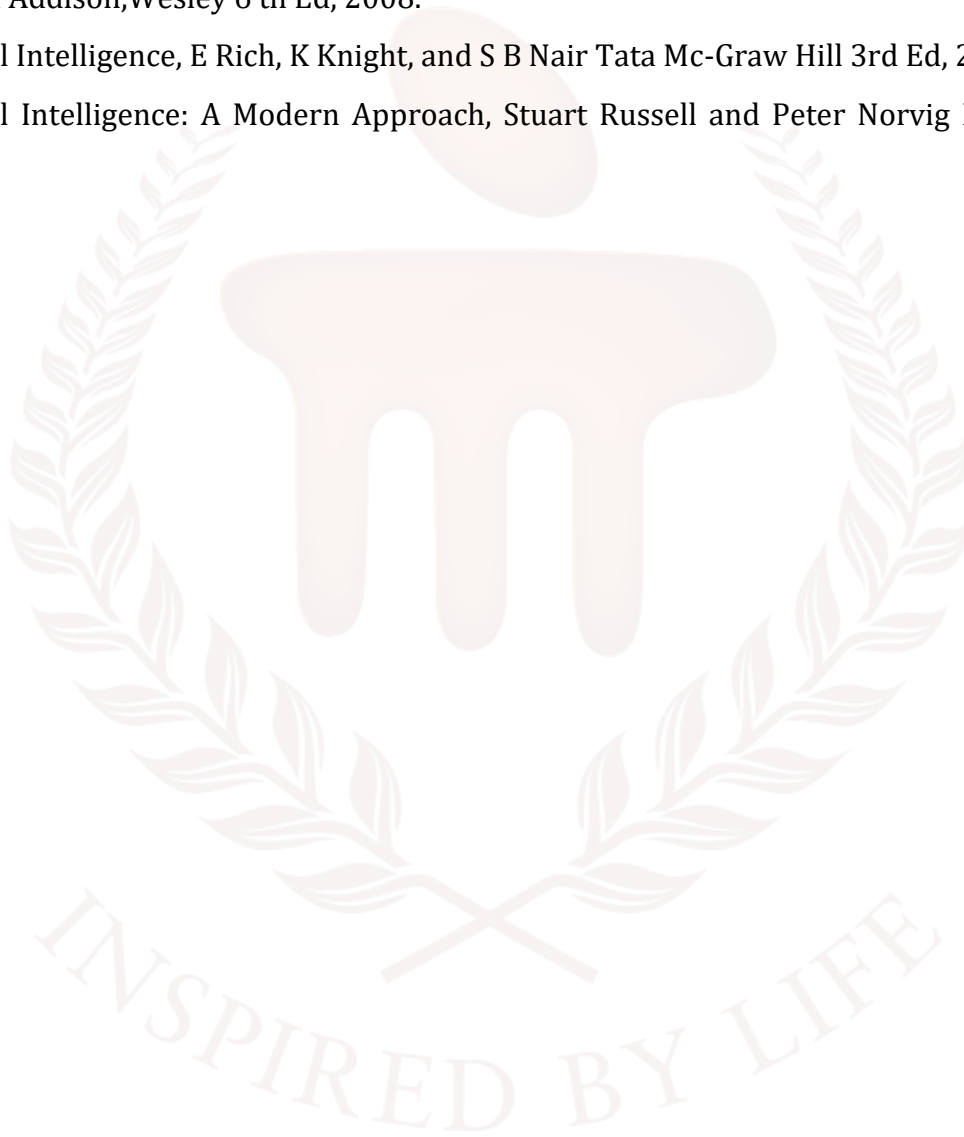
AI enables dynamic difficulty adjustment by analysing player behaviour and adjusting the game's challenge level in real-time to maintain engagement and prevent frustration or boredom. (Refer to section 6 for more information)

10. Answer:

Mini-Max and Alpha-Beta pruning are commonly used in board games like Chess and Tic-Tac-Toe to determine the best moves for AI-controlled players. (Refer to section 6 for more information)

11. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

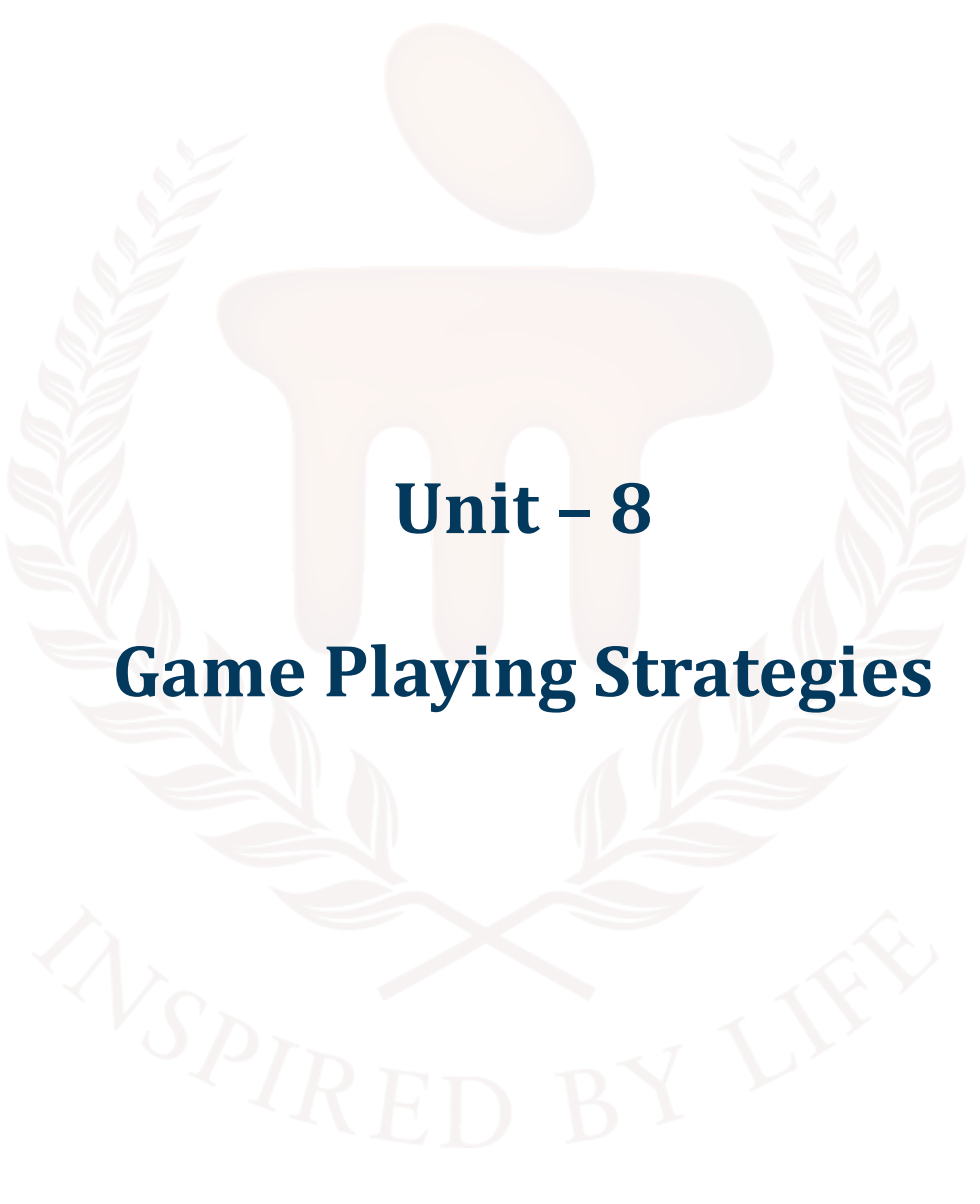


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109
ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING



Unit – 8

Game Playing Strategies

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Introduction to basic Games	1	-	6 - 8
3	Game Description	-	-	9 - 11
4	Game Management and Playing	-	1	12 - 15
5	Types of Games	2	2	16 - 19
6	Basic Game Strategies	3	3	20 - 23
7	Sample Game: Tic-Tac-Toe	4	-	24 - 27
8	Summary	-	-	28
9	Glossary	-	-	29 - 30
10	Terminal Questions	-	-	31
11	Answers	-	-	32 - 35
12	References	-	-	36

1. INTRODUCTION

In this chapter, we had studied how Artificial Intelligence (AI) is applied in gaming, particularly in two-player, turn-based games such as Tic-Tac-Toe, Chess, and Checkers. We had learnt how AI models these games using game trees, which represent all possible moves and their outcomes. The chapter introduced the Minimax Algorithm, a foundational concept in game-playing AI, which allowed the AI to make decisions by evaluating the best possible moves assuming the opponent also played optimally. This helped the AI choose moves that maximise its chance of winning while minimising the opponent's. We also explored Alpha-Beta Pruning, an advanced technique that improves the efficiency of the Minimax Algorithm by eliminating branches in the game tree that don't influence the final decision. This greatly reduces the computation time, making the AI faster and more practical for complex games. Then the chapter got included with examples, diagrams, and code implementations that helped us to understand how these algorithms work in real scenarios.

By the end of this chapter, we had understood how AI makes strategic decisions in games, how the Minimax Algorithm works, and how Alpha-Beta Pruning makes it faster and smarter. They will also gain hands-on experience by applying these techniques to simple games.

In this unit, we will study the fundamentals of game playing in artificial intelligence by exploring basic games and strategies. We will begin by examining common two-player, turn-based games such as tic-tac-toe, checkers, and chess, focusing on how these games are represented using game trees that map out possible moves and outcomes. The unit will introduce key concepts such as game states, legal moves, and how AI systems make decisions by anticipating opponent moves and planning strategically. Central to this study will be the Minimax algorithm, a decision-making method that helps AI select the best moves to maximise its chances of winning. By understanding these basic strategies and algorithms, you will gain insight into how AI approaches game playing, manages complexity, and formulates effective plans in competitive settings.

1.1. Objectives

After studying this unit, you should be able to:

- Interpret the basic concepts of game playing
- Apply strategic thinking of different games to recognise patterns
- Illustrate how a simple game like Tic-Tac-Toe can model decision-making processes and strategies.



2. INTRODUCTION TO BASIC GAMES

Competing in games of strategy like chess combines cerebral challenge with friendly rivalry. By engaging in such activities, we can enhance our cognitive abilities. We get a thrill out of seeing how our abilities stack up against the rest of the field in the competition.

The interest in using computer games as a platform for testing AI can be explained by the same rationale. Theoretically, more intelligent programs should have an advantage in competitions, allowing us to utilise them as a means of gauging AI performance.

There isn't much use in artificial intelligence for creating programs to play particular games. To start, the focus of specialised gaming players is extremely limited. They could excel at one game while struggling in another. Despite its victory over the global chess champion, Deep Blue is completely lost at checkers. (2) The fact that they are only partially effective is a further issue with specialised game playing systems. Their programmers do the bulk of the intriguing analysis and design ahead of time. The systems could be controlled remotely if needed.

There is yet hope. Bringing more of the design process in-house is necessary to successfully apply the gaming concept to AI inspiration and evaluation. Paying close attention to General Game Playing will accomplish this. Systems that can take runtime descriptions of any game and use them to play that game well without human input are called general game players. Put simply, the regulations will be revealed only when the games begin.

Regular players can't depend on game-specific algorithms like Deep Blue, who play exclusively in tournaments. Ultimately, it is the player's intelligence, not the programmer's, that determines how well they play games in general. Knowledge representation, thinking, and logical decision making are just a few areas that general game players need to excel in. It's important for these skills to complement one other.

On top of that, general game players, as opposed to specialised ones, need to be multi-talented. Games with complete or partial information, varying numbers of players, simultaneous or alternating play, with or without communication among the players, games in static or dynamic worlds, games as simple as Tic Tac Toe and as complex as Chess should be within their capabilities. There is a basic abstract framework that all games have, even if General Game Playing covers a wide range of games. The environment in which each game is played has a limited number of states, including an initial state and one or more terminal states. There is a fixed, finite number of players in every game, and

there is a finite number of actions that each player can do in any given state. Additionally, there is a goal value connected with each player's actions in each state.

A game can be seen as a state graph, similar to the one depicted below, because of this shared structure. Here we have a one-player game with eight states ($s_1, \dots, s_8$).

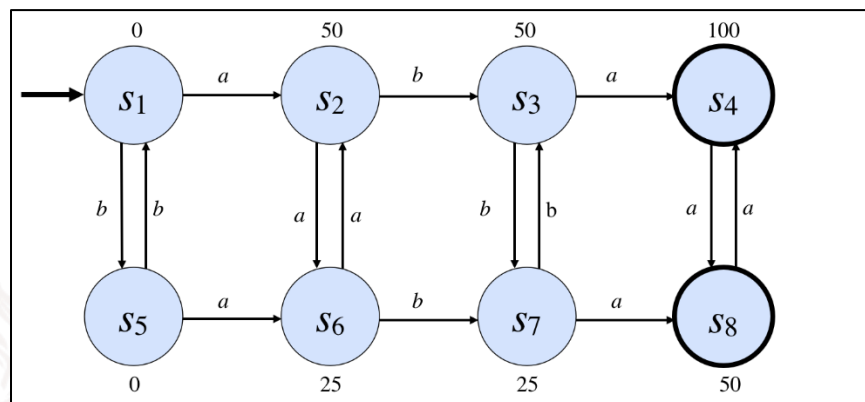


Figure 1: Representation of one player game with 8 states

This graph's arcs represent the game's transition function. When a player performs action a while the game is in state s_1 , for instance, the game will transition to state s_2 . The game will go to state s_5 if the player performs action b when the game is in state s_1 .

In this scenario, s_1 , there is only one starting state for the game. Nevertheless, s_4 and s_8 are just two of the many possible terminal states.

Each state's numerical value represents the utility that state provides to the player. Only in terminal states may players get those scores. Nevertheless, they are available for every state and, in certain games, they show how far you've come.

We can make two adjustments to this model to make it work for games with more than one participant. (1) All players in the game have their rewards annotated for each state. (Participants may receive the same or varied prizes.) (2) We identify the current player in control, or whose turn it is to play, for each state. Turns do not necessarily have to be consecutive; in fact, it is possible for one player to obtain multiple turns in a succession. (Since we are aiming for simplicity here, we will not get into the complexities of extending this model to games involving simultaneous moves.)

The conventional extensive normal form definition of games as game trees is superseded by this way of thinking about games. Every node in a game tree is connected to the ones that come after it through arcs that represent the legal actions in that state of the game. Nodes in the game tree might represent

many states, but they might also represent the same state at different periods. (When multiple paths ultimately result in the same state, this phenomenon occurs.)

Although there are analyses that are better suited to lengthy normal form, state-based representations provide benefits in general game playing. A state-based model allows for a more concise description of games and, in many cases, reduces the computing cost of game analysis due to the lesser size of game graphs compared to game trees.



3. GAME DESCRIPTION

In theory, state graphs can be used to describe the games we're thinking about because they're all finite. Regrettably, not every situation calls for such clear depictions. These sets can be enormous, and the accompanying graphs even larger, even if the number of states and actions is finite. In chess, for instance, there are over 1030 distinct possible positions.

A huge number of states and actions can be defined in terms of a smaller number of more fundamental elements in most games because of the composite structure of states and actions. For instance, in chess, states are not solid but can be represented in various ways, such as by pieces, squares, columns, diagonals, rows, etc.

Games can be compressed into a smaller format than direct representation by taking advantage of this structure. To back this up, Game Description Language (GDL) uses a database metaphor for game states and logic to define concepts like legality, among other things.

Take a look at the Tic-Tac-Toe rules to see an example of GDL in action. Please keep in mind that the goal of this example is to provide a high-level overview of GDL and not a comprehensive tutorial. The next chapter provides a comprehensive breakdown of the language.

We begin with an enumeration of roles. There are two players - x and o.

a. `role(x)`

b. `role(o)`

The starting state will be described next. All the cells are empty, and the x-player is in charge.

`init(cell(1,1,b))`

`init(cell(1,2,b))`

`init(cell(1,3,b))`

`init(cell(2,1,b))`

`init(cell(2,2,b))`

`init(cell(2,3,b))`

`init(cell(3,1,b))`

`init(cell(3,2,b))`

`init(cell(3,3,b))`

`init(control(x))`

We will now define legality. In a blank cell, the controlling player may make a mark. Variables in GDL are symbols with a capital letter initial, while constants are symbols with a lower case initial. According to the "if" principle, the expression to the left of the :-operand is true if and only if the expressions to the right of it are also true.

```
legal(mark(M,N)) :- cell(M,N,b)
```

The game's update rules will be examined next. In the following state, the player's mark is removed from the cell that retains it, and the cell is marked by the player in control. On top of that, control changes hands every play. If x is the master in one state and o the slave in the next, then x loses power in both states. By the time x enters a state from which o exits, o will no longer be in charge.

```
mark(M,N) :: control(R) ==> cell(M,N,R) & ~cell(M,N,b)
```

```
mark(M,N) :: control(x) ==> ~control(x) & control(o)
```

```
mark(M,N) :: control(o) ==> ~control(o) & control(x)
```

Goals. If there is an x-line and no o-line, the x player earns 100. If the line contains only o's and no x's, it receives no points. It receives 50 points if not. Similar benefits accrue to the o player. Below, we define the line relation.

```
goal(white,100) :- line(x) & ~line(o)
```

```
goal(white,50) :- ~line(x) & ~line(o)
```

```
goal(white,0) :- ~line(x) & line(o)
```

```
goal(black,100) :- ~line(x) & line(o)
```

```
goal(black,50) :- ~line(x) & ~line(o)
```

```
goal(black,0) :- line(x) & ~line(o)
```

Ideas that provide context. A line can be either a diagonal, a column of identical marks, or a row of them. When you see three marks in a row, it means they share the same first coordinate. Analogous definitions are given for the column and diagonal relations.

```
line(Z) :- row(M,Z)
```

```
line(Z) :- column(M,Z)
```

```
line(Z) :- diagonal(Z)
```

```
row(M,X) :- cell(M,1,X) & cell(M,2,X) & cell(M,3,X)
```

```
colun(N,X) :- cell(1,N,X) & cell(2,N,X) & cell(3,N,X)
```

```
diagonal(X) :- cell(1,1,X) & cell(2,2,X) & cell(3,3,X)
```

```
diagonal(X) :- cell(1,3,X) & cell(2,2,X) & cell(3,1,X)
```

End of contract. The game ends when there is a row of identical marks or when there are no more empty cells; in other words, when there are no blank cells.

terminal :- line(x)

terminal :- line(o)

terminal :- ~open

open :- true(cell(M,N,b))

It is easy to expand GDL to define partial information games, even though it is originally developed for complete information games. Sadly, processing the generated descriptions is more expensive and produces more verbose results. It is easy to expand GDL to define partial information games, even though it is originally developed for complete information games. Sadly, processing the generated descriptions is more expensive and produces more verbose results.

4. GAME MANAGEMENT AND PLAYING

a. Management:

The term "game management" refers to the steps used to oversee the running of a game in an informal gaming environment. Given that the focus is on managing specific matches rather than games as a whole, the term "match management" is more appropriate. Unfortunately, the term "Game Management" appears to be widely used, so we will have to stick with it.

Basically, this is how a game is run. The Game Manager will initially notify all players via start message to begin the match when they get a request to do so. When the game starts, it mimics the outcomes by sending play messages to every player. Until the game ends, this portion of the process is repeated. After then, every player gets a stop signal from the manager.

Included in the start message are the following details: the name of the match, the player's assigned role (such as white or black in chess), a formal description of the game (in GDL), and the start and play clocks for the match. Play will commence at the stroke of midnight, as shown on the start clock. Following the start of play, the amount of time each player has to complete each move is determined by the play clock.

Each participant receives a start message and uses that time to set up its data structures and do any analysis it sees fit. Once it's prepared to play, it notifies the Game Manager. The game manager has already sent out the start message and is now waiting for the players to respond. Play begins as soon as the Game Manager receives these responses OR the start clock runs out.

The Game Manager notifies the players of each step's progress through play messages. The message contains details regarding what each player did in the previous step. (The argument is meaningless in the initial stage.) Players attempt to plan their next movements as soon as they get a play message. The match's play clock will indicate how much time the controlling player has to respond.

The Game Manager is patiently waiting for the players to respond. When the timer runs out and the player in charge doesn't react, the game manager chooses a legal move at random. Whatever happens, after the player in control responds or the play clock runs out, the game manager decides what to do next by executing the requested move or the legal move it has decided for the player in control. After then, it determines if the game is over by evaluating the termination condition. The procedure

continues when the game management communicates its assumed move to all participants if the game is still ongoing.

When a game is over, the Game Manager notifies all players via a stop message that includes the last move made by the player in charge. In order to prepare for the match, players can clear out any data structures using the stop message. To help players with learning components make the most of their experience, we provide information about past plays. Once all players have been stopped, the game manager will calculate their prizes, add this data to the play history in its database, and then turn off the system.

Being able to use a formal description of a game to effectively play the game is far different from just having that description. Here we'll take a look at a few issues with developing general game players and talk about how to fix them.

First, we'll go over automated reasoning. Obviously, a player needs to be able to undertake some automatic reasoning since game descriptions are provided in logic.

Several options are available in this case. The game description can be used in an interpretive manner by the player throughout the game. (2) It is able to interpret the description by mapping it to a new representation. (3) The description can be used to create a game-specific program. This is essentially programming that runs itself.

The bright side is that Logic has robust and publicly accessible reasoners. Unfortunately, these reasoners do not address the fundamental issues of playing games in general, which remain constant regardless of the rules representation: namely, how to deal with uncertainty and limited resources.

A one-player game with a limited number of states and actions is considered the simplest type of game. Conventional AI planning methods work well in these situations. Finding a path from the current state to a desired end state can be accomplished by searching forward or backward, depending on the shape of the search space. I wish it were that easy for all games.

The first issue is the inherent unpredictability of games featuring more than two players. Remember that other players' activities can affect your current game state, and that no player can predict what other players will do. Then again, there are situations where a player can figure out how to reach a desired state regardless of what the other players do. Yet, instances like these are extremely unusual. The need to construct conditional plans, wherein a player's further moves are dictated by its past moves and the moves of its opponents, arises more frequently. More intricate methods of planning are required in these instances.

b. Game Playing:

Regrettably, there are instances where even this falls short. Not even a conditional plan can be guaranteed in certain situations. One example of this type of game is Tic-Tac-Toe. Winning is possible, but there is no surefire strategy. In certain cases, the best course of action is unclear. If you want to win, all you have to do is make a move and cross your fingers that the other players do the same. The issue of what steps to take before reaching this state remains unanswered by this approach, though. Reasoning based on probabilities is an option. The lack of a rational method for choosing a probability distribution for the other players' actions makes this far from ideal. Another strategy, often used in games with strong competition, is to make actions that make the other players seek more, increasing the likelihood that they will make a mistake due to time constraints.

Boundaries on available resources add another layer of complication. There are about 5,000 unique states in tic-tac-toe. Even though it's big, this is doable. There are over 1,030 different chess variations. Because it is finite, a state space of this size is completely searchable in theory but not in reality. In addition, players often have to choose acts without fully understanding their merit due to the time constraints imposed by most games.

Partial search, in which the game tree is examined to a specific depth, evaluated for possible outcomes at that point, and actions chosen accordingly, is the normal strategy in such instances. The availability of a non-terminal state evaluation function that is approximately monotonic in the real likelihood of success is, of course, a prerequisite for this approach. Although programmers can pre-implement evaluation methods for certain games like chess, this is not feasible for generic game playing due to the unknown structure of the game. Instead, the player needs to do some in-game analysis to figure out what an effective evaluation function is.

Another way to handle large numbers is by abstracting them. To make a state graph more abstract, one can sometimes reformulate it such that any solution to the abstract problem is a solution to the whole state graph. A good evaluation function for the whole graph or a guaranteed solution may be found in such instances. Additional research is required, however some researchers have suggested approaches in this direction.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the first step in managing a game effectively? A. Choosing a winner B. Understanding the rules C. Controlling the center D. Practicing regularly
2	Which of the following is NOT a component of game management? A. Rule enforcement B. Time tracking C. Random chance D. Turn management
3	Why is fair play important in game playing? A. To win faster B. To reduce thinking time C. To ensure a balanced and enjoyable game D. To confuse opponents
4	In a two-player game, what does taking alternate turns promote? A. Delay in the game B. Strategic imbalance C. Equal opportunity D. Random play
5	Which of the following is most essential to play games successfully? A. Memorising opponents B. Understanding and applying rules C. Avoiding all strategies D. Playing with multiple players only

5. TYPES OF GAMES

Games can be classified into various types based on how decisions are made, the role of chance, and the level of strategic thinking involved. Understanding these categories helps in designing intelligent agents in AI that can act optimally in different gaming environments.

A. Turn-Based Games

Turn-based games are those in which players take turns to make moves. Only one player acts at a time, and the game alternates between players in a sequential manner.

➤ Key Features:

- Moves are made in turns
- Players often have full knowledge of the game state
- Strategy and planning are crucial
- Games end when a goal or win condition is met

➤ Examples & Explanation:

1. Tic-Tac-Toe:

- Simple 3x3 grid game for two players (X and O)
- Players take turns placing their symbol to get three in a row
- Perfect for learning Minimax algorithm in AI

2. Chess:

- Highly strategic board game with complex movement rules
- Players alternate turns to move their pieces
- Used in AI to develop deep search trees and heuristic evaluation

3. Checkers:

- Played on an 8x8 board with diagonal moves and captures
- Turn-based and fully observable
- Often used to study AI strategies for endgame planning

➤ AI Relevance:

Turn-based games are commonly used in AI because they allow for modeling using state-space search, adversarial decision-making, and game trees.

B. Chance-Based Games

Chance-based games are games in which luck or randomness plays a key role in the outcome. Player skill may matter, but outcomes are also affected by elements beyond control.

➤ Key Features:

- Unpredictability due to random events (e.g., dice rolls, card draws)
- Often include elements of excitement or surprise
- Less emphasis on deep strategy
- Can be used to explore probabilistic reasoning in AI

➤ Examples & Explanation:

1. Snakes and Ladders:

- Progress is based on dice rolls
- Players cannot control or predict movement
- No strategy is involved; it's purely based on luck

2. Dice Games (e.g., Ludo, Pig):

- Dice determine the number of moves
- Players must make decisions after chance outcomes
- AI can use expected value and probability-based decision trees

3. Card Games like Uno:

- Shuffle and draw mechanics introduce randomness
- Some level of planning is possible, but luck dominates
- AI must learn to adapt to uncertain environments

➤ AI Relevance:

Chance-based games introduce non-determinism, useful for teaching AI concepts like:

- Stochastic modeling
- Bayesian decision theory
- Handling incomplete information

C. Strategy Games

Strategy games require planning, foresight, and tactical thinking. Players must consider the consequences of their moves and anticipate the opponent's actions.

➤ Key Features:

- Strong emphasis on player decision-making
- Low or no randomness (mostly deterministic)
- Can be zero-sum or cooperative
- Often involve multiple levels of planning (short-term and long-term)

➤ Examples & Explanation:

1. Chess:

- Mentioned again because it's one of the most strategic games
- Players think many moves ahead
- AI agents use Minimax with Alpha-Beta Pruning, heuristics, and evaluation functions

2. Connect Four:

- Two players take turns dropping discs into columns
- Objective: Get four in a row vertically, horizontally, or diagonally
- AI must analyse board configurations and block opponent moves

3. Sudoku:

- Single-player logic-based number placement game
- Strategy involves elimination and pattern recognition
- AI uses constraint satisfaction techniques

➤ AI Relevance:

Strategy games are essential for:

- Developing problem-solving AI agents
- Implementing rule-based reasoning
- Practicing adversarial search
- Training with reinforcement learning

Type of Game	Decision Style	Role of Luck	AI Concepts Involved
Turn-Based	Sequential turns	Low	Minimax, Game Trees, Adversarial Search
Chance-Based	Random + decision	High	Probability, Stochastic Models, Expected Utility
Strategy	Full decision-making	None or Low	Planning, Heuristics, Constraint Satisfaction, Learning

Table 1: Summary table for different types of gaming

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which game is an example of a chance-based game? A. Chess B. Sudoku C. Snakes and Ladders D. Connect Four
7	Turn-based games involve: A. Simultaneous play B. Timed races C. Players taking turns D. Pure luck
8	Strategy games typically require: A. Dice rolls B. Card shuffling C. Planning and foresight D. Guesswork
9	Which of the following is NOT a turn-based game? A. Chess B. Tic-Tac-Toe C. Uno D. Running race
10	Sudoku is best categorised as a: A. Chance-based game B. Multiplayer game C. Strategy game D. Turn-based game

6. BASIC GAME STRATEGIES

Effective strategies are essential for improving gameplay—whether you're a human player or designing an AI agent. The following strategies form the foundation for intelligent decision-making in both simple and complex games.

A. Know the Rules

Understanding the rules of a game is the first and most crucial step toward success. Rules define what is allowed and what is not, what moves you can make, and how the game can be won or lost.

➤ Why It's Important:

- Rules set the boundaries for decision-making.
- Missing or misinterpreting a rule can lead to poor choices or even disqualification.
- AI agents are coded based on rules, so a clear definition helps in algorithm design.

➤ Examples:

- In Tic-Tac-Toe, knowing that the game ends in a draw if no one gets three in a row is important for blocking strategies.
- In Chess, understanding the special rule of “castling” or “en passant” can change your approach dramatically.

AI must be programmed with complete knowledge of the rules to simulate intelligent play. This is part of the knowledge base in rule-based systems.

B. Think Ahead

Good players don't just react—they anticipate. Thinking ahead involves visualising the possible moves you or your opponent might make in future turns.

➤ Why It's Important:

- Helps in planning sequences of actions
- Prevents falling into traps
- Allows players to create winning opportunities and block threats

➤ Examples:

- In Chess, players often plan 3–5 moves in advance to control the board.
- In Tic-Tac-Toe, if your opponent has two symbols in a row, you must block them immediately—even if you could create your own row elsewhere.

This is the core of game tree search. Algorithms like Minimax simulate all possible future game states to choose the best move.

C. Control the Center

Controlling the center of the board is a common strategy in many games, particularly grid-based or board games. The center often provides the most flexibility and access to multiple paths or plays.

➤ Why It's Important:

- Offers maximum mobility in future moves
- Allows greater control over the opponent's actions
- Can lead to faster offensive or defensive options

➤ Examples:

- In Chess, central squares (d4, d5, e4, e5) allow your pieces to cover more area.
- In Tic-Tac-Toe, the centre square gives you access to 4 different winning paths—horizontal, vertical, and two diagonals.

Game-playing AIs often assign higher weights to centre control during evaluation. For example, in heuristic functions, central positions may get better scores.

D. Practice Makes Perfect

Like any skill, game strategy improves with experience. Practising helps players recognise patterns, learn from mistakes, and develop instinctive responses.

➤ Why It's Important:

- Reinforces pattern recognition
- Builds familiarity with common tactics
- Improves reaction time and confidence

➤ Examples:

- In Sudoku, experienced players quickly spot number-placement opportunities by remembering familiar configurations.
- In Chess, frequent players remember common openings like the "Sicilian Defense" or "Queen's Gambit".

Modern AI agents (like AlphaZero) learn through self-play, practicing millions of games to discover optimal strategies. This falls under reinforcement learning, where agents learn from trial and error.

Strategy	Purpose	Impact in AI
Know the Rules	Sets the framework for valid actions	Forms the knowledge base of AI agents
Think Ahead	Helps in planning and avoiding mistakes	Implemented via Minimax, Game Trees
Control the Center	Maximises options and positional strength	Central positions are prioritised in evaluation
Practice Makes Perfect	Builds instincts and pattern recognition	Achieved through self-play in reinforcement learning

Table 2: Summary of the game strategies

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	Why is "thinking ahead" important in games? A. It makes the game shorter B. It helps predict opponent's moves C. It removes rules D. It reduces complexity
12	Controlling the center in board games usually provides: A. More options for movement B. Faster win by luck C. Random placement D. Less complexity
13	Which of these helps in improving game skills over time? A. Luck B. Watching others C. Practicing regularly D. Avoiding play
14	What should you do before making a winning move? A. Celebrate B. Block opponent's winning chance C. Quit the game D. Break the rules

15 Understanding every rule in a game helps a player to:

- A. Lose gracefully
- B. Get confused
- C. Play strategically
- D. Depend on luck



7. SAMPLE GAME: TIC-TAC-TOE

Tic-Tac-Toe is a classic two-player game played on a 3×3 grid. The goal is simple:

Be the first to align three of your marks (either X or O) in a row—horizontally, vertically, or diagonally.

Basic Rules

1. Two players take turns: one uses X, the other uses O.
2. Players place their symbol in an empty cell on the 3×3 grid.
3. The game ends when one player gets three in a row or when all cells are filled.
4. If no player wins and the grid is full, it's a draw.

Why It's Important in AI

- Tic-Tac-Toe is a perfect information, zero-sum, deterministic game.
- It's simple enough to analyse completely, making it ideal for teaching AI concepts like:
 - Game trees
 - Minimax algorithm
 - Heuristics and evaluation
- It allows exploration of winning strategies, draw scenarios, and optimal decision-making.

Strategy Tips

1. Always Try to Go First
 - Reason: The player who moves first (usually “X”) has a slight advantage if they play optimally.
 - They control the pace of the game and can force a win or draw.
 - In AI terms, the first player has a better position in the game tree root.

Example: Choosing the center on your first move gives you access to the most winning paths.

2. Take the Center Square if It's Free
 - Reason: The center is the most valuable position on the board.
 - It allows the player to participate in four potential win paths:
 - Vertical
 - Horizontal
 - Two diagonals

Example: From the center, you can respond to any edge or corner play effectively, giving you more control.

3. Block Your Opponent Before Trying to Win

- Reason: Defensive play is critical when the opponent is about to complete a row.
- Always check if the opponent has two in a row, and immediately block the third spot—even if you have a chance to create your own line elsewhere.

Example: If your opponent has two O's in a row, and you fail to block the third, you lose on their next move.

- In AI, this is implemented using the Minimax algorithm, where the AI chooses moves that maximise its chance of winning while minimising the opponent's.

AI Agent Playing Tic-Tac-Toe

AI can be programmed to:

- Evaluate all possible moves
- Use the Minimax algorithm to simulate future game states
- Choose optimal moves that lead to a win or a draw
- Never lose, and always win if the opponent makes a mistake

Learning Outcome

Studying Tic-Tac-Toe helps in:

- Understanding basic game theory
- Practicing logic-based decision-making
- Learning how simple rules can lead to complex strategy
- Implementing AI agents that use search trees and decision rules

Aspect	Detail
Game Name	Tic-Tac-Toe
Grid Size	3 × 3
Objective	Get three Xs or Os in a row
First Move Tip	Always try to go first and take the center
Key Strategy	Block opponent before trying to win
AI Techniques	Game Tree, Minimax, Evaluation Functions

Table 3: Description of the game

Learning through Games

Games are more than just entertainment—they are powerful tools for developing cognitive and social skills. Whether playing traditional board games, digital games, or AI-driven simulations, players engage in experiences that promote learning by doing. Many educational methods now use games to teach logic, problem-solving, communication, and adaptability.

What Do Games Teach?

Let's explore the skills and values that games help develop:

1. Logic and Critical Thinking

Many games require players to analyse situations, identify patterns, and make reasoned decisions. These mental actions strengthen logical thinking and deductive reasoning.

➤ Examples:

- Tic-Tac-Toe: Teaches strategic planning and blocking opponent moves.
- Sudoku or Chess: Enhances pattern recognition, decision trees, and long-term planning.
- Puzzle games: Require logical sequencing and solution paths.
- Helps in learning problem-solving algorithms.
- Teaches how to break down complex tasks into smaller, manageable actions.

2. Focus and Concentration

Games demand attention to detail and sustained concentration. Players must follow rules, monitor moves, and anticipate outcomes—all of which train the brain to stay focused.

➤ Examples:

- Memory games improve short-term recall and attention span.
- Strategy games like Chess or Go require thinking several moves ahead.
- Improves performance in studies and tasks requiring deep work.
- Encourages mindfulness and reduced distraction.

3. Teamwork and Communication (in Multiplayer Games)

Multiplayer or team-based games promote collaboration, leadership, and effective communication. Players learn to coordinate, share information, and solve problems together.

➤ Examples:

- Board games like Monopoly teach negotiation and conflict resolution.

- Co-op video games (like Minecraft, Overcooked, or Among Us) foster division of roles and mutual support.
- Esports or sports simulations teach trust-building and leadership.
- Simulated multi-agent environments are used to model cooperative behavior, useful in swarm robotics and social AI.

4. Adaptability and Decision-Making Under Pressure

Games often present changing environments, random elements, or unexpected challenges. This teaches players how to adapt quickly, change tactics, and make good decisions in uncertain conditions.

➤ Examples:

- Card games like Uno or Poker involve unpredictability and chance.
- Role-playing games (RPGs) require adapting to plot twists and new challenges.
- Real-time strategy games involve quick thinking and dynamic resource management.

➤ Importance in AI and Life:

- Trains for real-world problem-solving, where conditions aren't always predictable.
- Encourages resilience and flexible thinking.

Why Play a Variety of Games?

Playing different types of games exposes individuals to:

- Different types of thinking (logical, creative, tactical)
- Different types of interaction (competitive, cooperative)
- Varied challenges (fixed rules vs. dynamic gameplay)

Encouragement: "Don't just stick to one type of game. Try puzzles, strategy, role-play, and even chance-based games to grow your mind in different directions."

In the Context of AI Education

Games are widely used to teach and train AI systems:

- Reinforcement Learning: Agents learn by interacting with game environments.
- Simulations: Used in autonomous driving, robotics, and decision-making training.
- Game-based environments (like OpenAI Gym) are used for AI model development.

8. SUMMARY

- Game playing is an important area in both recreational and educational contexts, as well as a foundational concept in Artificial Intelligence (AI). This topic explores how different types of games operate, the strategies behind them, and the skills they develop in players.
- Games can be broadly categorised into turn-based games (like chess and tic-tac-toe), chance-based games (like snakes and ladders or card games), and strategy games (such as Connect Four or Sudoku).
- Each type involves different levels of thinking and interaction, teaching players logic, planning, and adaptability. A key aspect of game playing is understanding the rules thoroughly, as even minor details can influence gameplay decisions.
- Thinking ahead is crucial, especially in games like chess, where anticipating an opponent's move can determine success.
- Controlling key positions—such as the center in a board game—can give players a strategic edge.
- Regular practice allows players to recognise patterns and improve their instincts over time.
- Sample games like Tic-Tac-Toe are often used in classrooms and AI to demonstrate concepts such as decision trees and the Minimax algorithm. Through these games, learners develop cognitive skills like focus and logical reasoning, as well as social skills such as teamwork and communication in multiplayer settings.
- Ultimately, exploring basic games and their strategies not only enhances problem-solving ability but also serves as a gateway to understanding intelligent decision-making processes, both for humans and AI systems.

9. GLOSSARY

Game	A structured activity or competition involving rules, goals, and interaction, designed for entertainment, learning, or problem-solving. In AI, games are often used to model intelligent decision-making.
Turn-Based Game	A type of game where players take turns to make their moves. Each player acts one at a time, allowing strategic planning. Examples: Tic-Tac-Toe, Chess, Checkers.
Chance-Based Game	Games where the outcome is heavily influenced by randomness or luck rather than pure skill or strategy. Examples: Snakes and Ladders, Dice games, Card games.
Strategy Game	Games that require planning, logical thinking, and foresight to win. Players must make decisions based on anticipated future moves. Examples: Chess, Sudoku, Connect Four.
Game Strategy	A plan or method developed by a player to achieve success in a game. Strategies often involve anticipating the opponent's moves and making optimal decisions accordingly.
Rules	A set of defined instructions or guidelines that govern how a game is played. Understanding rules is fundamental to effective gameplay and fair competition.
Objective	The goal or winning condition of a game. It defines what players must achieve to win, such as forming a line in Tic-Tac-Toe or reaching a target score.
Logic	The reasoning and systematic thinking used to analyse situations, solve problems, or plan moves. Games often enhance a player's logical thinking ability.
Focus	The mental concentration and attention are needed to track the game's progress, anticipate moves, and avoid mistakes. Essential in both fast-paced and turn-based games.
Teamwork	Collaborative play among multiple players where success depends on coordination and shared strategies. Found in multiplayer or co-operative games.

Adaptability	The ability to change strategies or approaches based on new information or game developments. Crucial for handling uncertainty or unexpected moves by opponents.
Practice	Repeated play or training that helps a player improve pattern recognition, decision-making, and reaction time. Increases familiarity with the game's mechanics and strategies.
Center Control	A strategy in many board games where occupying the central space provides greater flexibility and more winning opportunities. Example: In Tic-Tac-Toe and Chess, the center often provides access to more moves.
Blocking	A defensive tactic used to prevent the opponent from achieving their goal, such as stopping them from forming a winning row in Tic-Tac-Toe.
Tic-Tac-Toe	A simple turn-based game played on a 3×3 grid where two players take turns marking X or O. The goal is to get three in a row. Used widely in AI education for teaching decision-making.
Game Tree	A conceptual diagram that represents all possible moves in a game from the current state to terminal states (win/loss/draw). It is used in AI to evaluate and choose optimal paths.
Minimax Algorithm	A common decision-making algorithm in AI for two-player games. It aims to maximise the player's minimum gain (hence "minimax") by assuming the opponent plays optimally.
Heuristics	Rules or methods used to make decisions quickly when it's not practical to compute every possible outcome. Used in complex games where exhaustive search is too slow.
Zero-Sum Game	A situation in which one player's gain is exactly balanced by the other player's loss. Most competitive two-player games are zero-sum.
Educational Games	Games designed or used to teach specific skills such as math, logic, or teamwork. They are often interactive and goal-based, making learning more engaging.

10. TERMINAL QUESTIONS

1. What are the main types of games and how do they differ from each other?
2. How does understanding game rules affect strategy and performance?
3. What are some basic strategies that can be applied across most games?
4. Why is Tic-Tac-Toe considered a good example of a simple strategy game?
5. What role does randomness play in chance-based games, and how does it affect gameplay?
6. How do games help in developing real-world skills such as logic and adaptability?
7. Explain how the concept of 'controlling the center' provides a strategic advantage in board games.
8. Why is the first move important in games like Tic-Tac-Toe and how can it be used effectively?
9. How does practice improve gameplay, and can it replace strategy?
10. Why is game playing important in the field of Artificial Intelligence (AI)??

11. ANSWERS

Self-Assessment Questions

1. C. AND-OR Graph
2. B. AO* handles AND-OR relationships; A* does not
3. B. A choice between multiple path
4. C. All child nodes must be solve
5. C. $f(n) = g(n) + h(n)$
6. D. Minimum cost path estimate ($f(n)$)
7. C. All children must be expanded
8. D. Heuristic search and AND-OR reasoning
9. A. From goal nodes to root
10. C. It uses heuristics to guide the search
11. C. Actions
12. B. Assigning values to variables without violating constraint
13. A. Set of possible values for each variable
14. B. Sudoku puzzle
15. C. To eliminate inconsistent values from domains of unassigned variables
16. D. Minimum Remaining Values (MRV)
17. B. No digit may repeat in any row, column, or 3x3 block
18. C. Constraint propagation
19. C. Selects variable with most constraints on other unassigned variables
20. B. Forward checking

Terminal Questions Answers

1. Games can be broadly classified into three types:
 - **Turn-Based Games:** These are games where players take alternate turns to make decisions or moves. Each player has time to think before acting. Examples include Tic-Tac-Toe, Chess, and Checkers.
 - **Chance-Based Games:** In these games, outcomes are influenced heavily by luck or randomness. The players have limited control over the final result. Examples are Snakes and Ladders, Dice games, and card games like Uno.

- **Strategy Games:** These require critical thinking, foresight, and decision-making skills. Players must plan several steps to outmanoeuvre their opponent. Examples are Chess, Sudoku, and Connect Four. (Refer to section 2 for more information)

2. Answer:

Knowing the rules thoroughly is the foundation for effective gameplay. Even a small overlooked rule can significantly change the course of a game. For example, in Chess, knowing how each piece moves or the castling rule allows players to use more advanced tactics. Rules define what actions are legal and what strategies are possible. Therefore, without understanding them, a player cannot optimise moves or make informed decisions, leading to mistakes and missed opportunities. (Refer to section 2 for more information)

3. Answer:

Some universal strategies that can improve performance in most games include:

- **Think Ahead:** Anticipating what the opponent might do next helps in planning better moves.
- **Control the Center:** Especially in board games like Chess or Tic-Tac-Toe, controlling the central space increases influence and options.
- **Block Before Winning:** In competitive games, stopping the opponent's winning move can be more important than advancing your own.
- **Practice:** Repeated playing helps develop instincts, recognise patterns, and reduce errors. These strategies help improve consistency and decision-making. (Refer to section 7 for more information)

4. Answer:

Tic-Tac-Toe is easy to learn but introduces essential game-playing concepts like turn-taking, pattern recognition, and blocking strategies. It is played on a 3×3 grid, where the objective is to form a line of three Xs or Os. Though simple, it requires players to think ahead and block the opponent while trying to win. Because of its limited possibilities, it's also widely used in AI education to demonstrate decision trees and algorithms like Minimax. (Refer to section 7 for more information)

5. Answer:

Randomness (e.g., dice rolls, shuffled cards) determines major outcomes in chance-based games. This reduces the impact of a player's skill and introduces unpredictability. For instance, in Snakes and Ladders, a player's success largely depends on the numbers rolled, not their decisions. This makes such games less strategic but more accessible and exciting due to their unpredictability.

Players must learn to adapt to luck and use it wisely when possible. (Refer to section 5 for more information)

6. Answer:

- Games simulate real-life decision-making under rules and constraints.
- Logic: Games like Sudoku or Chess enhance analytical thinking and problem-solving.
- Adaptability: As game conditions change, players must quickly adjust their strategies.
- Teamwork: Multiplayer games encourage communication and coordination.
- Focus: Fast-paced games train players to maintain attention and react quickly.

By engaging players in rule-based challenges, games nurture a wide array of cognitive and interpersonal skills useful in daily life and work. (Refer to section 6 for more information)

7. Answer:

In many grid-based or board games, occupying the center gives players more freedom of movement and more options to form winning combinations. For instance, in Tic-Tac-Toe, the center square connects to the most lines (4) compared to corners or edges. In Chess, controlling central squares allows greater mobility and influence over the board. Thus, dominating the center increases strategic flexibility and forces opponents to play reactively. (Refer to section 6 for more information)

8. Answer:

In games like Tic-Tac-Toe, the first player (X) has a slight advantage due to the ability to initiate the strategy and control key spaces like the center. Taking the center as the first move increases chances of forming a winning line and puts pressure on the opponent. Statistically, if both players play optimally, the game ends in a draw, but making the first move gives a higher probability of forcing a win or at least not losing. (Refer to section 7 for more information)

9. Answer:

Practice develops pattern recognition, reaction time, and familiarity with game mechanics. Over time, players begin to “see” better moves automatically. However, practice complements, not replaces, strategy. For instance, playing Chess without learning tactical patterns or strategic principles may limit progress. Practice helps internalise strategies and apply them more effectively, but without understanding game logic, practice alone will not guarantee success. (Refer to section 4 for more information)

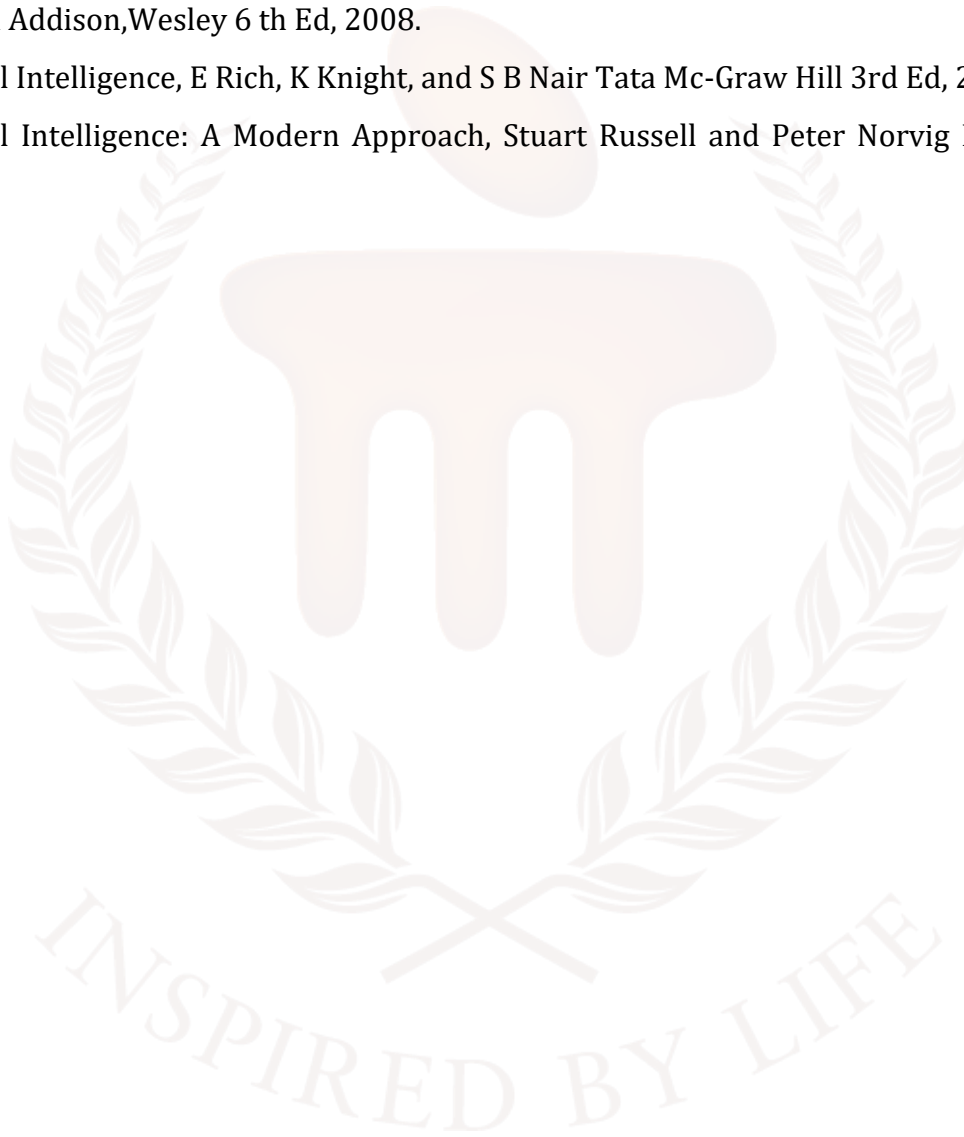
10. Answer:

Game playing provides a controlled, rule-based environment for AI to develop decision-making skills. Games like Tic-Tac-Toe, Chess, and Go have been used to test and improve AI algorithms like Minimax, Alpha-Beta pruning, and Reinforcement Learning. AI learns to predict outcomes, optimise moves, and adapt strategies—just as humans do. This research translates into real-world applications like autonomous driving, robotics, and finance where AI must evaluate options and make smart choices. (Refer to Section 3 for more information)



12. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

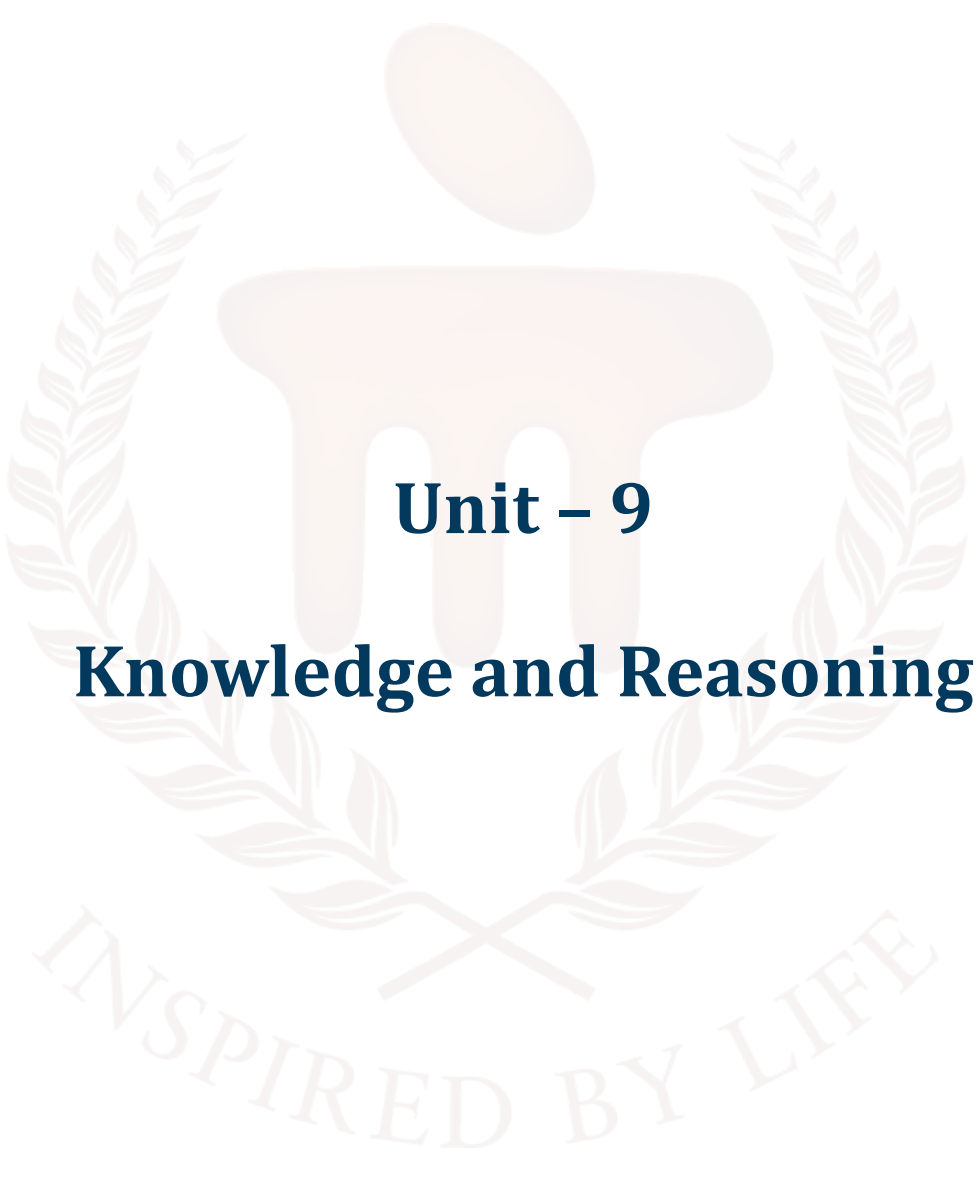


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109
ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING



Unit – 9

Knowledge and Reasoning

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Knowledge Representation and Reasoning	-	-	6 - 7
3	Knowledge Representation in AI	1	-	8 - 10
4	Core Methods of Knowledge Representation	-	1	11 - 13
5	The AI Knowledge Cycle	2	-	14 - 15
6	Types of Knowledge in AI	-	2	16 - 18
7	Applications of Knowledge representation in AI	-	-	19
8	Logical reasoning	-	-	20 - 23
9	Types of Logic used in AI	-	3	24 - 28
10	Basic concept of Propositional Logic	-	4	29 - 34
11	Summary	-	-	35
12	Glossary	-	-	36 - 37
13	Terminal Questions	-	-	38
14	Answers	-	-	39 - 41
15	References	-	-	42

1. INTRODUCTION

In previous unit, we had studied the fundamentals of game playing in artificial intelligence by exploring basic games and strategies. We started by examining common two-player, turn-based games such as tic-tac-toe, checkers, and chess, focusing on how these games are represented using game trees that map out possible moves and outcomes. We were introduced to key concepts such as game states, legal moves, and how AI systems make decisions by anticipating opponent moves and planning strategically. Central to this study will be the Minimax algorithm, a decision-making method that helps AI select the best moves to maximise its chances of winning. By understanding these basic strategies and algorithms, we had gained insight into how AI approaches game playing, manages complexity, and formulates effective plans in competitive settings.

In this unit, we will explore the foundational principles of how intelligent systems represent, process, and reason with knowledge. The focus will be on equipping you with the tools and techniques used in logical reasoning, a core aspect of artificial intelligence and decision-making systems. We will begin with an introduction to logical reasoning, where you'll understand how AI systems use logic to derive conclusions from known facts. You'll learn the importance of logic in AI, and how it helps in problem-solving, inference-making, and building knowledge-based systems.

Next, we will delve into the basic concepts of propositional logic, which is the simplest form of logic used to represent statements and their relationships. You will learn how to: Represent facts using propositional variables, Construct logical formulas using connectives (AND, OR, NOT, etc.), Understand the concepts of truth tables, tautologies, and logical equivalence, Apply inference rules like Modus Ponens, Modus Tollens, and more. By the end of the unit, you will be able to model knowledge using formal logic and perform basic reasoning tasks, laying a strong foundation for more advanced topics like predicate logic and automated reasoning systems.

1.1. Objectives

After studying this unit, you should be able to:

- Define key terms such as knowledge base, inference, and reasoning.
- Examine the effectiveness of different knowledge representation schemes
- Assess the limitations of various reasoning approaches
- Formulate a new reasoning strategy for a real-world AI problem using combined logic techniques.



2. KNOWLEDGE REPRESENTATION AND REASONING

The branch of artificial intelligence known as "Knowledge Representation and Reasoning" deals with teaching computers to interpret and apply material that is based on real-world examples in order to do complicated tasks. In KR&R, data is integrated into a knowledge base according to a set of rules for handling inference, guaranteeing data integrity and making it usable for decision-making. A cornerstone of AI, KR&R investigates the best practices for structured knowledge representation and the methods by which intelligent computers can reason with this representation to derive conclusions. In KR&R, Ontologies, Rules, and Semantics form its three main parts. Machines cannot interpret or manipulate knowledge meaningfully without all three components.

Components of Knowledge representation and reasoning

- a) **Ontologies:** Entities that represent concepts, establishing a system of knowledge by defining the key ideas of a domain.
- b) **Rules:** Control the reasoning structure that AI systems use to arrive at conclusions.
- c) **Semantics:** Data can be better understood with the help of semantics, which provide meaningful interpretation and relationship.

1. Ontologies

Ontologies provide a formal and structured framework for representing knowledge about a particular domain. They define:

- Concepts (e.g., Student, Course, Instructor),
- Categories (e.g., LivingBeing, Event),
- Relationships (e.g., enrolledIn, teaches),
- Properties or attributes of entities (e.g., age, name, credit hours).

For systems and people to have a common understanding, ontologies are useful for building a shared vocabulary and a logical structure. Semantic web, biomedical informatics, and natural language processing are just a few of the many areas that rely on them to facilitate collaboration and the sharing of information.

2. Rules

An information system's reasoning and inference processes are defined by its rules. They make it possible for computers to draw conclusions, make choices, and execute actions based on previously

acquired information. The usual format for rules is:

- If-Then statements (e.g., If a student has completed all required courses, then they are eligible for graduation),
- Constraints or conditions (e.g., No student can enroll in two classes that occur at the same time),
- Inference mechanisms like forward chaining (data-driven) or backward chaining (goal-driven).

These rules help the system reason logically, handle uncertainty, and ensure that decisions align with defined business or domain logic.

3. Semantics

The field devoted to the study of knowledge and its interpretation is known as semantics. It makes sure that computers can interpret data in context and not merely interpret it symbolically. Strictly speaking:

- Symbols are associated with real-world meanings,
- Relationships between entities become intelligible, not just syntactic,
- Machines can interpret ambiguity, resolve conflicts, and make inferences that align with human understanding.

For example, recognising that “car” and “vehicle” refer to related but different concepts, or that “doctor” implies someone who may treat “patients,” relies on semantic understanding.

Semantics is especially critical in areas like natural language understanding, semantic search, and machine learning models that rely on contextual data.

Intelligent systems are able to mimic human comprehension and decision-making because ontologies organise knowledge, rules propel reasoning, and semantics provide meaningful interpretation. To create AI applications that are resilient, understandable, and adaptable, a KR&R architecture must be well-integrated.

3. KNOWLEDGE REPRESENTATION IN AI

The process of encoding data about the environment into forms that AI systems can use to accomplish complicated tasks is called knowledge representation (KR) in artificial intelligence (AI). Machines are able to learn, reason, and make decisions using this approach because it structures data in a way that mimics human thinking.

The data is the lifeblood of artificial intelligence systems. Nevertheless, intelligence cannot be achieved from raw data alone. It is essential for AI to convert data into organised knowledge. KR accomplishes this by outlining specific rules for data organisation. In order to solve issues, make decisions, and learn from new experiences, AI systems rely on explicit representations.

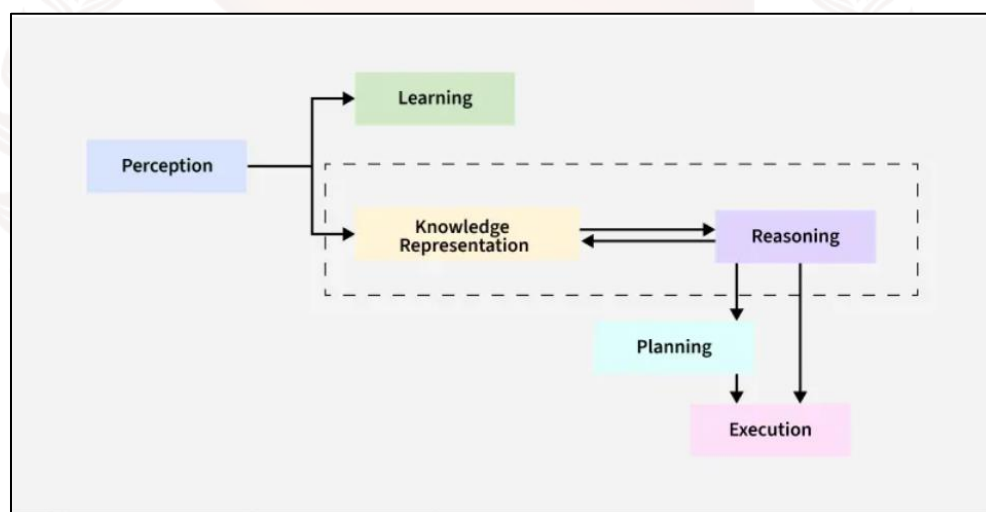


Figure 1: Knowledge Representation in AI

The above figure visually represents the relationship and flow between various components of an intelligent system, particularly emphasising the role of Knowledge Representation and Reasoning (KR&R) within the broader AI architecture. It illustrates how an intelligent agent operates by transforming raw sensory input into intelligent actions through the core use of Knowledge Representation and Reasoning. It highlights the interdependence of learning, planning, and reasoning in AI systems.

1. Perception

- This is the starting point of the intelligent system.
- It involves sensing the environment through various input sources (e.g., cameras, sensors, or user inputs).
- The data gathered from perception is raw and unstructured and needs to be processed.

2. Learning

- The perception data can be passed into a learning component.
- Here, the system can extract patterns, build models, or improve its performance over time.
- Learning updates the Knowledge Representation component by adding newly acquired information.

3. Knowledge Representation

- This is the core of the system, where structured information is stored.
- It acts as a central repository of facts, rules, and concepts about the environment or domain.
- It is bidirectionally connected to Reasoning, indicating that:
 - Reasoning draws upon structured knowledge to infer new facts.
 - Outcomes of reasoning (new conclusions) can update or modify the knowledge base.

4. Reasoning

- Reasoning allows the system to make inferences, draw conclusions, and solve problems based on the knowledge it possesses.
- It uses logic and rule-based mechanisms to simulate intelligent decision-making.
- Reasoning interacts with both Knowledge Representation and Planning.

5. Planning

- After reasoning, the system moves to the planning stage.
- It involves strategising a sequence of actions based on goals, constraints, and environmental factors.
- Planning helps determine how to act based on what the system knows and has reasoned.

6. Execution

- The final step is execution, where the system performs physical or digital actions based on the plan.
- This could mean moving a robot arm, responding to a user, or updating a database.

Intelligence and Knowledge Working Together

In artificial intelligence, knowledge and intelligence are interdependent:

- The Basis of Knowledge: Facts, regulations, and data are provided by knowledge (such as traffic laws for autonomous vehicles). It is essential for intelligence to have in order to take action.

- **Intelligence as application:** The ability to apply one's knowledge in order to solve issues is an example of intelligence in action (think of a robot that uses principles of physics to navigate a new environment).
- **Interdependence:** Adaptive intelligence is essential for the survival of our species, rendering static knowledge outdated. Knowledge is the antidote to intelligence; an AI that lacks a medical database, for instance, would be unable to make accurate diagnoses.
- **Synergy:** Strong knowledge bases and reasoning algorithms work together to form effective AI systems. For instance, ChatGPT is able to produce comprehensible text by integrating massive amounts of language data (knowledge) with transformer models (intelligence).



4. CORE METHODS OF KNOWLEDGE REPRESENTATION

1. Logic-Based Systems

Logic-based approaches model knowledge through the application of formal rules. For deterministic settings, these systems shine because of their emphasis on accuracy.

Propositional Logic

Uses logical operators such as AND, OR, and NOT to connect declarative statements (propositions) that represent knowledge. For example: "If it rains (A) AND the ground is wet (B), THEN the road is slippery (C)." Complex relationships are a challenge for its simplicity. Frequently use the structure of "IF condition THEN conclusion." An example of this would be a knowledge-based system that may include:

If something is both red and spherical, it could be an apple.

First-Order Logic (FOL)

Includes new concepts like quantifiers, predicates, and variables to expand propositional logic. All humans ($\forall x$) are mortal ($\text{Mortal}(x)$)," is one example of a statement that FOL can convey. While it does support complex reasoning, it is computationally intensive.

Analysing contracts for compliance is the job of legal AI technologies, which use logic-based rules.

2. Structured Representations

In order to simulate the way people classify data, these algorithms arrange knowledge hierarchically or via networks.

Semantic Networks

Map ideas and connections to a network of nodes. For example, "Dog" as an example; it connects to "Animal" through a "Is-A" relationship. Despite their lack of formal semantics, they simplify the understanding about inheritance.

Frames

Compose similar characteristics into organised "frames." Wheels, engine type, gasoline, and other such slots might be found in a "Vehicle" frame. Exceptions are a challenge for frames, but they do well with default reasoning.

Ontologies

Web Ontology Language (OWL) and similar standards can be used to define domain-level ideas, hierarchies, and relationships. Through the standardisation of terminology, ontologies enable semantic search engines and healthcare diagnoses.

To improve search accuracy and product categorisation, e-commerce platforms employ ontologies.

3. Probabilistic Models

By tying outcomes to probabilities, these systems deal with uncertainty.

Bayesian Networks

Model causal relationships using directed graphs. In this network, variables are represented by nodes, and conditional dependencies are shown by edges. As an example, by analysing maintenance records and current consumption, a Bayesian network may estimate when equipment will break down.

Markov Decision Processes (MDPs)

Decide in a systematic fashion in situations that are constantly changing. Through the evaluation of possible actions and incentives, MDPs assist robotic systems in navigating obstacles.

In order to foretell when storms will hit, weather prediction systems use probabilistic models that include data from sensors and past observations.

4. Distributed Representations

These days, artificial intelligence makes use of neural networks to capture hidden patterns in data by encoding information as numerical vectors.

- Embodies: Materialise concepts, pictures, or words as dense vectors. Word2Vec and similar word embeddings allow for semantic analysis by mapping synonyms to neighbouring vectors.
- Knowledge graphs: To represent entities (such as people or places) and the interactions between them, combine graph structures with embeddings. Google's Knowledge Graph improves search results by establishing connections between related ideas.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following is a method of knowledge representation in AI? a) Decision Trees b) Neural Networks c) Semantic Networks d) Graphical Models
2	Which knowledge representation technique uses if-then rules? a) Semantic Network b) Production Rules c) Frames d) Ontology
3	Frames in AI are used to represent: a) Events b) Heuristics c) Stereotyped situations d) Logic expressions
4	What does KR stand for in AI? a) Knowledge Rating b) Knowledge Representation c) Knowledge Recognition d) Knowledge Retrieval
5	Which of the following is not a core method of KR? a) Predicate Logic b) Rules c) Stacks d) Frames

5. THE AI KNOWLEDGE CYCLE

All AI systems go through the AI Knowledge Cycle, which includes acquiring, processing, using, and refining knowledge.

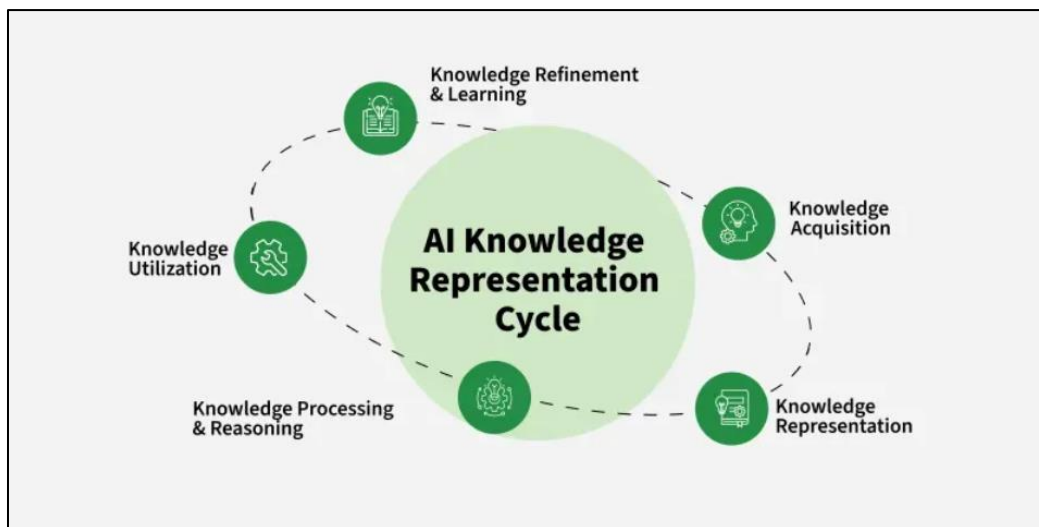


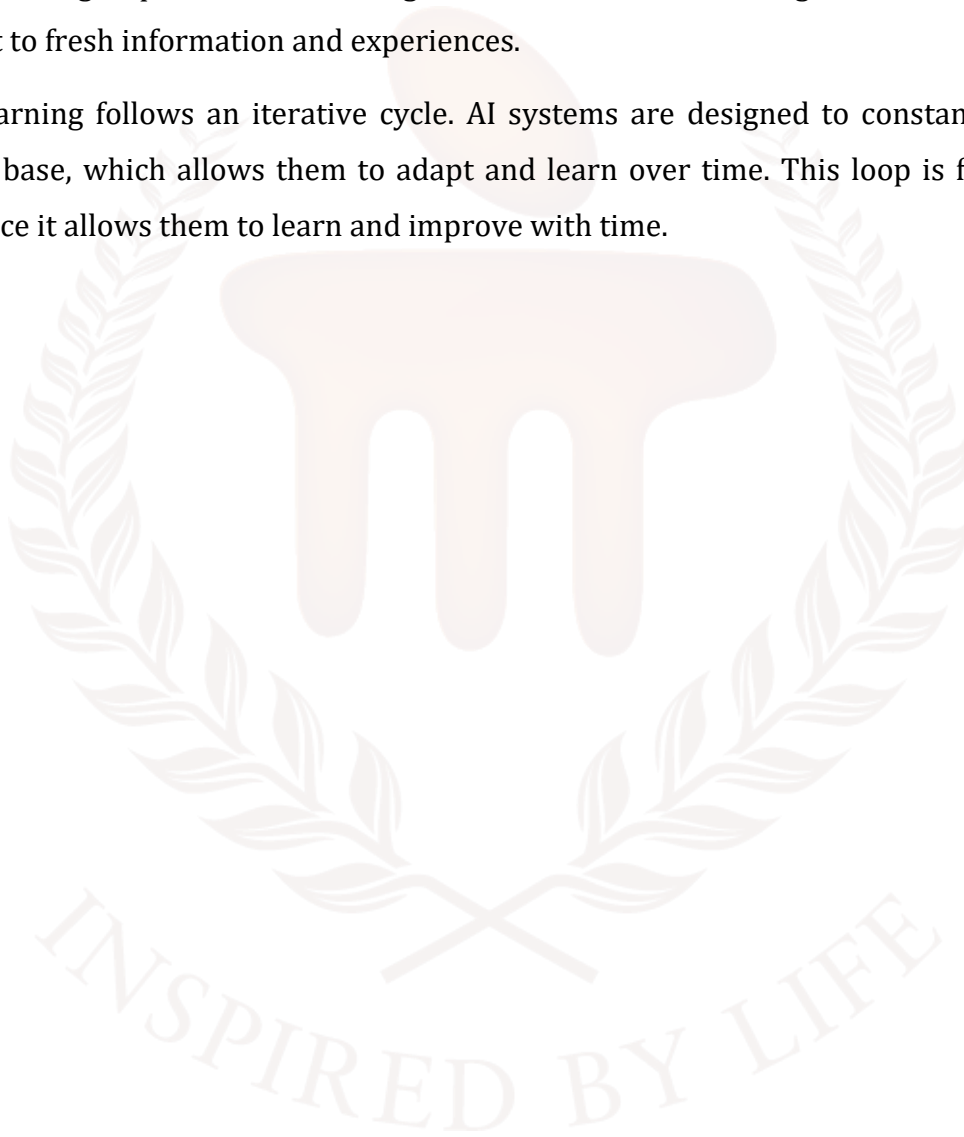
Figure 2: AI Knowledge Cycle

This iterative process guarantees that AI will continue to adapt and get better with time.

1. **Knowledge Acquisition:** AI collects information from a wide variety of sources, such as organised databases, photographs, unstructured text, and real-world encounters, among others. This process is known as learning. Computer vision, natural language processing (NLP), and machine learning are some of the techniques that make this acquisition possible.
2. **Knowledge Representation:** The second step is knowledge representation, which entails organising learnt material in a way that makes it easy to store and retrieve. As shown by the procedures outlined earlier:
3. **Knowledge Processing and Reasoning:** AI uses deep learning, probabilistic modelling, and logical inference to process knowledge. In this stage, AI is able to:
 - Apply heuristic search and optimisation to solve issues
 - Learn from experience and reinforcement learning
 - Draw conclusions (deductive and inductive reasoning)
4. **Knowledge Utilisation:** Fourth, AI uses what it knows to do things like automate routine operations, make predictions, and make decisions in the actual world. Some examples include:
 - AI-powered recommendation systems providing content suggestions

- Virtual assistants comprehending user queries
 - Autonomous vehicles' ability to make navigational judgements in real-time
5. Knowledge Refinement and Learning: Fifthly, AI is always learning and refining its knowledge base thanks to feedback loops. Accuracy and flexibility can be enhanced using techniques such as active learning, supervised fine-tuning, and reinforcement learning. As a result, AI is guaranteed to adapt to fresh information and experiences.

Machine learning follows an iterative cycle. AI systems are designed to constantly improve their knowledge base, which allows them to adapt and learn over time. This loop is fundamental to AI systems since it allows them to learn and improve with time.



6. TYPES OF KNOWLEDGE IN AI

For AI systems to work well, they need several kinds of knowledge. When it comes to thinking, making decisions, and solving problems, each kind is essential. The most common kind of information utilised by AI are as follows:

1. Self-Referential Information (Descriptive Information)

The data and facts about the world that AI systems keep and can retrieve when needed are called declarative knowledge. It stands for "what" is known as opposed to "how" to accomplish something. Knowledge graphs, databases, and ontologies are common places to find this kind of organised information.

"Paris is the capital of France" is an example of declarative knowledge. Applications that utilise artificial intelligence, such as virtual assistants and search engines, rely on this kind of information to respond to factual questions and deliver pertinent results.

2. Knowledge of Procedures (How-To Content)

What is needed to carry out particular activities is known as procedural knowledge. Instead than only providing a fact, it signifies "how" to do anything.

Procedural knowledge includes things like understanding how to operate a car or solve a quadratic equation. When it comes to carrying out tasks that include sequences of actions, AI systems like expert systems and robots rely on procedural knowledge. Machine learning models, decision trees, and rule-based systems frequently store this kind of information.

3. Meta-Knowledge, often known as "Knowledge About Knowledge," pertains to understanding the structure, usage, and validation of information. It aids AI in assessing the validity, usefulness, and relevance of knowledge in various contexts.

An AI system that uses meta-knowledge to determine the credibility of a medical recommendation vs. an unreliable blog post is one example. When it comes to enhancing decision-making, learning techniques, and filtering out misinformation, AI models rely heavily on this sort of information.

4. Knowledge Based on Heuristics (Experience)

This heuristic knowledge is based on intuition, trial and error, and experience. It enables AI systems to estimate solutions or make educated approximations when exact computations are challenging.

A navigation system that uses historical traffic data to propose a different route is using heuristic knowledge. In order to make better decisions in real-life situations, artificial intelligence search algorithms like genetic algorithms and A* search use heuristics to optimise problem-solving procedures.

5. Common-sense Knowledge:

AI has a hard time acquiring common-sense knowledge, which is fundamental understanding humans possess innately. Some examples of such statements are "water is wet" and "if you drop something, it will fall."

This kind of information necessitates contextual comprehension beyond explicit programming, which is why AI systems frequently fail to grasp it.

By utilising large-scale knowledge bases like ConceptNet, researchers are incorporating common-sense reasoning into AI. This allows robots to comprehend ordinary logic and enhance their relationship with people.

6. Knowledge of a Particular Domain

Medical, financial, legal, and engineering are examples of domains where domain-specific expertise is most useful. It contains organised and comprehensive data specific to a certain field.

In medicine, for example, information regarding symptoms, illnesses, and therapies is crucial for AI-driven diagnostic systems. Market movements, risk evaluations, and economic data are also utilised by financial AI models. In order to deliver precise forecasts and insights, industry-specific AI models and expert systems necessitate domain-specific information.

Despite its centrality to AI, knowledge representation presents a number of obstacles:

1. **Complexity:** Because it might be difficult to represent all potential knowledge about a domain, efficient management and processing of this information sometimes necessitates sophisticated approaches.
2. **Ambiguity and Vagueness:** It can be challenging to generate accurate representations of human language and concepts due to their tendency to be ambiguous or unclear.
3. **Scalability:** It is an important consideration for AI systems. It can be difficult to accommodate the increasing amount of knowledge without sacrificing storage and processing power.
4. **Acquiring Knowledge:** Collecting and transforming human-level expertise into a form that computers can understand is no easy feat, especially when dealing with highly dynamic or niche fields.

5. Reasoning and Inference: AI systems need to be able to reason and infer; this means they can't only memorise facts; they need to be able to create conclusions, solve issues, and store new information. To do this, we need complex reasoning algorithms that can efficiently process massive amounts of data.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which is the first stage of the AI Knowledge Cycle? a) Knowledge Inference b) Knowledge Acquisition c) Knowledge Sharing d) Knowledge Representation
7	Which type of knowledge tells us how to perform tasks? a) Declarative b) Procedural c) Meta d) Strategic
8	Which of the following is NOT part of the AI Knowledge Cycle? a) Acquisition b) Reasoning c) Compilation d) Refinement
9	Meta-knowledge refers to: a) Facts b) Knowledge about knowledge c) Actions d) Rules
10	Heuristic knowledge is based on: a) Facts b) Probabilities c) Common sense and experience d) Mathematics

7. APPLICATIONS OF KNOWLEDGE REPRESENTATION IN AI

Systems that are able to understand and reason like humans are made possible through the application of knowledge representation in several areas of artificial intelligence. Among the most prominent uses are:

1. **Expert System:** First, there are expert systems, which are able to offer guidance or make judgements in specialised fields (e.g., medical diagnosis, financial planning) by representing relevant knowledge.
2. **NLP (Natural Language Processing):** Chatbots, translation systems, sentiment analysis, and other applications that interpret and generate human language are made possible by knowledge representation.
3. **Robots:** In order to move around, communicate with their surroundings, and complete tasks without human intervention, robots employ knowledge representation.
4. **The Semantic online:** In order for machines to comprehend and analyse online information meaningfully, the Semantic Web depends on ontologies and other knowledge representation approaches.
5. **Cognitive Computing:** AI systems that can process and reason through massive volumes of data, such as IBM's Watson, are revolutionising industries including healthcare and research.

A key component of artificial intelligence, knowledge representation allows computers to comprehend, reason, and act upon the data they process. Complex tasks requiring human-like intellect can be tackled by AI systems by utilising various representation strategies. Nevertheless, there is an urgent need for more research into issues like complexity, ambiguity, and scalability. Improvements in knowledge representation will be crucial in creating smarter and more competent systems as AI develops further.

8. LOGICAL REASONING

In artificial intelligence, "logical reasoning" means that systems can think like humans by following predetermined rules and principles to reach conclusions and make decisions. It lays up the groundwork for AI systems to represent information, solve problems, and validate their results.

Key aspects of logical reasoning in AI:

As a cornerstone of AI, logical reasoning lays the groundwork for robots to reason, solve problems, and make decisions in a structured and interpretable manner. Similar to human reasoning, it entails processing information and drawing conclusions using formal logic.

1. Knowledge Representation

To reason logically, knowledge representation (KR) is essential. Preparing data for efficient processing by an AI system requires arranging and encoding it.

- In order for AI to process data, it has to be able to organise knowledge using forms like:
 - Propositional and predicate logic
 - Semantic networks
 - Ontologies
 - Frames
 - Production rules
- These structures aid the system in comprehending domain entities, attributes, and the interactions between them. As an illustration of the fact that "All humans are mortal," predicate logic might employ:

$\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

- For effective reasoning, an organised knowledge base is crucial. This will allow the AI to manage insufficient information, detect errors, and draw appropriate conclusions.

2. Inference

The logical process of drawing conclusions about new information from preexisting facts or rules is called inference. Artificial intelligence systems "think" and reason based on this fundamental principle.

Types of Inference:

- **Deductive Reasoning:**

- Derives specific conclusions from general principles.
- Example:
 - Rule: All birds can fly.
 - Fact: A sparrow is a bird.
 - Deduction: A sparrow can fly.
- Inductive Reasoning:
 - Infers general rules from specific examples.
 - Example: Observing that multiple swans are white and concluding that all swans are white.
- Abductive Reasoning:
 - Forms the most likely explanation from incomplete information.
 - Common in diagnosis systems (e.g., medical or fault detection systems).

AI systems use inference engines to process rules and facts to generate new insights or responses.

3. Problem Solving

Logical reasoning is central to AI problem-solving frameworks.

- A problem is defined by:
 - Initial state
 - Goal state
 - Set of possible actions
 - Rules/constraints
- The AI represents this information in a formal model and applies search and reasoning algorithms (e.g., forward chaining, backward chaining) to find a solution path.
- Example: In a chess game, the system evaluates possible moves using a tree of logical consequences to choose the optimal one.

Problem-solving using logic makes AI systems systematic, accurate, and efficient, especially in structured environments like games, planning tasks, or mathematical reasoning.

4. Decision Making

Logical reasoning supports decision-making by enabling the system to evaluate options against a set of criteria or rules.

- AI can:
 - Analyse the potential outcomes of different actions

- Apply decision rules or logic trees
- Select the action that leads to the best possible result
- Example: In autonomous vehicles, the AI must decide whether to stop, slow down, or proceed based on rules (traffic laws), sensor data, and predictions.

Unlike probabilistic decision-making, logical reasoning is rule-based, which enhances consistency and interpretability.

5. Explainability

One of the major advantages of logical reasoning is that it contributes to the explainability and transparency of AI systems.

- Because logical reasoning is explicit and rule-based, each decision made by the system can be traced back to the knowledge and rules used.
- This is particularly important in:
 - Medical diagnosis systems (e.g., explaining a diagnosis)
 - Legal expert systems
 - Finance and credit scoring tools

Explainable AI is crucial in gaining user trust, complying with regulations, and improving system debugging and training.

Summary table

Aspect	Description
Knowledge Representation	Structures information in a way that AI can process logically.
Inference	Derives new facts from existing ones using deductive, inductive, or abductive reasoning.
Problem Solving	Applies logic to find solutions by navigating rule-based problem spaces.
Decision Making	Uses logical rules to evaluate alternatives and select the best action.
Explainability	Provides transparency by making the reasoning process visible and understandable.

With the help of logical thinking, AI systems can behave in a reasonable and logical manner. It paves the way for computers to understand data, come up with answers, and articulate their reasoning

processes with clarity. Logical reasoning is still one of the most trustworthy AI methods for mission-critical applications when accuracy and explainability are paramount.



9. TYPES OF LOGIC USED IN AI

When it comes to representing information and reasoning, AI relies on logic. The formal basis it offers for intelligent system decision-making, conclusion-drawing, and correctness assurance is invaluable. The complexity and characteristics of the domain dictate the logic system that is utilised.

1. Propositional Logic (Boolean Logic)

The most fundamental kind of logic utilised by AI is propositional logic, which is sometimes called sentential logic. Combining statements that can be true or untrue using logical connectives such as AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$), etc. is what it's all about.

Characteristics

- Each statement (proposition) is atomic and does not consider internal structure.
- No variables or quantifiers.
- Example:
 - P = "It is raining"
 - Q = "I will carry an umbrella"
 - Rule: If it is raining, then I will carry an umbrella: $P \rightarrow Q$

Applications

- Basic decision-making systems
- Truth tables and logic circuits
- Formal verification of software

Limitations

- Cannot express relations between objects.
- Cannot handle variables, time, or complex data structures.

2. First-Order Logic (FOL) (Predicate Logic)

Quantifiers, predicates, variables, and functions are all extensions of propositional logic that First-Order Logic introduces. Because of this, it becomes much more emotional and impactful.

Characteristics:

- Can represent objects, their properties, and relationships.
- Includes quantifiers:

- Universal ($\forall$): "For all"
- Existential ($\exists$): "There exists"
- Syntax example:
 - $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
 - This means: "All humans are mortal"

Applications:

- Expert systems
- Natural language understanding
- Knowledge bases that need relational logic
- Theorem proving

Advantages:

- Expresses complex scenarios involving multiple entities.
- Suitable for structured knowledge domains.

Limitations:

- Higher computational cost than propositional logic.
- May become undecidable in some cases.

3. Rule-Based Systems

Knowledge is represented by rule-based logic systems through IF-THEN rules. To find out more information or make a judgement, these systems use logical inference to rules.

Structure:

- A typical rule looks like:
 - IF [condition] THEN [action/conclusion]
- Example:
 - IF temperature > 100 THEN alert = "High temperature"

Characteristics:

- Based on forward or backward chaining:
 - Forward chaining: Starts from known facts and applies rules to derive new facts.
 - Backward chaining: Starts from a goal and works backward to see if known facts support the goal.

Applications:

- Medical diagnosis systems
- Expert systems (e.g., MYCIN)
- Recommendation engines

Advantages:

- Easy to understand and implement.
- Modular: rules can be added or removed independently.

Limitations:

- Difficult to manage large rule sets.
- Struggles with uncertainty and contradictions.

4. Description Logic (DL)

Detailed description Logic, a branch of First-Order Logic, finds its main application in semantic web technologies and ontology modelling. It is ideal for reasoning over complicated hierarchies due to its strong expressiveness, structure, and decidability.

Key Features:

- Represents classes (concepts), relationships (roles), and individuals (instances).
- Basis for Web Ontology Language (OWL) used in Semantic Web.

Example:

- Classes: Person, Student
- Relationship: hasDegree
- Assertion: $\text{Student} \subseteq \text{Person}$ (every student is a person)

Applications:

- Ontologies (e.g., in bioinformatics or e-commerce)
- Semantic web (linked data, RDF, OWL)
- Knowledge graphs (e.g., Google Knowledge Graph)

Advantages:

- Balances expressiveness and computational feasibility.
- Supports automated reasoning: classification, consistency checking, instance retrieval.

Limitations:

- Not as expressive as full FOL.
- Requires structured knowledge modeling upfront.

Comparison Table:

Logic Type	Expressiveness	Use Cases	Strengths	Limitations
Propositional	Low	Basic AI, logic gates	Simplicity, fast computation	No object relationships or quantifiers
First-Order Logic	High	Expert systems, NLU	Expresses relations, uses variables	Computationally intensive
Rule-Based	Moderate	Expert systems, decision automation	Human-readable rules, easy to build	Hard to scale, lacks uncertainty handling
Description Logic	Structured & Decidable	Ontologies, Semantic Web	Semantic precision, used in OWL	Limited expressiveness vs full FOL

Examples of Logical Reasoning in AI:

- Expert System: Systems that diagnose medical issues, solve technical difficulties, or offer financial advice are known as expert systems.
- Robots: To move around, plan their next move, and interact with their surroundings, AI-powered robots rely on logic thinking.
- Logic is employed in Natural Language Processing (NLP) to decipher text, translate languages, and produce material that sounds human-like.
- Automated theorem provers are artificial intelligence systems that can do mathematical theorem proofs automatically.
- Legal AI: AI systems that scan contracts, find important language, and determine if everything is in order.

Benefits of Logical Reasoning in AI:

- Better Problem Solving: AI systems are capable of handling complicated challenges that

necessitate reasoning and analysis.

- Improved Decision-Making Capability: More trustworthy and predictable results from AI systems can be achieved through the use of logical reasoning.
- Increased Transparency: AI systems can be made more transparent and understandable through the explicit reasoning process, which increases transparency.
- An AI system's decision-making process can be better understood with the use of logic reasoning, which improves explainability.
- Unlocking New Possibilities for AI Applications: Logic can open doors to new AI applications in many different disciplines.

To put it simply, logical reasoning is an essential part of artificial intelligence that allows systems to think, solve issues, and make decisions in a smart and organised way.

SELF-ASSESSMENT QUESTIONS - 3

Fill in the Blanks:	
11	Deductive reasoning moves from _____ to _____.
12	Predicate logic is also known as _____ logic.
13	_____ reasoning helps in forming hypotheses from incomplete information.
14	Logical reasoning is essential in AI for _____ making.
15	In AI, logical rules are used to implement _____ systems.

10. BASIC CONCEPT OF PROPOSITIONAL LOGIC

By making use of basic assertions, complicated issues can be solved using propositional reasoning. These claims can either be true or untrue; they can't be true and false simultaneously. Artificial intelligence systems rely on these assumptions for knowledge representation, reasoning, and decision-making. This article will introduce you to propositional logic and its uses in artificial intelligence. Statements that can be true or untrue are dealt with in propositional logic, a branch of logic. Using logical connectives such as AND, OR, NOT, and IMPLIES, it constructs complex expressions and then uses rules to find their truth values. One of the main ways that artificial intelligence (AI) employs logical statements to describe knowledge is propositional logic. A proposition can only be true or false, not both. Using logical operators such as AND, OR, NOT, and IMPLIES to combine statements allows for the creation of more complicated expressions. An AI system can then use this information to improve its thinking and decision-making abilities.

When it comes to rule-based reasoning, automated theorem proving, and expert system decision-making, artificial intelligence uses propositional logic. Machines can infer conclusions from given facts using inference rules in machine learning, such as Modus Ponens and Resolution.

In the field of artificial intelligence, a proposition is a statement that states an assertion about the universe or a set of facts in order to facilitate the computers' ability to reason logically. The symbols that symbolise these truths and the logical operators that are often included in parenthesis make up the formulation of propositional logic.

Intelligence Graphs for Propositional Logic

Statements that can be true or untrue are used in propositional logic. A situation's facts or conditions are represented by these assertions. We represent the propositions with symbols and the logical procedures with symbols to connect them. It clarifies the interconnectedness of seemingly unrelated facts in complicated statements or problems. Combining several propositions to indicate logical relations is made easier with proposition operators such as conjunction ($\wedge$), disjunction ($\vee$), negation ($\neg$), implication ($\rightarrow$), and biconditional ($\leftrightarrow$).

Example of Propositional Logic

- P: "The sky is blue." (This statement can be either true or false.)
- Q: "It is raining right now." (This can also be true or false.)
- R: "The ground is wet." (This is either true or false.)

These can be combined using logical operations to create more complex statements. For example:

- $P \wedge Q$: "The sky is blue AND it is raining." (This is true only if both P and Q are true.)
- $P \vee Q$: "The sky is blue OR it is raining." (This is true if at least one of P or Q is true.)
- $\neg P$: "It is NOT true that the sky is blue." (This is true if P is false means the sky is not blue.)

Logical Equivalence

In any conceivable scenario, two assertions are considered logically equivalent if their truth values are same. As an illustration:

It is the same as saying " $\neg S \vee T$ " (not S or T) while stating " $S \rightarrow T$ " (if S then T). Thus, "either S is false or T is true" is equivalent to "if S is true, then T must be true."

- The biconditional " $P \leftrightarrow Q$ " (P in the event that Q is true) is the same as " $(P \rightarrow Q) \wedge (Q \rightarrow P)$ ". It follows that P implies Q and Q implies P.

These equivalents demonstrate that several logical phrases are capable of conveying the same idea. You can check them with truth tables or by applying logical principles to reduce the claims.

Basic Concepts of Propositional Logic

1. Propositions

Any statement that can be proven or disproven is called a proposition. Any statement that can be answered yes or no is a proposition, regardless of how complex it may be. As an illustration:

- "The sky is blue." - This is correct.
- "It is raining." - Not true

2. Logical Connectives

Logical connectives are used to combine simple propositions into more complex ones. The main connectives are:

- AND ($\wedge$): This operation is true if both propositions are true.
Example: "It is sunny $\wedge$ it is warm" is true only if both "It is sunny" and "It is warm" are true.
- OR ($\vee$): This operation is true if at least one of the propositions is true.
Example: "It is sunny $\vee$ it is raining" is true if either "It is sunny" or "It is raining" is true.
- NOT ($\neg$): This operation reverses the truth value of a proposition.
Example: " $\neg$ It is raining" is true if "It is raining" is false.
- IMPLIES ($\rightarrow$): This operation is true if the first proposition leads to the second.

Example: "If it rains then the ground is wet" ($\text{It rains} \rightarrow \text{The ground is wet}$) is true unless it rains and the ground is not wet.

- IF AND ONLY IF ($\leftrightarrow$): This operation is true if both propositions are either true or false together.

Example: "It is raining $\leftrightarrow$ The ground is wet" is true if both "It is raining" and "The ground is wet" are either true or both false.

3. Truth Tables

By verifying all conceivable combinations of truth values for its components, they are utilised to find the truth value of complex propositions. Their exhaustive listing of all conceivable permutations makes it simple to determine the impact of individual logical operators on the final result. By treating each combination equally, this method offers a full picture of the reasoning for the operation.

4. Tautologies, Contradictions and Contingencies

- Tautologies: No matter how you slice it, tautology is a notion that holds true in and of itself. " $P \vee \neg P$ " is a valid example since it can be true or false depending on the value of P.
- Contradiction: A statement that is always incorrect is called a contradiction. For instance, the statement " $P \wedge \neg P$ " is never true since P cannot simultaneously be true and untrue.
- Contingency: A proposition that depends on the truth values of its components to determine its truth or falsity is called a contingency.

If both P and Q are true, then the statement " $P \wedge Q$ " is true.

Properties of Operators

The many useful features of logical operators in propositional logic allow for the simplification and analysis of otherwise complicated claims.

1. Commutativity: Order of propositions doesn't matter when using AND ($\wedge$) or OR ($\vee$).
 - $P \wedge Q \equiv Q \wedge P$
 - $P \vee Q \equiv Q \vee P$
2. Associativity: Grouping of propositions doesn't matter when using multiple ANDs or ORs.
 - $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$
 - $(P \vee Q) \vee R \equiv P \vee (Q \vee R)$
3. Distributivity: AND ($\wedge$) and OR ($\vee$) can distribute over each other which is similar to multiplication and addition in math.

- $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$
 - $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$
4. Identity: A proposition combined with "True" or "False" behaves predictably.
 - $P \wedge \text{true} \equiv P$
 - $P \vee \text{false} \equiv P$
 5. Domination: When combined with "True" or "False" some outcomes are always fixed.
 - $P \vee \text{true} \equiv \text{true}$
 - $P \wedge \text{false} \equiv \text{false}$
 6. Double Negation: Negating a proposition twice cancels out the negation.
 - $\neg(\neg P) \equiv P$
 7. Idempotence: Repeating same proposition with AND or OR doesn't change its value.
 - $P \wedge P \equiv P$
 - $P \vee P \equiv P$

Applications of Propositional Logic in AI

1. Knowledge Representation: Knowledge can be structured using propositional logic. Because of this, AI systems can now store and alter data about the world. Expert systems, for instance, store information in the form of a collection of statements and logical rules.
2. Automated Reasoning: In order for systems to "think" logically and draw new conclusions from current facts, AI employs logical rules like Modus Ponens and Modus Tollens.

For example:

- Modus Ponens: If " $P \rightarrow Q$ " and " P " are true then " Q " must be true.
 - Modus Tollens: If " $P \rightarrow Q$ " and " $\neg Q$ " are true then " $\neg P$ " must be true.
3. Problem Solving and Planning: By describing goals, it enables AI planners to solve problems and construct action sequences. As an example, the STRIPS planning system is useful for representing actions' preconditions and effects using propositional logic.
 4. Decision Making: You can use it to figure out what to do next by comparing your alternatives. Truth tables can be utilised to evaluate the results of various options, and logical rules can be employed to encapsulate decision criteria.
 5. Natural Language Processing (NLP): In natural language processing, it is used for tasks such as semantic parsing, which involves transforming sentences from natural language into logical

forms. Sentence comprehension and reasoning are both aided by this.

Limitations of Propositional Logic

It has several benefits, but it also has some drawbacks:

1. It isn't expressive enough; for example, it can't deal with the idea that "All humans are mortal."
2. Scalability: As the number of propositions increases, truth tables grow exponentially.
3. Thirdly, it has limited inference capabilities; it can only deal with true/false truth values and not probabilities.
4. The use of quantifiers like "for all" (represented by $\forall$) or "there exists" (represented by $\exists$) is not permitted in it, unlike predicate logic.
5. It cannot handle circumstances with a high degree of uncertainty since it cannot represent probabilities or partial truths.
6. It can't understand complex situations since it doesn't take into account the meaning or context of utterances.

A straightforward and effective method for teaching computers to reason and make conclusions supported by evidence is propositional logic.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
16	Propositional logic uses: a) Variables and functions b) Propositions and connectives c) Graphs and trees d) Frames
17	Which symbol represents logical AND? a) $\vee$ b) $\rightarrow$ c) $\wedge$ d) $\neg$
18	Which of the following is NOT a logical connective? a) AND b) OR

	c) NOT d) ADD
19	What is the result of $\text{True} \wedge \text{False}$? a) True b) False c) Null d) Cannot be determined
20	In propositional logic, $\neg P$ means: a) P is True b) Not P c) P or Not P d) If P, then Q



11. SUMMARY

- Knowledge Representation and Reasoning provides an essential foundation for understanding how AI systems simulate human intelligence through structured data and logic.
- It begins by introducing the concept of Knowledge Representation (KR) in AI, explaining how information is captured, stored, and utilised by machines to make informed decisions.
- The core methods of KR such as semantic networks, frames, rules, and ontologies are discussed, showing how each supports various AI functions.
- The AI Knowledge Cycle, which outlines the process of acquiring, representing, reasoning, and learning from knowledge. It also differentiates between various types of knowledge like declarative, procedural, heuristic, and meta-knowledge.
- Real-world applications of KR are examined, including its role in expert systems, natural language processing, and robotics.
- AI systems draw conclusions from facts using logic. Different types of logic such as propositional logic and first-order logic are introduced, with emphasis on how they enable machines to handle truth, inference, and deduction.
- AI uses different types of knowledge: declarative, procedural, heuristic, and meta-knowledge. Applications of KR are found in expert systems, robotics, and natural language processing. Logical reasoning helps AI make decisions using facts and inference rules.
- Types of logic used in AI include propositional logic and first-order logic. Propositional logic deals with statements and their truth values using logical connectives. The chapter concludes with a basic introduction to propositional logic, covering syntax, semantics, and the use of inference rules to perform automated reasoning.

12. GLOSSARY

Knowledge Representation (KR)	- A method used in AI to encode information about the world in a form that a computer system can understand and use.
Semantic Networks	- A graph-based structure that represents relationships between concepts using nodes and edges.
Frames	- A data structure for dividing knowledge into substructures by representing "stereotyped situations."
Scripts	- A structured representation of a sequence of events in a particular context, often used for understanding narratives.
Ontology	- A formal representation of a set of concepts and relationships within a domain.
Declarative Knowledge	- Knowledge that describes facts and information (i.e., "knowing that").
Procedural Knowledge	- Knowledge about how to perform tasks (i.e., "knowing how").
Heuristic Knowledge	- Rule-of-thumb knowledge based on experience and judgment rather than fixed facts.
Meta-Knowledge	- Knowledge about other knowledge—used to evaluate or control reasoning processes.
AI Knowledge Cycle	- The process of acquiring, organising, utilising, and revising knowledge in AI systems.
Logical Reasoning	- The process of deriving conclusions from known facts or premises using logical methods.
Propositional Logic	- A form of logic where statements (propositions) are combined with logical operators to determine truth values.

First-Order Logic (Predicate Logic)	- An extension of propositional logic that includes quantifiers and predicates for expressing more complex statements.
Inference Engine	- A component of an AI system that applies logical rules to the knowledge base to draw new conclusions.
Rule-Based System	- A system that applies a set of "if-then" rules to data to make decisions or inferences.



13. TERMINAL QUESTIONS

1. What is Knowledge Representation in AI?
2. Why is Knowledge Representation important in AI?
3. Name any four core methods of Knowledge Representation.
4. What is the AI Knowledge Cycle?
5. List four types of knowledge used in AI.
6. Give two real-world applications of Knowledge Representation in AI.
7. Define Logical Reasoning in the context of AI.
8. Differentiate between Propositional Logic and Predicate Logic.
9. What is the use of inference in Knowledge Representation?
10. List down the applications of Propositional Logic in AI
11. What is a Frame in KR, and how is it used?
12. Write an example of a propositional logic expression and its interpretation.

14. ANSWERS

Self-Assessment Questions

1. C: Semantic Networks
2. B: Production Rules
3. C: Stereotyped situations
4. B: Knowledge Representation
5. C: Stacks
6. B: Knowledge Acquisition
7. B: Procedural
8. C: Compilation
9. B: Knowledge about knowledge
10. C: Common sense and experience
11. general, specific
12. first-order
13. Abductive
14. decision
15. expert
16. B: Propositions and connectives
17. C: $\wedge$
18. D: ADD
19. B: False
20. B: Not P

Terminal Questions Answers

1. Answer:

Knowledge Representation (KR) is the process of encoding information about the real world into a form that an AI system can understand and utilise to solve complex problems. (Refer to Section 2 for more information).

2. Answer:

It enables machines to reason, learn, and make decisions by simulating human understanding through structured information. (Refer to Section 3 for more information)

3. Answer:

- Semantic Networks
- Frames
- Logic (Propositional & Predicate)
- Rules and Scripts. (Refer to Section 9.4 for more information)

4. Answer:

The AI Knowledge Cycle includes stages such as knowledge acquisition, representation, reasoning, and refinement to maintain and update the knowledge base. (Refer to Section 9.5 for more information)

5. Answer:

- Declarative Knowledge
- Procedural Knowledge
- Meta-Knowledge
- Heuristic Knowledge (Refer to Section 6 for more information)

6. Answer:

- Chatbots and virtual assistants
- Expert systems in medical diagnosis (Refer to Section 7 for more information)

7. Answer:

Logical Reasoning is the process of using formal logic rules to deduce new information or verify the truth of given statements. (Refer to Section 8 for more information)

8. Answer:

Propositional Logic deals with simple, declarative statements (true/false), while Predicate Logic handles complex statements involving variables and quantifiers. (Refer to Section 6 for more information)

9. Answer:

Inference allows AI systems to derive new facts from existing knowledge, helping them make decisions and solve problems. (Refer to Section 8 for more information)

10. Answer:

Semantic Networks represent knowledge as a graph with nodes (concepts) and edges (relationships), making it easy to visualise associations. (Refer to section 10 for more information)

11. Answer:

A frame is a data structure for representing a concept with a collection of attributes (slots) and values. It is used to model stereotypical situations. (Refer to section 4 for more information)

12. Answer:

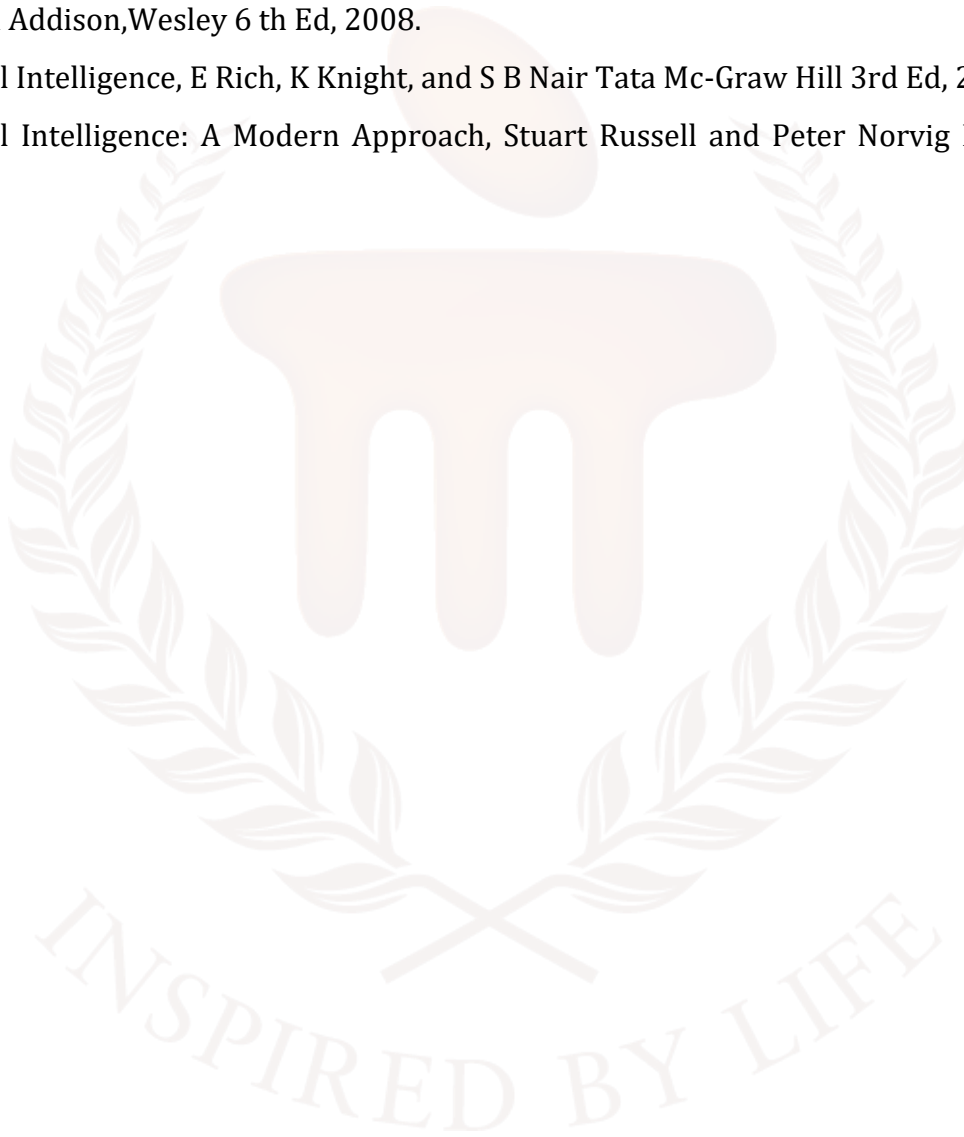
Expression: $P \wedge Q$

Interpretation: Both propositions P and Q must be true for the entire expression to be true (logical AND). (Refer to section 10 for more information)



15. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.



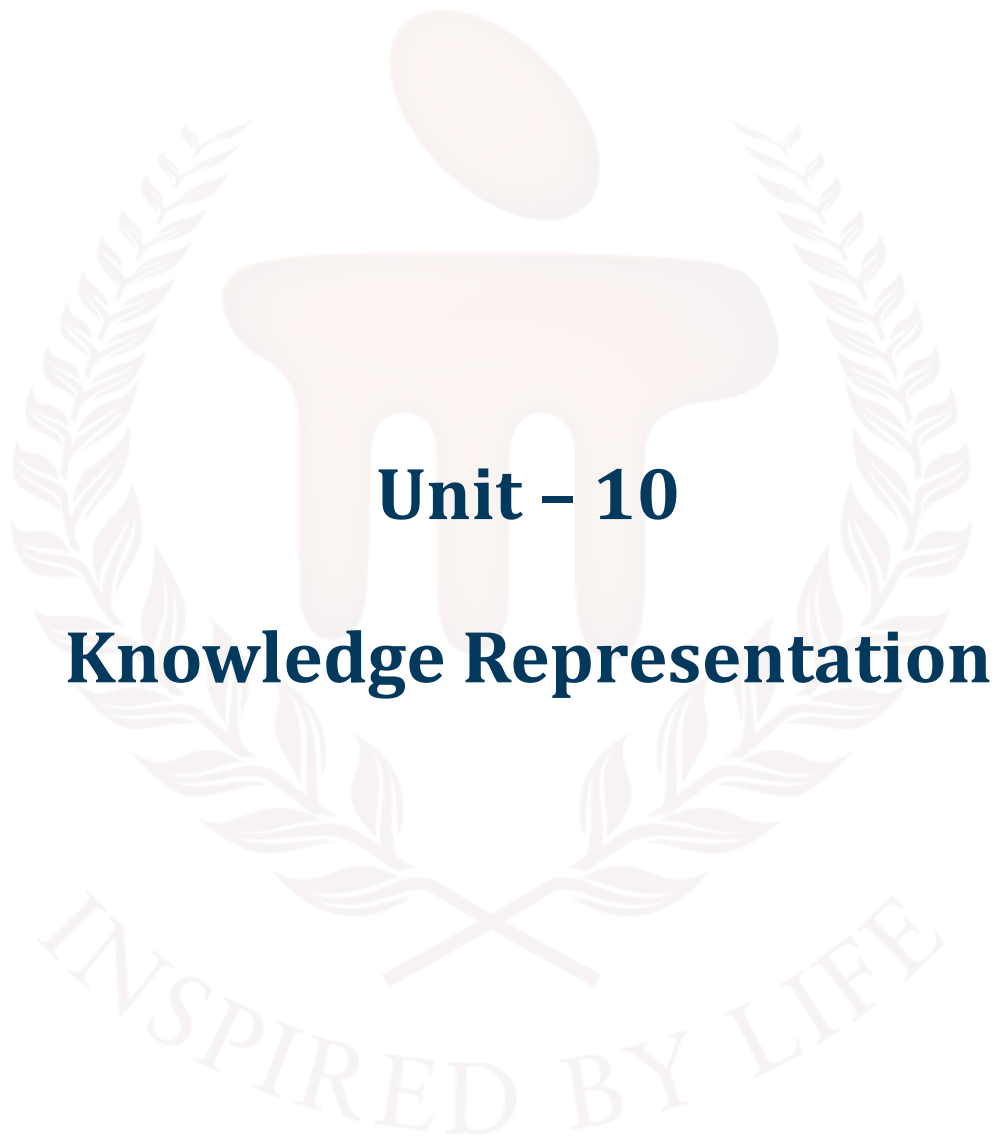
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109

**ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING**



Unit – 10

Knowledge Representation

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Knowledge representation	-	-	6 - 7
3	Semantic Networks in Artificial Intelligence	-	-	8
4	Types of Semantic Networks	1	1	9 -16
5	Frame script in AI	2	2	17 -24
6	Script Theory in Artificial Intelligence	-	-	25 - 28
7	Situation Calculus	-	-	22 - 31
8	Summary	-	-	32
9	Glossary	-	-	33 - 34
10	Terminal Questions	-	-	35
11	Answers	-	-	36 - 38
12	References	-	-	39

1. INTRODUCTION

In previous unit, we had explored the foundational principles of how intelligent systems represent, process, and reason with knowledge. The focus will be on equipping you with the tools and techniques used in logical reasoning, a core aspect of artificial intelligence and decision-making systems. We began by understanding the introduction to logical reasoning, where we learnt how AI systems uses logic to derive conclusions from known facts. We learnt the importance of logic in AI, and how it helps in problem-solving, inference-making, and building knowledge-based systems.

We also delved into the basic concepts of propositional logic, which is the simplest form of logic used to represent statements and their relationships. We also learnt how to represent facts using propositional variables, Construct logical formulas using connectives (AND, OR, NOT, etc.), we understood the concepts of truth tables, tautologies, and logical equivalence, Apply inference rules like Modus Ponens, Modus Tollens, and more. By the end of the unit, we were able to model knowledge using formal logic and perform basic reasoning tasks, laying a strong foundation for more advanced topics like predicate logic and automated reasoning systems.

In this unit, we will explore the essential concepts and techniques involved in Knowledge Representation (KR), which is the foundation for enabling artificial intelligence systems to understand, reason, and act based on the information they possess. We'll study how knowledge can be visually represented using graph structures, where concepts are nodes and relationships are edges. This helps AI systems infer facts and build meaningful connections. We'll explore frames as structured data forms that organise knowledge into slots and values. Frames are useful for representing stereotypical situations or objects and support inheritance and default reasoning.

Then the script section will introduce scripts—predefined, structured representations of sequences of events in specific contexts (e.g., visiting a restaurant). Scripts help AI predict and understand actions in everyday scenarios. We will examine this formal logic-based method to represent dynamic worlds, where actions cause changes over time. It is key for modelling actions, events, and their effects in decision-making systems.

Together, these approaches equip AI with the ability to reason like humans, adapt to new information, and respond intelligently in complex environments. By the end of this unit, you'll understand how different KR models work and how they contribute to the design of smarter AI systems.

1.1. Objectives

After studying this unit, you should be able to:

- Interpret the concepts of Semantic networks in AI
- Apply strategic thinking to different types of semantic networks
- Assess the functionalities of frame scripts
- Differentiate between frames and semantic networks
- Recall the ontology of situation calculus



2. KNOWLEDGE REPRESENTATION

Knowledge Representation Techniques in Artificial Intelligence

Various knowledge representation strategies have been developed in AI to enable machines to analyse, reason, and comprehend information.

- **Logical Representation:** When it comes to artificial intelligence knowledge representation, logical representation is among the most rigorous and formal approaches. It encodes knowledge using formal logic so that reasoning can be exact and clear.
- **Semantic Networks:** Semantic networks, as we've already established, are a graphical representation of knowledge in which ideas are nodes and connections are edges. They stand for associative and hierarchical linkages and are quite intuitive.
- **Production Rules:** Expert systems in particular make use of production rules as a means of representing knowledge in AI. A common way to represent these rules is as "if-then" statements, which specify what to do when specific conditions are satisfied.
- **Frames Representation:** In the same way that objects are used in object-oriented programming, frames are data structures that store stereotypical knowledge about things, situations, or occurrences. Attributes (slots) and the values assigned to them make up a frame.

Components of Knowledge Representation:

Establishing a vocabulary of ideas (nodes) and their interconnections (edges) is the initial stage in developing a semantic network. Making a network that is reflective of the field of study is an important part of this process.

1. **Inference and Reasoning:** Artificial intelligence systems are able to navigate networks and draw conclusions from the connections between nodes. Finding new information or answers to queries is accomplished by following the edges between nodes in this process.
2. **Querying the Network:** It is possible to retrieve particular data by querying the network. Finding categories, extracting specific data points, or discovering links between concepts are all possible with queries.
3. **Updating the Network:** Adding, editing, or removing nodes and edges allows the network to be updated as new information becomes available. In this way, the network is able to accurately reflect the most recent information.

4. **Reasoning Mechanisms:** Regularly, reasoning processes like forward chaining and backward chaining are employed by semantic networks. Forward chaining involves starting with known facts and applying inference rules to derive new facts, while backward chaining involves starting with a goal and working backward to determine if known facts can support it.



3. SEMANTIC NETWORKS IN ARTIFICIAL INTELLIGENCE

Semantic networks are a powerful method in AI for representing knowledge and the connections between concepts. In these visual structures, nodes represent ideas and edges represent relationships. Natural language processing (NLP), reasoning systems, and knowledge representation all make extensive use of semantic networks.

Semantic networks are a key component of artificial intelligence (AI), and this section will attempt to give a thorough introduction to them by outlining their types, components, applications, and how they help with knowledge representation and reasoning.

What is a Semantic Network?

One way to describe knowledge is through a semantic network, which provides a graphical depiction of the relationships between concepts. As a tool in artificial intelligence, it facilitates data organisation and structure for robots to understand, process, and make decisions based on. A semantic network is made up of nodes that stand for ideas and edges that describe the connections between them, using words like "is a," "part of," and "related to."

For example, a basic semantic network may represent the concept 'Dog' linked to 'Animal' by an 'is a' relationship.

4. TYPES OF SEMANTIC NETWORKS

The sort of relationship that a semantic network represents determines its classification.

Some important kinds are listed below:

1. Definitional Networks

Taxonomies and ontologies frequently make use of definitional networks to depict hierarchical relationships. Ideas are defined in relation to one another, whether those ideas be more general or more particular.

2. Assertional Networks

Particular facts or assertions regarding specific instances of concepts are represented by assertional networks. They usually list the characteristics of particular things.

Using an assertional network, we can express the ideas that "Rex is a Dog" and "Rex has Brown Fur."

3. Implicational Networks

Concepts' logical implications are the primary subject of representation in implications networks. They help draw conclusions about new information based on preexisting relationships.

Given the sentence "All Dogs are Mammals" and the fact that "Rex is a Dog," an implicational network can deduce that "Rex is a Mammal."

4. Executable Networks

Relationships in executable networks comprise actions or sequences that an AI system may execute, making them a good representation for procedural knowledge.

You may think of an executable network as a recipe with instructions like "Add Water to Pot" and "Boil Water."

5. Learning Networks

Learning networks are adaptable and change over time in response to new data learnt by the AI. After gathering fresh information, they revise the connections and nodes accordingly.

An AI's definition of "Dog" could be updated in a learning network when it comes across different breeds or traits.

6. Hybrid Networks

Hybrid networks are able to express knowledge more complexly and versatily since they mix features from many categories.

By combining definitional and assertional features, a hybrid network might stand in for both the abstract idea of "Dog" and concrete examples like "Rex."

Components of Semantic Networks

There are numerous essential parts that make up semantic networks:

1. **Nodes:** are the building blocks of a semantic network; they stand in for ideas, things, and entities in the domain of knowledge. Some examples include "Dog," "Animal," and "Tree."

- Labels: Identifiers or descriptive names attached to the nodes that allow us to refer to the ideas they stand for.

2. Structural Components

- Edges/Links: The relationships represented by the connections between nodes, such as "is a," "part of," "causes," or "associated with."
- Types of Relationship: There are many different kinds of relationships, such as hierarchical ones (like "is a"), associative ones (like "related to"), and functional ones (like "causes" or "results in").

3. Semantic Components

- Node Interpretations or Meanings: How each node is understood in relation to the rest of the network
- Relationship Interpretation: Verifying that the relationships are meaningful and appropriately represent the domain by comprehending the real-world significance of the edges or ties between nodes.

4. Procedural Part

- Inference Rule: A set of rules called "inference rules" that the network can use to learn new things by drawing on previously established connections. If we have "Dog is a Mammal" and "Mammal is an Animal," for instance, the network can deduce that "Dog is an Animal."
- Query Mechanisms: Approaches to retrieve data from the network according to predetermined criteria or queries.

- **Update Mechanisms:** Procedures and rules for incorporation of new information through the addition, modification, or deletion of nodes and links.

Working of Semantic Networks

For artificial intelligence systems to represent, infer, and reason about knowledge, semantic networks must undergo a number of processes. Semantic networks are a knowledge representation technique used in artificial intelligence to visually and structurally represent relationships between concepts. They consist of nodes (representing entities or concepts) and edges (representing the relationships between them). The working of semantic networks involves several key processes that enable AI systems to represent, infer, reason, and update knowledge efficiently.

1. Knowledge Representation

Semantic networks represent knowledge as a graph structure. Each node denotes a concept (e.g., Dog, Animal, Tail), and each edge represents a relationship (e.g., is-a, has-a, part-of). These labeled links describe how concepts are connected and help structure information meaningfully.

Example:

- Dog → is-a → Animal
- Dog → has-a → Tail

This structure helps machines understand and process relationships among different entities.

2. Inference Through Link Traversal

AI systems use semantic networks to infer new information by following the relationships between nodes. If it is known that:

- Dog is an Animal, and
 - Animals breathe air,
- then the system can infer:
- Dog breathes air

This form of chaining relationships is essential for knowledge-based reasoning.

3. Hierarchical Inheritance

Semantic networks support inheritance, meaning lower-level nodes can automatically acquire the properties of higher-level nodes. For example, if the concept Animal has the property can move, all its

subclasses like Dog, Cat, or Elephant inherit this property. This reduces redundancy and improves efficiency in storing and retrieving knowledge.

4. Querying and Reasoning

One of the core functionalities of semantic networks is the ability to respond to queries. When a user or system asks a question (e.g., "Is a dog a mammal?"), the AI navigates through the network to find relevant links and respond accordingly. This capability supports applications in search engines, expert systems, and intelligent assistants.

5. Knowledge Updating

Semantic networks are dynamic and can be modified over time. New concepts can be added, existing relationships can be updated, and obsolete links can be removed. This allows the AI system to learn from new information and adapt to changing environments or user inputs.

Examples of Semantic Networks in AI

One useful way to describe classifications and relationships across different domains is with a semantic network. To better interpret and organise complex information, semantic networks have many potential applications; below are a few examples from several fields.

1. Technology Stack Classification

Nodes: API, Django, Python, JavaScript, HTML, CSS, and frontend/backend/html.

Labels: Web Development, Framework, Language Links: "is a" relation, "uses" relation

The many parts of a technology stack are modelled in this semantic network. "HTML," "CSS," and "JavaScript" are connected to "Frontend" by a "is a" relation, whereas "Python" and "Django" are connected to "Backend." The "uses" relation establishes a connection between "API" and "Frontend" and "Backend," signifying its function in the creation of websites.

2. Food Hierarchy

- Nodes: Lion, Apple, Banana, Fruit
- Hyperlinks: connection "eaten by"
- Meat Eater, Predator, Herbivore

An ordered set of foods is represented by this semantic network. "Apple" and "Banana" are related to "Fruit," while "Lion" is related to "Animal" via a "is a" link. "Fruit" is linked to "Herbivore" and

"Animal" to "Carnivore" via the "eaten by" link, demonstrating the nutritional linkages within the food chain.

3. Programming Concepts

- Nodes: Integer, Python, and Java as programming languages
- Connections: "is a" and "has" relations
- Categories: Numeric Type, Variable, High-Level Language

Here we see a programming-related example of a semantic network in action. The "is a" relation connecting "Programming Language" to "Python" and "Java" indicates that both terms refer to programming languages. "Data Types" is associated with "Integer" via a "has" relation, and both are related to "Numeric Type," illustrating how data types are categorised in programming.

The Distinction between Frames and Semantic Networks

When it comes to representing knowledge, there are two main approaches: semantic networks and frames.

Table 1: Differences between Frames and Semantic Networks

Aspect	Semantic Networks	Frames
Representation	Graphical representation with nodes and edges.	Data structures with slots and values.
Components	Nodes (concepts), edges (relationships).	Slots (attributes) and values (fillers).
Structure	Hierarchical or associative network of concepts.	Structured, often hierarchical, but focused on specific entities or scenarios.
Purpose	Represents relationships between concepts, often used for reasoning.	Represents stereotypical knowledge about objects or situations.
Usage	Used in knowledge representation, reasoning, and inference.	Used to model structured knowledge, like objects, events, or scenarios.

Aspect	Semantic Networks	Frames
Flexibility	More flexible in representing complex relationships.	More rigid, with predefined slots and values.
Example	“Dog is a Mammal” and “Mammal is an Animal” relationship.	A frame for “Car” might have slots for “Make,” “Model,” “Color.”
Handling of Default Values	Does not inherently handle default values.	Can include default values for slots, which can be overridden.
Reasoning Capability	Supports inferencing through relationships (e.g., inheritance).	Typically involves simple procedural attachments for reasoning.
Ease of Modification	Can be more complex to modify due to interconnected relationships.	Easier to modify specific frames by adjusting slots and values.

Applications of Semantic Networks in AI

Among the many artificial intelligence (AI) uses for semantic networks are:

1. **Natural Language Processing (NLP):** Semantic networks aid in NLP by representing the links between various words and concepts, which helps in interpreting the meaning of phrases and words.
2. **Expert Systems:** To enable the system to make decisions or offer suggestions based on human experts' knowledge, expert systems use semantic networks to represent that information.
3. **Ontology Development:** Semantic networks are commonly used by ontologies to depict the connections between concepts in a given domain, which is responsible for defining the structure of knowledge in that domain.
4. **Information Retrieval:** Better search results are produced by systems that are able to comprehend the context and connections between various bits of information, which is made possible by semantic networks, which improve information retrieval.

5. Machine Learning: By offering a structured representation of the knowledge the model has learnt, semantic networks enhance the interpretability of models in some machine learning applications.

Advantages of Semantic Networks

- **Clear and Intuitive Representation:** Semantic networks streamline the process of representing knowledge, facilitating the understanding of complicated relationships by both humans and machines.
- **Flexibility:** Because of their adaptability, they can be used in a wide variety of contexts and represent a wide range of connection kinds.
- **Help with Reasoning:** Semantic networks make it easier for AI systems to reason by letting them infer new information from preexisting relationships.

Challenges and Limitations

Despite their usefulness, semantic networks present a number of obstacles:

- **Scalability:** Problems with complexity and management arise when the number of concepts and relationships in a semantic network grows.
- **Ambiguous or imprecise relationships** can cause misunderstandings in AI systems.
- **Complexity in computation:** Processing and reasoning about complex networks with many relationships can demand large computational resources.

Because they provide a natural and organised way to describe information and connections between ideas, semantic networks are vital to AI. Natural language processing (NLP), expert systems, and other AI applications rely on them heavily. Semantic networks continue to be a vital AI tool for systems to comprehend and handle the massive volumes of data they encounter, despite their limitations.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What does a semantic network primarily represent? A. Arithmetic operations B. Sequential processes C. Relationships between concepts D. Hardware configurations

2	In a semantic network, concepts are represented as: A. Tables B. Nodes C. Rules D. Sets
3	The “Is-A” relationship in semantic networks is used for: A. Describing ownership B. Denoting similarity C. Expressing hierarchical classification D. Showing time sequence
4	Which of the following is a key advantage of semantic networks? A. High computation speed B. Structured inference through relationships C. Random data mapping D. Error-free communication
5	What does the edge in a semantic network signify? A. An index value B. A condition C. A relationship D. A data type

INSPIRED BY LIFE

5. FRAME SCRIPT IN AI

An essential idea in AI, frames allow computers to comprehend and make sense of complicated real-world situations. Frames are used to organise information so that AI systems can reason, infer, and make decisions. They originate from cognitive science and knowledge representation.

This unit explores frames, what they are, how they are important in AI, and some real-world uses for them.

What Are Frames in AI?

In artificial intelligence, "frames" are data structures that stand in for common, generalised cases. Within a specific context, they contain data regarding things, occurrences, and how they are related to one another. Assembled from a collection of characteristics and principles, each frame serves as a blueprint for comprehending particular cases.

Take a "restaurant" frame as an example; it may contain properties like "menu," "waitstaff," and "tables," each of which could have its own array of details.

Concept of Frames

Knowledge representation relies on Minsky's 1974 introduction of the frame notion. With the use of frames, we may more easily retrieve and manipulate data by providing an organised approach to capture its fundamental elements. They help break down large amounts of information into more manageable pieces, much like a schema or blueprint.

Key Components of Frames

Understanding the basic components of frames assists in properly utilising them, which is crucial for AI knowledge structure.

Here are the key elements of frames and some examples of how to use them:

1. Slots

A frame's slots are its characteristics. They stand for various features or qualities of the concept represented by the frame.

As an illustration: A "Person" frame may have the following slots:

- Name: The name of the person
- Household Address: The person's residential address

- Occupation: The person's line of work

2. Facets

The facets of a slot define its permissible values and the rules for its usage, among other things.

In the "Person" frame, for instance, in the "Age" field:

- Integer type
- \From zero to one hundred twenty
- Valuation by default: 30

3. Default Values

In the absence of an explicit value, predetermined default values are applied to the corresponding slots. The information that is more particular can be used to override their baseline.

As an illustration: Within a "Car" image:

"Unknown" could be the default setting for "Make."

For this model, "Unknown" might be the default value.

The current year could be the default value for "Year."

4. Procedures

A procedure is a set of related methods or functions that specify how to handle or use the data contained within a frame.

As an illustration: Inside the "Account" window:

One way to calculate interest using the account balance is the "CalculateInterest" procedure.

Example of a Complete Frame

The following is a comprehensive library management system "Book" framework:

- Frame Name: Book
 - Slots:
 - Title: "To Kill a Mockingbird"
 - Author: "Harper Lee"
 - Publication Year: 1960
 - ISBN: "978-0-06-112008-4"
 - Genre: "Fiction"

- Facets:
 - Publication Year:
 - Type: Integer
 - Range: 1450 to current year (reasonable range for publication years)
 - ISBN:
 - Format: 13-digit number
- Default Values:
 - Genre: "Unknown" (if not specified)
- Procedures:
 - You may see if the book is available at the library right now by using the "CheckAvailability" method.
 - A method to update the book's record when it is borrowed or returned is called UpdateRecord

Book Frame
Slots: Title: "To Kill a Mockingbird" Author: "Harper Lee" Publication Year: 1960 ISBN: "978-0-06-112008-4" Genre: "Fiction"
Facets: Publication Year: - Type: Integer - Range: 1450 to current year - ISBN: - Format: 13-digit number
Default Values: Genre: "Unknown" (if not specified)
Procedures: Check Availability: Method to check if the book is currently available in the library. Update Record: Method to update the book's record when it is borrowed or returned.

Figure 1: Complete Frame of a Book-Details

This container holds all the essential details about a book and offers ways to engage with those details.

Frame Inheritance: A First Look, One way that knowledge representation systems handle and organise information efficiently is through frame inheritance. The ability for one frame (the kid) to inherit characteristics and settings from another frame (the parent) establishes a hierarchical framework. This approach makes it easier to build upon and repurpose prior knowledge.

Key Concepts of Frame Inheritance

1. A parent frame defines global attributes and properties, which are inherited by all its child frames.
2. The parent frame bestows its attributes and properties onto the child frame. In order to express more precise information, it can either add new attributes or replace current ones.
3. Bequests Structured like a tree, hierarchies allow frames to be arranged in a certain order. There can be a web of relationships formed when a child frame inherits from more than one parent frame.
4. When an attribute is overridden, it means that a child frame has replaced the inherited value or specification with a more specific one.

Fifthly, extension is the process of giving a child frame features that its parent frame does not have.

How Frame Inheritance Works?

1. Establish Parent Frame: Make a generic frame with shared characteristics. Frames with the class "Vehicle" may contain information such as "Make," "Model," and "Year."
2. Make a New Frame by Defining a Specific Offspring Frame. As an illustration, the "Car" frame may take on characteristics from the "Vehicle" frame and additionally include details like "Number of Doors."
3. Make use of inherited attributes; doing so provides an organised approach to building upon prior knowledge, as the child frame incorporates all attributes from the parent frame automatically.
4. Override or Extend: You can enhance the representation by modifying or adding attributes in the child frame as appropriate. To define a range of permissible values, the "Car" frame may, for instance, supersede the "Year" property.

Example of Frame Inheritance

Let's consider an example with a hierarchy of frames in a library system:

- Parent Frame: "LibraryItem"
 - Attributes:
 - Title
 - Author
 - Publication Year
- Child Frame 1: "Book" (inherits from "LibraryItem")

- Inherited Attributes: Title, Author, Publication Year
- Extended Attributes:
 - ISBN
 - Genre
- Child Frame 2: "Magazine" (inherits from "LibraryItem")
 - Inherited Attributes: Title, Author, Publication Year
 - Extended Attributes:
 - Issue Number
 - Publisher

In this example:

- The "Book" frame takes on the general properties of the "LibraryItem" frame and augments them with book-specific properties.
- Similar to the "LibraryItem" frame, the "Magazine" frame takes on magazine-specific properties.

Applications of Frames in AI

1. First, there's natural language processing (NLP), which makes use of frames to decipher the meaning of phrases and sentences. To understand reservation requests and extract pertinent information like time, date, and number of persons, a "booking" frame could be utilised.
2. Expert Systems: In order to store information pertaining to particular domains, expert systems make use of frames. To illustrate different illnesses, symptoms, and treatment choices, a medical diagnosis system, for instance, may use frames.
3. Frames provide organised information about things and their properties, which robots use to understand their environment. Robots can now manipulate and recognise objects because to this.
4. Fourthly, cognitive modelling makes use of frames to mimic the way people think. Researchers can build models that simulate human thinking and decision-making by expressing information in frames.

Advantages of Using Frames

- Knowledge Organisation: Frames facilitate the organisation of data in a manner that is analogous to real-world situations, which aids AI systems in comprehending and processing the data.

- Adaptability to new information and contexts: Frames can be simply extended or adjusted to accommodate new details.
- You can reuse frames across different applications or scenarios once they are defined, which promotes reusability, consistency and efficiency.

Challenges and Limitations

- Complexity: Managing and maintaining the frames can grow complex as the number of them and their interrelationships rise.
- Context Sensitivity: In uncertain or rapidly changing environments, established frameworks might not work as well as intended.
- Scalability: Performance and resource management can be made more difficult for large-scale systems due to the sheer number of frames and how they interact with one another.

Difference between Frames and Ontologies

Table 2: Frames Vs. Ontologies

Aspect	Frames	Ontologies
Definition	Data structures representing specific situations	Formal representations of knowledge domains
Structure	Slots, facets, default values, procedures	Classes, subclasses, properties, instances
Flexibility	Adaptable to specific contexts and scenarios	Formal and standardised, designed for consistency across domains
Usage	NLP, expert systems, cognitive modelling	Semantic web, knowledge management, data integration
Context	Context-specific, can vary in structure	Domain-wide, provides a shared understanding

Aspect	Frames	Ontologies
Formalism	Less formal, more flexible	Highly formal, uses specific languages (e.g., OWL)

Although they have distinct functions, ontologies and frames are equally useful for AI knowledge representation. Applications that need to be flexible and adaptable commonly make use of frames because of how effective they are for portraying specific, context-dependent scenarios. In contrast, ontologies offer a formal, standardised method of representing knowledge across whole domains, which promotes consistency and interoperability. If you know what to look for in a tool, you can pick the right one for any job.

In artificial intelligence, frames play a crucial role in representing and organising data about the environment. Frames improve AI systems' reasoning, inferring, and decision-making capabilities by offering a systematic approach to encapsulating information. Frames continue to be an essential part of many AI applications, such as robotics and natural language processing, despite their difficulties. The importance of frames in supporting intelligent systems is projected to grow as AI develops further.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	A frame in AI is primarily used to represent: A. Logical rules B. Facts and figures C. Structured knowledge about objects D. Sound waves
7	In a frame, a “slot” is: A. A storage location for video B. A communication channel C. An attribute of an object D. A type of memory
8	Which of the following is NOT typically a feature of a frame? A. Default values

	B. Inheritance C. Procedural attachments D. Time complexity algorithm
9	Frames are particularly useful for: A. Representing sequential events B. Representing stereotyped situations C. Mathematical calculations D. Data compression
10	What allows a frame to inherit information from another frame? A. Slot sharing B. Default templates C. Subclass relationship D. Rule chaining



6. SCRIPT THEORY IN ARTIFICIAL INTELLIGENCE

Script theory, introduced by Roger Schank in the 1970s, applies cognitive psychology concepts to AI by representing human behaviours as structured sequences of actions (scripts). This helps AI systems comprehend and predict typical behaviours in specific contexts.

What is Script Theory?

Script theory originates in cognitive psychology and is applied in artificial intelligence to model human conduct according to preexisting scripts, allowing robots to understand and anticipate human actions. By outlining a framework for recording archetypal sequences of behaviours in certain contexts, cognitive scientist Roger Schank proposed script theory in the 1970s as a means by which AI systems may imitate human understanding and interaction.

Key Components of Script Theory

Scripts consist of various essential parts:

1. **Scenes:** Scenes are the building blocks of a script that describe individual events or actions.
2. **Actors:** The things (humans, machines, etc.) that are involved in the situations and doing things.
3. **Props:** Things that are part of the performance.
4. The prerequisites that must be satisfied before a script may begin running are called entry conditions.

The outcomes or goals that are reached by completing the script are called results.

Scenes like "entering the restaurant," "ordering food," "eating," and "paying the bill." are common in restaurant scripts. The client and the server could play the roles of the actors, the menus and food items could represent the props, the customer's hunger could be the entry condition, and the customer's satisfaction after eating could be the result.

Applications of Script Theory in AI

Several areas of artificial intelligence have found success using script theory to improve understanding, prediction, and interaction capacities:

- **Data Science for Natural Language Processing:**
NLP systems rely on scripts to comprehend context and produce text that is relevant to it. As an example, chatbots can grasp typical conversational sequences such as welcomes,

information requests, and farewells since they employ scripts to mimic human discussions.

- **Robotics:** Machines carry out complicated operations according to predefined scripts. Using the following cleaning script as an example: "vacuuming the living room," "dusting the shelves," and "mopping the kitchen floor." This would be a typical sequence of operations for a domestic robot.
- **Artificial intelligence in video games:** NPCs act realistically and in predictable patterns by following scripts. Non-playable characters (NPCs) in simulation games, for instance, may have set schedules that include things like "waking up," "going to work," "eating lunch," and "returning home." Using scripts and past data, AI systems can foretell what's going to happen next.
- **Industrial predictive maintenance systems,** for instance, utilise scripts to foretell when machinery would break down in response to consumption trends and data collected from sensors.

Advantages of Using Script Theory in AI

Scripts are useful in AI systems for many reasons:

- Save time and cut down on manual labour by automating complicated operations. This will increase efficiency.
- Make sure procedures can be reliably and repeatedly executed for consistency's sake.
- Efficiently manage complex models and massive datasets; scalable.
- Sharing and reproducing results across systems and environments is made easy with reproducible scripts.

Challenges and Limitations of Using Script Theory in AI

Despite scripts' numerous benefits, they can pose a number of difficulties:

- **Adaptability:** Rigid scripts make it hard to deal with out-of-the-ordinary situations.
- **Difficulty:** It can be tedious and time-consuming to create thorough scripts for every conceivable scenario.
- **Upkeep:** It takes continuous work to keep scripts updated with fresh information and adapt them to different situations.

Highly Efficient AI Systems Built on Scripts-more complex script-based systems have been developed as a result of recent advances in AI. To make scripts more versatile and adaptable, these systems use

deep learning and machine learning.

Dynamic Script Generation:

- AI systems are capable of producing scripts on the fly using context and real-time data.
- Analysing the user's calendar, emails, and preferences, an AI personal assistant may generate a script for the user's daily schedule.

Adaptive Scripts:

- Scripts with the ability to adjust to new circumstances or user actions.
- As an example, when driving autonomously, the navigation script can change the route depending on things like road closures and traffic.

Hierarchical Scripts:

- Programs that can execute at many levels of abstraction, enabling behaviours that are more intricate and subtle.
- As an example, in the medical field, a diagnostic AI may utilise broad scripts for general patient evaluation and more particular scripts for individual diagnostic procedures.

Case Study: Script-Based AI in Healthcare

The use of scripts by AI systems to aid in medical care and diagnostics is one prominent example of script theory's application in AI. As an example, a doctor may be led by a scripted series of questions and tests when a patient presents with certain symptoms by an AI-powered diagnostic assistance.

1. Initial Assessment Script:

- The AI starts by requesting the patient's medical history and main symptoms in an initial assessment script.
- The AI chooses relevant follow-up questions and assessments depending on the answers.

2. Diagnostic Testing Script:

The AI will initiate a diagnostic testing script that specifies the required medical tests (such as blood tests and imaging) and their proper sequence if the initial evaluation suggests a possible condition.

3. Treatment Recommendation Script:

After a diagnosis has been determined, the AI will follow a script that suggests treatments based

on evidence. The script will detail how to put the chosen treatment plan into action.

Improving diagnosis accuracy and treatment outcomes, healthcare AI systems can use scripts to guarantee a consistent and comprehensive approach to patient care.

Future Directions

Exciting prospects are ahead for script theory in artificial intelligence. Systems that rely on scripts are anticipated to advance in complexity, adaptability, and domain integration as artificial intelligence (AI) technology does. Possible developments encompass:

- AI systems with enhanced context awareness can interpret context better and incorporate it into scripts for more adaptable and personalised interactions.
- Multidisciplinary Use: Bringing script-based AI to more domains including teaching, banking, and customer service.
- Collaborative AI: creating AI systems that can work with other AI systems and humans, coordinating complicated interactions between multiple agents using scripts.

To model and automate sequential behaviour in AI systems, script theory offers a powerful framework. Artificial intelligence (AI) can improve efficiency and consistency across applications while achieving more human-like comprehension and interaction by employing scripts. Although there are still obstacles, the continuous progress in AI is creating opportunities for script-based systems that are smarter, more adaptive, and more versatile.

7. SITUATION CALCULUS

To model and reason about domains that are in a constant state of flux, logicians developed situation calculus.

In 1963, cognitive scientist and computer scientist John McCarthy initially proposed it.

The core principle is that the steps needed to get to a certain state (here called a situation) may be described in terms of those steps. The scenario can be used as an argument to define what holds true in terms of relations.

To put it simply, situational calculus is the action representation based on features.

Here, we only think about deterministic behaviours, fully observable environments, and solitary agents.

States can be linked to situations. Situations can be linked to states. Two key distinctions are:

- State-to-state equality and situation-to-situation equality are distinct concepts. There may be multiple contexts referring to the same state if multiple acts lead to the same state.
- Every state does not have to have the same circumstances. Unreachable states do not have comparable situations; they are exclusive to accessible states. To be accessible, a state must have an existing sequence of activities that may be used to go from the starting state to that state.

A situation is either:

- **init:** This is the initial situation, or
- **do(A,S):** This is the situation that arises because of performing action A in situation S, if it is possible to do action A in situation S.

The $do(A,S)$ function does not map to every possible state. However, there are situations where an agent must consider such a scenario without knowing if A is feasible in state S or if S is doable overall.

In AI, why do representations matter so much?

When it comes to AI, knowledge representation is crucial. Thinking and how thinking contributes to the intelligent behaviour of artificial intelligence systems is the focus of knowledge representation and reasoning (KR, KRR), a topic of artificial intelligence.

Data mining is the process of transforming raw, real-world data into a form that computers can interpret, analyse, and apply to real-world issues and tasks.

Machines are able to comprehend, interpret, and reason thanks to knowledge representation. Also covered is the topic of knowledge representation in AI. Not only can it store information in a database, but it also enables smart robots to learn from their own experiences and knowledge, allowing them to mimic human intelligence in their behaviour.

Simply said, knowledge representation is crucial because it gives AI-powered computers the ability to learn from existing information, experience, or experts—basically making them more human-like.

Systems driven by AI require representations of the following types of knowledge:

- **Things:** This category includes all the information regarding things in our physical reality.
- **Event:** Things that happen in the world are called events. To put it simply, performance is the application of one's understanding of how to accomplish things.
- **Meta Knowledge:** What we call "meta-knowledge" is really just information about our own knowledge.
- **Truths:** These are verifiable facts about the world around us and our representation of it.
- **Knowledge-Base (KB):** This is the meat and potatoes of knowledge-based agents. It is a string of phrases.

The ontology of situation calculus

The situations of ontology consists of the building blocks of calculus are fluent, situations, and actions.

One way to look at a situation is as a quick picture of the world around you. This logical phrase encompasses both the original circumstance and any subsequent situations that develop as a result of an action being taken.

Any variable-valued entity could be considered fluent. You can think of them as situationally dependent functions and predicates. The value of a fluent is true when it applies to a certain context and false when it does not. You can also have functions that are not time-bound or external. The function `LeftLegOf(Wumpus)` and the predicate `Gold (G1)` are two such instances.

Two axioms define each action in the most basic form of situation calculus. The effect maximum and the possibility axiom are two such concepts.

The possibility axiom specifies the conditions under which the action can be carried out, and the effect axiom describes the results of carrying out the action.

Frame Problem in AI

One problem with expressing information about a robot in the real world using first-order logic (FOL) is shown by the frame problem in artificial intelligence (AI). Many axioms implying that items in the environment do not change arbitrarily are necessary when trying to represent a robot's state using traditional first-order language.

A 1969 essay by John McCarthy and Patrick J. Hayes titled "Some Philosophical Problems from the Standpoint of Artificial Intelligence" was the first to define the problem. Knowledge representation for AI is notoriously difficult, and this problem served as a jumping off point for other broader talks on the topic.

To have a workable description of a robot's environment, the frame problem boils down to finding enough suitable sets of axioms.

Even in relatively straightforward areas, this issue persists. The issue is that it is not sufficient to state which conditions are altered by the acts; this does not guarantee that no other conditions are altered. Incorporating frame axioms can solve it. It is explicitly stated in these frame axioms that while an action is being executed, all circumstances that are not affected by it are preserved.

Each action-condition pair must adhere to one of these frame axioms in order for the action to have no effect on the condition. The challenge lies in formalising a domain of dynamics without providing explicit specifications for the frame axioms.

The Hayes–McCarthy definition of the frame problem was finally resolved in the late 1980s.

Several formalisations resolve the frame problem. Some examples of solutions that illustrate this process are as follows:

- Fluent occlusion solution
- Predicate completion solution
- Successor state axioms solution
- Fluent calculus solution
- Event calculus solution
- Default logic solution
- Answer set programming solution
- Separation logic solution
- Action description languages (these pretty much elude the problem instead of solving it)

8. SUMMARY

- This unit focuses on the foundational concepts and techniques used in Knowledge Representation (KR) within Artificial Intelligence. KR is the core of intelligent behaviour, as it determines how knowledge about the world is stored, processed, and used by AI systems. It ensures that machines can simulate human-like reasoning and decision-making.
- We begin with an overview of Knowledge Representation, exploring why structured knowledge is essential for AI to perform tasks such as inference, learning, and planning. We cover different representation methods—declarative, procedural, semantic, and logical—and emphasise the need for clear, machine-interpretable models.
- In Semantic Networks, we study graph-based models where concepts (nodes) are connected by relationships (edges). These networks mimic human memory structures and are useful for expressing taxonomies, inheritance, and conceptual associations.
- Next, Types of Semantic Networks such as definitional networks, assertional networks, and implicational networks are discussed, each tailored to a specific type of reasoning—definitional hierarchies, factual assertions, or rule-based implications.
- The concept of Frames builds upon semantic networks by organising knowledge into units with slots and values. Frames represent stereotypical knowledge and help simplify reasoning about objects and events by encoding default values and expectations. They are highly effective in systems that require a structured but flexible representation of objects and contexts.
- Script Theory extends frames to dynamic scenarios. Scripts model common human activities (like going to a restaurant) as a sequence of events with roles, actions, and outcomes. This allows AI to understand and predict events based on prior knowledge, making it useful for natural language processing and behavioural prediction.
- Finally, Situation Calculus introduces a logical framework for modelling changes over time. It represents actions, their preconditions, and their effects in a formal way, allowing AI systems to perform reasoning in dynamic environments. This is critical in domains like robotics and automated planning.
- Together, these techniques offer a comprehensive toolkit for representing structured knowledge in AI. They allow systems to not only store and retrieve facts but also to reason, learn from interactions, and adapt over time—making them indispensable for building truly intelligent applications.

9. GLOSSARY

Knowledge Representation (KR)	- A field in AI focused on how information and rules about the world can be stored in a format that machines can process, reason with, and act upon.
Semantic Network	- A graph-based structure where nodes represent concepts and edges represent relationships. Used for expressing hierarchical and associative knowledge.
Node (in Semantic Networks)	- A representation of a concept, object, or event within a semantic network (e.g., "Dog", "Vehicle").
Edge (in Semantic Networks)	- A labelled link between two nodes showing a relationship (e.g., "Is-A", "Has-Part").
Inheritance (in Semantic Networks)	- A mechanism allowing a concept to inherit properties from a more general category (e.g., "A robin inherits the ability to fly from birds").
Frame	- A data structure for representing a stereotyped situation, object, or concept, consisting of slots (attributes) and fillers (values).
Slot (in Frames)	- An attribute or property within a frame (e.g., color, size, location).
Filler (in Frames)	- The value or content assigned to a slot (e.g., color: red).
Default Value (in Frames)	- A common or expected filler for a slot unless stated otherwise (e.g., default fuel for car: petrol).
Script	- A structured representation of a sequence of events in a stereotypical scenario (e.g., dining at a restaurant), used to understand and predict actions.
Scene (in Scripts)	- A subdivision of a script representing a specific phase of an activity (e.g., "Ordering food" in the restaurant script).
Actor (in Scripts)	- A participant or role in the script (e.g., customer, waiter).

Slot-and-Filler Structure	- A common format used in both frames and scripts to store and retrieve knowledge efficiently.
Situation Calculus	- A logical language for representing and reasoning about dynamic worlds, using formal descriptions of actions, states (situations), and their effects.
Action (in Situation Calculus)	- An event that changes the state of the world (e.g., “OpenDoor”, “Move”).
Fluent	- A function or predicate that describes how a property can change over time (e.g., “LightIsOn” may be true or false depending on the action “ToggleSwitch”).
Precondition	- A condition that must be true before an action can be performed (e.g., a door must be unlocked before it can be opened).
Effect	- The outcome or result of an action performed in a situation (e.g., after “OpenDoor”, the door’s state becomes “open”).
Logical Reasoning	- The process of using formal logic to derive conclusions from known facts and rules; includes propositional, predicate, and modal logics.
Ontology	- A structured set of terms and categories that define the properties and relationships of entities in a particular domain (e.g., medical ontology).

10. TERMINAL QUESTIONS

1. Define Knowledge Representation in AI. Explain its significance with examples.
2. Explain Semantic Networks. Illustrate how they represent knowledge with a diagram.
3. Differentiate between Types of Semantic Networks with suitable examples.
4. What is a Frame in AI? Describe its structure with an example.
5. Compare and contrast Frames and Scripts in AI.
6. What is Script Theory? Explain with an example scenario.
7. Define Situation Calculus. How does it help in dynamic decision-making?
8. Discuss the challenges faced in Knowledge Representation.
9. Explain the concept of inheritance in Semantic Networks with an example.
10. What are the real-world applications of Knowledge Representation in AI?

11. ANSWERS

Self-Assessment Questions

1. Answer: C. Relationships between concepts
2. Answer: B. Nodes
3. Answer: C. Expressing hierarchical classification
4. Answer: B. Structured inference through relationships
5. Answer: C. A relationship
6. Answer: C. Structured knowledge about objects
7. Answer: C. An attribute of an object
8. Answer: D. Time complexity algorithm
9. Answer: B. Representing stereotyped situations
10. Answer: C. Subclass relationship

Terminal Questions Answers

1. Answer: Knowledge Representation (KR) in AI refers to the method used to encode information about the world into a format that a computer system can utilise to solve complex tasks such as diagnosing diseases, understanding language, or controlling robots. KR allows machines to simulate intelligent reasoning. For example, a medical diagnosis system uses rules and facts to infer diseases based on symptoms. The significance lies in enabling machines to make informed decisions, understand context, and explain reasoning, which is essential for intelligent behaviour. (Refer to Section 2 for more information)
2. Answer: A Semantic Network is a graph-based structure used to represent relationships between concepts. Concepts are represented as nodes, and relationships are represented as edges. For instance, in a semantic network:
 - "Dog" → (Is-A) → "Animal"
 - "Dog" → (Has) → "Tail"

This structure supports inheritance and associative reasoning. Semantic networks are used in NLP, knowledge graphs, and intelligent tutoring systems. (Refer to Section 4 for more information)

3. Answer:

Types of Semantic Networks include:

- Definitional Networks: Focus on hierarchical "Is-A" relationships. E.g., "Car" is a "Vehicle".
- Assertional Networks: Store factual knowledge. E.g., "Paris is the capital of France".

- Implication Networks: Show conditional relationships. E.g., “If it rains, the ground gets wet”.

Each type serves a different purpose depending on whether the focus is on classification, facts, or logic-based reasoning. (Refer to Section 4 for more information)

4. Answer: A Frame is a data structure that holds knowledge about a stereotyped object or situation. It consists of slots (attributes) and fillers (values).

Example – Frame: Car

- Slots:
 - Color: Red
 - Engine: Petrol
 - Wheels: 4

Frames support inheritance and default reasoning. They are widely used in robotics, NLP, and expert systems. (Refer to Section 5 for more information)

5. Answer:
- Frames represent static knowledge about objects or concepts using attributes.
 - Scripts represent dynamic, event-based knowledge using sequences of actions in a scenario.
 - Example:
 - Frame: Restaurant (has menu, waiter, kitchen)
 - Script: Going to a restaurant (enter → order → eat → pay → exit)

While frames capture object features, scripts capture temporal sequences and expectations. (Refer to Sections 4 and 6 for more information)

6. Answer:

Script Theory represents structured sequences of events in routine situations. Each script has roles (actors), props (objects), and scenes (steps).

Example – Restaurant Script:

- Scene 1: Enter the restaurant
- Scene 2: Order food
- Scene 3: Eat
- Scene 4: Pay and leave

AI systems use scripts in understanding natural language and planning tasks, where events follow predictable orders. (Refer to Section 6 for more information)

7. Answer:

Situation Calculus is a logical language used to describe and reason about change in dynamic environments. It models the world as a series of actions and resulting situations.

Example:

- Action: "TurnOn(Light)"
- Situation: Light becomes "on"

It uses actions, preconditions, and effects to infer future states. It's useful in robotics and planning systems where decision-making depends on the sequence and outcome of actions. (Refer to Section 7 for more information)

8. Answer:

Major challenges include:

- Complexity: Representing the vast knowledge of a domain
- Ambiguity: Interpreting vague human language
- Scalability: Handling growing knowledge bases
- Reasoning: Efficient inference from stored knowledge
- Acquisition: Converting raw data into machine-readable knowledge

Solving these requires advanced representation models and machine learning techniques.

9. Answer:

Inheritance in Semantic Networks allows nodes to inherit properties from parent nodes.

Example:

- "Bird" → has wings, can fly
- "Sparrow" → (Is-A) → "Bird"

The system infers that a sparrow can also fly and has wings, without explicitly stating it. This reduces redundancy and supports efficient reasoning. (Refer to Section 4 for more information)

10. Answer:

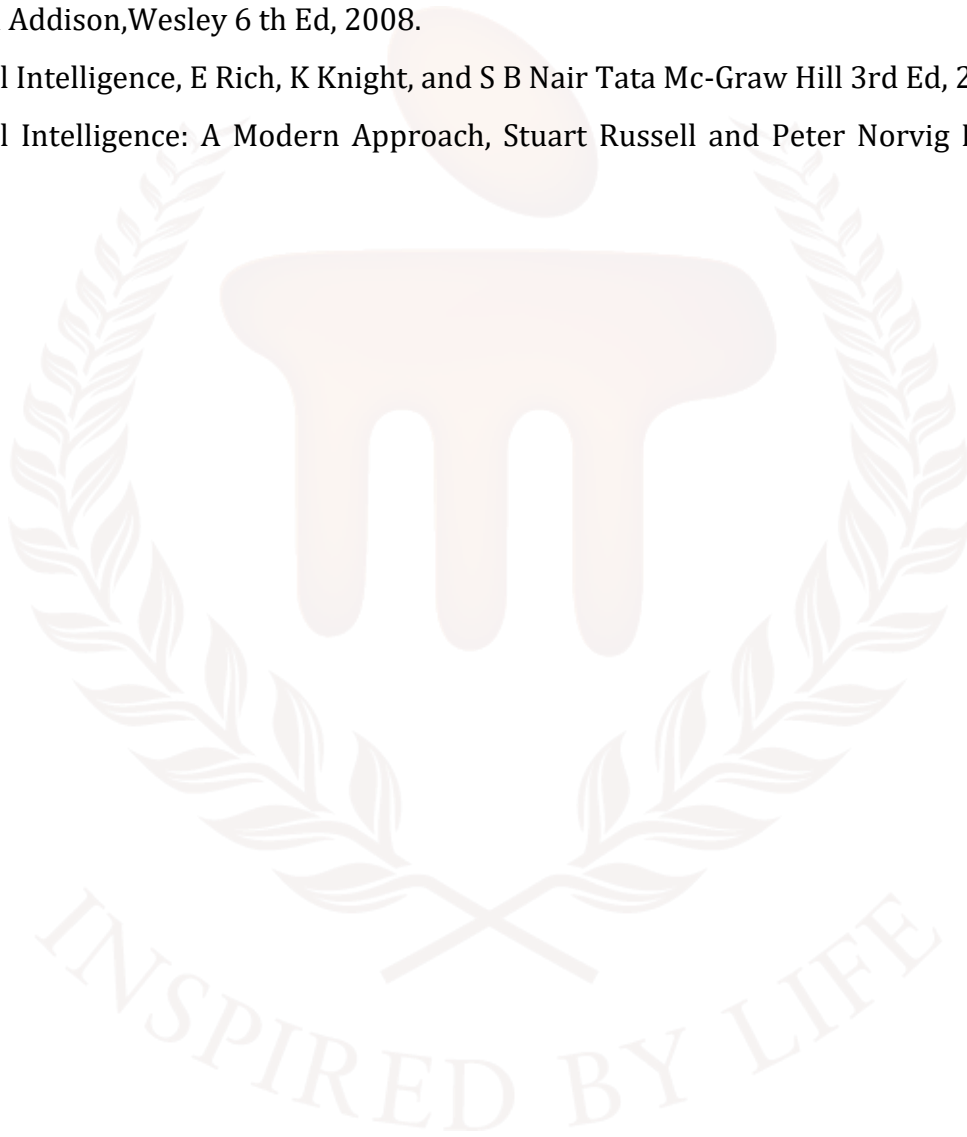
Key applications include:

- Expert Systems (e.g., medical diagnosis)
- NLP systems (e.g., chatbots, sentiment analysis)
- Robotics (e.g., path planning, object manipulation)
- Semantic Web (e.g., linked data, search engines)
- Cognitive Computing (e.g., IBM Watson)

AI uses KR to store, access, and reason about information in all these areas. (Refer to Section 2 for more information)

12. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

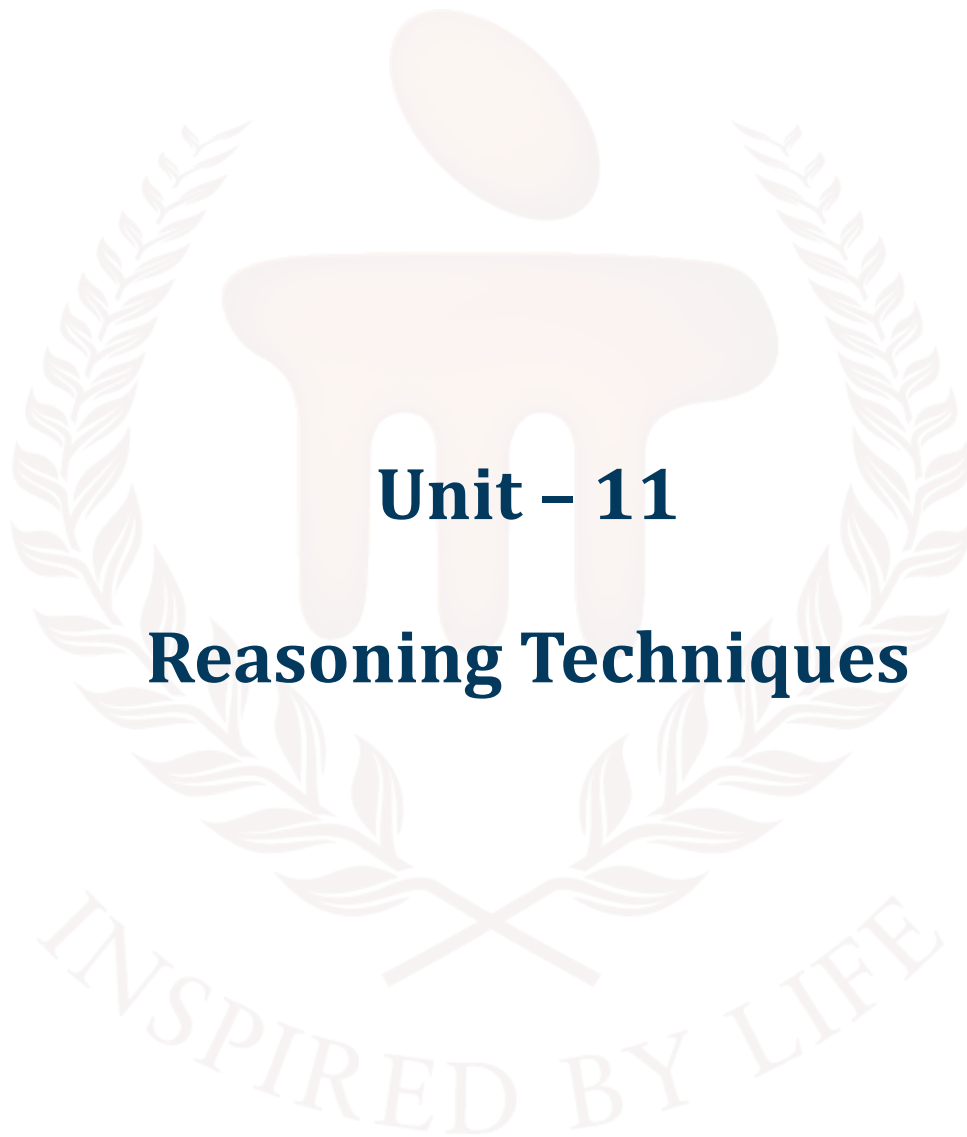


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109
ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING



Unit – 11

Reasoning Techniques

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Reasoning Techniques	-	-	6 - 8
3	Types of reasoning in AI	-	-	9 - 15
4	Forward chaining	1	1	16 - 20
5	Backward chaining	2	2	21 - 24
6	Resolution	-	3	25 - 31
7	Summary	-	-	32 - 33
8	Glossary	-	-	34 - 35
9	Terminal Questions	-	-	36
10	Answers	-	-	37 - 39
11	References	-	-	40

1. INTRODUCTION

In the previous unit, we had explored the essential concepts and techniques involved in Knowledge Representation (KR), which is the foundation for enabling artificial intelligence systems to understand, reason, and act based on the information they possess. We'll study how knowledge can be visually represented using graph structures, where concepts are nodes and relationships are edges. This helps AI systems infer facts and build meaningful connections. We had explored frames as structured data forms that organise knowledge into slots and values. Frames are useful for representing stereotypical situations or objects and support inheritance and default reasoning. Then the script section had introduced scripts—predefined, structured representations of sequences of events in specific contexts (e.g., visiting a restaurant). Scripts help AI predict and understand actions in everyday scenarios.

In this unit, learners will explore fundamental reasoning techniques that enable intelligent systems to draw logical conclusions and make decisions. These techniques are essential for tasks such as expert systems, rule-based reasoning, and automated theorem proving. Learners will study Forward Chaining, a data-driven approach where inference starts from known facts and applies rules to derive new facts until a goal is reached. This method is widely used in systems that operate by continuously updating their knowledge as new information becomes available.

They will also learn about Backward Chaining, a goal-driven reasoning technique. Here, the system starts from the goal and works backward to determine what facts or conditions need to be satisfied. This approach is particularly common in diagnostic systems and query answering. Finally, learners will explore Resolution, a powerful rule of inference used primarily in propositional and first-order logic. It enables systems to automatically prove logical statements by refuting their negations, forming the basis of many logic-based AI algorithms.

By the end of this unit, learners will understand how these reasoning techniques work, where they are applied, and how they help AI systems think logically, solve problems, and make informed decisions.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the concepts of Reasoning in AI
- Assess the functionalities of different types of reasoning techniques
- Differentiate between the forward and backwards reasoning
- Recall the ontology of the resolution



2. REASONING TECHNIQUES

Reasoning is the process of using what you already know to come to conclusions, make predictions, or come up with solutions. Reasoning is very important in AI because it helps us understand how the human brain thinks, comes to conclusions, and solves issues. AI systems can act like humans when it comes to making decisions and solving problems by using reasoning. The following are different types of reasoning used in AI.

A software platform called an AI reasoning system uses logical methods like deduction and induction to come to conclusions based on what it knows. Large Language Models (LLMs) have evolved with the advent of Generative AI. Earlier, they produced quick, pre-trained responses, but now modern systems can also engage in deeper, real-time reasoning to solve complex problems.

This move from gut reactions to well-thought-out choices based on logical thinking is a game-changer. It opens up a whole new set of Agentic AI applications that can deal with the complexities of the actual world.

It's evident that just looking for information and making content isn't adequate as business problems get more complicated. AI needs to stop, think, and come to conclusions in real time. Pre-trained models use "training-time computing" to guess the next word based on a lot of data they have already seen.

That works for easy things. But rapid, instinctual answers don't work for hard, high-stakes challenges. It takes time, ingenuity, and deliberate thought to make real progress. The same is true for AI.

This is when AI reasoning comes in. A model doesn't only repeat patterns or make predictions based on past data when it "stops to think." It's about considering multiple options, thinking through the results, and making choices based on reason. Inference-time computing is a type of thinking that takes a little more work but gives you much more useful results.

We're challenging AI to go beyond simple answers by making it "stop and think." This lets it solve the big, complicated problems that actually matter, and that's where AI is going in the future.

Importance of Reasoning in Artificial Intelligence

Reasoning changes AI from a simple tool for automating tasks to a smart partner for making decisions. Reasoning lets AI predict outcomes, weigh trade-offs, and change strategies based on changing inputs,

just like how people think in business settings where problems are complicated and not always clear-cut.

This makes the findings more accurate, dependable, and aware of the situation. It's not enough to just fix problems; you also need to really understand them, use logic, and make sure that your solutions fit with the bigger goals of the business. Adding reasoning skills to AI makes it better able to handle unclear or contradictory information and come up with answers that are in line with company goals.

How reasoning in AI works

Reasoning in AI is depicted as a system typically made up of two core components:

- Knowledge base
- Inference engine

Knowledge base

The knowledge base is what an AI reasoning system is built on. It has paradigms for representing knowledge, such as knowledge graphs, ontologies, semantic networks, and more. These structured forms turn things that exist in the actual world, like concepts, domain-specific information, events, facts, objects, relationships, rules, and situations, into a structure that AI models can understand and work with.

Engine for inference

The inference engine is the part of an AI reasoning system that thinks. It runs on machine learning models that have been trained. The inference engine has the logic and reasoning tools it needs to look at data from the knowledge base and make a choice.

Techniques of Knowledge Representation

There are different methods that make KR&R systems work, and each one has its own purpose:

- Logical Representation: Uses formal logic to clearly and logically explain how things are related to each other. It lets you use logical thinking and helps you check the conclusions you've drawn.
- Semantic Networks: Use graphical notations to show relational knowledge, which makes it easier to show how concepts are related to each other.
- Frames and Scripts: Use template structures to show common situations so that systems can predict what will happen and respond appropriately.

- Rule-Based Systems: Use groups of conditional statements to tell the inference engine how to make judgements, just like an expert would.



3. TYPES OF REASONING IN AI

The reasoning is classified into the following types:

1. Deductive Reasoning

Deductive reasoning starts with general principles or premises that are known or thought to be true and then draws conclusions from them. This kind of reasoning uses known data to come to valid conclusions.

If all people are mortal and Socrates is a person, then Socrates is mortal. Deductive reasoning is commonly employed in expert systems and rule-based AI systems, where rules (if-then statements) show what knowledge is. These systems use general rules to figure out how to solve problems or make judgements.

2. Inductive Reasoning

Inductive reasoning begins with specific observations and derives broader generalisations or hypotheses, which are less certain than conclusions from deductive reasoning. For example, if we see that the sun rises in the east every day, we can guess that it will rise in the east again tomorrow.

Inductive reasoning is a common part of machine learning algorithms. Models that have been trained on data patterns can use this information to make predictions about fresh, unseen data.

3. Abductive Reasoning

Abductive reasoning starts with a small number of observations and then tries to find the most likely explanation. Instead than looking for an exact fact, it looks for the most plausible conclusion based on what is known.

For instance, if a patient has a fever and a cough, a doctor might think they have the flu, even if other illnesses could have the same symptoms.

Abductive reasoning is used in AI for diagnostic systems like medical diagnostics tools and fault detection systems, which try to find the most likely source of a problem when there isn't enough evidence.

4. Analogical Reasoning

Analogical reasoning involves comparing two similar situations and applying knowledge from one to solve problems in the other, allowing AI systems to draw parallels between different scenarios.

If flying a drone is like flying a helicopter, then anything you learn about flying a helicopter can be used to fly a drone.

Use in AI: Analogical reasoning is helpful in AI for figuring things out, making choices, and sharing knowledge, notably in robotics and cognitive systems.

5. Common Sense Reasoning

Common sense thinking uses what we know and have experienced in everyday life to get to conclusions. It mimics how people employ common sense to deal with everyday problems, which is something that AI often has trouble with because this information isn't always clear.

We can expect the ground to get wet if it rains, even if we don't say it out loud.

Application in AI: Developers are working on AI systems like conversational bots (e.g., Siri, Alexa) that leverage common sense reasoning to make interactions with users more natural and complicated.

6. Monotonic Reasoning

Monotonic thinking is a type of reasoning in which conclusions can't be changed, even if new evidence comes to light. This makes sure that the conclusions stay the same even when the knowledge base is changed.

For example, the sentence "The Sahara is a desert" is still valid even if we learn more about deserts around the world.

Monotonic reasoning is used in traditional reasoning systems and logic-based AI, where consistency is very important. This kind of reasoning is used by systems like formal verification tools to make sure that conclusions don't alter over time.

7. Nonmonotonic Reasoning

Nonmonotonic reasoning, on the other hand, lets AI systems change their findings when they learn new things. This is quite helpful in places where the knowledge base is always changing.

For example, you might first think that all birds can fly, but then change your mind when you find that penguins can't fly.

In AI, nonmonotonic reasoning is employed in systems that make decisions that change based on new information or contexts, like real-time traffic management or adaptive learning systems.

8. Fuzzy Reasoning

Fuzzy reasoning deals with uncertainty and imprecision by letting the truth be a range of values instead of just true or false. This makes it a good choice for real-world situations where data isn't always clear or full.

For example, "It is warm outside" is a vague statement in human language. Instead of being either true or untrue, fuzzy reasoning can give a degree of truth, like 0.7 warm.

Use in AI: Fuzzy reasoning is used a lot in control systems, including in air conditioners, washing machines, and self-driving cars, where exact measurements aren't always possible.

Challenges and Problems of AI Reasoning Systems

Challenges:

Reasoning can lead to more powerful AI applications, but it also has its limits. Here are some challenges associated with AI reasoning systems:

- Bias
- Computational costs
- Interpretability

a. Bias

AI reasoning systems can inherit biases from training data. Reducing bias requires diverse data sources, human oversight, ethical algorithm development, and strong AI governance.

b. Computational costs

It is hard to scale these systems since complex reasoning processes need a lot of processing power. Businesses need to make AI models more efficient while keeping their accuracy. They also need to be ready to spend money on the tools they need to build, train, and use these reasoning systems.

c. Interpretability

A lot of the time, AI reasoning systems, especially the more complicated ones, are black boxes. They don't make their reasoning and decision-making processes clear. There are a number of ways to make AI models easier to understand, and making systems that are easy to understand can help consumers trust them.

Reasoning vs. Learning

Reasoning and learning are two fundamental yet distinct pillars of artificial intelligence.

Reasoning is the process where an AI system applies pre-existing knowledge and logical rules to solve new problems. For example, if an AI knows that "all vehicles must stop at a red light," it can reason that when it detects a red light, it should stop. Reasoning allows AI to make sense of new situations by connecting facts and following established logic, even when it hasn't seen that exact scenario before.

On the other hand, learning focuses on acquiring new knowledge from data and experience. This is the foundation of machine learning. By analysing large volumes of data, an AI system can learn patterns — for example, learning to recognise cats in photos or predict stock market trends. Learning empowers AI systems to improve over time as they process more data.

In modern AI, these two approaches are often combined. Learning enables the AI to continuously acquire and update knowledge, while reasoning helps it apply that knowledge logically and explain its decisions. This combination makes AI systems more adaptable, more reliable, and capable of handling real-world complexities in a human-like way.

Logical Reasoning Frameworks

Logical reasoning frameworks provide the foundational structure that AI systems use to process and draw conclusions from knowledge:

- The most basic framework is propositional logic, which deals with simple statements called propositions. Each proposition can either be true or false, and complex logical expressions can be built using operators like AND, OR, and NOT. For example, the propositions 'It is raining' and 'The ground is wet' can be connected to form logical rules. This type of logic is straightforward but limited in expressing detailed relationships.
- Moving beyond this, predicate logic (also known as first-order logic) allows us to represent more complex information. It introduces variables, functions, and quantifiers, enabling AI systems to describe objects, their properties, and relations among them. For example, "All humans are mortal" can be formally expressed using predicates, which is not possible with propositional logic alone. Predicate logic is crucial for representing structured knowledge about the world in a more flexible way.
- Finally, knowledge representation refers to the methods AI uses to organise and encode knowledge so it can reason effectively. This includes semantic networks that connect concepts,

frames that describe stereotypical situations, and ontologies that define categories and relationships. Strong knowledge representation allows AI to understand context, interpret new information, and support advanced reasoning tasks.

Inference Mechanisms

Inference mechanisms are the core methods that AI systems use to draw conclusions from a set of facts and rules. The two main types are forward chaining and backward chaining, and they're widely used in expert systems

Forward Chaining

- Data driven approach: starts from known facts and fires rules whose conditions match
- Continues applying rules to infer new facts until no more rules apply or a goal fact is reached
- Ideal when the complete dataset is available up front

Backward Chaining

- Goal driven approach: starts with the goal (hypothesis) and works backward to find supporting facts
- Selects rules that could produce the goal and recursively verifies their prerequisites
- Efficient when the target conclusion is clear and the fact base is large

Use in Expert Systems

- Expert systems embed both mechanisms in their inference engine
- Forward chaining for monitoring and real time alerts (e.g., process control)
- Backward chaining for decision support and diagnostics (e.g., medical or legal advice)
- Hybrid strategies combine both to balance exploration and goal focus

Problems:

Reasoning can help make AI programs more useful, but it also has its limits. Here are some problems that come up with AI reasoning systems:

- Prejudice
- Costs of computation
- Understandability

Prejudice

AI reasoning systems can pick up on biases that may be present in the training data. Using a variety

of data sources can help reduce bias. Also, making sure that these reasoning systems make conclusions in a fair and ethical way requires human monitoring, AI ethics to be built into algorithm development, and AI governance to be put in place.

Costs of computation

It is hard to scale these systems since they need a lot of processing power to do complex reasoning tasks. Companies need to make AI models more efficient without losing accuracy. They also need to be ready to spend money on the tools and people they need to build, train, and use these reasoning systems.

Understandability

A lot of the time, AI reasoning systems are black box models, especially the more complicated ones. They don't make it clear how they come to their decisions or how they think about things. There are a number of ways to make AI models easier to understand, and making systems that are easy to understand can help consumers trust them.

AI applications for reasoning:

Reasoning in AI may help businesses solve problems and automate hard tasks, which is why it is useful. AI reasoning systems can be useful in many areas, such as:

- Help with customers
- Cybersecurity
- Health care
- Making things
- Robots

Service to customers

Chatbots and virtual agents are types of conversational AI that can use AI reasoning to deliver better solutions to customer concerns. Retailers can also use logic in their recommendation engine to suggest goods that are right for each user, which makes the experience more personal and better overall.

Cybersecurity

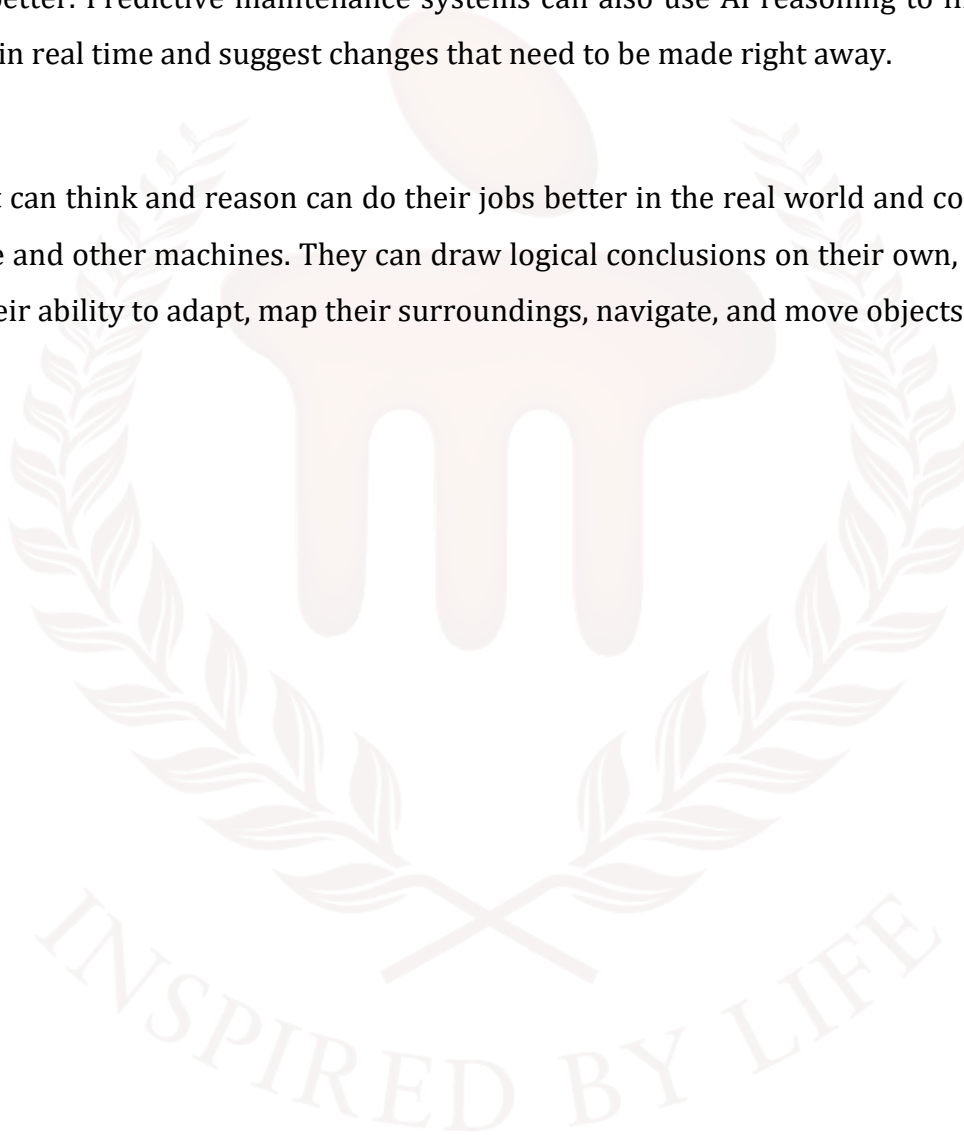
AI reasoning systems can assist cybersecurity tools keep an eye on things and discover threats. They can also rapidly recommend the best thing to do, which helps people react faster. Healthcare AI

reasoning models can help doctors figure out what's wrong with patients and how to treat them. They can also assist researchers select the best compounds to test for drug development, which speeds up the process of developing new medications.

Manufacturing AI reasoning systems can help with demand predictions to help keep track of inventory better. Predictive maintenance systems can also use AI reasoning to find problems with equipment in real time and suggest changes that need to be made right away.

Robots

Robots that can think and reason can do their jobs better in the real world and communicate better with people and other machines. They can draw logical conclusions on their own, which helps them improve their ability to adapt, map their surroundings, navigate, and move objects.



4. FORWARD CHAINING

Reasoning is very important for coming to conclusions based on what we already know while making smart systems. Forward and Backward Chaining are two main ways that AI uses to think. These methods let machines think logically and solve difficult issues quickly.

Forward Chaining starts with what is known and uses rules to identify new facts and get to a conclusion. Backward chaining starts with a conclusion and works its way back to see if there are any facts that support it.

Rule-based AI systems, such as decision support systems, computer-based diagnosis, and expert systems, usually use these methodologies. We can make smart systems that can do logical thinking and make wise decisions if we know how they work.

Inference Engine

The Inference Engine is a very important part of AI systems. It takes the information it already has to come to conclusions and then uses reasoning to come up with new ideas based on that information. This engine is very important for AI to be able to solve problems and make decisions. Inference engines were first employed in expert systems to make logical conclusions automatically. They usually work in two ways:

- Forward chaining
- Chaining backward

Forward Chaining

Forward chaining is a way to make inferences based on data. It starts with the data that is already there and uses rules to figure out additional data until a goal is accomplished. People often utilise this strategy when they have a lot of data to start with and want to draw conclusions from it.

Forward chaining is a way of reasoning that starts with facts that are already known and employs rules of inference to come up with fresh information until it reaches a conclusion. This procedure goes step by step from known information to a final conclusion. Forward chaining is a data-driven method that means the system looks at all feasible rules based on the data it has before making a final decision.

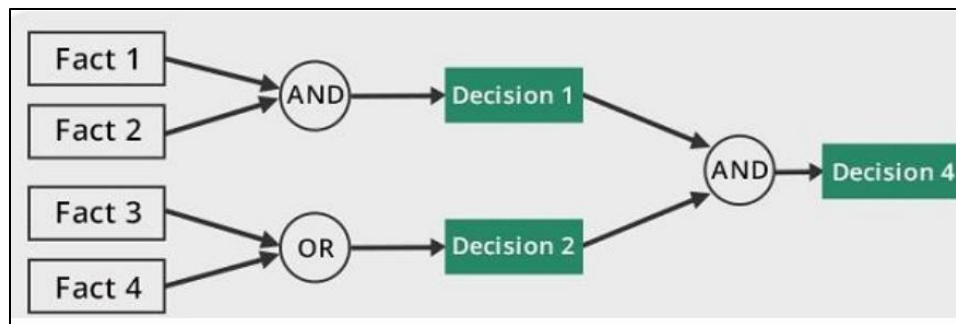


Figure 1: Working of the Forward chain

This diagram illustrates how forward chaining works by starting from known facts and applying rules step by step to derive decisions.

Facts as Inputs

- The process begins with basic facts (Fact 1, Fact 2, Fact 3, Fact 4).
- These facts are stored in working memory as the initial knowledge.

Applying Logical Conditions (AND/OR)

- Facts 1 and 2 are combined using an AND condition.
 - This means both Fact 1 and Fact 2 must be true to proceed.
 - Once satisfied, this leads to Decision 1.
- Facts 3 and 4 are combined using an OR condition.
 - This means either Fact 3 or Fact 4 being true is enough to proceed.
 - This leads to Decision 2.

Combining Intermediate Decisions

- Decision 1 and Decision 2 are then both required together (AND condition) to reach Decision 4.
- This shows how intermediate conclusions (decisions) can themselves become new "facts" in the next stage of reasoning.

Forward Chaining Process

- The system starts from available facts and moves forward by firing applicable rules.
- At each step, new decisions or facts are derived and added back into working memory.
- This continues until final goals or conclusions (like Decision 4) are reached.

Working of forward chaining

Step: 1

- Start with a set of initial facts
Facts can come from sensors, user inputs, or historical data.
- Store these in a working memory
Working memory acts as a dynamic area to hold all known facts and new conclusions as they are derived.

Step: 2

- Identify all rules whose conditions match current facts
Evaluate the conditions (IF parts) of rules using the current facts.
- Known as "firing" or "triggering" rules
Rules that match can be selected for execution in the next step.

Step: 3

- Apply the selected rule
Execute the rule to generate new knowledge.
- Add the resulting new fact(s) to the working memory
Expand the known set of facts, enabling further reasoning in the next iteration.

Step: 4

- Repeat the process
- Continue checking for new fireable rules after each update.
- Stop when no more rules can be applied or when the goal is achieved
- The process naturally ends when no further inferences are possible or a desired result is reached

Example of Forward Chaining

Consider a medical diagnosis system where rules are used to diagnose diseases based on symptoms:

- Fact: The patient has a fever.
- Rule: If a patient has a fever and a rash, they might have measles.

Starting with the known fact (fever), the system checks for other symptoms (rash). If the patient also has a rash, the system infers the possibility of measles.

Example 1

Let us imagine there is a customer complaint system designed to handle various issues. When a consumer submits a complaint (fact), the system follows specific rules such as –

- If the issue is technical, proceed with troubleshooting steps.
- If the issue is related to billing, review the payment history.
- If the problem remains unresolved, escalate it to a manager.
- The system advances by utilising facts until it identifies the appropriate solution.

Advantages of Forward Chaining

1. **Simplicity:** Forward chaining is straightforward and easy to implement.
2. **Automatic Data Processing:** It processes data as it arrives, making it suitable for dynamic environments where new data continuously becomes available.
3. **Comprehensive:** It explores all possible inferences, ensuring that all relevant conclusions are reached.
4. **Efficiency in Certain Scenarios:** It can be efficient when all possible inferences need to be made from a set of data.

Disadvantages of Forward Chaining

1. **Inefficiency in Goal-Oriented Tasks:** It can be inefficient if only a specific goal needs to be achieved, as it may generate many irrelevant inferences.
2. **Memory Intensive:** It can consume significant memory, storing a large number of intermediate facts.
3. **Complexity with Large Rule Sets:** As the number of rules increases, the system may become slow due to the need to check many conditions.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions

- | | |
|---|---|
| 1 | Which of the following best describes forward chaining? |
| | a) Goal-driven reasoning |
| | b) Data-driven reasoning |
| | c) Random rule application |
| | d) Backward search |

2	In forward chaining, reasoning proceeds: a) From goal to facts b) From facts to goal c) Randomly between facts and goals d) Only on facts without rules
3	Forward chaining is most suitable when: a) Only one specific goal needs to be proven b) We have incomplete information c) All facts are known upfront and multiple conclusions are required d) We have no rules defined



5. BACKWARD CHAINING

In AI, backwards chaining is a way of reasoning that starts with a goal or conclusion and goes backwards to identify the evidence and rules that support it. This is a "top-down" method in which the system starts with a hypothesis and then seeks to verify it by breaking it down into smaller, verifiable sub-goals.

In AI, backward chaining is a way of reasoning that is based on goals. It begins with a goal or hypothesis and then looks at the facts to see if they support the aim. The procedure goes on until all the facts are checked or no more supporting data is found by repeatedly breaking the objective down into smaller sub-goals.

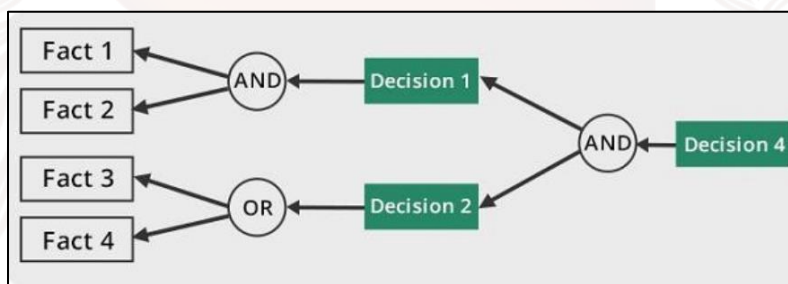


Figure 2: Working of the Backward chain

This diagram illustrates how backwards chaining works by starting from a goal and working backwards to check supporting facts.

Start with the Goal

- The final goal here is Decision 4.
- The system starts by asking: What is needed to achieve Decision 4?

Trace Back to Sub-Decisions

- Decision 4 depends on both Decision 1 AND Decision 2.
- These two intermediate decisions become sub-goals that must be proven.

Further Breakdown of Sub-Goals

- Decision 1 requires both Fact 1 and Fact 2 to be true (AND condition).
 - The system now checks if these two facts can be established.
- Decision 2 requires either Fact 3 OR Fact 4 to be true (OR condition).
 - The system checks if at least one of these facts holds.

Recursive Checking

- The process recursively checks each required fact.
- If all necessary facts for Decision 1 and at least one fact for Decision 2 are verified, then Decision 4 is confirmed.

Backward Reasoning

- Unlike forward chaining (which starts from facts and builds forward), here the system starts from the goal (Decision 4) and moves backward through the logical structure.
- This approach avoids exploring irrelevant facts and focuses only on what is necessary to prove the final goal.

Properties of Backward Chaining:

- Goal-Driven: Reasoning starts with a goal and looks for proof that it is true.
- Top-Down Approach: The system begins with the aim and then looks for facts that are related to it.
- Depth-First Search Strategy: The inference engine goes deep down a path before looking at alternative options, giving each objective or sub-goal a priority in order.
- Infinite Loops Are Possible: If not done right, backward chaining may become caught in loops while trying to find proof that the aim is true.

How Backward Chaining Works

1. Set a goal: The inference engine starts with the objective or hypothesis it wishes to prove.
2. Find Rules: It looks for rules that could help it reach its aim.
3. Check requirements: For each rule, it checks to see if the requirements are met. This may mean showing that other sub-goals have been met.
4. Recursive Process: This process goes back through the rule set until it reaches the original facts or decides that the aim is impossible.

Example of Backward Chaining:

Think about the same medical diagnosis system, but this time the purpose is to find out if a patient has the flu.

1. What is the goal? To find out if the patient has the flu?
2. If the patient has a sore throat and a fever, they may have the flu.
3. Sub-goals:

- Check to see if the patient has a fever.
- Check to see if the patient has a sore throat.

The system starts with the aim (flu) and checks to see if the patient's symptoms (fever and sore throat) match. If all of the sub-goals are met, the system decides that the patient probably has the flu.

Advantages of Backward Chaining

1. **Aim-Oriented:** It is useful for jobs that have a clear aim since it only gives you the information you need to reach that goal.
2. **Efficient with resources:** It usually consumes less memory because it just looks at certain goals instead of all potential inferences.
3. **Interactive:** It works effectively for interactive apps where the system has to answer certain questions or fix certain problems.
4. **Good for Diagnostic Systems:** It works especially well in diagnostic systems when the goal is to find out what is wrong based on the symptoms.

Disadvantages of Backward Chaining

1. **Difficult to implement:** It can be harder to put into action since it needs advanced tactics to deal with the recursive structure of the inference process.
2. **Needs Known Goals:** It needs goals that are already set, which might not always be possible in changing situations where the goals aren't known ahead of time.
3. **Inefficiency with Multiple Goals:** If you have to reach more than one goal, you may have to do backward chaining more than once for each objective, which could make things less efficient.
4. **Problems with Big Rule Sets:** It can be harder to manage the backward chaining process as the number of rules grows.

Comparison of Forward and Backward Chaining

Feature	Forward Chaining	Backward Chaining
Approach	Data-driven	Goal-driven
Starting Point	Known facts	Specific goals
Efficiency	Efficient for exploring all possible inferences	Efficient for achieving specific goals
Memory Usage	Can be memory intensive due to storing intermediate facts	Typically requires less memory

Implementation Complexity	Simple to implement	More complex to implement due to recursive nature
Suitability	Suitable for dynamic environments with continuously arriving data	Suitable for diagnostic systems and interactive applications
Handling Large Rule Sets	May become slow with large rule sets	Can be complex to manage with large rule sets

In rule-based systems, both forward chaining and backward chaining are useful ways to draw conclusions. Each has its own pros and cons. Forward chaining works best in situations where there is a lot of data and the goal is to learn something new. Backward chaining works best in situations where the goal is clear and the path to the conclusion needs to be traced. By learning how these technologies work and how they might be used, we can develop AI systems that are better and more useful for different situations.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
4	Backward chaining starts from: a) Known facts b) All possible data c) Desired goal or hypothesis d) A random rule
5	Backward chaining is most efficient when: a) Many possible goals need to be derived b) Only one or few goals need verification c) All data is available at the start d) No goals are defined
6	In backward chaining, if a condition is not a known fact, it is treated as: a) An irrelevant piece of information b) A new sub-goal c) A failed condition d) Automatically true

6. RESOLUTION

Resolution is a logical inference method that proves statements by refutation—assuming the opposite of a statement and showing this leads to a contradiction.

Resolution algorithm is a rule that AI uses to make logical decisions. It helps our AI system decide whether or not a proposition is logically proven by a group of known facts. It mostly works with assertions written in Conjunctive Normal Form (CNF), and it is most often employed in Propositional Logic and First-Order Predicate Logic.

"The main motivation behind using the resolution algorithm in AI is its ability to provide a mechanical, step-by-step method for proving logical statements.

Unlike humans, who rely on intuition and experience to navigate complex proofs, the resolution method is completely systematic. It can be programmed and executed by a machine without needing creative insight, which makes it ideal for automation.

Secondly, resolution simplifies the reasoning process in AI systems. Rather than having to consider all possible interpretations and logical manipulations, AI can break down reasoning into smaller, clear steps using resolution. This makes problem-solving more manageable and easier to implement in software.

Finally, resolution serves as the foundation for many automated proof systems and logic-based agents. Systems like automated theorem provers, expert systems, and certain types of logic-based planners all rely on resolution to formally validate conclusions or derive new facts from existing knowledge.

This mechanical and reliable nature of resolution is what makes it so valuable for building intelligent agents that can reason and solve problems independently."

Basic Idea for Resolution in AI

"The basic idea behind the resolution algorithm is to prove a statement by refutation. This means we try to show that the negation of our target statement leads to a contradiction.

This approach is known as proof by contradiction. Instead of directly proving a statement is true, we assume it is false and then show that this assumption conflicts with the known facts. If we reach a contradiction, we conclude that our original statement must be true.

To do this, the resolution method combines pairs of clauses to eliminate a literal and produce new,

simpler clauses. This process uses the resolution rule to systematically simplify the set of clauses in our knowledge base.

The algorithm continues this process until either a contradiction is found, represented by an empty clause, or until no further new clauses can be derived. If we reach an empty clause, it means our proof is successful, and we have confirmed the statement.

On the other hand, if no contradiction is found, it indicates that the statement cannot be logically derived from the given knowledge.

This refutation-based approach is powerful because it provides a clear, mechanical path to proving logical statements, making it suitable for AI and automated reasoning systems."

For example:

The algorithm will try to figure out if a statement like "It is raining OR it is sunny" is always true, sometimes true, or never true based on the information given.

The algorithm works by putting together logical statements in a way that either finds a contradiction, which means the statement is untrue, or affirms that the statement is true.

How the Resolution Algorithm Works

The Resolution Algorithm works with clauses, which are short logical statements that are linked by "AND" or "OR." It works by

Step 1: Convert Statement into Logical Forms

- First, we change the input sentence into a standard format called Conjunctive Normal Form (CNF). This means that we break down the complicated sentences into basic sections that are linked by "AND" and "OR."
- The CNF of the original sentence, "If it is raining then the ground is wet," is "NOT(Raining) OR Wet."

Step 2: Combine Clauses Using Resolution Rule

- The resolution rule, which merges two sentences to make a new clause, is the main notion behind the Resolution Algorithm.
- If you have two clauses, like A OR B and NOT(A) OR C, you can put them together to make B OR C. our step gets rid of one variable (A in our case) and makes the problem easier.

Step 3: Repeat Until You Find an Answer

The algorithm keeps using the resolution rule on pairs of clauses until one of two possibilities happens:

- **Found a Contradiction:** If the algorithm gives you an empty clause (expressed as FALSE), it signifies that the original collection of statements can't all be true at the same time.
- **No Contradiction:** If there is no empty clause after trying all the permutations, then the statements are consistent and can live together.

What does "Resolution" mean in First-Order Logic?

In first-order logic (FOL), a resolution is a rule that lets us come to new conclusions based on data that has already been established. Proof by contradiction says that if you assume something is wrong and it leads to a contradiction, then that assumption must be true.

The resolution method is commonly used in automated reasoning and logic-based problem solving because it creates clear, complete, and quick proofs of logical claims.

The resolution method is commonly used in logic-based problem solving and automated reasoning because it gives systematic, detailed, and quick demonstrations of logical claims.

Why Do We Use Resolution?

Resolution is often employed in different areas of AI and automated reasoning. Here are some important reasons to choose resolution:

- **Prove Statements** In a logical way Resolution checks if a statement is true or false by looking at its contradictions.
- **Automate Logical Reasoning** lets computers automatically come to conclusions based on the facts they are given.
- **Fix Problems in AI and Logic Programming:** used for rule-based reasoning in Prolog and other AI programs.
- **Make sure that knowledge-based systems are consistent.** This helps find errors and keep information accurate.
- **Check Software and Systems**—makes sure that hardware, software, and security mechanisms are all working as they should.

Important Parts of Resolution

Using First-Order Logic (FOL) makes the inference process work better in a few critical ways. They are—

a. Clause

A clause is a disjunction that is the most important part of resolution-based proofs. Here are some examples to help you understand the Clauses better:

- $P(x) \neg Q(x) \rightarrow$ This expression has two literals that are connected by OR $()$.
- $\neg ABC$: This phrase has three literals, which means that at least one of them must be true.

b. Literals

A literal is either the atomic proposition (fact) or the opposite of it. Literals are the parts that make up clauses. This is an example of literals.

- $P(x)$ is a positive literal that can be either true or untrue.
- $\neg Q(y)$ is a negative literal that means the opposite of a fact.

c. Unification

Unification is the process of making two logical expressions equal by finding the right values for the variables in each one. This is an example of Unification.

- Let's look at two predicates: Loves (x, Mary) is the first predicate.
- Predicate 2: Loves (John,y).

We need to determine the values for x and y that make these two predicates the same in order to combine them. If we replace x with John and y with Mary, we get Loves(John, Mary). So, after they are unified, both predicates are the same.

d. Substitution

Substitution is when you substitute variables with specific values or words in a logical statement to make it more clear. It is employed in rules for unification, resolution, and inference to make logical statements more precise. This is an example of substitution:

- Think about this predicate with a function: Teaches(Prof, Subject). If you replace "Prof/Dr. Smith" with "Subject/Mathematics," you get "Teaches(Dr. Smith, Mathematics)."

e. Skolemisation

Adding Skolem functions or constants to a statement removes existential quantifiers ($\exists$). This is called Skolemization.

- Let's look at the sentence "for every x, there exists some y such that x loves y."
- We add a Skolem function $f(x)$ to get rid of y, which gives us $x \text{ Loves}(x, f(x))$, where $f(x)$ stands for a specific person who x loves. This makes the statement completely free of quantifiers.

Applications of the Resolution Algorithm in AI

Among the many artificial intelligence uses for the Resolution Algorithm are:

1. **Automated Theorem Proving:** Without human interaction, computers can now automatically prove mathematical theorems or logical reasoning.
2. **Knowledge Representation:** Artificial intelligence systems use logical assertions to express knowledge. Thanks to the Resolution Algorithm, these systems are able to make good use of that information when reasoning.
3. **Problem Solving:** You may think of a lot of real-life difficulties as logical puzzles. Logical reasoning employing resolution can be useful in many contexts, such as task scheduling, system issue diagnosis, and route planning.
4. **Foundation for Advanced Techniques:** Further developments in artificial intelligence, including logic programming languages and SAT solvers for Boolean satisfiability problems, have their roots in the Resolution Algorithm.

Limitations of the Resolution Algorithm

Another restriction of the Resolution Algorithm is:

1. **Efficient:** The algorithm's wide search space makes it computationally expensive, particularly for complicated systems.
2. **Second, you'll need to convert all of your logical formulas to CNF before you can use the algorithm.** This process can be somewhat time-consuming.
3. **There Is No Simple Solution:** The resolution process achieves its results via disproving the goal's negation, rather than by directly building a solution.

An important tool in artificial intelligence (AI) for logical reasoning, resolution algorithms allow systems to draw conclusions and solve complicated issues. The computational cost and conversion needs, however, can be somewhat onerous, particularly in systems that are vast and complicated.

Optimisations and Improvements

Although the resolution algorithm can be quite powerful, it needs optimisations to handle large or complex problems effectively.

- One important approach is the use of strategies like unit resolution and subsumption to reduce complexity. Unit resolution focuses on resolving using clauses that contain only a single literal, which simplifies the process and can help reach a contradiction faster.
- Subsumption is another technique that removes redundant clauses from the clause set, preventing unnecessary growth and reducing the overall search space.
- Another optimisation involves using heuristics for clause selection to speed up the proof. Instead of blindly picking any pair of clauses, the algorithm can prioritise certain clauses that are more likely to lead to a successful resolution. This targeted approach avoids wasting time on less promising combinations and makes the reasoning process more efficient.

Lastly, integration with other reasoning techniques to create hybrid approaches can further improve performance. By combining resolution with other methods like forward chaining, backward chaining, or model checking, we can leverage the strengths of each technique. This makes it possible to solve more complex problems and build more robust AI reasoning systems. These optimisations and improvements are essential to make resolution practical and scalable in real-world applications."

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
7	The resolution technique in AI is primarily used for: a) Pattern recognition b) Refutation proofs c) Heuristic search d) Data mining
8	The resolution algorithm requires all knowledge to be represented in: a) Disjunctive Normal Form (DNF) b) Conjunctive Normal Form (CNF) c) Negation Normal Form (NNF) d) Natural language
9	Which of the following is a limitation of resolution? a) Cannot handle first-order logic

- | | |
|--|---|
| | <ul style="list-style-type: none">b) Works only with forward chainingc) Computationally expensive for large clause setsd) Cannot be automated |
|--|---|



7. SUMMARY

- Reasoning is a fundamental capability that allows artificial intelligence systems to draw new conclusions, solve problems, and make decisions based on existing knowledge and rules. By using various reasoning techniques — including deductive, inductive, abductive, probabilistic, and non-monotonic reasoning — AI can handle both certain and uncertain information, simulate expert-level thinking, and adapt to real-world complexities.
- Logical frameworks like propositional and predicate logic, combined with robust knowledge representation methods, provide a structured foundation for reasoning processes. Inference mechanisms such as forward and backward chaining power expert systems in areas like diagnosis, troubleshooting, and legal advice.
- Despite challenges like high computational cost and dependence on complete knowledge, advancements such as neuro-symbolic integration and explainable AI are making reasoning more powerful and transparent.
- Forward chaining is a data-driven reasoning technique widely used in artificial intelligence, especially in expert and rule-based systems. It starts with a set of known facts and iteratively applies inference rules to derive new facts, expanding its knowledge step by step.
- This approach is particularly useful in situations where all initial data is available upfront and when the system needs to explore all possible conclusions rather than prove a specific goal. Examples include medical diagnosis, industrial monitoring, cybersecurity, and environmental alert systems.
- Forward chaining offers advantages like thorough analysis and automatic discovery of all potential outcomes. However, it can face challenges such as generating large numbers of intermediate facts and high computational costs in large rule bases.
- With the help of optimisations — like rule prioritisation, indexing, and conflict resolution strategies — these challenges can be effectively managed.
- Looking forward, forward chaining is expected to become even more powerful by integrating with machine learning, enabling more adaptive and intelligent real-time decision-making in dynamic applications.
- Backward chaining is a goal-driven reasoning technique used in artificial intelligence and expert systems. Instead of starting with all known data, it begins with a specific hypothesis or goal that

needs to be proven. The system then works backwards, identifying which rules could lead to that goal and what conditions (facts) must be satisfied for those rules to apply.

- Backwards chaining is widely used in areas like medical diagnosis, legal reasoning, troubleshooting systems, and personalised recommendation engines. While it offers speed and focus, it can face challenges such as missing data, managing complex recursive searches, and handling multiple branching sub-goals.
- By using optimisations like caching results, limiting search depth, and applying heuristics, backwards chaining systems can become even more practical and robust.
- Overall, backward chaining is a powerful strategy for targeted, hypothesis-driven reasoning — enabling AI systems to deliver fast, precise, and explainable conclusions.
- The resolution algorithm is a powerful, systematic reasoning technique used in artificial intelligence to automatically prove logical statements. It works by refutation — showing that the negation of a statement leads to a contradiction — and operates on formulas expressed in Conjunctive Normal Form (CNF).
- Resolution is complete and mechanical, making it suitable for automated theorem proving, expert systems, and logic programming languages like Prolog. Its ability to systematically derive new knowledge from known facts makes it fundamental to logic-based AI systems.
- However, resolution also has limitations: it can be computationally expensive for large knowledge bases, requires conversion to CNF (which can increase complexity), and may be less efficient for certain first-order logic problems.
- Through optimisations like unit resolution, clause selection heuristics, and hybrid reasoning approaches, resolution can be made more practical and scalable.
- Overall, resolution provides a strong foundation for enabling machines to reason logically, prove statements, and solve complex problems automatically, reinforcing its importance in symbolic and knowledge-based AI

8. GLOSSARY

Reasoning	- The process of drawing conclusions or making inferences from known facts or rules.
Forward Chaining	- A data-driven reasoning technique that starts from known facts and applies inference rules to derive new facts until a goal or conclusion is reached. Commonly used in expert systems when all data is available at the start.
Backward Chaining	- A goal-driven reasoning technique that starts from a specific goal or hypothesis and works backward to see if known facts and rules can support it. Often used in diagnostic and problem-solving systems to prove specific conclusions.
Resolution	- A rule of inference used mainly in automated theorem proving and logic-based AI. It works by refutation — combining clauses to eliminate contradictory literals — until a contradiction is found (or no more new clauses can be derived). Essential for formal proofs and logic programming.
Inference Rule	- A logical rule that describes how new conclusions can be drawn from existing statements or facts.
Clause	- A disjunction of literals (e.g., $A \vee \neg B$). In resolution, knowledge is typically represented in Conjunctive Normal Form (CNF), which is a conjunction of such clauses.
Conjunctive Normal Form (CNF)	- A standardised way of writing logical formulas as a conjunction (AND) of disjunctions (OR) of literals, required for applying the resolution algorithm.
Working Memory	- A temporary storage area where facts and intermediate conclusions are kept during reasoning processes, especially in forward and backward chaining.
Refutation	- A proof technique where you show that the negation of a statement leads to a contradiction, thus proving the original statement true.

Expert System	- A computer program that uses reasoning techniques and a knowledge base to solve complex problems by emulating human expert decision-making.
----------------------	---



9. TERMINAL QUESTIONS

1. Define reasoning in the context of artificial intelligence. Why is it important?
2. Differentiate between forward chaining and backward chaining with examples.
3. Explain the main steps involved in forward chaining.
4. What are the advantages and limitations of backward chaining?
5. Describe how the resolution technique works in AI. What is its main purpose?
6. Why is Conjunctive Normal Form (CNF) important in resolution? Provide an example.
7. List real-world applications where forward chaining is commonly used.
8. In what scenarios is backward chaining preferred over forward chaining? Explain why.
9. What are the key differences between data-driven and goal-driven reasoning approaches?
10. Discuss the computational challenges associated with the resolution algorithm. Suggest possible optimisations.

10. ANSWERS

Self-Assessment Questions

1. Answer: b) Data-driven reasoning
2. Answer: b) From facts to goal
3. Answer: c) All facts are known upfront and multiple conclusions are required
4. Answer: c) Desired goal or hypothesis
5. Answer: b) Only one or few goals need verification
6. Answer: b) A new sub-goal
7. Answer: b) Refutation proofs
8. Answer: b) Conjunctive Normal Form (CNF)
9. Answer: c) Computationally expensive for large clause sets

Terminal Questions Answers

1. Answer:

Reasoning is the process of drawing logical conclusions or inferences from known facts or knowledge. In AI, it enables systems to make decisions, solve problems, and derive new knowledge, forming the foundation of intelligent behaviour. (Refer to section 2 for more information)

2. Answer:

- Forward chaining is data-driven: it starts from known facts and applies rules to infer new facts until a goal is reached.

Example: Diagnosing diseases by starting from observed symptoms and identifying possible conditions.

- Backward chaining is goal-driven: it starts from a specific goal and works backward to check if supporting facts exist.

Example: Verifying if a patient has a specific disease by checking if the required symptoms are present. (Refer to section 5 for more information)

3. Answer:

1. Start with a set of known facts stored in working memory.
2. Find all rules whose conditions match the current facts ("fire" the rules).
3. Apply these rules to derive new facts.
4. Add new facts to working memory.

5. Repeat the process until no new facts can be added or a goal is achieved.

(Refer to section 4 for more information)

4. Answer:

Advantages:

- Efficient when only specific goals need to be proved.
- Reduces unnecessary exploration of unrelated facts.

Limitations:

- Not ideal when multiple goals need to be explored simultaneously.
- May require extensive search if supporting facts are deeply nested or missing.

(Refer to section 5 for more information)

5. Answer:

Resolution works by refutation: it proves a statement by showing that its negation leads to a contradiction. It systematically combines clauses to eliminate complementary literals, generating new clauses until a contradiction (empty clause) is found or no new clauses can be derived. Its main purpose is to automate logical proofs and theorem proving. (Refer to section 3 for more information)

6. Answer:

CNF is required because the resolution algorithm operates on clauses expressed as a conjunction of disjunctions.

Example:

- Statement: $(A \Rightarrow B) \wedge (B \Rightarrow C)$
- In CNF: $(\neg A \vee B) \wedge (\neg B \vee C)$ (Refer to section 6 for more information)

7. Answer:

- Medical diagnosis expert systems.
- Industrial process control systems.
- Real-time monitoring and alarm systems.
- Rule-based recommendation engines. (Refer to section 4 for more information)

8. Answer:

Backward chaining is preferred when a specific goal or hypothesis needs to be tested, such as confirming a particular disease or verifying a legal condition. It avoids unnecessary derivations and focuses resources on proving only what is needed. (Refer to section 5 for more information)

9. Answer:

- Data-driven (Forward chaining): Starts from available facts, explores all possible conclusions.
- Goal-driven (Backward chaining): Starts from a target goal, works backward to check if it can be supported by known facts.

10. Answer:

Challenges:

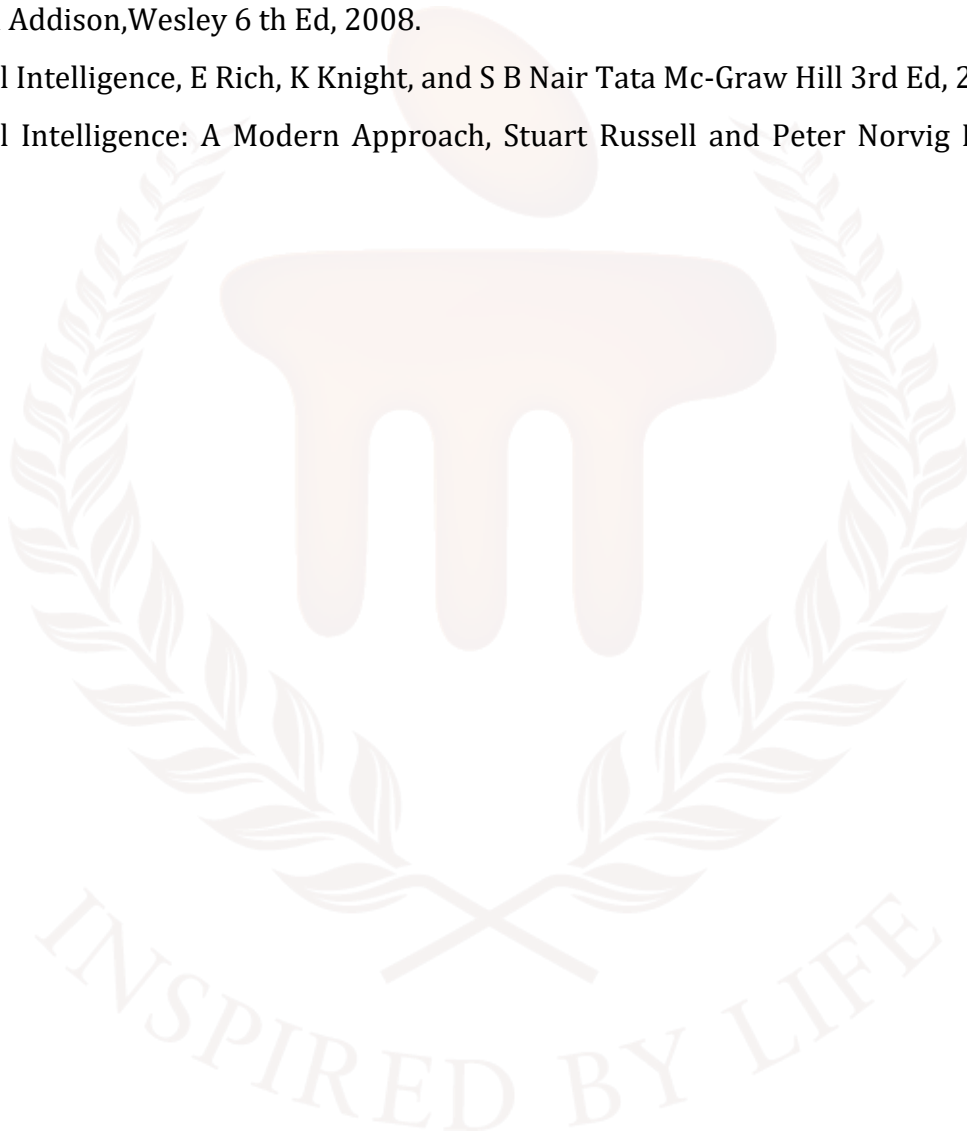
- High computational cost with large clause sets.
- CNF conversion can significantly increase formula size.

Optimisations:

- Using unit resolution and subsumption to simplify clauses.
- Heuristics for clause selection to prioritise promising paths.
- Combining resolution with other reasoning methods in hybrid systems.

11. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

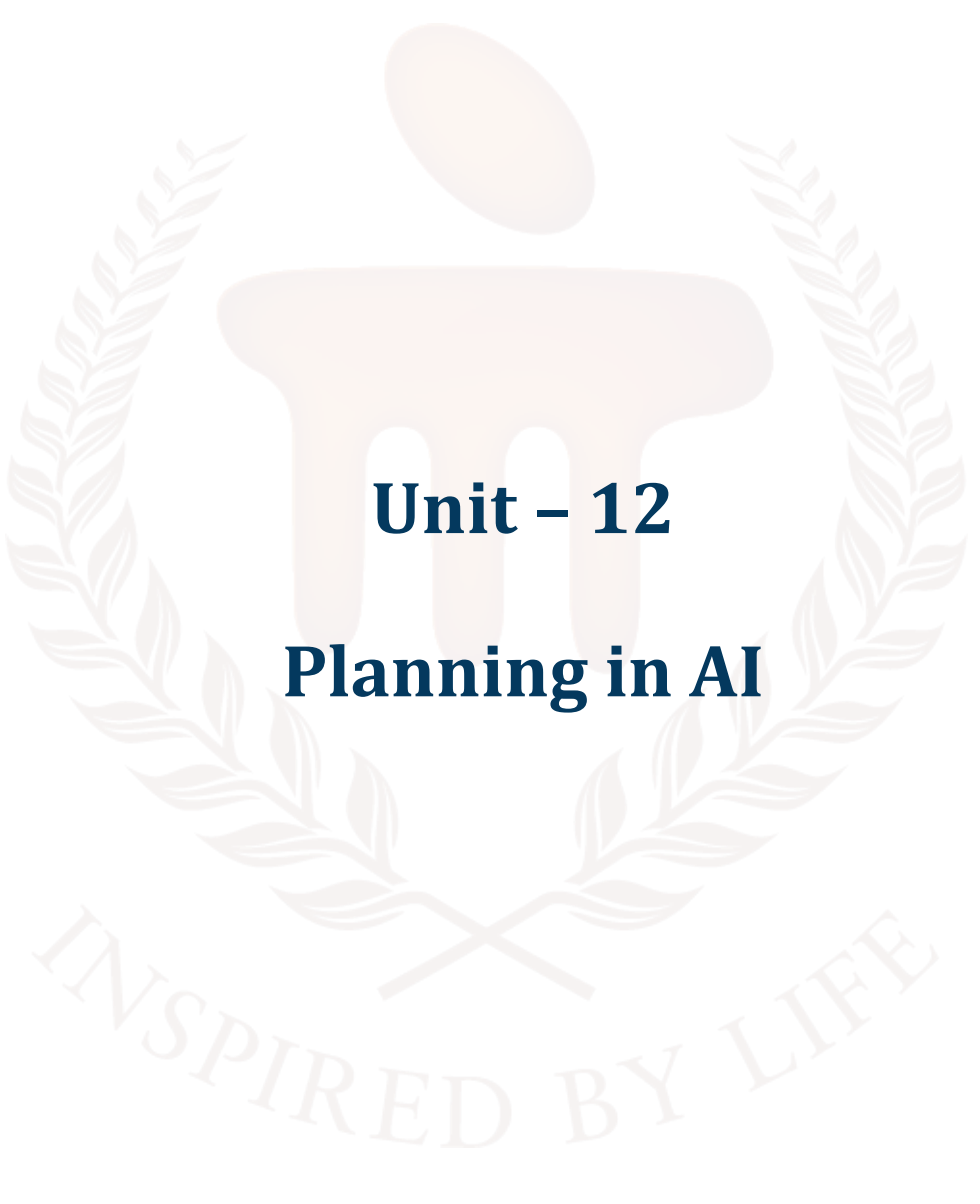


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109
ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING



Unit – 12

Planning in AI

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Planning in AI	-	-	6 - 7
3	Types of Planning in Artificial Intelligence	-	-	8 - 10
4	Importance of Planning in AI	-	1	11 - 12
5	Planning problem	-	-	13 - 16
6	Partial order Planning	1 , 2	-	17 - 19
7	Usage of Partial Order Planning in AI	3	2	20 - 22
8	Planning Graphs	4	-	23 - 25
9	Mutual Exclusion in Planning Graph	-	3	26 - 29
10	Summary	-	-	30 - 31
11	Glossary	-	-	32 - 33
12	Terminal Questions	-	-	34
13	Answers	-	-	35 - 37
14	References	-	-	38

1. INTRODUCTION

In previous unit, learners explored fundamental reasoning techniques that enabled intelligent systems to draw logical conclusions and make decisions. These techniques are essential for tasks such as expert systems, rule-based reasoning, and automated theorem proving. We also studied Forward Chaining, a data-driven approach where inference starts from known facts and applies rules to derive new facts until a goal is reached. This method is widely used in systems that operate by continuously updating their knowledge as new information becomes available.

They also learnt about Backward Chaining, a goal-driven reasoning technique. Here, the system started from the goal and works backward to determine what facts or conditions need to be satisfied. This approach is particularly common in diagnostic systems and query answering. Finally, learners explored Resolution, a powerful rule of inference used primarily in propositional and first-order logic. It enables systems to automatically prove logical statements by refuting their negations, forming the basis of many logic-based AI algorithms.

In this unit, learners will explore the essential concept of planning, a core ability that allows intelligent agents to decide on a sequence of actions to achieve their goals effectively and efficiently. First, learners will study the Planning problem, which involves defining an initial state, a set of actions, and a desired goal state. They will learn how AI systems determine plans that move from the starting point to the goal, handling constraints and selecting optimal paths.

Next, learners will dive into Partial order planning, a flexible approach where the exact order of some actions is not strictly fixed from the start. This method allows for concurrent and parallel actions, making planning more efficient and adaptable to real-world situations. Finally, learners will learn about Planning graphs, a powerful representation that uses layered graphs to capture possible actions and resulting states over time. Planning graphs help identify achievable goals, estimate the cost of reaching them, and support efficient plan extraction.

By the end of this unit, learners will understand how AI systems generate and organise plans, handle complex tasks, and ensure robust execution in dynamic environments.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the key concepts and components involved in planning within AI systems
- Assess different types of planning problems and planning approaches
- Distinguish between classical planning and partial-order planning techniques
- Recall the structure and working principles of planning representations such as STRIPS and state-space models



2. PLANNING IN AI

Planning is a fundamental aspect of Artificial Intelligence (AI), enabling systems to solve complex problems and achieve goals autonomously. In AI, planning refers to the process of formulating a sequence of actions to transition from an initial state to a desired goal state. By leveraging planning, AI systems can navigate uncertain environments, optimise resource usage, and make real-time decisions.

From robotics and autonomous vehicles to logistics and game playing, planning plays a pivotal role in enhancing AI's ability to operate efficiently and effectively. By simulating various outcomes and selecting the best course of action, planning helps AI systems operate in dynamic and unpredictable environments.

In this unit, we'll explore the concept of planning in AI, its significance, the different types of planning techniques, key challenges, and real-world applications.

What is a Plan?

In AI, a plan is a sequence of actions designed to achieve a specific goal. It is a blueprint for decision-making that outlines the steps necessary to transition from an initial state to a goal state. Planning is essential for solving problems where decisions must be made in a structured and systematic manner, ensuring that the chosen actions are both efficient and effective.

Planning enables AI systems to break down complex problems into smaller tasks and determine the most efficient sequence of actions to achieve a given objective.

Planning is essential for AI applications because it enables computers to:

- **Anticipate:** AI systems consider many outcomes before choosing the best one.
- **Change with the times:** AI systems can adjust their strategies in response to fresh knowledge when it comes to surroundings that are unclear.
- **Carry out activities without human intervention:** AI planning allows computers to execute tasks without human intervention, whether it's a smart assistant or an autonomous robot.

Example:

A robot's ability to navigate from one place to another, obstacle-free, is the goal of robot navigation. The robot's decisions on its movements, such turning left, going forward, and pausing at specific

spots, would be part of the plan. In a similar vein, AI systems can optimise routes to save travel time and expenses without sacrificing coverage of all locations.

Importance of Planning:

- AI systems use planning to tackle difficult problems by dividing them into smaller, more manageable pieces. This enables them to develop solutions that accomplish the provided goals.
- Making a Call: AI systems are able to plan ahead, which allows them to consider many courses of action and choose the optimal one based on expected results.
- The ability of AI systems to function autonomously in complicated and unpredictable real-world settings depends on their ability to plan ahead.

3. TYPES OF PLANNING IN ARTIFICIAL INTELLIGENCE

1. Classical Planning

Assuming a static, completely observable world where all activities are deterministic, classical planning is the standard approach of artificial intelligence planning. With full awareness of its environment, the AI agent pursues a predetermined objective: to discover a path from its current state to its desired end state.

i. Forward State Space Planning (FSSP)

- Conventional AI planning practices adhere to classical planning's tenets, which hold that all actions are deterministic and the world is static and fully observable. In its current state, the AI agent knows what it wants to do, so it uses that knowledge to explore its surroundings in search of a way to get there.

Key Features:

- **Exploration Path:** It begins at the beginning and continues on its way to the end.
- **Guarantee:** If a solution exists, the algorithm will find it.
- **Difficulty:** The computing cost of FSSP can be significant due to the enormous branching factor, which means that the number of possible actions can expand exponentially.

Example: Discovering the way out of a labyrinth by going around it all the way back to the beginning

ii. Backward State Space Planning (BSSP)

- Another kind of classical planning, BSSP operates in the opposite direction. It begins at the desired condition and finds the steps to get back to the starting point by working backwards. When there is a well-defined target state and a particular set of steps that must be undone to get back to the starting point, this method is frequently more efficient.

Key Features:

- **Exploration Direction:** Backwards-working towards the original state is the starting point for the exploration.
- **Computational Efficiency:** BSSP is known for its smaller branching factor, which results in better efficiency.

- **Difficulty:** Inconsistencies and faulty algorithms might make solution searching a losing proposition.

Example: Working backwards from the checkmate situation, a chess player plans the actions needed to attain an objective.

2. Probabilistic Planning

- Probabilistic planning is the best approach to use when confronted with situations that are unclear. Every action has several possible repercussions, and each of those consequences has its own probability. The AI system needs to account for that. Markov decision processes (MDPs) and Partially Observable Markov decision processes (POMDPs) are popular models used in probabilistic planning to resolve this uncertainty.

Key Features:

- When dealing with uncertainty, it is important to consider the probability of several outcomes for each action.
- Applying Probabilistic Models: Makes use of models such as MDPs and POMDPs to make decisions when faced with uncertainty.
- The necessity to think about every conceivable action result and the likelihood of each one makes for more complicated computations, which in turn increases the challenges.

Example: Autonomous vehicles negotiating congested roadways, with unpredictable road conditions and the actions of other drivers.

3. Reactive Planning

- Environments that are very unpredictable and dynamic are best suited for reactive planning. The AI agent doesn't have a set strategy; instead, it responds instantly to new information or changes in its surroundings. Rather of relying on a comprehensive plan to be created in advance, this strategy prioritises quick answers to the present circumstance.

Key Features:

AI is able to adapt in real-time to changes in its surroundings.

- **No Pre-computed Plan:** Prioritises Short-Term Efforts Over Comprehensive Strategy.
- **Difficulty:** May not plan ahead, instead focusing on short-term responses

Example: An AI in a video game that learns from the player's moves or a robot that skilfully navigates an unfamiliar area.

4. Hierarchical Task Network (HTN) Planning

- Planning using a hierarchical task network (HTN) involves dividing large tasks into smaller ones and then developing a strategy for each of those. When dealing with large-scale challenges, this hierarchical method works well since it allows for the division of goals into achievable chunks. It frequently entails breaking down complex activities into smaller, more manageable steps.

Key Features:

With hierarchical decomposition, large-scale projects are divided into smaller, more controllable chunks.

- A Layered, Structured Approach: Helps with Planning in a Layered Fashion.
- Problems: Things might get ugly if the sub-goal hierarchy isn't clear or is very complicated.

Example: A robot that disassembles a product into its component pieces, puts them together, and then tests it.

4. IMPORTANCE OF PLANNING IN AI

Planning is essential in AI for several reasons:

AI systems can optimise their use of time and resources by planning ahead to determine the best way to accomplish a task. By optimising delivery routes, an AI logistics system can cut expenses and fuel usage.

- **The ability to adapt:** AI systems encounter unpredictable situations in real-world applications. With the help of planning, AI can change its strategy in reaction to fresh data or unexpected challenges. When faced with obstacles, a self-driving automobile, for example, can simply change its route.
- **Autonomy:** AI systems are able to behave independently thanks to planning. Artificial intelligence (AI) has the capacity to plan, which allows it to execute tasks autonomously, whether it's a manufacturing plant robotic arm or a personal assistant such as Siri or Alexa.

Planning-based AI systems evaluate the possible outcomes of different actions to support informed decision-making. This is critical in domains such as game AI, financial forecasting, and medical diagnosis.

Challenges in AI Planning

Despite its importance, AI planning presents several challenges:

- **Computational Complexity:** Planning, particularly in intricate settings, can become rather costly in terms of computational resources. Large, dynamic systems often have an ideal sequence of operations that requires a lot of time and processing capacity to find. Making a strategy that can account for every conceivable event is difficult in unpredictable or uncertain circumstances. This is still a challenging topic, despite the efforts of probabilistic and reactive planning methods to address it.
- **Scalability:** The difficulty of planning grows in direct proportion to the size of the problem or endeavour. There is a technological barrier to scaling planning algorithms to deal with big datasets or settings with lots of variables.

Applications of AI Planning

The role of planning in AI is visible across various industries and applications, such as:

Planning enables robots to navigate surroundings efficiently, avoid obstacles, and carry out tasks without human intervention in the robotics field. An example of this would be a warehouse robot that can avoid obstacles by carefully plotting its route.

In healthcare, AI planning systems are utilised for treatment planning. Algorithms consider patients' medical history, present health, and success likelihood when suggesting ideal medicines.

- **Self-Driving Cars:** These vehicles are able to steer themselves, change lanes, stop at traffic lights, and avoid other vehicles and pedestrians thanks to extensive planning.
- **Video games:** AI-powered NPCs in these games can act intelligently thanks to planning. The game becomes more difficult and unexpected as NPCs plot their moves in real-time.

AI-powered supply chain management strategies boost productivity while cutting expenses by enhancing inventory management, transportation, and logistics. Artificial intelligence can figure out when to refill items or the most economical ways to ship them.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following is true about classical planning problems? a) The environment is partially observable and stochastic b) The environment is deterministic and fully observable c) Actions have probabilistic outcomes d) The environment changes dynamically during execution
2	In a planning problem, the sequence of actions designed to achieve the goal is called: a) Heuristic b) Policy c) Plan d) State transition
3	What is the main challenge in solving planning problems? a) Limited memory of the agent b) Combinatorial explosion of state space c) Lack of sensors in the environment d) Too many agents collaborating

5. PLANNING PROBLEM

A planning problem has an optimal goal, based on limited resources and under specific constraints.

Optimal goals can be any number of things, such as:

- Maximised profits - the optimal goal results in the highest possible profit.
- Minimised ecological footprint - the optimal goal has the least amount of environmental impact.
- Maximised satisfaction for employees or customers - the optimal goal prioritises the needs of employees or customers.

The ability to achieve these goals relies on the number of resources available. For example, the following resources might be limited:

- The number of people
- Amount of time
- Budget
- Physical assets, for example, machinery, vehicles, computers, buildings, and so on

Consider the unique limitations of these resources as well, such as the amount of time an employee can put in, their familiarity with particular machines, and the degree to which their tools are compatible with one another. When it comes to solving constraint satisfaction challenges, the Business Optimiser is a lifesaver for Java programmers. Along with efficient score calculation, it incorporates optimisation heuristics and metaheuristics.

A planning problem has a huge search space

A planning problem has a number of solutions. There are several categories of solutions:

Possible solution

A possible solution is any solution, whether or not it breaks any number of constraints. Planning problems often have an incredibly large number of possible solutions. Many of those solutions are worthless.

Feasible solution

A feasible solution is a solution that does not break any (negative) hard constraints. The number of feasible solutions tends to be relative to the number of possible solutions. Sometimes there are no

feasible solutions. Every feasible solution is a possible solution.

Optimal solution

An optimal solution is a solution with the highest score. Planning problems usually have a few optimal solutions. They always have at least one optimal solution, even in the case that there are no feasible solutions and the optimal solution is not feasible.

Best solution found

The best solution found is the solution with the highest score found by an implementation in a given amount of time. The best solution found is likely to be feasible and, given enough time, may be optimal. Even with a small dataset, the number of possible solutions can be enormous.

For the most part, there is an abundance of options in the planner-engine distribution folder examples. Because finding the best answer is never a sure thing, every implementation must test some subset of all the possibilities.

In order to effectively sort through that enormous pool of potential solutions, the Business Optimiser is compatible with multiple optimisation techniques.

Although it is hard to predict in advance, some optimisation strategies work better than others depending on the use case. With only a few lines of code or XML in the Business Optimiser's solver settings, you can change the optimisation algorithm.

A planning problem has constraints

Usually, a planning problem has at least two levels of constraints:

- A (negative) hard constraint must not be broken.
For example, one teacher can not teach two different lessons at the same time.
- A (negative) soft constraint should not be broken if it can be avoided.
For example, Teacher A does not like to teach on Friday afternoons.

Some problems also have positive constraints:

- A positive soft constraint (or reward) should be fulfilled if possible.
For example, Teacher B likes to teach on Monday mornings.

Only rigid limitations apply to some fundamental issues. There are situations where there are three tiers of restrictions, such as when there are hard, medium, and soft constraints.

A planning problem's score calculation, or fitness function, is defined by these limitations. It is possible to assign a score to each possible answer to a planning problem. You can use an object-oriented language like Java or Drools rules to write score constraints in the Business Optimiser.

STRIPS Representation

"STRIPS, which stands for Stanford Research Institute Problem Solver, was developed in the early 1970s at Stanford Research Institute. It is a foundational framework in AI for representing planning problems, especially in classical planning.

In STRIPS, every action is defined using two main components:

- Preconditions, which specify the conditions that need to be true before the action can be executed. For example, to 'pick up' an object, the precondition might be that the robot's hand is empty and the object is on the table.
- Effects, which describe what changes in the environment once the action is completed. In our example, after the 'pick up' action, the effect would be that the robot is now holding the object, and the object is no longer on the table.
- STRIPS simplifies the representation of complex tasks by clearly specifying these conditions and changes for each action. It allows planners to sequence actions logically to reach a goal state from an initial state.

Thanks to its simplicity and power, STRIPS has been widely used in classical planning systems and has influenced many later planning frameworks and algorithms. It remains an essential concept when discussing action representation in AI planning."

Consider the following example illustrating how STRIPS works: the Move(A, B) action. The action here is 'Move', where the agent moves from point A to point B. This action is very common in robot navigation or path-planning tasks. The preconditions define what must be true for this action to be performed. In this case:

- At(A) means the agent must currently be at location A.
- Path (A, B) ensures there is a valid path from A to B, so the move is actually possible.
- Next, we have the effects, which describe how the world changes after the action:
- At(B) indicates that after moving, the agent will now be at location B.
- $\neg$ At(A) (read as 'not At(A)') shows that the agent is no longer at location A.

This clear breakdown of preconditions and effects makes it easier for the AI planner to determine when and how actions can be taken to reach a goal. STRIPS uses this logical structure to systematically build a sequence of actions from an initial state to the desired goal state, ensuring each step is valid and achievable."

Planning vs Search

"When we talk about Planning vs Search in AI, it's important to understand how these concepts differ and where they overlap. Planning focuses on creating a goal-driven sequence of actions. Here, actions are explicitly defined with detailed preconditions and effects, which allows the system to predict what will happen after each action. This makes planning suitable for tasks where an agent needs to carefully consider each move, like in robotics, logistics, or automated assistants.

On the other hand, Search explores paths through a problem space but usually does not explicitly model individual actions and their effects. Instead, it focuses on finding a path from a start state to a goal state, like when solving a maze or playing simple puzzle games.

A key difference is that planning incorporates constraints and effects. For example, it might consider limitations on resources (like fuel or time), dependencies between actions, and the specific changes each action makes to the environment. In short, while both techniques aim to reach a goal, planning provides a more detailed, structured, and constraint-aware approach, making it more powerful for complex, real-world tasks where precise execution is critical."

6. PARTIAL ORDER PLANNING

Automated planning techniques like Partial Order Planning (POP) build plans by organising operations with minimal ordering limitations. Unless some dependencies specify otherwise, POP permits actions to remain unordered, in contrast to total-order planners that strictly sequence all actions. Because of the approach's adaptability, the plan's actions can be carried out in a variety of legitimate ways. Artificial intelligence (AI) Agents, who must change their behaviour in response to new information, frequently make use of it.

How Partial Order Planning Works

By creating plans with just the essential ordering constraints between operations, Partial Order Planning (POP) makes execution sequences flexible.

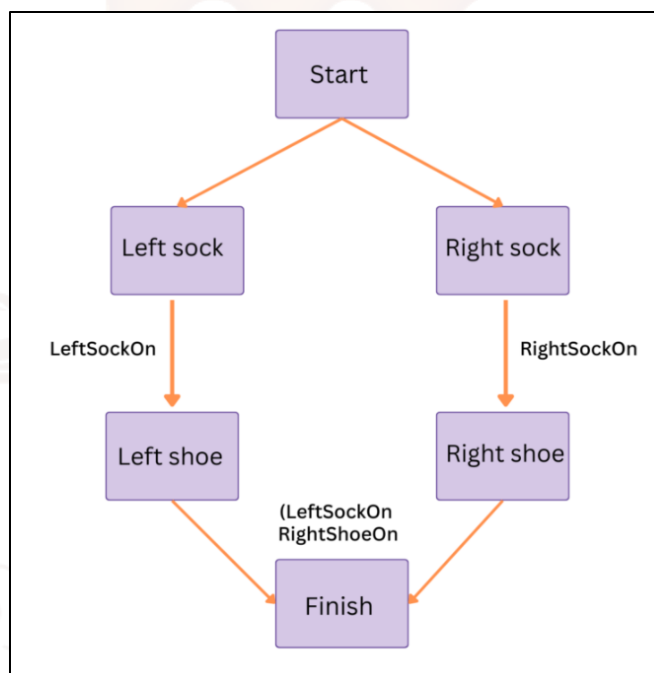


Figure 1: Working of partial planning

This approach prioritises goal-attainment over action-sequence rigidity, adhering to certain orderings only when dependencies demand it.

Partial Order Planning Essentials:

- **Actions:** What needs to be done in order to reach the objective.
- **Partial Order:** Restrictions that outline the required order of specific operations.

An action's consequence fulfils the precondition of an additional action when there is a causal link

between the two.

- **Open Preconditions:** These are requirements that the strategy does not yet address.

Planning process:

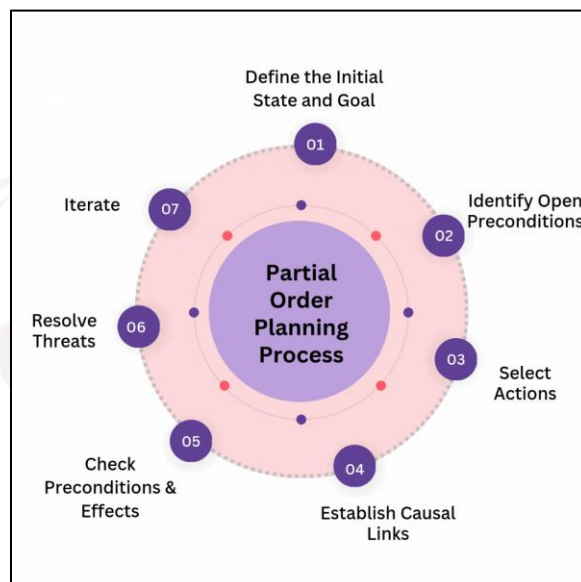


Figure 2: Process of Partial Order

Here's a step-by-step explanation of how POP works with examples:

1. **Initialisation:** Start with an initial plan containing only the initial state and the goal state.
2. **Identify Open Preconditions:** Determine which preconditions of the actions in the plan are not yet satisfied.
3. **Select Actions:** Choose actions that can fulfill these open preconditions.
4. **Establish Causal Links:** Create causal links between actions to indicate that one action's effect satisfies another's precondition.
5. **Introduce Ordering Constraints:** Add ordering constraints only when necessary to resolve conflicts or satisfy dependencies.
6. **Resolve Threats:** Address any conflicts where an action might invalidate the precondition of another action, using strategies like promotion (ordering the threatening action after the threatened causal link) or demotion (ordering it before).
7. **Iterate:** Repeat the process until all preconditions are satisfied and no conflicts remain.

How Does Partial Order Planning Compare to Total Order Planning?

When it comes to automated planning, there are two main methods: partial-order planning (POP) and total-order planning (TOP). Consider this contrast:

Aspect	Partial-Order Planning (POP)	Total-Order Planning (TOP)
Action Ordering	Orders actions only when necessary, allowing multiple valid sequences.	Uses a strict, linear sequence with a single execution path.
Commitment Strategy	Follows a “least commitment” approach, reducing backtracking.	Commits to a fixed sequence early, which may require revisions.
Handling of Subgoals	Develop independent subplans, making complex tasks manageable.	Requires a predefined sequence, complicating interdependent goals.
Search Space	Explores fewer plans, improving efficiency.	Searches through a larger set of fully ordered plans.
Execution Flexibility	Allows multiple valid execution paths, adapting to changes.	Has a single execution path, limiting adaptability.

TOP offers a systematic method that is better suited to simpler, well-defined problems, whereas POP is more flexible and efficient for complicated jobs.

Advantages of Partial Order Planning

In the context of automated scheduling and planning, partial-order planning (POP) provides numerous significant benefits:

POP delays action sequencing until necessary, minimising unnecessary constraints and reducing backtracking by following the least commitment strategy.

The development of separate subplans for various objectives facilitates the resolution and integration of complicated problems, leading to efficient management of subgoals. Completeness and Soundness - Assures that plans are valid and that a solution is discovered if one exists within the specified limitations. Efficient plans are generated by minimising superfluous processes, which results in streamlined execution, thanks to Optimal Plan Length. Execution Flexibility - Enables many legitimate action sequences, adjusting effectively to changing situations and unforeseen changes

7. USAGE OF PARTIAL ORDER PLANNING IN AI

When it comes to AI, Partial Order Planning (POP) is crucial since it lets smart systems create efficient and adaptable plans. With POP, there is an organised way to sequence events without imposing additional constraints, which is useful for AI-driven processes that commonly entail numerous interdependent tasks.

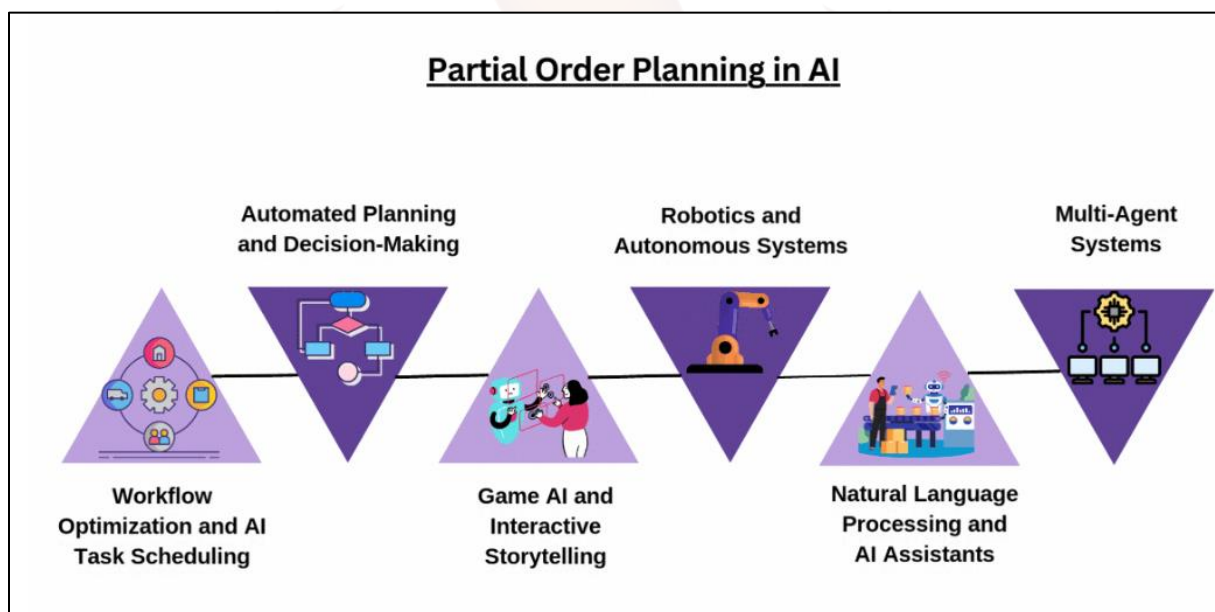


Figure 3: Partial order Planning

AI systems employ POP to create flexible action plans, automating the planning and decision-making processes, and limiting execution to the necessary ordering constraints. In the field of robotics and autonomous systems, POP allows robots to execute numerous jobs simultaneously, modifying their action sequences in response to current circumstances. In multi-agent systems, POP is used to schedule activities and avoid conflicts by AI-driven logistics, collaborative AI agents, and drone coordination.

POP contributes to the creation of interactive storytelling, realistic NPC behaviours, and dynamic game narratives, as well as adaptable opponent techniques. Artificial intelligence (AI) applications in healthcare, manufacturing, and customer service utilise POP to optimise workflows and eliminate bottlenecks in operations.

The usage of POP allows virtual assistants to handle user requests in a flexible order while preserving conversation context, which is important for natural language processing and AI helpers.

Challenges of partial planning

Automated planning with partial-order planning (POP) is efficient and flexible, but it comes with a few drawbacks:

- Managing goals that interact in an unpredictable way, such as Sussman's anomaly, can be difficult.
- The processing time per node increases due to the higher computational cost caused by evaluating several possible orderings.
- Finding and handling inconsistencies in actions increases the level of complexity in threat detection and resolution.
- It is challenging to integrate numerical constraints with partial-order structures when dealing with numerical effects.
- Planning gets computationally costly as the number of activities and interactions rises. This poses a problem with scalability.

An automated planning strategy, Partial Order Planning prioritises efficiency and adaptability by committing to action orders only when absolutely required.

This technique is highly beneficial in domains, particularly AI, that demand quick and adaptive planning solutions. It can handle complex planning scenarios and even execute actions in simultaneously.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
4	Partial order planning allows: a) Only a fixed sequence of actions b) Actions to be executed strictly one after another c) Some actions to remain unordered initially d) Only single-agent planning
5	Partial order planning is also known as: a) Linear planning b) Total-order planning c) Non-linear planning d) Reactive planning

- | | |
|---|--|
| 6 | <p>What is an advantage of partial order planning?</p> <ul style="list-style-type: none">a) Always faster than total-order planningb) Supports concurrent execution of independent actionsc) Requires no preconditions for actionsd) Does not handle conditional branches |
|---|--|



8. PLANNING GRAPHS

To help with decision-making, planning graphs graphically depict potential states and actions, which is crucial in AI planning. The compact structure known as graph planning is constructed and analysed in this work within STRIP-like domains. Using a network planning method as an example, we will also explore the function of mutual exclusion.

When it comes to AI and automated planning, a Planning Graph is the data format of choice for solving planning challenges. It is a representation of the evolution of a planning problem through levels that characterise world situations and possible actions. The key parts and how it works are as follows:

Levels: Levels representing actions and levels representing states alternate in a planning graph. Every planning problem begins with a state level as its first level.

1. **State Levels:** Nodes at these levels stand in for world facts or logical statements. Every new level of states incorporates all the prior levels' propositions as well as any that can be deduced from the levels of actions that came before it.
2. **Action Levels:** Nodes on these levels stand in for many kinds of actions. If a state has all the prerequisites for an action, then an action node can connect to that state. The conditions of a state can be changed by actions, which in turn affect the level of states that follow.
3. **Edges:** There are two types of edges: those linking states to actions (indicating the state satisfies the action's prerequisites) and those linking actions to resulting states (indicating the action's effects).
4. **Mutual Exclusion (Mutex) Relationships:** There may be certain states or activities that cannot coexist at any given level because of competing conditions or consequences; this is called a mutual exclusion. By restricting the possible action-state combinations, these mutex interactions play a crucial role in simplifying the planning problem.

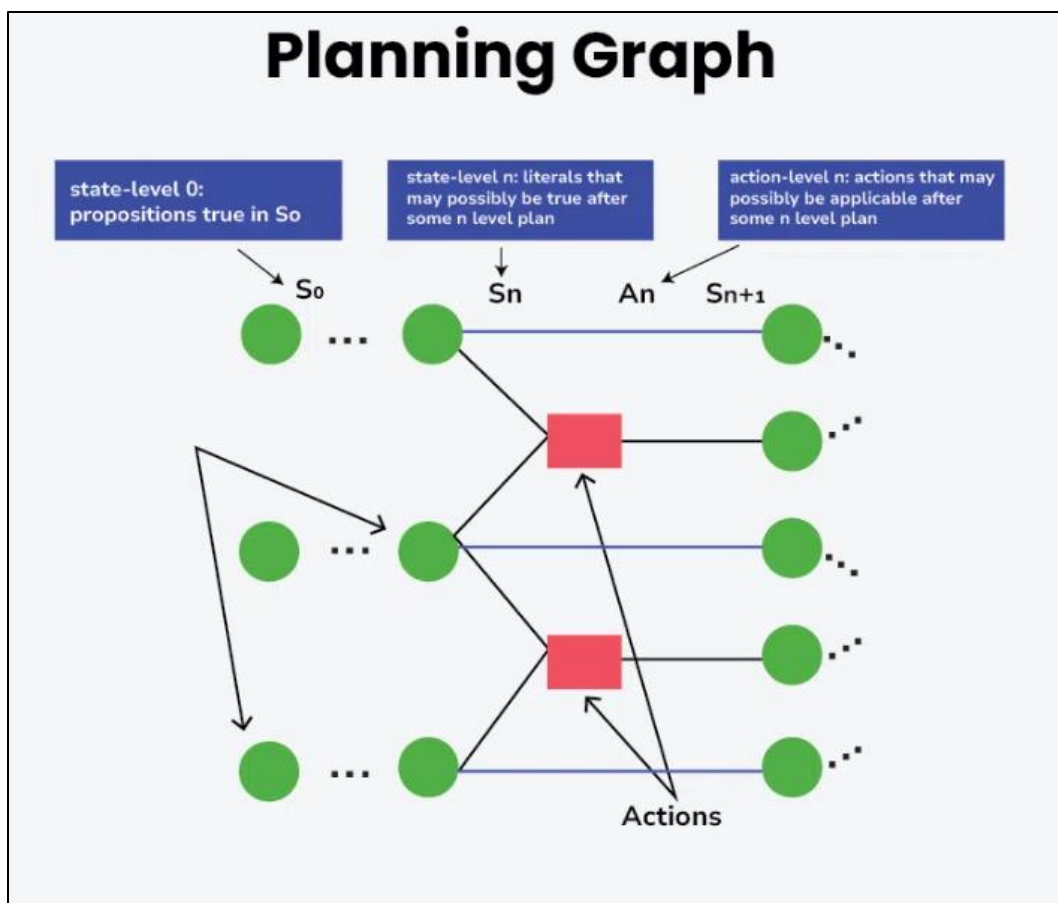


Figure 4: Presentation of Planning Graph

Levels in Planning Graphs

- Level S0: It is the initial state of the planning graph that consists of nodes each representing the state or conditions that can be true.
- Level A0: Level A0 consists of nodes that are responsible for taking all specific actions in terms of the initial condition described in the S0.
 - Si: It represents the state or condition which could hold at a time i, it may be both P and $\neg P$.
 - Ai: It contains the actions that could have their preconditions satisfied at i.

Working of Planning Graph

1. A single proposition level comprises all the initial circumstances in the planning graph. Below, we will go over each stage of the planning graph and how it works:
 - Extending the Planning Graph: Currently, we are at stage i, where the graph plan extends the planning graph from stage i-1 by one time step. Next, we have the proposition level, which represents the states that have resulted from actions, and the action level, which represents all the conceivable actions given the states in the preceding level.

- Valid Plan Found: The graph plan will stop planning once it discovers a valid plan.
- Proceeding to the Next Stage: In the absence of a workable strategy, the algorithm proceeds to stage i , where it decides that none of the objectives will be met by time i .



9. MUTUAL EXCLUSION IN PLANNING GRAPH

As a general rule, the planning problem's constraints and dependencies make it such that no two actions or propositions may be mutually exclusive. This notion is known as "mutual exclusion" in graph planning. Under some circumstances, mutex connections between literals and actions can hold.

Mutex Conditions between Actions

One deed cancels out another deed, leading to inconsistent effects.

Interference occurs when one action removes a precondition or causes another action to have an add-effect. Incompatible Requirements: Action a's prerequisite and action b's prerequisite cannot both be true at the same time.

Mutex Conditions between Literals

Two literals are incompatible with each other if and only if they are mutually negational. No two non-mutex operations can simultaneously make both literals true; this is how it is accomplished.

Planning a Graph for a CAKE Problem

To understand the concept of planning a graph, let's use the CAKE example. This illustration demonstrates various states and actions to achieve the goal of successfully eating the cake. By examining these states, we can see how actions transition from one state to another to reach the desired outcome.

a. Initial State (S0)

- Have(Cake): We have the cake.
- $\neg$ Eaten(Cake): The cake has not been eaten yet.

b. Action Level (A0)

The action level represents possible actions that can transition the state from S0 to the next state. In our example, the available action is:

- Eat(Cake): Represents the action to eat the cake.

c. Next State (S1)

The next state is determined by the action taken at the action level based on the propositional variables. In the CAKE example, we have four propositional variables in the next state:

- Have(Cake): If no action is performed (no-op), we will still have the cake.
- $\neg$ Have(Cake): If the action Eat(Cake) is performed, we will no longer have the cake.
- Eaten(Cake): If the action Eat(Cake) is performed, the cake will have been eaten.
- $\neg$ Eaten(Cake): If no action is performed (no-op), the cake will still be uneaten.

d. Performance of Mutual Exclusion in Planning Graph

Any subset of actions at A0 can be used to acquire all literals at state level S1. Put another way, after the activities in A0 are taken into account, all potential outcomes are stored in state level S1. Since our lone action is Eat(Cake) at A0, S1 will display all outcomes, with and without the action, in our example.

e. Mutual Exclusion

The occurrence of two literals that are incompatible with each other is known as mutual exclusion. Mutex connections, which show which propositions in S1 are mutually exclusive, stand in for these disagreements.

If we consume the cake, for example, we can't have any more cake. Therefore, it would be impossible to have both Have(Cake) and Eaten(Cake). The collection of states is defined by the mutex connections, which show which literal combinations are not conceivable together.

For example:

- If Eat(Cake) is performed, Have(Cake) and $\neg$ Eaten(Cake) cannot be true simultaneously.
- If Eat(Cake) is not performed, $\neg$ Have(Cake) and Eaten(Cake) cannot be true together.

Second Action Level (A1) and State Level (S2)

Continuing from the above example, we introduce two more levels: Action-level (A1) and State-level (S2).

Actions Available at A1 from S1

- Bake(Cake): The precondition for this action is $\neg$ Have(Cake). If $\neg$ Have(Cake) is true in S1, applying Bake(Cake) results in Have(Cake) in S2. This action does not affect the Eaten(Cake) or $\neg$ Eaten(Cake) states.
- Eat(Cake): The precondition for this action is Have(Cake). If Have(Cake) is true in S1, applying Eat(Cake) results in $\neg$ Have(Cake) and Eaten(Cake) in S2.

Steps in the Graph Plan Algorithm

1. The first step is to enlarge the planning graph so that all of the goals are visible and non-mutex at level n.
2. Secondly, make sure there is a plan: the algorithm will not work if the planning graph stops moving before all non-mutex goals are present.
3. Valid Plan Search: The method, without breaking mutex constraints, searches backwards from the last level to the initial state to discover a sequence of actions that will lead to the goals.
4. If a legitimate plan cannot be found, the search is expanded to include another level and continues until either a plan is discovered or the graph reaches a plateau.

```
function GRAPHPLAN(problem) returns solution or failure
  graph ← INITIAL-PLANNING-GRAPH(problem)
  goals ← CONJUNCTIONS(problems.GOAL)
  loop do
    if goals all non-mutex in last level of graph then do
      solution ← EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph))
      if solution ≠ failure then return solution
      else if NO-SOLUTION-POSSIBLE(graph) then return failure
    graph ← EXPAND-GRAPH(graph, problem)
```

Properties of Graph Plan

A monotonic increase or decrease characterises the components of the planning graph: The growth of the literary canon is monotonic.

- Actions Progress in a Uniform Manner
- Mutagens Decline in a Stable Manner

These characteristics allow the planning graph to converge to a level when there are finitely many literals and actions.

To approach and fix planning issues methodically, the graph plan algorithm uses planning graphs. Consistency and practicality are guaranteed by adopting the principles of mutual exclusion. Planning graphs easily handle complicated circumstances, as exemplified by the CAKE example.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
7	Who introduced the concept of planning graphs? a) Alan Turing b) Blum and Furst c) John McCarthy d) Marvin Minsky
8	In a planning graph, what does the mutex relation represent? a) A guaranteed achievable state b) A pair of actions or facts that cannot occur together c) The initial set of propositions d) A measure of action cost
9	What is the main purpose of using a planning graph? a) To increase the search space size b) To estimate heuristics and guide search more efficiently c) To represent only deterministic problems d) To eliminate goal propositions



10. SUMMARY

Planning in Artificial Intelligence is the process by which an intelligent agent decides on a sequence of actions to achieve a specific goal. It starts with understanding the current situation—called the initial state—and identifying the desired outcome, or goal state. The agent then evaluates possible actions and selects the most effective strategy to reach that goal, considering factors such as available resources, time constraints, and uncertainties. Different types of planning handle varying levels of complexity: classical planning assumes a predictable and fully known environment, while conditional planning deals with uncertainty and incomplete information. Hierarchical planning breaks complex tasks into manageable sub-tasks, and reactive planning allows the agent to adapt in real time to dynamic changes. Planning is a crucial capability in AI systems and is widely applied in areas such as robotics, gaming, autonomous vehicles, and virtual assistants, where intelligent decision-making is essential for efficient and purposeful behavior.

Planning in Artificial Intelligence (AI) refers to the process of generating a sequence of actions that an intelligent agent must perform to achieve specific goals, starting from an initial state. It involves deciding what actions to take and in what order to accomplish objectives effectively and efficiently. Unlike simple problem solving, planning explicitly considers future actions and their consequences.

In AI, planning often uses a model of the environment, which describes possible actions (operators), their preconditions, and effects. Classical planning assumes a deterministic, fully observable environment, where the planner knows exactly what actions will result from its moves. Examples include pathfinding for robots, task scheduling, and game playing.

A key component is the representation of states and actions. States represent snapshots of the world at a given moment, while actions transform one state into another. The planning problem can be formally described as finding a path from an initial state to a goal state in a state space.

Popular planning algorithms include state-space search (such as forward and backward search), partial-order planning (which allows for flexible action ordering), and hierarchical planning (which decomposes complex tasks into simpler subtasks). Modern planning systems, like STRIPS and PDDL, provide formal languages to specify planning domains and problems.

Planning is crucial for autonomous agents, robots, logistics, and intelligent systems, enabling them to act autonomously and adapt to complex environments.

Different types of planning exist depending on the nature of the environment. Classical planning assumes a deterministic and fully observable world, where outcomes of actions are predictable. Conditional planning accounts for uncertainty and includes contingencies for different possible observations during execution. Hierarchical planning simplifies complex tasks by breaking them down into smaller, manageable sub-tasks, enabling step-by-step refinement. Probabilistic planning, on the other hand, deals with environments where actions have probabilistic outcomes and focuses on maximising expected utility. Various algorithms are used to perform planning effectively, such as state-space search (forward or backward), partial-order (plan-space) planning, and graph-based approaches like GraphPlan. The heuristic techniques are often used to guide the search process and improve efficiency. Formal languages like STRIPS (Stanford Research Institute Problem Solver) and PDDL (Planning Domain Definition Language) are used to describe planning problems and domains in a standardised way. Planning has wide applications in robotics (motion and task planning), game AI (strategic behaviour generation), logistics and supply chain management (route and schedule optimisation), autonomous vehicles (path and manoeuvre planning), and business process automation (workflow scheduling). Despite its power, planning faces challenges such as scalability due to the exponential growth of possible states and actions, dealing with uncertainty, and coping with dynamic environments where the world may change during plan execution. Overall, planning is a core capability that enables AI systems to act intelligently and autonomously by carefully considering future actions and their consequences to achieve specified goals efficiently.

11. GLOSSARY

Agent	- An entity that perceives its environment and takes actions to achieve specific goals.
State	- A snapshot or description of the environment at a particular point in time, defined by the values of relevant variables.
Initial State	- The starting condition from which the planning process begins.
Goal State	- A condition or set of conditions that define the successful completion of a plan.
Action (Operator)	- A basic operation that causes a transition from one state to another. Each action has preconditions and effects.
Preconditions	- The set of conditions that must be true in a state for an action to be applicable.
Effects	- The changes to the state resulting from executing an action.
Plan	- A sequence (or partially ordered set) of actions designed to move from an initial state to a goal state.
State-Space Search	- A method of planning that involves exploring all possible states and transitions to find a path from the initial state to the goal state.
Forward Search	- A planning approach that starts from the initial state and applies actions forward to reach the goal state
Backward Search	- A planning approach that starts from the goal state and works backward to find possible actions that lead to it.
Partial-Order Planning (Plan-Space Planning)	- A strategy where actions are only ordered when necessary, allowing for flexibility in plan execution.

Hierarchical Planning	-	A technique that decomposes complex tasks into simpler subtasks, creating a hierarchy of actions.
Conditional Planning	-	Planning that accounts for possible uncertainties or different observations during execution and includes contingency actions.
Probabilistic Planning	-	Planning that deals with actions having uncertain (probabilistic) outcomes, focusing on maximising the expected success.
STRIPS (Stanford Research Institute Problem Solver)	-	A formal language for specifying actions with preconditions and effects, widely used in classical planning.
PDDL (Planning Domain Definition Language)	-	A standardised language used to define planning problems and domains, commonly used in AI planning research
Planning Graph	-	A data structure used in some algorithms (like GraphPlan) that layers actions and propositions to efficiently check goal reachability.
Heuristic	-	A technique or rule of thumb used to guide search algorithms toward more promising parts of the search space and improve efficiency.
Scalability	-	The ability of a planning system to handle large or complex problems as the number of states and actions increases.
Dynamic Environment	-	An environment that can change while an agent is executing its plan, requiring the agent to adapt.
Deterministic Environment	-	An environment where the outcome of each action is certain and predictable.
Fully Observable Environment	-	An environment in which the agent has complete knowledge of the current state at all times.
Execution Monitoring	-	The process of observing whether a plan is proceeding as expected during execution and making adjustments if necessary.

12. TERMINAL QUESTIONS

1. What is the planning problem in AI?
2. What are the main components of a planning problem?
3. What is partial order planning?
4. What is a planning graph?
5. How does a planning graph help in solving planning problems?
6. Explain "mutex" in the context of planning graphs.
7. What advantage does partial order planning have over total order planning?
8. What is the initial level of a planning graph?
9. Describe the backward extraction process in GraphPlan.
10. What is the main difference between classical planning and partial order planning?

13. ANSWERS

Self-Assessment Questions

1. Answer: b) The environment is deterministic and fully observable
2. Answer: c) Plan
3. Answer: b) Combinatorial explosion of state space
4. Answer: c) Some actions to remain unordered initially
5. Answer: c) Non-linear planning
6. Answer: b) Supports concurrent execution of independent actions
7. Answer: b) Blum and Furst
8. Answer: b) A pair of actions or facts that cannot occur together
9. Answer: b) To estimate heuristics and guide search more efficiently

Terminal Questions Answers

1. Answer:

The planning problem in AI involves finding a sequence of actions (plan) that transforms an initial state into a goal state, given a set of possible actions, their preconditions, and effects. The challenge is to determine which actions to take and in what order to achieve the desired goal efficiently. (Refer section 2 for more information)

2. Answer:

The main components are:

- Initial state — the starting condition of the system.
- Goal state(s) — the condition(s) that represent the successful outcome.
- Actions/operators — available actions with their preconditions and effects.
- State space — all possible states reachable from the initial state. (Refer section 4 for more information)

3. Answer:

Partial order planning is a planning approach in which actions are ordered only when necessary, rather than strictly sequencing every step from start to finish. This allows for more flexible execution, as some actions can occur in parallel or in any order, provided constraints are respected. (Refer section 5 for more information)

4. Answer:

A planning graph is a layered data structure used in planning algorithms (like GraphPlan) to represent the progression of possible actions and resulting states over time. It alternates between levels of possible actions and levels of propositions (states), helping efficiently identify reachable goals and conflicts. (Refer section 6 for more information)

5. Answer:

A planning graph helps by compactly representing the possible sequences of actions and by making it easier to detect if goals are achievable without conflicts. It allows planners to quickly identify mutex (mutually exclusive) actions and states, prune infeasible plans, and guide backward plan extraction. (Refer section 8 for more information)

6. Answer:

"Mutex" (mutual exclusion) in planning graphs refers to a relationship between actions or propositions that cannot occur simultaneously. Two actions are mutex if they interfere with each other, and two propositions are mutex if they cannot both be true in the same state level. Identifying mutex relations is critical for finding valid plans. (Refer section 8 for more information)

7. Answer:

Partial order planning allows for flexibility by permitting independent actions to be unordered and potentially executed in parallel, which can lead to more efficient and adaptable plans. In contrast, total order planning imposes a strict, linear sequence, which may be unnecessarily restrictive. (Refer section 7 for more information)

8. Answer:

The initial level of a planning graph consists of the propositions (facts) that represent the initial state of the problem. This is the starting point from which action levels and subsequent proposition levels are iteratively added. (Refer section 8 for more information)

9. Answer:

Backward extraction in GraphPlan starts from the goal propositions in the final proposition level of the planning graph and works backward, selecting actions that achieve these goals and recursively adding their preconditions as new subgoals, until actions that are applicable from the initial state are identified. This process builds a valid plan. (Refer section 8 for more information)

10. Answer:

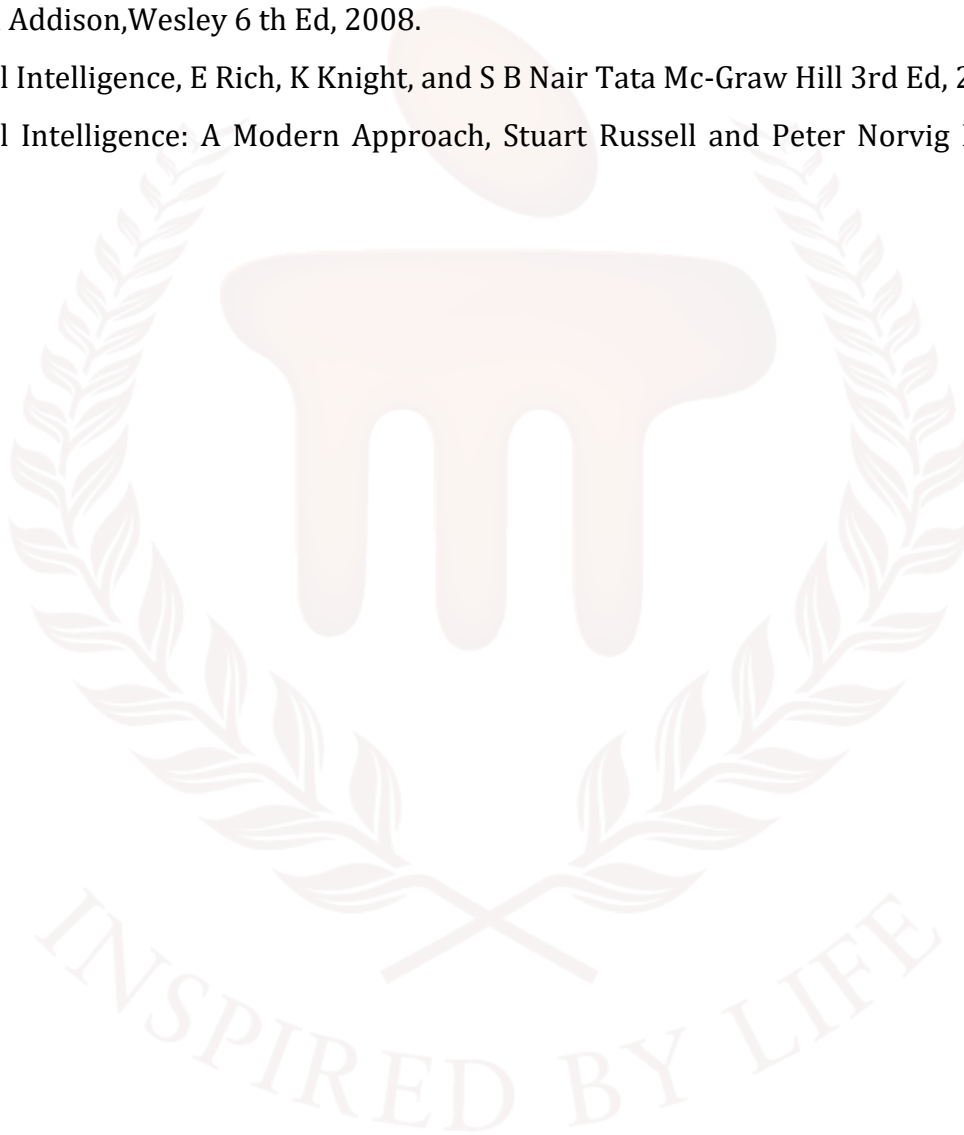
Classical planning typically produces a totally ordered sequence of actions where every action's position is fixed, assuming a linear execution path. Partial order planning, on the other hand, generates a plan where actions are only ordered when necessary, providing more flexibility and

allowing some actions to be executed concurrently or in different orders.(Refer section 6 for more information)



14. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

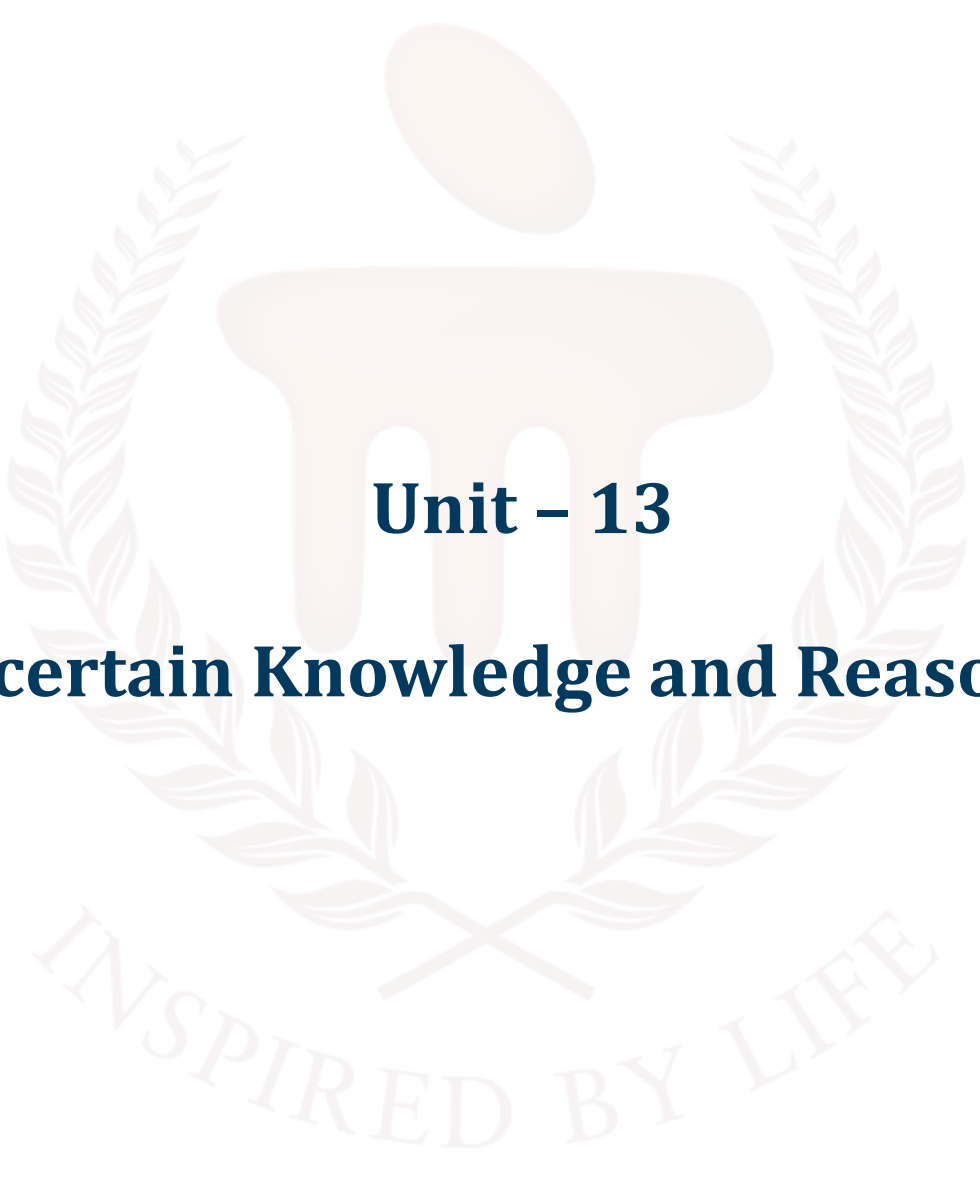


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109
ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING



Unit – 13

Uncertain Knowledge and Reasoning

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Learning Objectives	-	-	
2	Reasoning with Uncertain Information	-	-	6
3	Review of Probability Theory	1, i	-	7 - 10
4	Probabilistic reasoning	-	-	11 - 14
5	Applications of Probabilistic Reasoning in AI	-	-	15 - 17
6	Probability Reasoning in AI	-	-	18 - 19
7	Bayes theorem in AI	-	-	20 - 22
8	Importance of Bayes' Theorem in AI	-	1	23 - 25
9	Bayesian Inference	-	2	26 - 28
10	Summary	-	-	29
11	Glossary	-	-	30 - 31
12	Terminal Questions	-	-	32
13	Answers	-	-	33 - 34
14	References	-	-	35

1. INTRODUCTION

In previous unit, learners had explored the essential concept of planning, a core ability that allows intelligent agents to decide on a sequence of actions to achieve their goals effectively and efficiently. First, learners studied the Planning problem, which involved defining an initial state, a set of actions, and a desired goal state. They also learnt how AI systems determine plans that move from the starting point to the goal, handling constraints and selecting optimal paths.

Learners also dived into Partial order planning, a flexible approach where the exact order of some actions is not strictly fixed from the start. This method allows for concurrent and parallel actions, making planning more efficient and adaptable to real-world situations. Finally, we learnt about Planning graphs, a powerful representation that uses layered graphs to capture possible actions and resulting states over time. Planning graphs help identify achievable goals, estimate the cost of reaching them, and support efficient plan extraction.

In this unit, learners will explore how artificial intelligence handles situations where information is incomplete, ambiguous, or uncertain. They will begin by studying reasoning with uncertain information, learning why traditional true-or-false logic is not always enough in real-world scenarios. A review of probability theory will provide them with the mathematical foundation needed to quantify uncertainty and understand how to work with likelihoods and events. Building on this, learners will examine probabilistic reasoning, which enables AI systems to make predictions and decisions even when data is noisy or partially missing.

The unit will also cover practical applications of probabilistic reasoning in AI, such as medical diagnosis, spam detection, and recommendation systems, highlighting its critical role in modern intelligent systems. Learners will then focus on probability reasoning in AI, emphasising how probabilities guide learning and adaptive behaviour. A key part of the unit will be learning about Bayes' theorem, a fundamental tool for updating beliefs when new evidence arrives. The importance of Bayes' theorem in AI will be discussed in depth, explaining how it allows systems to integrate prior knowledge with new data for better decision-making. Finally, learners will study Bayesian inference, a method that uses Bayes' theorem to continuously refine predictions and improve AI performance over time.

By the end of the unit, learners will understand how to model, interpret, and reason under uncertainty, making them better equipped to design intelligent systems that can function effectively in dynamic and unpredictable environments.

1.1. Objectives

After studying this unit, you should be able to:

- Outline the role of probabilistic reasoning in AI
- Analyse the Concept of Bayesian theory and Bayesian networks
- Demonstrate probabilistic inference using Bayesian Networks
- Explain the other Paradigm of Uncertain Reasoning



2. REASONING WITH UNCERTAIN INFORMATION

The most fundamental concepts in probability theory for uncertain reasoning include independence, probability notation, inference using joint distributions, Bayes' rule, and the semantics of Bayesian networks.

For many kinds of decisions, reasoning is a crucial first step. Accurate and specific information enables effective reasoning and decision-making. However, when knowledge is uncertain, the likelihood of incorrect decisions increases. The major goal of this unit is to examine how decisions are formed in this circumstance with some uncertainty, specifically with the information that is not yet known.

There are a number of potential causes for data uncertainty, such as human error, faulty instruments, untrustworthy sources, and so on. We can't depend on models that employ specific data after we acquire it since then we'll have to make decisions with knowledge that isn't guaranteed. Probabilistic reasoning seems to be one possible solution for these kinds of situations. When faced with ambiguous knowledge, we can utilise probabilistic models to reason with a degree of certainty. Before we get into probabilistic reasoning.

Representing Knowledge in an Uncertain Domain, Basic Probability Notation, Inference Using Full Joint Distributions, Independence, Bayes' Rule and Its Use Bayesian Networks and Their Semantics

We have been confident in our understanding of the predicates up until this point since we have learnt knowledge representation using first-order and propositional logic. If we have this knowledge representation, we may say that if A is true, then B is true, but if we are unsure about the truth of A, we cannot say this; this is known as uncertainty. Therefore, uncertain reasoning or probabilistic reasoning is required to describe knowledge that is not certain, when the predicates are unknown.

Causes of uncertainty:

Following are some leading causes of uncertainty to occur in the real world.

1. Information occurred from unreliable sources.
2. Experimental Errors
3. Equipment fault
4. Temperature variation
5. Climate change.

3. REVIEW OF PROBABILITY THEORY

You should now understand the logic and its potential applications in probability theory. A brief overview of probability theory is in order before we go into the specifics of Bayes' theorem and its uses. All of the subsequent material in this unit will be built upon these ideas.

The many trials, sample spaces, and events that make up a random experiment are probably not new to you. This is known as a trial.

Consider the following examples

Example 1: Assume, for the sake of argument, that the experiment will include recording the outcomes of three separate coin flips. The experiment consists of three trials, with each throw serving as a trial, and the experiment is not considered complete until the third trial has concluded.

Example 2: The second example involves a specific testing method that is applied sequentially to a large number of manufactured objects. Ten items are selected from this pool. Obviously, there are ten trials in the experiment, and the underlying experiment doesn't finish until item number ten is selected.

Example 3: Each trial, or toss, in Example 1 determines whether the outcome is a head (H) or a tail (T). The experiment can have 8 possible outcomes: $s_1 = (H,H,H)$, $s_2 = (H,H,T)$, $s_3 = (H,T,H)$, $s_4 = (T,H,H)$, $s_5 = (T,T,H)$, $s_6 = (T,H,T)$, $s_7 = (H,T,T)$, and $s_8 = (T,T,T)$.

The sample space ζ is fixed. An event is already known to be a set of sample points from ζ . Picture yourself as an observer in the experiment that forms the basis of ζ . The outcome of the experiment is included in the event E if "the event E occurs" is taken at face value. The same holds true for the absence of event E : if it does not happen, the experiment will produce a result that is not related to event E . So, E_c is the set of all sample points that aren't part of E , another event that's complementary to E . E_c is the event that includes all the ζ sample points that are not in E . Therefore, it is intuitive that E happens only in the absence of E_c . E and E_c are two sides of the same coin; when combined, they make up the whole sample space, or $E \cup E_c = \zeta$.

All the sample points make up ζ , as you might remember. Therefore, the null event, typically represented as $\emptyset$, is the empty set that does not contain any sample points; so, $\forall c \quad \emptyset \subset \zeta$.

Let us once again consider

Thirdly, let's pretend (E) that all three throws result in a head. Hence, the event of not scoring a head at all, $E^c = \{s8\}$, is complementary to $E = \{s1, s2, s3, s4, s5, s6, s7\}$. The event that the white marble is picked up at least once is defined as $E = \{(r1, w), (r2, w), (w, r2), (w, r1)\}$ in Example 3, when replacement is not taken into account. Therefore, the occurrence of deciding against selecting the white marble altogether is represented by $E^c = \{(r1, r2), (r2, r1)\}$.

Two occurrences, E and F, will be considered now. $E \cup F$, which stands for "E union F," is used to represent the set of data points that are accountable for the presence of either E or F, or both. So, $E \cup F$ is a brand-new event that happens when either E or F takes place, or both—that is, when at least one of E or F happens. Looking at this from a broader perspective, we may construct a new event E_j as the "union" of the k events $E_1, E_2, \dots, E_k$. This new event has all the sample points that are in one of the events $E_1, E_2, \dots, E_k$, and it only happens if at least one of those events happens.

Once again, consider two events, E and F. $E \cap F$, which can be read as E "Intersection" F, represents the set of sample points where the presence of any one of these points implies the presence of both E and F. Hence, the union of E and F is an event that happens only when both E and F happen. In a more general sense, we can say that all k events $E_1, E_2, \dots, E_k$ must occur at the same time for a new event E_j to be defined as the "intersection" of all k events. This new event will contain sample points that are common to all k events. Additionally, if two events E and F do not have a common sample point, i.e. $E \cap F = \emptyset$, we say that they are mutually exclusive or disjoint.

You can't have two events that aren't compatible happening at the same time. Consider the coin-tossing experiment: if heads happens, tails cannot. These two outcomes are incompatible with one another. Re-examining Example 3 will help us make sense of what's happening. For example, if the number of heads is odd, we have $E = \{s1, s5, s6, s7\}$ and if the number of tails is odd, we have $F = \{s5, s8\}$. The case where only the third throw produces a head is denoted by $E \cap F = \{s5\}$. Therefore, E and F can both happen.

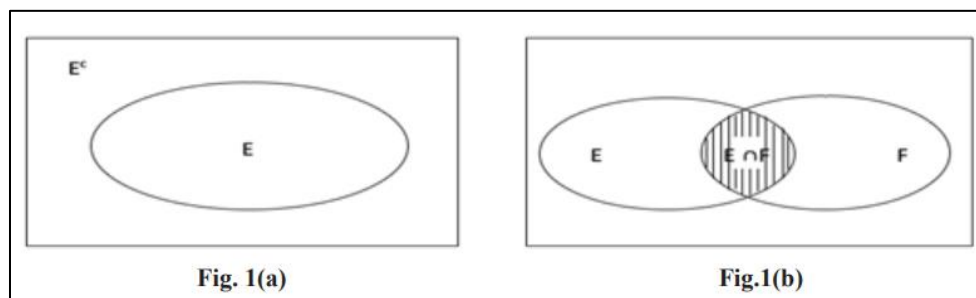


Figure 1: Venn diagram to show the occurrence of connections

The preceding connections between occurrences are best illustrated by means of a Venn diagram. The sample space $\mathcal{Y}$ is symbolised by a rectangle. The rectangle contains all of the sample points represented by points. A closed curve enclosing all the sample points that lead to an event represents the event in a graphical sense. The area within the rectangle that is not covered by the closed curve, which stands for E , stands for the supplementary event E_c . (As seen in Figure 1(a)) In Figure 1(b), the event is also represented by the empty space inside the broken line. Both $E \cup F$ and the shaded area stand for $E \cap F$.

Given the inherent uncertainty in random experiment outcomes, it is impossible to anticipate with absolute confidence any of the occurrences within a sample space prior to conducting the underlying experiment and recording its results. Some outcomes, nevertheless, may appear more probable than others based on your gut feelings. A person's life expectancy of 20 years appears to be higher than their life expectancy of 200 years, when considering only humans as an example. Thinking about this makes us want to find out whether there's a way to quantify the chances of different outcomes. A minor but crucial piece of information helps us out with this. A number of definitions are necessary before we go into detail on this.

If we take a random experiment and run it n times under the same conditions, we may see that event E —which is unlikely to happen every time—happens $fn(E)$ times in these runs. Next, we have $fn(E)$, which is the event's frequency in n iterations of the experiment and $rn(E)$ is the relative frequency of event E in n repeats of the experiment, and it is defined as $(E)/n = fn$. Now, think about the following scenario.

Fourthly, think about the experiment with tossing a coin. Consider the following scenario: we toss a coin five times, and the frequencies of heads are listed in **Table-1**:

No. of repetitions (n)	Frequency of head ($fn(H)$)	Relative frequency of head $rn(H)$
1	0	0
2	1	$1/2$
3	2	$2/3$
4	3	$3/4$
5	3	$3/5$

Look at Table-1's third column; it shows the relative frequencies of heads, $rn(H)$. Table 1 can be used to calculate the values of $rn(H)$ as the number of repetitions (n) increases. We offer below a very simplistic method that is by no means rigorous, but it aids in seeing things more clearly at this stage; all it does is fix notions about the concept of event probability.

Check Your Progress- 1

Problem - 1. In each of the following exercises, an experiment is described. Specify the relevant sample spaces:

- a) A machine manufactures a certain item. An item produced by the machine is tested to determine whether or not it is defective.
- b) An urn contains six balls, which are colored differently. A ball is drawn from the urn and its color is noted.
- c) An urn contains ten cards numbered 1 through 10. A card is drawn, its number noted and the card is replaced. Another card is drawn and its number is noted.

Problem - 2. Suppose a six-faced die is thrown twice. Describe each of the following events:

- i. The maximum score is 6.
- ii. The total score is 9.
- iii. Each throw results in an even score.
- iv. Each throw results in an even score larger than 2.
- v. The scores on the two throws differ by at least 2.

4. PROBABILISTIC REASONING

Applying the concept of probability to show the uncertainty in knowledge is known as probabilistic reasoning, which is a technique of representing knowledge. To deal with ambiguity, probabilistic reasoning integrates logic and probability theory. Probability is a tool in probabilistic thinking that helps us deal with the kind of uncertainty that arises when people are careless or don't know enough. Lots of things can happen in the actual world that we don't know for sure, such "It will rain today," "some people's behaviour in certain situations," and "a match between two teams or two players." Probabilistic reasoning is applied in situations where outcomes are possible but not certain.

Why AI Necessitates Probabilistic Reasoning:

The use of AI for probabilistic reasoning is crucial for many applications, including:

Machine learning algorithms benefit from probabilistic reasoning to learn from noisy or incomplete data.

Robotics: Allows robots to respond and engage with settings that are constantly changing and filled with uncertainty.

Natural Language Processing: Allows machines to comprehend the nuances and context-dependence of human speech.

Decision making systems: Probabilistic reasoning enables AI systems to make informed decisions by evaluating the likelihood of different possibilities.

By introducing uncertainty through probabilistic reasoning, the AI system can make accurate predictions and act rationally in the real world.

Key Concepts in Probabilistic Reasoning

1. Bayesian Networks

A Bayesian network is a graphical model that visually represents the probabilistic relationships between variables

Advantages: To convey thinking on missing data and cause and effect, Bayesian Networks operate exceptionally well. Their use in medical diagnosis has been extensive, since symptoms are variables with varying degrees of connection with diseases.

2. Markov Models

Think of a forecast for the weather. The present and historical states of a system are used to generate predictions about its future by a Markov model. Example: Bayesian Networks perform quite well when it comes to conveying reasoning about missing data and cause and effect, as shown by a basic Markov model. Due to the fact that symptoms are variables with varied degrees of association to diseases, their use in medical diagnosis has been considerable.

Weather-wise, there's a better chance that two consecutive sunny days will occur than that one sunny day will be followed by rain.

Advantages: Markov models are simple to use and produce good results. One common application is speech recognition, but they also include predictive capabilities that take into account the user's word selection history to determine the likelihood of the following word.

3. Hidden Markov Models (HMMs)

Hidden Markov Models (HMMs) are used when some variables are hidden or unobservable, such as hidden factors influencing the weather. One variant of Markov models that include hidden states is the Hidden Markov Model (HMM).

Advantages: When hidden variables are considered, HMMs show their true power. Predicting stock market values is a common example of this kind of work, and the variables that influence these prices are often opaque.

4. Probabilistic Graphical Models

A more comprehensive framework that includes Bayesian networks and HMMs is provided by Probabilistic Graphical Models. Generally speaking, PGMs are a way to describe and reason with ambiguous data that is presented in a graphical fashion.

Advantages: Complex relationships that may encompass various kinds of uncertainty are well-suited to PGMs because they provide a strong, versatile, and expressive vocabulary for undertaking probabilistic reasoning.

- These methods are not exclusive of one another; rather, they can be enhanced and combined to address an ever-growing list of AI-related challenges. For example, the nature of the uncertainty and the desired outcome will determine the specific method that can be employed. Consequently, AI systems can improve their decision-making capabilities through the use of probabilistic reasoning, which enables them to make quantitative predictions.

Techniques in Probabilistic Reasoning

Inference: Calculating the distribution of a set of variables based on known values of other variables is called probability theory. Methods for precise inference include the junction tree algorithm and variable elimination, while methods for approximate inference include MCMC and belief propagation.

Learning: Using data from observations to revise the parameters and framework of a probabilistic model. Three methods are available: expectation-maximisation (EM), maximum likelihood estimation, and Bayesian estimation.

Decision Making: To maximise expected benefit, use probabilistic models to guide decision-making. A common framework for implementing these techniques is the POMDP, which entails calculating predicted benefits and choosing actions accordingly.

How Probabilistic Reasoning Empowers AI Systems?

Just pretend for a second that you're lost in a labyrinth and have a hazy map to guide you. You would come to a standstill trying to use the type of conventional, rule-based reasoning that cannot account for the possibility of a dead end or an ambiguous path forward. In situations where the future is unclear, probabilistic reasoning can illuminate the way ahead like a strong torch. This is how AI systems are empowered by probabilistic reasoning:

Quantifying Uncertainty: Uncertainty is not a problem for probabilistic reasoning. It uses probability theory's tools to depict uncertainty by assigning likelihood levels. As an illustration, probabilistic reasoning could provide a 60% likelihood that tomorrow will see rain instead of a straightforward "true" or "false" answer.

Reasoning with Evidence: The luxury of autonomous decision-making is out of the question for AI systems. To aid in refining the probability, they must take into account the evidence that is available and act accordingly. For instance, if ominous clouds roll in during the afternoon, the likelihood of a wet day can be fine-tuned to 80%.

Based on Past Experience: Preceding instances can teach AI systems new things. Knowing the decision-making process inside and out is an important component of probabilistic reasoning. So, for instance, when determining the likelihood of rain, an AI system trained on local meteorological records from the past might take seasonal trends into account.

Effective Decision-Making: Additionally, AI systems will be able to make educated conclusions utilising data, past knowledge, and measured uncertainty through the use of probabilistic reasoning.

Going back to our maze example, the AI may improve its chances of reaching the goal by weighing the likelihood of different paths based on the map at each location and whatever it has already traversed. The goal of probabilistic thinking is not to arrive at an ideal solution in an uncertain world, but rather to accept the limitations of our knowledge and make the most of what we have. Thus, AI systems are able to function well in the actual world, which is rife with ambiguity and lacking in complete data.



5. APPLICATIONS OF PROBABILISTIC REASONING IN AI

Probabilistic reasoning is widely applicable in a variety of domains:

Robotics: It is possible for robots to traverse and engage with unpredictable surroundings through the use of probabilistic reasoning. One example is the SLAM method, which tracks the robot's position in real time while also using probabilistic techniques to build maps of unfamiliar areas.

Healthcare: Probabilistic models are useful in medical diagnosis since they allow one to evaluate the probability of diseases based on symptoms and test findings. One example is the ability of Bayesian networks to predict the links between different medical problems and diagnostic signs.

Natural Language Processing (NLP): Applications of probabilistic models include machine translation, named entity recognition, and part-of-speech tagging. Examples of such models are HMMs and Conditional Random Fields (CRFs).

Finance: Modelling market behaviour, assessing risks, and making investment decisions all make use of probabilistic reasoning. Financial modelling frequently utilises methods such as Bayesian inference and Monte Carlo simulations.

Advantages of Probabilistic Reasoning

- **Flexibility:** Probabilistic models can handle a wide range of uncertainties and are adaptable to various domains.
- **Robustness:** These models are robust to noise and incomplete data, making them reliable in real-world applications.
- **Interpretable:** Probabilistic models provide a clear framework for understanding and quantifying uncertainty, which can aid in transparency and explainability.

In order to make rational conclusions in the face of problem ambiguity, probabilistic reasoning is a popular and crucial tool for artificial intelligence applications. Artificial intelligence systems are able to improve their performance and dependability by utilising the built-in probability to handle real-world complexity.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

- Bayes' rule
- Bayesian Statistics

Uncertain Knowledge and Reasoning: Acting under Uncertainty, Basic

Probability Notation, Inference Using Full Joint Distributions,

Independence, Bayes' Rule and Its Use, Representing Knowledge in an

Uncertain Domain, The Semantics of Bayesian Networks

Till now, we have learned knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates. With this knowledge representation, we might write $A \rightarrow B$, which means if A is true then B is true, but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called uncertainty.

So to represent uncertain knowledge, where we are not sure about the predicates, we need uncertain reasoning or probabilistic reasoning.

Causes of uncertainty:

Following are some leading causes of uncertainty to occur in the real world.

1. Information occurred from unreliable sources.
2. Experimental Errors
3. Equipment fault
4. Temperature variation
5. Climate change.

Probabilistic reasoning:

Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge. In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.

We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.

In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "It will rain today," "behavior of someone for some situations," "A match between two teams or two players." These are probable sentences for which we can assume that it will happen but not sure about it, so here we use probabilistic reasoning...

Need of probabilistic reasoning in AI:

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

- Bayes' rule
- Bayesian Statistics

As probabilistic reasoning uses probability and related terms, so before understanding probabilistic reasoning, let's understand some common terms:

Probability: Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties

As probabilistic reasoning uses probability and related terms, so before understanding probabilistic reasoning.

6. PROBABILITY REASONING IN AI

Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties

$0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A .

$P(A) = 0$, indicates that the event A will not occur.

$P(A) = 1$, indicates total certainty in an event A .

We can find the probability of an uncertain event by using the below formula.

$P(\neg A)$ represents the probability that event A does not occur

$$P(\neg A) = 1 - P(A)$$

- Event: Each possible outcome of a variable is called an event.
- Sample space: The collection of all possible events is called sample space. Random variables:
- Random variables are used to represent the events and objects in the realworld.
- Prior probability: The prior probability of an event is probability computed before observing new information.
- Posterior Probability: The probability that is calculated after all evidence or information has taken into account. It is a combination of prior probability and new information.

Conditional probability:

Conditional probability is a probability of occurring an event when another event has already happened

Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B ", it can be written as:

$$P(E) = \frac{\text{Number of favorable outcomes}}{\text{Total number of possible outcomes}}$$

- **P(E):** Probability of event E occurring.
- **Number of favourable outcomes:** The count of outcomes that satisfy event E .
- **Total number of possible outcomes:** All outcomes in the sample space (the complete set of possible results).

If the probability of A is given and we need to find the probability of B, then it will be given as

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

- **P(B|A):** Probability of event B occurring given that event A has occurred.
- **P(A∩B):** Probability that both A and B occur (joint probability).
- **P(A):** Probability of event A.

It can be explained by using the below Venn diagram, where B is occurred event, so sample space will be reduced to set B, and now we can only calculate event A when event B is already occurred by dividing the probability of P(A∩B) by P(B).

Example: In a class, there are 70% of the learners who like English and 40% of the learners who like English and mathematics, and then what is the percentage of learners who like English also like mathematics

Solution:

Let A be an event that a student likes Mathematics, and B be an event that a student likes English.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the learners who like English also like Mathematics.

7. BAYES' THEOREM IN ARTIFICIAL INTELLIGENCE

Bayes' Theorem, also known as Bayes' rule or Bayes' law, is fundamental in probability and artificial intelligence. It allows for updating the likelihood of an event as new information is obtained.

Find out how artificial intelligence makes use of the Bayes theorem in this article.

An Artificial Intelligence Application of Bayes' Theorem

The relation between the conditional probability and marginal probability of two random events is discussed in Bayes' theorem, a result of probability theory. Simply said, it gives a method to determine the value of $P(B|A)$ given the information about $P(A|B)$.

To determine the conditional probability, one uses the formula known as Bayes' theorem. Here is the formula:

$$P(A|B) = P(A \cap B) / P(B) = (P(A) * P(B|A)) / P(B)$$

where,

- $P(A)$ is the probability that event A occurs.
- $P(B)$ defines the probability that event B occurs.
- $P(A|B)$ is the probability of the occurrence of event A given that event B has already occurred.
- $P(B|A)$ can now be read as: Probability of event B occurring given that event A occurred.
- $p(A \cap B)$ is the probability events A and B will happen together.

The Bayes' Theorem encompasses four major elements:

1. **Prior Probability $P(A)$:** The likelihood or conviction in an occurrence A before taking any new evidence into account; it stands for our existing understanding of A.
2. **Likelihood $P(B|A)$:** how strongly the evidence points towards the event, given the occurrence of event A, is determined by the probability of evidence B.
3. **Evidence $P(B)$:** Whether or not A is true, the likelihood of detecting evidence B is evidence. In order for the posterior probability to be a legitimate probability distribution, it normalises the distribution.
4. **Posterior Probability $P(A|B)$:** A change in belief about event A based on new evidence B is known as the posterior probability. It provides a response to the query, "Given evidence B, what is the likelihood that A is true?"

These parts are used by Bayes' Theorem to calculate the posterior probability $P(A|B)$, which is our revised belief in A in light of the new evidence.

When it comes to AI, the Bayes Theorem and probability really shine when it comes to drawing conclusions or making decisions with incomplete or questionable data. Artificial intelligence (AI), machine learning (ML), and decision-making all rely on it since it allows us to logically revise our opinions in light of fresh facts.

Bayes' theorem in AI

This capacity of AI to deal with ambiguity and base decisions on probabilities makes Bayes' theorem very relevant to the field. I'll explain the importance of it here:

Probabilistic Reasoning: A lot of the time, AI systems have to reason with unknowns in the real world. Based on new evidence, AI systems can alter their views thanks to Bayes' theorem. Applications such as driverless vehicles rely on this, as their surroundings are dynamic and sensor readings might be noisy.

Machine Learning: Bayesian machine learning techniques are based on Bayes' theorem. Using these techniques, AI models can learn from their mistakes and adjust their assumptions in light of new evidence. This is invaluable when working with complicated interrelationships between variables or in situations where data is scarce.

Classification and Prediction: Applying Bayes' theorem to classification problems like medical diagnosis or spam email detection allows one to determine the likelihood that an input belongs to a specific class. Because of this, AI systems may use the evidence at their disposal to make better decisions.

Anomaly Detection: The application of Bayes' theorem in anomaly detection allows AI systems to spot out-of-the-ordinary patterns in data. Bayes' theorem can be used to model a system's typical behaviour and identify any departures from this norm, which can indicate possible security issues or anomalies.

In sum, Bayes' theorem is foundational for numerous artificial intelligence applications, such as pattern recognition and decision-making, since it gives a robust framework for reasoning under uncertainty.

Proof of Bayes' Rule through Mathematics

Bayes' Rule is derived from the definition of conditional probability. Let's start with the definition:

$$P(A|B) = P(A \cap B) / P(B)$$

This equation states that the probability of event A given event B is equal to the probability of both events happening (the intersection of A and B) divided by the probability of event B.

Similarly, we can write the conditional probability of event B given event A:

$$P(B|A) = P(A \cap B) / P(A)$$

By rearranging this equation, we get:

$$P(A \cap B) = P(B|A) \cdot P(A) = P(A|B) \cdot P(B)$$

Now, we have two expressions for $P(A \cap B)$. Since both expressions are equal to $P(A \cap B)$, we can set them equal to each other:

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

To get $P(A|B)$, we divide both sides by $P(B)$:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

8. IMPORTANCE OF BAYES' THEOREM IN AI

Many branches of artificial intelligence (AI) rely heavily on Bayes' Theorem.

- **Probabilistic Reasoning:** This method is crucial in artificial intelligence since many AI problems include uncertainty. With the help of Bayes' Theorem, AI systems can represent and reason with uncertainty by revising their beliefs based on fresh evidence. Making decisions, recognising patterns, and using predictive modelling all rely on this.
- **Machine learning,** and more especially Bayesian machine learning, rely on Bayes' Theorem. For the purpose of modelling complicated relationships, estimating model parameters, and making predictions, Bayesian approaches are employed. When dealing with classification, regression, or clustering problems, Bayesian models allow for the principled handling of uncertainty.
- In the field of data science, Bayes' Theorem finds widespread use in Bayesian statistics. Hypothesis testing, Bayesian inference, and Bayesian optimisation are just a few of the many applications that make use of it to estimate and update probability. It provides a uniform structure for data modelling and understanding.

Example of Bayes' Rule Application in AI

Application of Bayes' Rule in spam email classification is one of the classic examples of AI. Here we see an application of Bayes' Theorem to the problem of email spam classification using keyword presence.

Think about a spam filter that checks all incoming emails for the word "win" to see if they are spam or not. Here are the possibilities that have been provided to us:

- $P(S)$: The prior probability that any given email is spam.
- $P(H)$: The prior probability that any given email is not spam (ham).
- $P(W|S)$: The probability that the word "win" appears in a spam email.
- $P(W|H)$: The probability that the word "win" appears in a non-spam email.
- $P(W)$: The probability that the word "win" appears in any email.

Given Data

- $P(S)=0.2$ (20% of emails are spam)
- $P(H)=0.8$ (80% of emails are not spam)

- $P(W|S)=0.6$ (60% of spam emails contain the word "win")
- $P(W|H)=0.1$ (10% of non-spam emails contain the word "win")

We want to find $P(S|W)$, the probability that an email is spam given that it contains the word "win".

Applying Bayes' rule, we get:

$$P(S|W) = P(W)P(W|S) \cdot P(S) / P(S)P(S|W) = [P(W|S) \cdot P(S)] / P(W)$$

First, we need to calculate $P(W)$, the probability that any email contains the word "win". Using the law of total probability:

$$P(W) = P(W|S) \cdot P(S) + P(W|H) \cdot P(H)$$

Substituting the given values:

$$(0.6 \times 0.2) + (0.1 \times 0.8) = 0.12 + 0.08 = 0.20$$

Now, we can use Bayes' Rule to find $P(S|W)$:

$$P(S|W) = [P(W|S) \cdot P(S)] / P(W)$$

Substituting the values:

$$P(S|W) = [0.6 \times 0.2] / 0.2 = 0.12 / 0.2 = 0.6$$

We can deduce that an email with the word "win" in it is likely spam with a probability of 0.6, or 60%. So, there's a 60% probability that an email with the word "win" in it is spam.

This computation would be a component of a bigger model that takes into account various properties (words) within an email in a real-world AI system, such a spam filter. In order to effectively and accurately categorise emails, the filter makes use of these probabilities in conjunction with other algorithms. Adapting to new types of spam and improving its accuracy over time is achieved by continuously changing the probability based on incoming data.

Uses of Bayes Rule in Artificial Intelligence

Probabilistic inferences, belief updates, and information-based decision-making are all enabled by Bayes' theorem in AI. We will look at several significant AI uses of Bayes' rule here.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the main purpose of Bayes' theorem in AI? A) To classify data without using prior knowledge B) To update the probability of a hypothesis when new evidence is introduced C) To generate random numbers for simulation D) To store data efficiently
2	In AI applications, Bayes' theorem is mainly used for: A) Reducing memory storage B) Making decisions under uncertainty by combining prior knowledge and evidence C) Sorting databases alphabetically D) Converting continuous data into discrete
3	Which of the following best describes a practical use of Bayes' theorem? A) To determine the exact outcome of a coin flip B) To predict the probability of a patient having a disease given certain symptoms C) To create static rule-based expert systems D) To build relational database tables



9. BAYESIAN INFERENCE

For the purpose of updating the probability distribution across a collection of parameters or hypotheses using observed data, the Bayes' rule is employed in Bayesian statistics. For machine learning tasks such as Bayesian network parameter estimation, hidden Markov models, and probabilistic graphical models, this is of utmost importance.

Naive Bayes Classification: There is extensive use of the Naive Bayes classifier in the domain of text classification and natural language processing. By applying Bayes' theorem, it determines, from a document's word count alone, the probability that it belongs to a certain category. In spite of its "naive" belief in the independence of features, it performs admirably when put into practice.

Bayesian Networks: Graphical models known as Bayesian networks depict and forecast probabilistic correlations between variables by applying Bayes' theorem. Medical diagnosis, fault detection, and decision support systems are just a few of the many AI applications that use them.

Spam Email Filtering: To ascertain whether a received email constitutes spam or not, email filtering systems employ Bayes' theorem. Based on its calculations, the model changes the probability based on whether it is more likely to find certain phrases or traits in spam or non-spam emails.

Reinforcement Learning: One way to model the world probabilistically is using Bayes' rule. With the use of Bayesian reinforcement learning techniques, agents can improve their decision-making by estimating and updating their beliefs regarding state transitions and rewards.

Bayesian Optimisation: The objective function can be represented as a probabilistic surrogate using Bayes' theorem in optimisation problems. In order to efficiently identify the optimal solution, Bayesian optimisation techniques use this model to iteratively explore and exploit the search space. Hyperparameter tuning and optimisation of algorithm parameters often make advantage of this.

Anomaly Detection: One application of Bayes' theorem is in detecting outliers or anomalies in datasets. Anomaly detection is useful for many things, like spotting fraud and ensuring the safety of networks, because it allows one to quantify outliers by modelling the normal distribution.

Personalisation: Applying Bayes' theorem to recommendation systems allows for the updating of user preferences and the provision of personalised recommendations. The system can provide better recommendations by continuously adjusting a user's preferences according to their behaviour.

Robotics and Sensor Fusion: Combining sensors is done using the Bayes' rule in robotics. It

estimates the robot's or the environment's status using data from numerous sensors. Jobs like mapping and localisation require this.

Medical Diagnosis: Medical decision support systems implement Bayes' theorem to revise the probability of different diagnosis in light of patient symptoms, laboratory findings, and medical history.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
4	Which of the following best describes "uncertain knowledge" in AI? A) Knowledge that is always false B) Knowledge that is complete and precise C) Knowledge that involves incomplete, noisy, or ambiguous data D) Knowledge stored only in rule-based systems
5	What is the primary reason for using probability theory in AI? A) To perform faster arithmetic calculations B) To ensure all data is always correct C) To quantify and manage uncertainty in data and reasoning D) To avoid using any logical operators
6	In probability theory, what is the range of probability values? A) 0 to infinity B) 0 to 10 C) -1 to 1 D) 0 to 1
7	Which of the following is an example of probabilistic reasoning? A) Deductive logic using strict true/false rules B) Calculating exact solutions in algebra C) Estimating the likelihood of rain based on weather data D) Executing a predefined sequence of instructions
8	In AI, why is probabilistic reasoning preferred over deterministic reasoning in many real-world applications? A) Because it uses fewer mathematical operations B) Because real-world data is often uncertain and incomplete

- | | |
|--|---|
| | <p>C) Because it requires no data at all</p> <p>D) Because it always produces exact answers</p> |
|--|---|



10. SUMMARY

In this unit, you have learnt:

- In artificial intelligence (AI), reasoning with uncertain information is crucial because real-world environments are often incomplete, noisy, or unpredictable.
- Unlike traditional logic-based systems that rely on exact facts, probabilistic approaches allow systems to handle uncertainty and make decisions even when information is imperfect. A solid review of probability theory is fundamental for this, as it provides the mathematical basis to quantify uncertainty using probabilities.
- Probabilistic reasoning enables AI systems to infer the likelihood of outcomes, update beliefs when new evidence arrives, and choose actions that maximise expected success. This reasoning framework is widely applied in AI applications such as diagnostic systems, speech recognition, autonomous driving, and spam detection, where clear-cut data cannot always be expected.
- The use of probability reasoning in AI further allows for robust decision-making by integrating evidence from different sources and weighing the likelihood of various scenarios.
- Bayes' theorem is a key tool in this context, offering a mathematical way to update the probability of a hypothesis given new evidence. The importance of Bayes' theorem in AI lies in its ability to formally incorporate prior knowledge and continuously refine predictions, making it fundamental for dynamic learning systems.
- Bayesian inference, which extends Bayes' theorem, is used to compute posterior probabilities, enabling systems to adaptively learn from data and improve performance over time. Overall, these probabilistic techniques empower AI systems to function intelligently in uncertain, real-world conditions, making them more reliable and human-like in their reasoning processes.

11. GLOSSARY

Uncertain Information	Information that is incomplete, ambiguous, or unreliable. In AI, systems - must handle and reason about such information to make practical decisions.
Reasoning	The process of drawing conclusions or making inferences from facts or evidence. In AI, reasoning helps machines "think" and solve problems.
Probability Theory	A branch of mathematics that deals with quantifying the likelihood of different outcomes. Forms the foundation for probabilistic reasoning in AI.
Probabilistic Reasoning	A reasoning approach that uses probabilities to handle uncertainty and make predictions or decisions when exact information is unavailable.
Prior Probability	The initial probability assigned to an event or hypothesis before new evidence is considered. Important in Bayesian reasoning.
Posterior Probability	The updated probability of a hypothesis after incorporating new evidence, calculated using Bayes' theorem.
Bayes' Theorem	A fundamental formula used to update the probability of a hypothesis based on new evidence. Expressed as: $P(H E) = \frac{P(E H) \times P(H)}{P(E)}$ Where $P(H E)$ is the posterior probability, $P(E H)$ is the likelihood, $P(H)$ is the prior, and $P(E)$ is the evidence probability.
Bayesian Inference	A statistical method using Bayes' theorem to update the probability of hypotheses as more evidence becomes available. Widely used in AI for adaptive learning.
Likelihood	The probability of observing the given evidence assuming a specific hypothesis is true.
Evidence	The observed data or information used to update beliefs or probabilities in probabilistic models.

Diagnostic System	- An AI application that uses probabilistic reasoning to infer causes from observed symptoms (e.g., medical diagnosis).
Inference Network (Bayesian Network)	- A graphical model representing probabilistic relationships among variables. Used to compute joint probabilities and update beliefs efficiently.
Decision Making Under Uncertainty	- The process in AI where systems select the best action based on uncertain or incomplete information, often using probabilities to guide choices.
Spam Detection	- An AI application where probabilistic reasoning (often using Bayesian classifiers) is used to classify emails as spam or not spam based on observed patterns.
Dynamic Learning	- The ability of a system to continuously improve or adapt its predictions and decisions as new data becomes available, often using Bayesian inference.

12. TERMINAL QUESTIONS

1. What is the main advantage of using probabilistic reasoning in AI
2. Define probability in the context of AI.
3. What is the formula for Bayes' theorem?
4. What is prior probability?
5. What is posterior probability?
6. Name one real-world application of probabilistic reasoning in AI.
7. What is a Bayesian network?
8. Why is Bayes' theorem important in AI?
9. What does reasoning with uncertain information mean?
10. What is likelihood in the context of Bayes' theorem?

13. ANSWERS

Self-Assessment Questions

1. Answer: B
2. Answer: B
3. Answer: B
4. Answer: C
5. Answer: C
6. Answer: D
7. Answer: C
8. Answer: B

Terminal Questions Answers

1. Answer:
It allows AI systems to handle uncertainty and make decisions even when information is incomplete or noisy.
2. Answer:
Probability quantifies the likelihood of an event or hypothesis occurring, allowing systems to represent and manage uncertainty numerically
3. Answer:
$$P(H|E) = [P(E|H) \times P(H)] / P(E)$$

Where $P(H|E)$ is the posterior probability, $P(E|H)$ is the likelihood, $P(H)$ is the prior probability, and $P(E)$ is the evidence probability. (Refer section 7 for more information)
4. Answer:
The initial belief or probability of a hypothesis before new evidence is taken into account.
5. Answer:
The updated probability of a hypothesis after considering new evidence, calculated using Bayes' theorem. (Refer section 5 for more information)
6. Answer:
Medical diagnosis systems, where probabilities are used to infer diseases based on symptoms. (Refer section 5 for more information)
7. Answer:

A graphical model that represents probabilistic relationships among variables, used to perform reasoning and update beliefs in AI.

8. Answer:

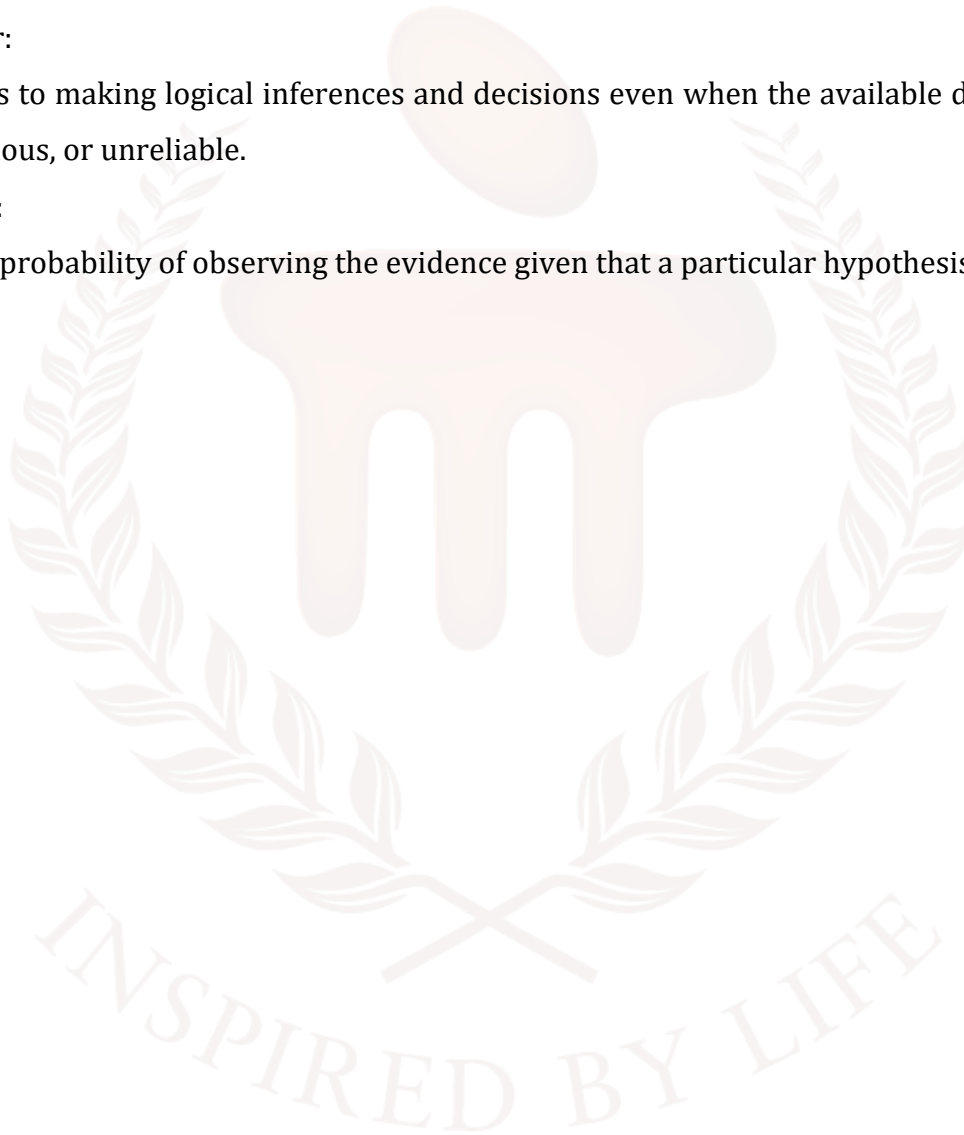
It provides a systematic way to update beliefs as new evidence is received, making AI systems more adaptive and intelligent.(Refer Section5 for more information)

9. Answer:

It refers to making logical inferences and decisions even when the available data is incomplete, ambiguous, or unreliable.

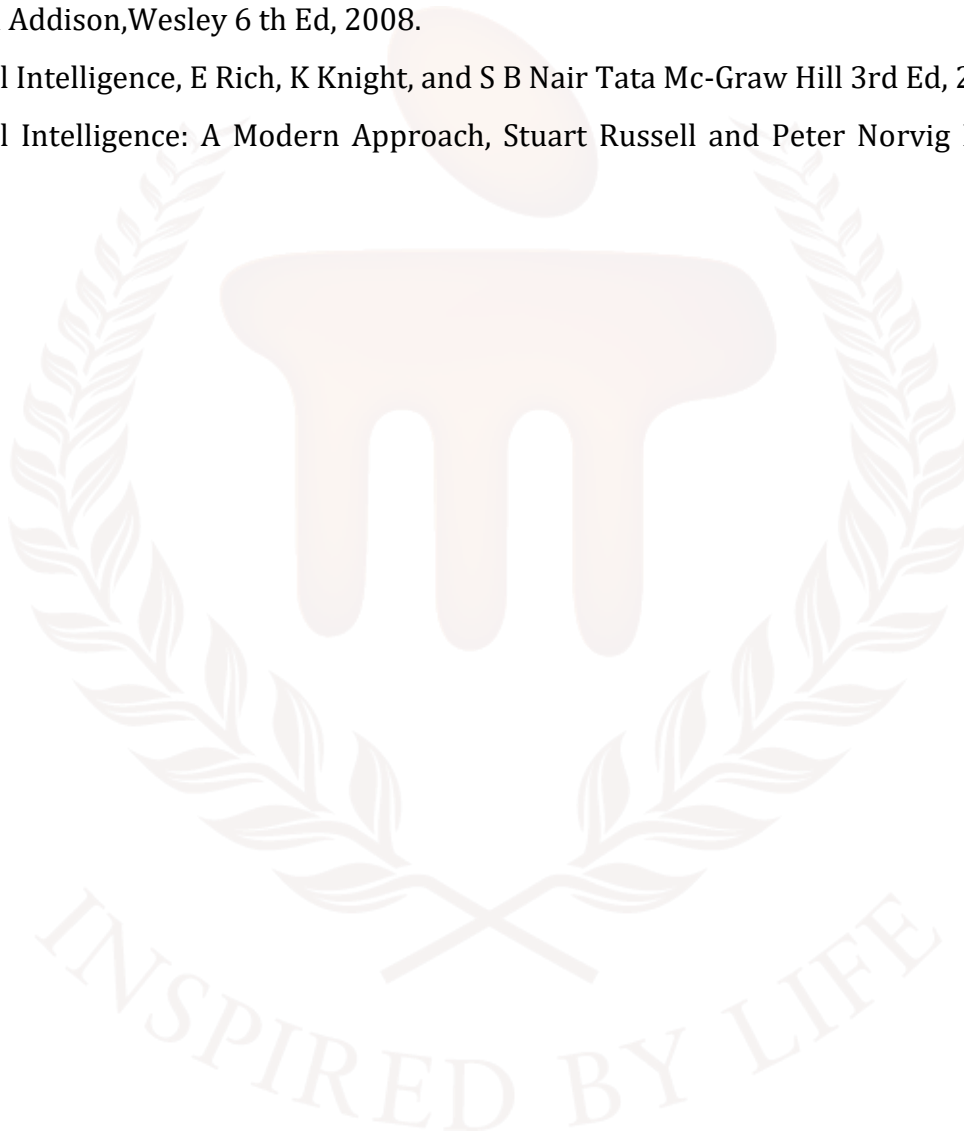
10. Answer:

It is the probability of observing the evidence given that a particular hypothesis is true.



14. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison,Wesley 6 th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

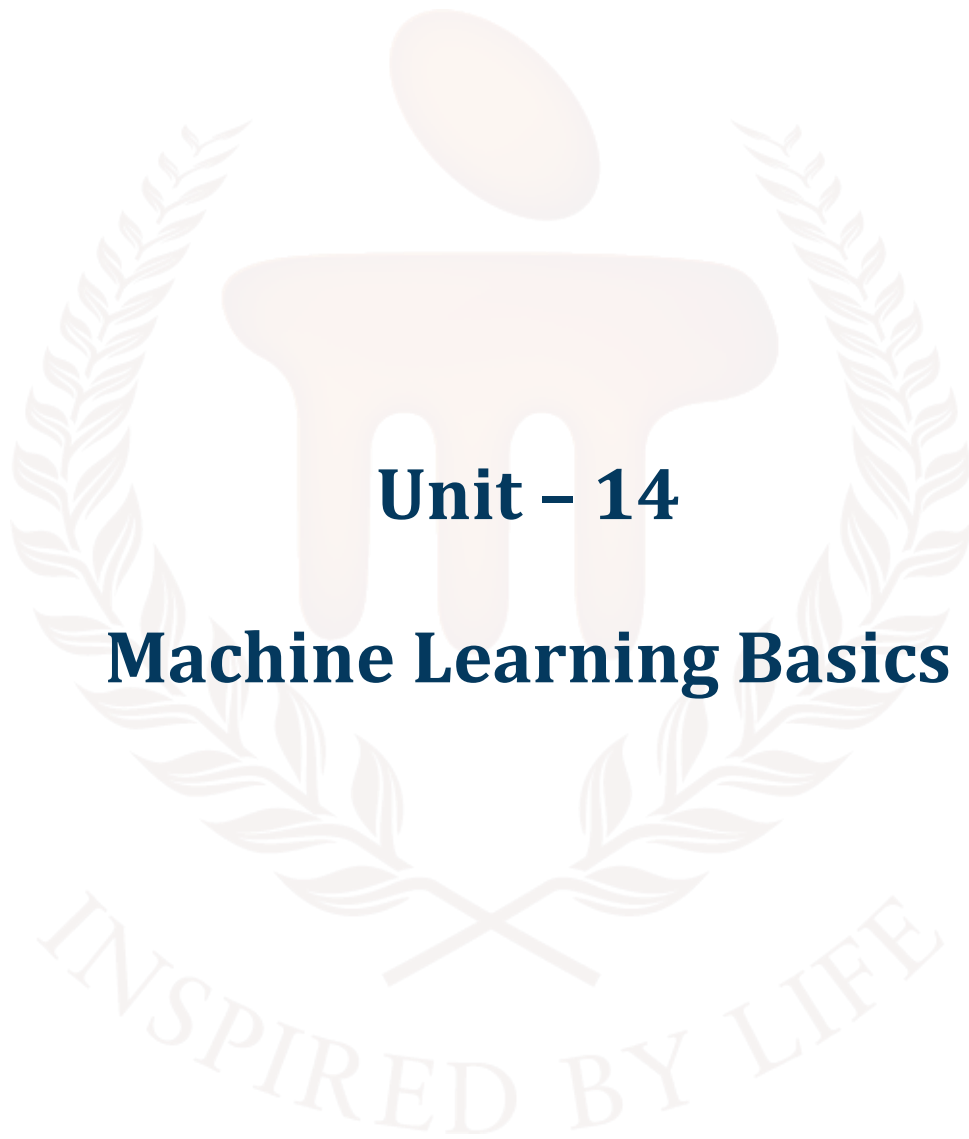


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2109
ARTIFICIAL INTELLIGENCE FOR
PROBLEM SOLVING



Unit – 14

Machine Learning Basics

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Introduction to Machine Learning	1	-	5 - 8
3	Working of Machine Learning	2	-	9 - 11
4	Types of Machine Learning	-	1	10 - 14
5	Supervised Learning	3, 4, 5	-	15 - 21
6	Training a Supervised Learning Model	-	2	22 - 24
7	Unsupervised Learning	6, 7	3	25 - 30
8	Reinforcement Learning	8	4	31 - 34
9	Summary	-	-	35 - 36
10	Glossary	-	-	37 - 38
11	Terminal Questions	-	-	39
12	Answers	-	-	40 - 41
13	References	-	-	42

1. INTRODUCTION

In previous unit, learners had explored how artificial intelligence handles situations where information is incomplete, ambiguous, or uncertain. We began by studying reasoning with uncertain information, learning why traditional true-or-false logic is not always enough in real-world scenarios. A review of probability theory will provide them with the mathematical foundation needed to quantify uncertainty and understood how to work with likelihoods and events. Building on this, we were will examined probabilistic reasoning, which enables AI systems to make predictions and decisions even when data is noisy or partially missing.

The previous unit also covered the practical applications of probabilistic reasoning in AI, such as medical diagnosis, spam detection, and recommendation systems, highlighting its critical role in modern intelligent systems. Learners will then focus on probability reasoning in AI, emphasising how probabilities guide learning and adaptive behaviour. A key part of that unit will be learning about Bayes' theorem, a fundamental tool for updating beliefs when new evidence arrives. The importance of Bayes' theorem in AI was discussed in depth, explaining how it allows systems to integrate prior knowledge with new data for better decision-making. Finally, learners will study Bayesian inference, a method that uses Bayes' theorem to continuously refine predictions and improve AI performance over time.

1.1. Objectives

After studying this unit, you should be able to:

- Outline the fundamental concepts and importance of machine learning
- Differentiate between supervised, unsupervised, and reinforcement learning approaches.
- Explain the key algorithms and techniques used in each type of machine learning.
- Analyse the advantages and limitations of different machine learning methods.



2. INTRODUCTION TO MACHINE LEARNING

The field of artificial intelligence known as machine learning allows algorithms to discover previously unseen patterns in data sets. Machine learning algorithms can make predictions on new, unseen data without requiring explicit task-specific programming. Image and audio recognition, NLP, recommendation systems, fraud detection, optimisation of portfolios, and job automation are just a few of the many areas that machine learning finds use in.

Autonomous cars, drones, and robots are all impacted by machine learning, which makes them more adaptable in changing settings. This method, which is highly related to data mining and data science, represents a significant advance in the area where computers may learn from data examples in order to produce reliable results.

It also allows for developing complex algorithms that may be used to train certain machines to carry out specific tasks. In order to take a set of input data and generate outputs based on specific patterns, one needs a dataset and then uses that dataset to investigate correlations, uncover patterns, and apply algorithms. You can generalise models for specific tasks from this dataset because each item has its unique set of features.

Machine learning, in its most basic form, may be thought of as computers learning from large datasets autonomously. Overarchingly, machine learning is an AI subfield that aims to investigate algorithmic structures that enable autonomous understanding; this, in turn, opens the door to the prospect of recognising and extracting patterns from massive amounts of data, thereby constructing a learning model.

The field of artificial intelligence known as machine learning allows for the development of complex algorithms that may be used to train certain machines to carry out specific tasks. In order to take a set of input data and generate outputs based on specific patterns, one needs a dataset and then uses that dataset to investigate correlations, uncover patterns, and apply algorithms. You can generalise models for a given job using this dataset because each item has its unique collection of features

Machine learning, in its most basic form, may be thought of as computers learning from large datasets autonomously. Machine learning, in its broadest sense, is an area of AI that aims to study algorithmic structures that enable autonomous understanding; this, in turn, opens the door to the prospect of recognising and extracting patterns from massive amounts of data, thereby constructing a learning model.

Learning from examples—whether formal education or experience—enables systems to improve over time. After learning, the algorithm can handle dynamic and difficult tasks with ease, improve its predictions, adapt to new situations, and behave intelligently. Thus, machine learning can be described as a field of study that enables computers to learn on their own, without being explicitly programmed to do so.

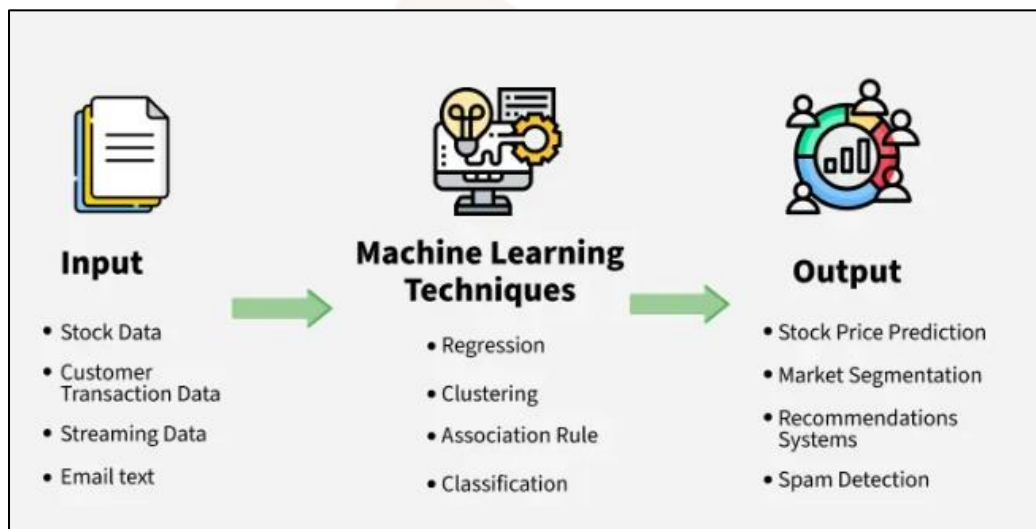


Figure 1: Representation of Machine Learning Process

Need for Machine Learning

The ability for computers to automatically learn from data and become better at certain activities is known as machine learning, and it is crucial for many reasons. Machine learning is highly effective for jobs requiring handling massive volumes of data, intricate decision-making, and ever-changing settings due to its data-learning and adaptability capabilities.

A few examples of concrete applications of machine learning are these:

Predictive modeling: Better decision-making for organisations is possible with the use of predictive models built using machine learning. To illustrate the point, machine learning can be employed to forecast which consumers are most inclined to purchase a specific item or which individuals are most prone to contracting a specific illness.

Natural language processing: The development of systems capable of comprehending and interpreting human language is facilitated by machine learning. Applications like language translation, chatbots, and voice recognition rely on this.

Computer vision: Systems that can identify and understand visual content are created using machine learning. This is crucial for uses like medical imaging, surveillance systems, and self-driving cars.

Fraud detection: Machine learning has several potential applications, one of which is the detection of fraudulent activity in online advertising and financial transactions.

Recommendation systems: Recommendation systems use machine learning to provide users with content, products, or services based on their preferences and previous actions.

Because it helps organisations use data more effectively, make better decisions, and provide customers with more personalised experiences, machine learning has become a crucial tool for many businesses and industries.

Machine Learning, Conventional Programming, and AI: What They Have in Common

Machine Learning	Traditional Programming	Artificial Intelligence
A subset of AI focusing on creating algorithms that learn from data and make predictions.	Writing rule-based, deterministic code based on specific problem statements.	Technology that enables machines to perform tasks that typically require human intelligence.
Data-driven, learns from historical data to predict future outcomes.	Rule-based and deterministic, relies on explicit instructions from developers.	Uses a mix of data-driven techniques and predefined rules, incorporating ML, deep learning, and traditional programming.
Capable of finding patterns and insights in large datasets, learning and improving over time.	Lacks self-learning capabilities; output is directly tied to input and predefined rules.	Adapts and evolves to perform complex tasks with high accuracy, often exceeding human capabilities in specific domains.
Used in predictive analytics, autonomous vehicles,	Used to build applications with specific functionalities like	Broad applications including natural language processing,

Machine Learning	Traditional Programming	Artificial Intelligence
chatbots, and other AI-based applications.	software tools and systems.	computer vision, robotics, and more.
Dependent on the quality and diversity of data. Can perform poorly if data is not representative.	Dependent on the intelligence and foresight of developers. Limited to known scenarios.	Combines the strengths of both ML and traditional programming to tackle complex, multi-faceted problems.



3. WORKING OF MACHINE LEARNING

Machine learning algorithms are able to learn and adapt to new situations automatically by spotting patterns and correlations in large amounts of data.

The typical steps in a machine learning process are as follows:

1. Data Collection

The first step is collecting relevant data, which may include examples, features, or attributes such as images, text, or numerical values.

2. Data Preprocessing

Data often needs to be preprocessed before being sent into the algorithm. At this stage, the data may be cleaned up (by dealing with outliers and missing values) and transformed (by normalising or scaling) before being divided into a training set and a test set.

3. Choosing a Model

A machine learning model is selected according to the task at hand, such as classification, regression, or clustering. Decision trees, SVMs, neural networks, and more complex models with deep learning architectures are just a few examples.

4. Training the Model

With the help of the training data, the chosen model is trained. The system picks up on data patterns and correlations during training. In order to do this, the model parameters are adjusted in an iterative fashion until the disparity between the expected and actual outputs (labels or targets) in the training data is minimised.

5. Evaluating the Model

After the model has been trained, its performance is assessed by comparing it to the test data. To measure the model's ability to generalise to new, unknown data, metrics like accuracy, precision, recall, or mean squared error are employed.

6. Fine-tuning

Hyperparameters, such as the learning rate or the number of hidden layers in a neural network, are not directly learnt during training but can be fine-tuned to improve the performance of a model.

7. Prediction or Inference

As a last step, new data is fed into the trained model in order to generate predictions or judgements. Class labels in classification tasks and numerical values in regression tasks are examples of outputs that can be generated by applying the learnt patterns to new inputs.

Machine Learning Lifecycle

The machine learning lifecycle includes:

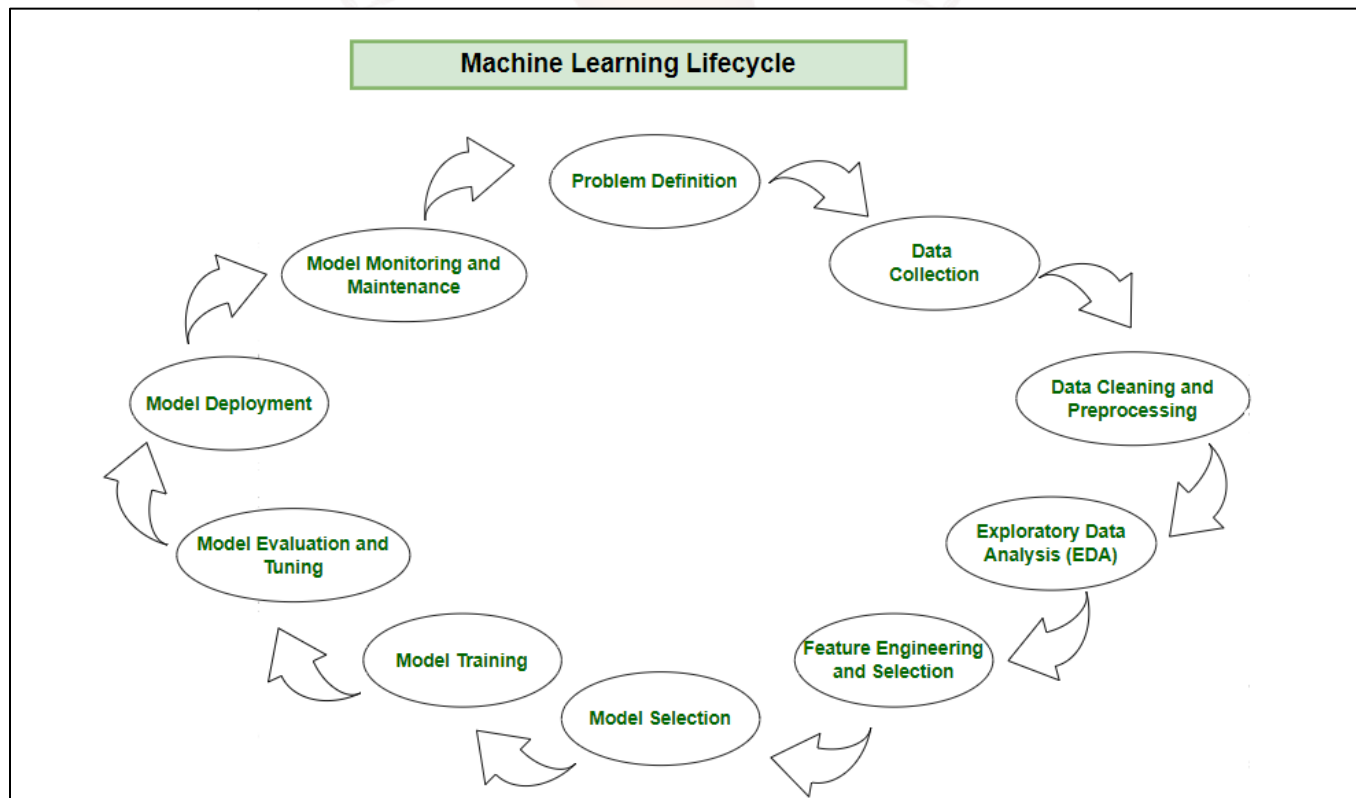


Figure 2: Life cycle of Machine Learning

- Defining the Problem: Find the actual issue that needs fixing.
- Data Collection: Get the info you need from all the right places.
- Data Cleaning and Pre-processing: Get the data ready for analysis by fixing any problems with its quality.
- Exploratory Data Analysis (EDA): Find trends, outliers, and patterns in the data.
- Feature Engineering and Selection: If you want your model to perform better, you need to improve its data features and pick the right ones.
- Model Selection: Based on the nature of the problem and the data, select appropriate models.
- Model Training: Split the dataset in half and use each half to train the model.

- **Model Evaluation and Tuning:** Evaluate and fine-tune the model by utilising pertinent metrics.
- **Model Deployment:** To get predictions in real time, put the model to use in a production setting.
- **Model Monitoring and Maintenance:** Keep the model up-to-date and accurate by checking it regularly.



4. TYPES OF MACHINE LEARNING

1. Supervised Machine Learning

Supervised learning algorithms are taught using the dataset that has labels. Using labelled training data, they figure out how to associate input features with targets. In supervised learning, there are primarily two approaches:

Regression: A regression algorithm takes in information as input and uses them to learn how to predict continuous values.

Classification: A classification algorithm is a programme that, given some features, can learn to place pieces of data into predefined categories. Classification produces discrete values as its output labels.

2. Unsupervised Machine Learning

Unsupervised learning algorithm discovers patterns in data without receiving direct training with labelled examples. The objective is to unearth the data's fundamental distribution or structure. There are two main types of unsupervised learning:

Clustering: Data points with similar characteristics are grouped together via clustering algorithms. Find sets of data points that are comparable but unique from one another; this is called a cluster

Dimensionality reduction: Algorithms for dimensionality reduction minimise the amount of input variables in a dataset while maintaining the majority of the original data. To simplify a dataset for better visualisation and analysis, this is helpful.

3. Reinforcement Machine Learning

An agent learns to interact with its environment in Reinforcement Learning by doing actions and then either getting rewards or punishments depending on its actions. Learning a policy—a mapping from states to actions—that maximises the predicted cumulative reward over time is the goal of reinforcement learning.

There are two main types of reinforcement learning:

Model-based reinforcement learning: The environment is modelled by the agent, who learns the probabilities of transitioning between states and the rewards linked to each state-action pair. Following this model's instructions, the agent acts in a way that maximises its expected return.

Model-free reinforcement learning: Because it doesn't bother to construct an explicit model of its surroundings, the agent learns policies by doing. In response to environmental cues and reward computation, the agent modifies its policy.

Various Applications of Machine Learning

- **Automation:** Machine learning, a kind of automation, may operate independently in any domain with little to no oversight from a human. As an example, in manufacturing plants, robots carry out the necessary process processes.
- **Finance Industry:** Machine learning is gaining traction in the financial sector. Mainly, banks are utilising ML to detect patterns within data and also to forestall fraud.
- **Public sector agency:** ML is utilised by the public sector to oversee utility and public safety operations. We can look at China's extensive use of face recognition as an example. In order to curb jaywalking, the government has begun to employ artificial intelligence.
- **Healthcare:** Early adopters of machine learning for image identification were in the healthcare business.
- **Marketing:** With the abundance of data available, AI is being widely used in marketing. Researchers used sophisticated mathematical methods, such as Bayesian analysis, to determine a customer's value before the era of mass data. The marketing department is utilising AI to enhance consumer connections and marketing strategies in response to the deluge of data.
- **One sector that makes use of machine learning is retail,** specifically in the areas of consumer behaviour analysis, demand prediction, and inventory management. Retailers can also use it to make product recommendations to customers based on their tastes and previous purchases, making shopping more personalised.
- **Transportation:** The transportation industry makes use of machine learning to enhance system efficiency, optimise routes, and decrease fuel usage. Additionally, it is utilised in autonomous vehicles, where ML algorithms are employed to determine navigational and safety-related judgements.

Limitations of Machine Learning

Data Availability: In order for machines to learn, there must be enough data for them to do so. One of the biggest problems with machine learning is a dataset that isn't diverse enough. Heterogeneity is required: In order to derive useful insights, diverse and varied data is essential. When datasets have very little variation, algorithms struggle to extract useful information. Typically, at least twenty observations per group are recommended for effective learning.

Lastly, gaining an understanding of machine learning unveils a reality where computers analyse and acquire new knowledge from data in order to generate predictions and conclusions. Combining CS with statistics, this area enables systems to gradually improve performance without direct programming. Machine learning is becoming more important in our daily lives due to the potential it has to revolutionise how we interact with technology.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following best describes machine learning? a) Programming all rules explicitly b) Learning from data without explicit programming c) Only using mathematical equations d) Only storing and retrieving data
2	Which of the following is not a type of machine learning? a) Supervised learning b) Unsupervised learning c) Reinforcement learning d) Descriptive learning
3	Which field is machine learning a part of? a) Networking b) Artificial intelligence c) Database administration d) Web development

5. SUPERVISED LEARNING

The two most common forms of machine learning are supervised and unsupervised. In supervised learning, each input is paired with an output, and the model is trained using this labelled data. In contrast, unsupervised learning does not rely on labels to guide the model's training; instead, it uses the data itself to discover hidden structures, patterns, and correlations.

At its core, supervised machine learning is an AI and machine learning methodology. Using labelled data, in which each input has an associated correct outcome, is what is used to train the model. The term 'supervised' refers to the use of labelled data, guiding the model during training. In this post, we will go into the fundamentals of supervised learning, the various supervised machine learning algorithms, and their applications through real-world examples.

Training a model in supervised learning, as we discussed before, involves associating each input with its corresponding output using labelled data. As it compares its predictions to the actual answers in the training data, the model learns. It learns and adapts over time to become more precise and error-free. Making reliable predictions using previously unseen data is the holy grail of supervised learning. When presented with new numbers, a model that has been trained to recognise handwritten digits will be able to accurately identify them.

The versatility of supervised learning makes it an indispensable tool for supervised data mining and artificial intelligence applications such as supervised learning classification and supervised learning regression.

Learning a class from examples is a basic idea in supervised machine learning. For example, to teach a model to distinguish between pictures of cats and dogs, you would show it examples of each type of image and label them accordingly. Afterwards, the model uses what it has learnt about the unique characteristics of each class to the classification of fresh photos.

To illustrate the point, consider a labelled dataset consisting of pictures of elephants, camels, and cows. Each image would be tagged with one of these words.

Example to Understand

For example, to teach a model to recognise different fruits, labelled images with features such as texture, colour, and shape are used during training. The trained model then compares new images to these learned characteristics to classify unknown fruits.

Take the following scenario as an example: we train the machine by exposing it to fruits sequentially:

- An apple is a red fruit that is spherical, with a little indentation on top;
- A banana is a long, curved fruit that is greenish-yellow; and so on.

Now that it has been trained, we can test its knowledge by asking it to recognise a different fruit (let's say a banana) from the basket. Based on its appearance, it will determine that the new fruit is a banana and assign it to the appropriate group. Machines can learn to identify new, unlabelled fruits by applying what they've learnt from the training data, which is a basket of labelled fruits.

How Supervised Machine Learning Works?

In a supervised learning method, each input feature is accompanied by an associated label for the output. This procedure is carried out by:

- **Training Data:** The model receives its input data in the form of features and its output data in the form of labels or target variables from a training dataset.
- **Learning Process:** By analysing the training data, the algorithm learns how to correlate the attributes inputted with their corresponding labels. The model's parameters are fine-tuned in this way such that the discrepancy between the predicted and real labels is kept to a minimum.

The model is then tested on a test dataset to see how well it performs and how accurate it is after training. After that, methods like cross-validation are used to optimise the model's performance by balancing the variance and bias. This guarantees that the model remains applicable to novel, previously unknown data.

To review, supervised machine learning is teaching a model to accurately predict outcomes from unlabelled data by first learning patterns and correlations in the labelled data.

Let's have a look at the steps involved in training a supervised machine learning model to learn the mapping function between input and output:

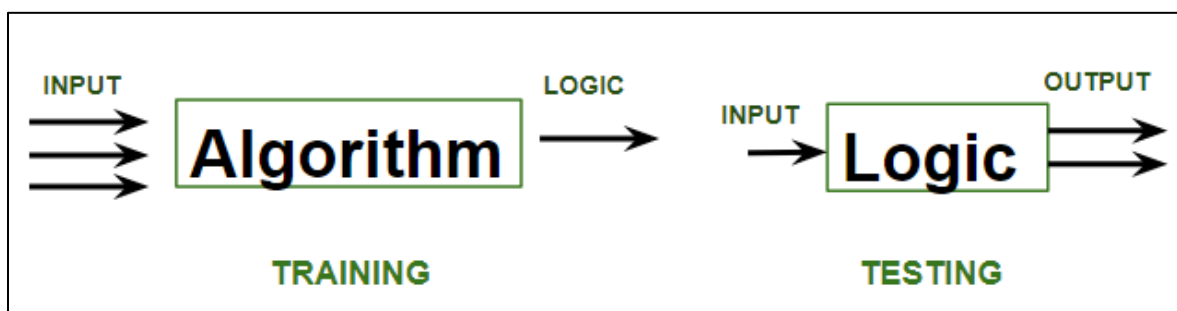


Figure 3: Machine Learning model mapping function between input and output

In the image above,

- **Training:** Each data point is associated with its correct output during the training phase of the algorithm, which is known as labelled data. Over time, the algorithm becomes adept at recognising correlations and patterns in both the input and output data.
- **Testing:** In the testing step, we give the algorithm new, unseen data and check how well it can use the patterns it learnt to predict the right output.

Types of Supervised Learning in Machine Learning

Supervised learning can now be used for two primary problem types:

- Classification:** If the result is a yes or no answer, for example, we say that the output is a categorical variable; this is the case in classification.
- Regression:** In regression, the output is a continuous variable, such as the price of a house or the value of a stock.

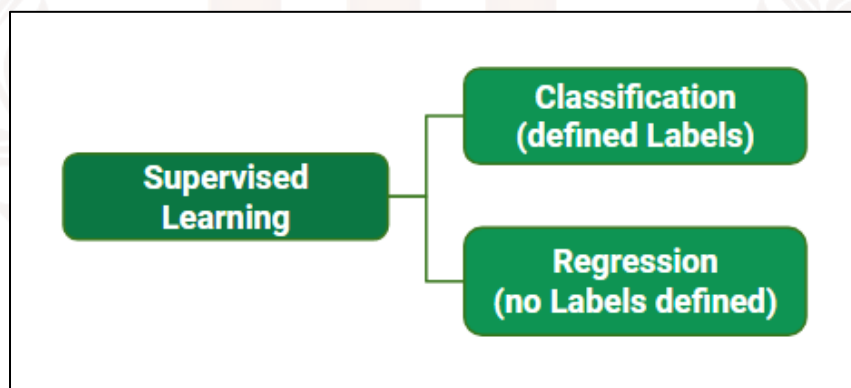


Figure 4: Types of Supervised Learning

Typically, when training the model, the data is divided into two parts: 80% for training and 20% for testing. We include both input and output for 80% of the data in the training set. All the model needs to learn is the training data. In order to construct our model, we employ various supervised learning techniques, which will be covered in depth in the section that follows. Take a look at the table below to get a feel for the regression and classification data:

User ID	Gender	Age	Salary	Purchased	Temperature	Pressure	Relative Humidity	Wind Direction	Wind Speed
15624510	Male	19	19000	0	10.69261758	986.882019	54.19337313	195.7150879	3.278597116
15810944	Male	35	20000	1	13.59184184	987.8729248	48.0648859	189.2951202	2.909167767
15668575	Female	26	43000	0	17.70494885	988.1119385	39.11965597	192.9273834	2.973036289
15603246	Female	27	57000	0	20.95430404	987.8500366	30.66273218	202.0752869	2.965289593
15804002	Male	19	76000	1	22.9278274	987.2833862	26.06723423	210.6589203	2.798230886
15728773	Male	27	58000	1	24.04233986	986.2907104	23.46918024	221.1188507	2.627005816
15598044	Female	27	84000	0	24.41475295	985.2338867	22.25082295	233.7911987	2.448749781
15694829	Female	32	150000	1	23.93361956	984.8914795	22.35178837	244.3504333	2.454271793
15600575	Male	25	33000	1	22.68800023	984.8461304	23.7538641	253.0864716	2.418341875
15727311	Female	35	65000	0	20.56425726	984.8380737	27.07867944	264.5071106	2.318677425
15570769	Female	26	80000	1	17.76400389	985.4262085	33.54900114	280.7827454	2.343950987
15606274	Female	26	52000	0	11.25680746	988.9386597	53.74139903	68.15406036	1.650191426
15746139	Male	20	86000	1	14.37810685	989.6819458	40.70884681	72.62069702	1.553469896
15704987	Male	32	18000	0	18.45114201	990.2960205	30.85038484	71.70604706	1.005017161
15628972	Male	18	82000	0	22.54895853	989.9562988	22.81738811	44.66042709	0.264133632
15697686	Male	29	80000	0	24.23155922	988.796875	19.74790765	318.3214111	0.329656571
15733883	Male	47	25000	1					

Figure A: CLASSIFICATION

Figure B: REGRESSION

Figure 5: Output of classification and regression- Dataset

Both the above figures have labelled data set as follows:

- Figure A: This dataset from a retail establishment can be utilised to forecast a customer's propensity to buy a specific product, taking into account their gender, age, and income level.
 Input: Gender, Age, Salary
 Output: The customer's intention to buy, expressed as a number between zero and one; zero indicates no intention to buy, and one indicates yes.
- Figure B: Predicting wind speed from a variety of criteria is the job of this meteorological dataset.
 Input: Dew Point, Temperature, Pressure, Relative Humidity, Wind Direction
 Output: Wind Speed

A small number of real-world applications of supervised machine learning in several fields:

Fraud Detection in Banking: Training models with labelled datasets of normal and fraudulent transactions allows for accurate prediction of fraud patterns using supervised learning techniques applied to historical transaction data.

Parkinson Disease Prediction: The central nervous system and the muscles and organs that are under its control are gradually weakened in Parkinson's disease.

Customer Churn Prediction: Effectively predicts customer retention by analysing previous customer data using supervised learning approaches to uncover variables related with churn rates.

Cancer cell classification: Uses supervised learning to determine whether cancer cells are "malignant" or "benign" according to their characteristics.

Stock Price Prediction: Uses supervised learning to make a signal prediction about the usefulness of a stock purchase.

Supervised learning is further subdivided into a number of varieties, each of which has its own distinct traits and uses. Most supervised learning algorithms fall into the following categories:

Linear Regression: To forecast a continuous output value, supervised learning regression algorithms like linear regression are employed. Among supervised learning algorithms, it is among the most popular and easiest to understand.

Logistic Regression Predicting a binary output variable is the job of logistic regression, a kind of supervised learning classification technique.

Decision Trees: A decision tree is a graphical representation of a decision-making process and the outcomes that may result from it. The tree's internal nodes stand for decisions, and the leaf nodes for potential outcomes.

Random Forests: One further way that several decision trees collaborate to produce predictions is in random forests. The input characteristics and data are trained on distinct subsets of trees in the forest. All the projections from the trees in the forest are combined to make one final prediction.

Support Vector Machine (SVM): For the purpose of classifying new data points into the appropriate category, the SVM method constructs a hyperplane to divide n-dimensional space into classes. Support Vector Machine gets its name from the fact that it uses support vectors, which are the most extreme examples, to assist build the hyperplane.

K-Nearest Neighbors (KNN): Finding the k training samples that are geographically nearest to an input is the first step in KNN's prediction process. From there, the model takes the average or majority class of these neighbours into account. The distance metric used to determine proximity and the value of k can both affect KNN's performance.

Gradient Boosting Gradient Boosting is a method for building stronger models by merging weak learners, such as decision trees. It builds new models iteratively, fixing mistakes produced by earlier ones.

Naive Bayes Algorithm: Assuming, "naively," that features are unrelated to one another in light of the class label, the Naive Bayes algorithm applies Bayes' Theorem to supervised machine learning problems.

Let's summarise the supervised machine learning algorithms in a table:

Algorithm	Regression, Classification	Purpose	Method	Use Cases
Linear Regression	Regression	Predict continuous output values	Linear equation minimising sum of squares of residuals	Predicting continuous values
Logistic Regression	Classification	Predict binary output variable	Logistic function transforming linear relationship	Binary classification tasks
Decision Trees	Both	Model decisions and outcomes	Tree-like structure with decisions and outcomes	Classification and Regression tasks
Random Forests	Both	Improve classification and regression accuracy	Combining multiple decision trees	Reducing overfitting, improving prediction accuracy
SVM	Both	Create hyperplane for classification or predict continuous values	Maximising margin between classes or predicting continuous values	Classification and Regression tasks
KNN	Both	Predict class or value based on k closest neighbours	Finding k closest neighbors and predicting based on majority or average	Classification and Regression tasks, sensitive to noisy data
Gradient Boosting	Both	Combine weak learners to create strong model	Iteratively correcting errors with new models	Classification and Regression tasks to improve prediction accuracy

Algorithm	Regression, Classification	Purpose	Method	Use Cases
Naive Bayes	Classification	Predict class based on feature independence assumption	Bayes' theorem with feature independence assumption	Text classification, spam filtering, sentiment analysis, medical

Various forms of supervised learning in machine learning can be employed according to the nature of the problem and the dataset at hand. Assigning inputs to established classes is the goal of classification issues, whereas numerical results are the focus of regression problems.

6. TRAINING A SUPERVISED LEARNING MODEL

Key Steps

Being able to generalise effectively to new data is the holy grail of supervised learning. There are a number of critical processes in training a model for supervised learning, all with the goal of preparing the model to make good predictions or judgements using labelled data. Here are the main things you need to know to train a supervised machine learning model:

- **Data Collection and Preprocessing:** Collect input features and labels for the desired output to create a labelled dataset. For supervised learning algorithms to work well, it is necessary to clean the data, deal with missing values, and scale features as required.
- **Splitting the Data:** Make a training set with 80% of the data and a test set with 20%.
- **Choosing the Model:** Consider the nature of the problem while choosing an algorithm. For AI supervised learning to be effective, this stage is important.
- **Training the Model:** Provide the model with input data and output labels so it can learn patterns through parameter adjustments.
- **Evaluating the Model:** Evaluate the trained model's performance using several measures by running it on the unseen test set.
- **Hyperparameter Tuning:** Apply methods such as grid search and cross-validation to fine-tune the training process's parameters (such as the learning rate).
- **Final Model Selection and Testing:** To make sure the model is ready for deployment, retrain it on the whole dataset with the optimal hyperparameters and test its performance on the test set.
- **Model Deployment:** Use the verified model to forecast using fresh, unlabelled data.

Supervised learning models can be trained to do a wide range of tasks, from learning a class from examples to making predictions in real-world applications, by following these stages.

Advantages of Supervised Learning

In many contexts, supervised learning shines because of its precision in pattern prediction and data-driven decision-making. The benefits of supervised learning are as follows:

- Pattern prediction and data-driven decision-making are areas where supervised learning truly shines.
- For supervised learning models to efficiently learn input-output correlations, labelled training

data is important.

Examples of supervised machine learning tasks include supervised learning regression and supervised learning classification.

- Image recognition and NLP are two examples of the complicated challenges that can be solved with these applications.
- When evaluating the performance of a supervised learning model, it is vital to use the established evaluation measures, which include accuracy, precision, recall, and F1-score.
- Supervised learning relies on large amounts of labelled training data and relies on data that is both representative and of high quality in order to create complicated models that can accurately predict outcomes from fresh data.

Disadvantages of Supervised Learning

There are some serious drawbacks to supervised learning, even though it has many advantages:

1. **Overfitting:** In supervised machine learning, models have a tendency to overfit training data, which results in subpar performance when presented with new data.
2. **Feature Engineering:** In supervised learning applications, extracting relevant features is vital, but it can be time-consuming and requires domain expertise.
3. **Model Bias:** Unfair predictions in supervised learning algorithms can be caused by bias in the training data.
4. **Depending on Labelled Data:** One of the biggest problems with supervised learning is how dependent it is on labelled training data, which may be difficult and expensive to acquire.

An effective subfield of machine learning, supervised learning is based on training a class using pre-existing examples. Models can be taught to provide predictions from labelled data through the use of supervised learning techniques. Supervised machine learning is great for many things, like picture identification and financial forecasting, because it can generalise from the training data to new, unknown data. If you want to pick the right algorithm to fix certain problems, you need to know what kinds of supervised learning algorithms are out there and how supervised machine learning works. Supervised learning is already having a significant influence on machine learning, and it will continue to play an important role in the advancement of AI-driven solutions as we delve deeper into the various forms of supervised learning and improve these methods.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
4	In supervised learning, the training data includes: a) Only input data b) Only output data c) Input data and corresponding labels d) No data at all
5	Which of the following is an example of a supervised learning task? a) Customer segmentation without known groups b) Predicting house prices based on features c) Discovering frequent item sets in transactions d) Grouping news articles automatically
6	Which algorithm is commonly used for classification in supervised learning? a) K-Means b) Decision Tree c) Principal Component Analysis (PCA) d) Apriori algorithm

7. UNSUPERVISED LEARNING

Machine learning's unsupervised learning subfield handles data without labels. In unsupervised learning, the algorithms are not given any predetermined meaning for the data; instead, they are instructed to discover patterns and correlations within the data without any labels or labels applied to it. Bypassing human oversight, or "output" in the case of supervised machine learning, allows unsupervised algorithms to discover previously unseen patterns and data. The algorithm automatically finds clusters or patterns in the data using only input features, without output labels.

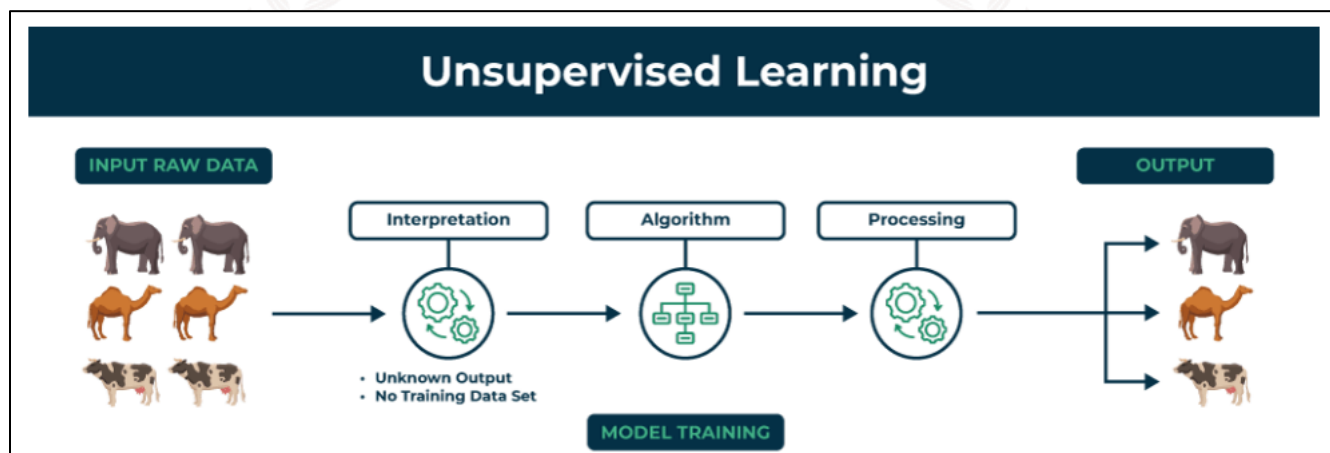


Figure 6: Representation of unsupervised learning

The picture depicts a group of animals, including elephants, camels, and cows, which stand in for the raw data that the unsupervised learning algorithm will analyse. The algorithm lacks predetermined labels or categories for the data when it reaches the "Interpretation" step. Finding ways to categorise or arrange the data according to underlying patterns is something it must accomplish.

Clustering, dimensionality reduction, and anomaly detection are algorithms that unsupervised learners use to find structures and patterns in data. The algorithm is shown processing the data during this stage. The unsupervised learning process's outcomes are displayed in the output. The algorithm may have done something similar here, classifying the animals into groups according to their species (e.g., cows, camels, and elephants).

How does unsupervised learning work?

To find patterns and correlations in unlabelled data, unsupervised learning analyses it. Since no labels or results have been applied to the data, the algorithm is left to its own devices to discover the linkages

and patterns. This process can be challenging, but it often reveals data insights that might be overlooked in labelled datasets.

In Figure A, we can see a data collection representing a mall that includes information about its subscribers. After signing up, customers receive a membership card that allows the mall to track all of their purchases. With this information and some unsupervised learning algorithms, the shopping centre can now classify customers according to the criteria we provide.

CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Female	30	19	72
11	Male	67	19	14
12	Female	35	19	99
13	Female	58	20	15
14	Female	24	20	77
15	Male	37	20	13
16	Male	22	20	79
17	Female	35	21	35

Figure 7: Representation of the dataset for the unsupervised learning

The following data is fed into the unsupervised learning models:

- **Unstructured Data:** Data that is not organised can include things like unknown or noisy numbers, missing information, or both.
- **Data without labels:** This type of data merely includes the values of the input parameters and does not include any intended output values. Its collection is simpler than the supervised method's labelled one.

Unsupervised Learning Algorithms

There are mainly three types of Algorithms which are used for Unsupervised dataset.

- Clustering
- Association Rule Learning
- Dimensionality Reduction

1. Clustering Algorithms

The term "clustering" refers to the step in unsupervised machine learning whereby related pieces of unlabelled data are grouped together. Clustering works by first attempting to discover links and patterns in the data without knowing what the data actually means.

Data is typically grouped using this method when our computer model identifies certain patterns, such as similarities or differences. The purpose of these algorithms is to categorise raw, unclassified data objects. We will use this method to classify clients according to the input parameters supplied by our data; for instance, we haven't specified any values for the output parameters in the previous figure.

Some common clustering algorithms:

- K-means Clustering: Groups data into K clusters based on how close the points are to each other.
- Hierarchical Clustering: Creates clusters by building a tree step-by-step, either merging or splitting groups.
- Density-Based Clustering (DBSCAN): Finds clusters in dense areas and treats scattered points as noise.
- Mean-Shift Clustering: Discovers clusters by moving points toward the most crowded areas.
- Spectral Clustering: Groups data by analysing connections between points using graphs.

2. Association Rule Learning

One popular method in unsupervised machine learning for finding associations is association rule learning, which is also called association rule mining. As a rule-based ML method, it can sift through mountains of data in search of meaningful correlations between parameters. The primary application of this method is in market basket analysis, which seeks to clarify the interrelationships between various goods.

For instance, this method is used by algorithms at retail establishments to determine the correlation between the sales of different products in relation to each other, all based on client behaviour. For example, a consumer may also purchase bread, eggs, and butter in addition to milk. In order to boost sales, these models might be trained to plan various offers.

Some common Association Rule Learning algorithms:

- Apriori Algorithm: Finds patterns by exploring frequent item combinations step-by-step.

- **FP-Growth Algorithm:** An Efficient Alternative to Apriori. It quickly identifies frequent patterns without generating candidate sets.
- **Eclat Algorithm:** Uses intersections of itemsets to efficiently find frequent patterns.
- **Efficient Tree-based Algorithms:** Scales to handle large datasets by organizing data in tree structures.

3. Dimensionality Reduction

Reducing the number of features in a dataset while retaining as much information as feasible is called dimensionality reduction. Data visualisation and machine learning algorithm performance can both benefit from this method.

Suppose there was a dataset containing one hundred attributes pertaining to pupils, such as their weight, height, grades, etc. If you want to zero in on important characteristics, you can simplify the data by removing all but two variables: height and grades.

A few well-known algorithms for dimensionality reduction are as follows:

- **Principal Component Analysis (PCA):** Reduces dimensions by transforming data into uncorrelated principal components.
- **Linear Discriminant Analysis (LDA):** Reduces dimensions while maximizing class separability for classification tasks.
- **Non-negative Matrix Factorization (NMF):** Breaks data into non-negative parts to simplify representation.
- **Locally Linear Embedding (LLE):** Reduces dimensions while preserving the relationships between nearby points.
- **Isomap:** Captures global data structure by preserving distances along a manifold.

Challenges of Unsupervised Learning

Here are the key challenges of unsupervised learning:

- Outliers and noise in the data might cause patterns to be distorted and algorithms to be less effective.
- It's not uncommon for algorithms to depend on assumptions (such cluster shapes) that don't always correspond to the real data structure. This is known as assumption dependence.
- The danger of overfitting arises when models identify irrelevant patterns in the data as overfitting.

- Insufficient Direction: Without labels, it's difficult to direct the algorithm to produce desired results.
- There may be a lack of clarity or congruence with real-world categories in the results, such as clusters, which can affect their interpretability.
- Parameter Sensitivity: The number of clusters in k-means and other algorithms like it need to be fine-tuned.
- Inadequate Ground Truth: Since unsupervised learning does not use labelled data, it is difficult to assess the reliability of the outcomes.

Applications of Unsupervised learning

There are many different sectors and domains that can benefit from unsupervised learning. Here are some important uses:

- Customer Segmentation: Algorithms group clients according to their demographics or buying habits, allowing for more specific advertising.
- Anomaly Detection: This feature helps with fraud detection, cybersecurity, and preventing equipment failure by identifying unexpected patterns in data
- Recommendation Systems: These systems analyse user behaviour and preferences to suggest items, movies, or music. For purposes such as organisation, categorisation, and content suggestion, image and text clustering can be useful.
- Social network analysis: it finds groups or patterns in how people are interacting on social media.
- Climate and Astronomy: Assists with study by categorising weather patterns or galaxies

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
7	Which of the following tasks is best suited for unsupervised learning? a) Classifying emails as spam or not spam b) Grouping similar customers together c) Predicting stock prices d) Diagnosing diseases from patient data with known outcomes
8	Which algorithm is commonly used for clustering? a) Linear Regression

	b) K-Means c) Support Vector Machine d) Random Forest
9	Unsupervised learning works with: a) Labeled data b) Unlabeled data c) Only numerical data d) Only text data



8. REINFORCEMENT LEARNING

The goal of the machine learning subfield known as Reinforcement Learning (RL) is to teach agents to maximise their cumulative rewards by making decisions based on past mistakes. RL enables robots to acquire knowledge through engaging with their surroundings and obtaining responses that are derived from their activities. Rewards or punishments are the forms this feedback takes.

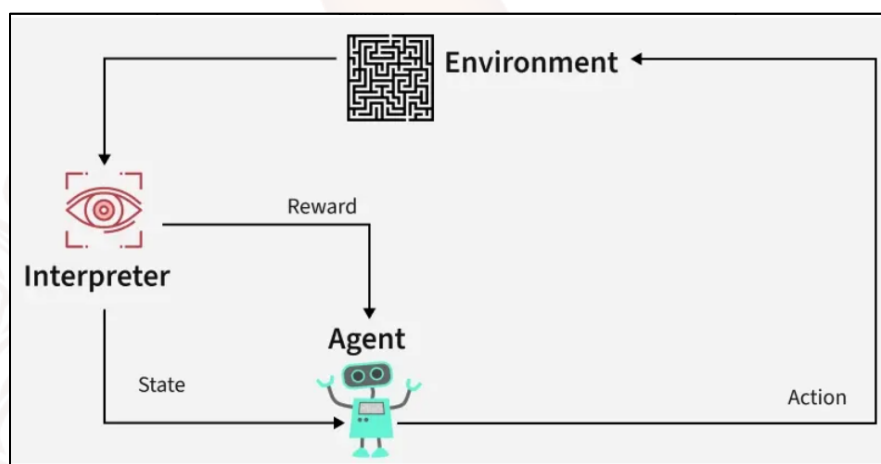


Figure 8: Representation of Reinforcement learning

Reinforcement An agent (the learner or decision-maker) engages with their environment to accomplish a goal; this is the core tenet of learning. In order to improve its decision-making over time, the agent takes action and gets feedback.

- **Agent:** The decision-maker that performs actions.
- **Environment:** The world or system in which the agent operates.
- **State:** The situation or condition the agent is currently in.
- **Action:** The possible moves or decisions the agent can make.
- **Reward:** The feedback or result from the environment based on the agent's action.

How Reinforcement Learning Works?

In reinforcement learning (RL), an agent acts in a given environment, gets feedback in the form of rewards or penalties, and then adjusts its behaviour in response. In order to maximise the cumulative reward, this loop aids the agent in making better decisions as time goes on.

A list of the parts that make up RL is as follows:

- **Policy:** The agent's method for deciding what to do next given its current state.
- The reward function is a tool for leading agents towards their goals by providing feedback on

their behaviour.

- The Value Function predicts the total future rewards that an agent can expect to get from a specific condition.
- Environmental Model: A depiction of the environment that helps with planning by predicting future conditions and rewards.

Reinforcement Learning Example: Navigating a Maze

Envision a robot avoiding fire risks as it solves a maze in search of a diamond. Finding the best route that maximises reward with the fewest risks is the objective:

- When the robot does the right thing, it gets a reward; when it messes up, it loses points.
- As it explores the maze, the robot learns its way around. It calculates the benefits and drawbacks of each possible manoeuvre by testing them out.
- The optimal path is gradually determined by the robot by observing which activities result in the biggest cumulative reward.

What follows is a synopsis of the robot's learning procedure:

1. First, there's exploration. The robot will first try to find every conceivable way out of the maze by doing something new at each turn (such as turning left, right, up, or down).
2. Ecosystem Feedback: Following every action, the robot gets environmental feedback in the form of: A bonus for approaching the diamond in close proximity and a negative outcome for approaching a fire hazard.
3. Adapting Actions: The robot takes this information into account and changes its actions in order to maximise the total reward. It will choose routes that minimise danger and get it closer to the diamond.
4. Optimal Path: Using its prior experiences as a guide, the robot eventually finds the path that has the fewest hazards and the most reward.

Types of Reinforcements in RL

1. Positive Reinforcement

When a desired action is rewarded with a stronger and more frequent occurrence, this is called positive reinforcement. Put simply, it influences behaviour in a favourable way. Maximises performance and aids in maintaining change over time; these are the advantages. One potential drawback is that it can become less effective if used too much.

2. Negative Reinforcement

The term "negative reinforcement" describes the process by which a behaviour is reinforced when a bad circumstance is either eliminated or mitigated. Benefits: raises the frequency of behaviour and guarantees a baseline level of performance. Drawbacks: activity may be incentivised to the point where penalties are avoided, but not beyond that.

CartPole in OpenAI Gym

In OpenAI Gym's CartPole environment, one of the fundamental RL challenges is to maintain a pole's equilibrium on a cart. So long as the agent keeps the cart from toppling over, he or she can steer it to the left or right.

- The state space provides a description of the cart-pole system's four critical variables: position, velocity, angle, and angular velocity.
- The action space: Two choices, left and right, to move the cart.

The agent receives a reward for every timestep the pole remains balanced, encouraging effective strategies.

Application of Reinforcement Learning

1. First, robotics: RL helps robots automate jobs in structured contexts like factories by teaching them to move more efficiently and with less effort.
2. Gaming: State-of-the-art RL algorithms have surpassed human players at numerous games, including video games, chess, and Go.
3. Industrial Control: RL is useful for optimising and adjusting industrial operations in real-time, like oil and gas refining processes.
4. Systems for Personalised Training: RL allows for the personalisation of course materials according to each learner's unique learning style, which boosts interest and efficiency.

Advantages of Reinforcement Learning

RL can solve extremely complicated problems that are intractable using more traditional methods.

- Error Correction: The model is able to fix training-related mistakes because it learns from its surroundings continuously.
- Direct Environment Interaction: RL agents are able to learn through adaptive learning through real-time interactions with their environment.
- Dealing with Non-Deterministic Environments: RL shines in practical settings where results are unpredictable or subject to change, making it a valuable tool for several industries.

Disadvantages of Reinforcement Learning

RL is frequently overkill for simple jobs where simpler algorithms would be more effective, hence it's not a good fit for these types of problems.

- **Excessive Computing Needs:** RL model training is extremely resource-intensive because to the large amounts of data and computing resources it demands.
- The success of RL is highly dependent on how the reward mechanism is designed. Inadequately planned incentives have the potential to elicit undesirable or suboptimal actions.
- Complicated debugging and troubleshooting due to difficulty in understanding RL agents' decision-making processes

When dealing with uncertain or changing conditions, Reinforcement Learning is an effective method for optimisation and decision-making. Nevertheless, RL's intricacy calls for well-thought-out reward mechanisms and a lot of computing power. Learning RL's ins and outs will allow you to use it to drive innovation across many different industries while solving complex real-world problems.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
10	In reinforcement learning, the agent learns through: a) Fixed rules only b) Predefined outputs c) Rewards and penalties from the environment d) Labelled data
11	Which of the following is a key challenge in reinforcement learning? a) Designing reward functions b) Handling labelled data c) Performing clustering d) Calculating static values only
12	What is a common application of reinforcement learning? a) Predicting housing prices b) Image segmentation c) Training game-playing agents like AlphaGo d) Grouping shopping items

9. SUMMARY

- Machine learning is a powerful and rapidly evolving field of artificial intelligence that enables computers to learn patterns and make decisions from data without being explicitly programmed step by step.
- Inspired by the way humans learn from experience, machine learning models analyse large amounts of data to recognise hidden trends and relationships, which they then use to make predictions or automate tasks.
- There are three major types: supervised learning, where models learn from labelled data; unsupervised learning, which finds patterns in unlabelled data; and reinforcement learning, where systems learn through rewards and penalties by interacting with their environment.
- Machine learning is at the heart of many modern technologies we use daily — from personalised recommendations on streaming services to fraud detection in banking, medical diagnosis, and self-driving cars.
- Despite its enormous potential, machine learning also comes with challenges such as the need for large, high-quality datasets, avoiding biases, and ensuring that models remain interpretable and ethical.
- Overall, machine learning represents a transformative approach to solving complex problems, enabling smarter, adaptive, and more efficient systems that continuously improve over time.
- Supervised learning offers a powerful and practical approach to building predictive models by learning from labelled examples. Its ability to generalise from past data allows it to make accurate predictions on new, unseen cases, making it highly valuable in real-world applications.
- As more data is collected and the model is retrained, its accuracy and reliability continue to improve over time.
- Supervised learning is versatile, successfully handling both classification tasks — such as sorting emails into spam or non-spam — and regression tasks like predicting prices or medical measurements.
- Because of this flexibility and strong performance, supervised learning has become one of the most widely used techniques across various fields, including healthcare, finance, marketing, and beyond.
- Unsupervised learning is a type of machine learning that works with data without any predefined labels or outputs. The main goal of unsupervised learning is to uncover hidden patterns,

structures, or relationships within the data on its own. It is widely used for tasks like clustering, dimensionality reduction, association rule learning, and anomaly detection.

- Popular applications include customer segmentation, image and video compression, recommendation systems, and fraud detection. A major advantage of unsupervised learning is that it does not require labelled data, saving time and effort, and it is excellent for exploring and understanding new datasets.
- However, it also has limitations, such as difficulty in evaluating accuracy, sensitivity to data scaling and parameters, and the possibility of creating meaningless groups without human interpretation.
- Overall, unsupervised learning provides powerful tools to analyse and make sense of large and complex datasets, helping organisations gain valuable insights and make data-driven decisions.
- Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards or penalties. Unlike supervised learning, RL does not require labelled data; instead, it relies on a trial-and-error approach to discover the best strategies for maximising cumulative rewards.
- Key components include the agent, environment, states, actions, and rewards. RL methods can be divided into value-based, policy-based, and model-based approaches, each offering different ways for the agent to learn optimal behaviours.
- It has been successfully applied in various real-world scenarios such as game playing, robotics, self-driving cars, and automated trading. The main advantages of RL include its ability to solve complex problems, adapt through direct interaction, and handle uncertain environments.
- However, it also has challenges, such as high computational requirements, dependence on carefully designed reward functions, and difficulties in debugging and interpreting agent decisions.
- Overall, reinforcement learning is a powerful technique that enables machines to learn from experience and improve their decision-making capabilities over time.

10. GLOSSARY

Machine Learning (ML)	- A field of artificial intelligence where computers learn from data to make predictions or decisions without being explicitly programmed.
Supervised Learning	- A type of machine learning where the model is trained on labeled data, meaning each input has a known output. The goal is to learn a mapping from inputs to outputs.
Unsupervised Learning	- A type of machine learning that works with unlabeled data. The model tries to find hidden patterns or groupings in the data without predefined outputs.
Reinforcement Learning (RL)	- A learning approach where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards or penalties.
Label	- The correct output or target value in supervised learning used to train the model.
Feature	- An individual measurable property or characteristic of the data used as input for machine learning models.
Classification	- A supervised learning task where the goal is to assign inputs into predefined categories or classes.
Regression	- A supervised learning task focused on predicting continuous numeric values.
Clustering	- An unsupervised learning technique that groups similar data points together into clusters.
Dimensionality Reduction	- An unsupervised technique used to reduce the number of input variables while retaining important information, often for simplification or visualisation.
Association Rule Learning	- An unsupervised method to find relationships or associations among variables in large datasets (e.g., items frequently bought together).

Agent	- In reinforcement learning, the entity that makes decisions and learns from interactions with the environment.
Environment	- The system or world in which the reinforcement learning agent operates and makes decisions.
Action	- A choice made by the agent that affects the environment.
State	- A representation of the current situation of the agent within the environment.
Reward	Feedback from the environment to the agent, indicating the success or failure of an action.
Policy	A strategy used by the reinforcement learning agent to decide which action to take in each state.
Value Function	An estimate of the expected cumulative reward an agent will receive starting from a particular state.
Exploration	The process of trying new actions to discover their effects and gather more information.
Exploitation	Using known information to take the best action and maximise reward based on current knowledge.

11. TERMINAL QUESTIONS

1. What is the main difference between supervised and unsupervised learning?
2. Give two examples of applications that use supervised learning.
3. What is clustering, and which type of learning does it belong to?
4. What is reinforcement learning, and how does it differ from supervised learning?
5. What is the role of a reward in reinforcement learning?
6. Mention one advantage of using unsupervised learning.
7. What is a policy in reinforcement learning?
8. Name one common algorithm used in supervised learning and one used in reinforcement learning.

12. ANSWERS

Self-Assessment Questions

1. Answer: b) Learning from data without explicit programming
2. Answer: d) Descriptive learning
3. Answer: b) Artificial intelligence
4. Answer: c) Input data and corresponding labels
5. Answer: b) Predicting house prices based on features
6. Answer: b) Decision Tree
7. Answer: b) Grouping similar customers together
8. Answer: b) K-Means
9. Answer: b) Unlabelled data
10. Answer: c) Rewards and penalties from the environment
11. Answer: a) Designing reward functions
12. Answer: c) Training game-playing agents like AlphaGo

Terminal Questions Answers

1. Answer:

In supervised learning, the model is trained using labelled data where the correct output is known.

In unsupervised learning, the model works with unlabelled data and tries to find patterns or groupings without predefined outputs. (Refer to Section 3 for more information)

2. Answer:

Examples include email spam detection (classifying emails as spam or not spam) and house price prediction (predicting continuous price values based on features). (Refer Section 3 for more information)

3. Answer:

Clustering is the process of grouping similar data points together into clusters. It belongs to unsupervised learning. (Refer to Section 5 for more information)

4. Answer:

Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards or penalties. Unlike supervised learning, it does not use labelled data but learns through trial and error. (Refer Section 8 for more information)

5. Answer:

The reward provides feedback to the agent about the effectiveness of its actions, helping it learn to maximise long-term cumulative rewards. (Refer Section 8 for more information)

6. Answer:

Unsupervised learning can find hidden patterns or structures in data without the need for labeled data, making it useful for exploratory data analysis. (Refer Section 7 for more information)

7. Answer:

A policy is a strategy that defines how an agent selects actions based on the current state to maximise rewards over time. (Refer Section 8 for more information)

8. Answer:

A common supervised learning algorithm is the Decision Tree. A common reinforcement learning algorithm is Q-Learning. (Refer Section 5 for more information)

13. REFERENCES

- Artificial Intelligence, Saroj Kaushik Cengage Learning 2014 Edition.
- Artificial Intelligence: Structures and Strategies for Complex Problem Solving, George F Luger Pearson Addison, Wesley 6th Ed, 2008.
- Artificial Intelligence, E Rich, K Knight, and S B Nair Tata Mc-Graw Hill 3rd Ed, 2009.
- Artificial Intelligence: A Modern Approach, Stuart Russell and Peter Norvig Prentice Hall 3rd, 2009.

